

ARM PrimeCell

MultiPort Memory Controller (PL172)

Design Manual

ARM PrimeCell MultiPort Memory Controller (PL172) Design Manual

© Copyright ARM Limited 2002-2004. All rights reserved.

Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.
All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is ARM Confidential (Distributed to ARM staff, product licensees & NDA signatories only).

Product status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com/>

Feedback

ARM Limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on the ARM PrimeCell

If you have any comments or suggestions about this product, please contact your supplier giving:

- The Product name
- A concise explanation of your comments.

Contents

1	ABOUT THIS DOCUMENT	1
1.1	Change history	1
1.1.1	Current status and anticipated changes	1
1.1.2	Change history	1
1.2	References	1
1.3	Terms and abbreviations	1
1.4	Conventions used in this document	2
1.4.1	Signals	2
1.4.2	Bytes, Half-words and Words	2
1.4.3	Bits, bytes, K and M	2
2	SCOPE	3
3	INTRODUCTION	3
4	MPMC DESIGN OVERVIEW	3
4.1	Signal Description	3
4.2	Programmer's Model	14
4.3	Design Hierarchy	21
4.4	MPMC Block Partitioning	23
4.5	AHB Memory Interfaces top level block	24
4.5.1	AHB Memory Interface	26
4.5.2	Endian Block	33
4.6	AHB Register Interface	37
4.7	Arbiter	43
4.8	Buffer Unit	46
4.8.1	Buffer	50
4.8.2	Buffer Controller	52
4.9	Memory Controller	57
4.10	Dynamic Memory Controller	58
4.10.1	Dynamic memory register block	60
4.10.2	Dynamic Memory Read Request Generation Block	62
4.10.3	Dynamic memory data-access-state-machine	65
4.10.4	Dynamic memory mode State Machine	72
4.10.5	Dynamic memory refresh State Machine	74
4.10.6	Dynamic memory SyncFlash State Machine	77

4.10.7	Dynamic memory read data FIFO block	80
4.10.8	Dynamic memory internal register block	83
4.10.9	Dynamic memory latency control block	89
4.11	Static Memory Controller	91
4.11.1	Static memory register block	92
4.11.2	Static memory controller internal interface block	95
4.11.3	Static memory transfer State Machine	103
4.11.4	Static memory timing control block	108
4.11.5	Static memory controller external interface	111
4.12	Memory Controller Interface	116
4.13	Memory Bus Granting Logic	118
4.14	Pad Interface	120
4.15	Clock gating logic	122
4.16	Revision Designator Block	123
4.17	Test Interface Controller	124
5	MPMC IMPLEMENTATION DETAILS	127
5.1	MPMC Top Level Module (Mpmc)	127
5.2	MPMC Core (MpmcCore)	127
5.3	MPMC AHB Memory Interfaces Top Level (MpmcAhbTop)	127
5.3.1	Ahb Memory Interface Block (MpmcAhbif)	128
5.3.2	Endian Block (MpmcEndian)	134
5.4	AHB Register Interface (MpmcRegIf)	135
5.5	Arbiter (MpmcArbiter)	141
5.6	Buffer Unit (MpmcBufUnit)	142
5.6.1	Buffer (MpmcBuffer)	142
5.6.2	Buffer Controller (MpmcBufCntl)	143
5.7	Memory Controller (MpmcMemCntl)	147
5.8	Dynamic Memory Controller (MpmcDyMemCntl)	148
5.8.1	Dynamic Memory Controller Register Block (MpmcDyRegBlk)	148
5.8.2	Dynamic Memory Read Request Generation Block (MpmcDyIntRdGen)	150
5.8.3	Dynamic Memory Controller Data Access State Machine (MpmcDyAxsSM1 and MpmcDyAxsSM2)	151
5.8.4	Dynamic Memory Mode State Machine (MpmcDyModSM)	158
5.8.5	Dynamic Memory Refresh State Machine (MpmcDyRefSM)	161
5.8.6	Dynamic Memory Controller SyncFlash State Machine (MpmcDySyFlash)	163
5.8.7	Dynamic Memory Controller Receive FIFO (MpmcDyFIFO)	166
5.8.8	Dynamic Memory Controller Internal Register Block (MpmcDyIntRegBlk)	167
5.8.9	Dynamic Memory Latency Counter Block (MpmcDyLatCntl)	167
5.9	Static Memory Controller (MpmcStMemCntl)	168

5.9.1	Static memory register block (MpmcStRegBlk)	169
5.9.2	Static memory controller internal interface (MpmcStIntIf)	171
5.9.3	Static memory transfer State Machine (MpmcStTSM)	174
5.9.4	Static memory timing control block (MpmcStTimCntl)	182
5.9.5	Static memory controller external interface (MpmcStExtIf)	184
5.10	Memory Controller Interface (MpmcMemCntlIf)	188
5.11	Memory Bus Granting Logic (MpmcMemBGnt)	189
5.12	Pad Interface (MpmcPadIf)	191
5.13	Clock gating logic (MpmcClkOr)	193
5.14	Revision Designator block (MpmcRevAnd)	194
5.15	MPMC Test Interface Controller (MpmcTIC)	194
6	DESIGN ASSUMPTIONS	195
6.1	Software Assumptions	195
6.1.1	Mode programming/ Initialisation	195
6.1.2	Self Refresh Entry and Exit	196
6.1.3	SyncFlash	196
6.1.4	Low Power Deep Sleep	196
6.1.5	Static memory extended wait transfers	197
6.1.6	Byte Lane State	197
6.1.7	General	197
6.2	System Assumptions	197
6.2.1	Power-on reset memory map	197
6.2.2	Power-on reset values of Input pins	197
6.2.3	Self-refresh mode	198
6.2.4	Write enable connection of static memory devices	198
6.2.5	Address connection of static memory device	198
6.2.6	Entry into Test mode	198
7	MPMC VERIFICATION TESTBENCH	200
7.1	MPMC VHDL Verification Testbench	200
7.1.1	Testbenches	202
7.1.2	Unit Under Test	202
7.1.3	Bus Master Protocol Checker	203
7.1.4	AHB Master/Arbiter Model	203
7.1.5	Decoder	203
7.1.6	Default Slave	207
7.1.7	Protocol Checker	207
7.1.8	Trickbox	207
7.1.9	Trickbox Memory Model	207
7.1.10	Denali Memory Models	208
7.1.11	Simulation environment	208
7.2	MPMC Verilog Verification Testbench	209
7.2.1	Testbenches	211
7.2.2	Unit Under Test	211

7.2.3	Bus Master Protocol Checker	212
7.2.4	AHB Master/Arbiter Model	212
7.2.5	Decoder	212
7.2.6	Default Slave	216
7.2.7	Protocol Checker	216
7.2.8	Trickbox	216
7.2.9	Trickbox Memory Model	216
7.2.10	Denali Memory Models	217
7.2.11	Simulation Environment	217
7.3	Trickbox Description	218
7.3.1	Trickbox Signal Description	219
7.3.2	Trickbox register summary	221
7.3.3	MPMC Static Memory Model (TrickMem)	227
7.4	Testbench Description	236
7.4.1	Denali Memory Models parts used in test benches	236
7.4.2	Testbenches	239
8	VERIFICATION TESTS	245
8.1	Functional Tests	248
8.1.1	HReset value Test	248
8.1.2	POReset Test	248
8.1.3	MpmcRegisterTest	251
8.1.4	Initialisation Routine Check Test	254
8.1.5	Address Toggle Test	255
8.1.6	MPMC enable Test	255
8.1.7	Clock control Test	255
8.1.8	Clock Enable Test	256
8.1.9	Read and refresh alignment Test	256
8.1.10	Refresh Frequency Check Test	257
8.1.11	SelfRefreshChk	257
8.1.12	SelRefPriority Test	257
8.1.13	Chip select polarity Test	258
8.1.14	Busy Insertion Test	258
8.1.15	Bus Degrant Test	258
8.1.16	Endianness Test	259
8.1.17	BigEndian Pin Test	259
8.1.18	Write protect Test	260
8.1.19	Address Mirror Test	260
8.1.20	Controller Busy Test	260
8.1.21	Memory Test	261
8.1.22	Transfer Test	266
8.1.23	Low power mode Test	267
8.1.24	Delay Value Test	267
8.1.25	StWtPg Test	267
8.1.26	StWtPgBufEn Test	268
8.1.27	ExdWaitDelTest	268
8.1.28	BLS Delay Value Test	268
8.1.29	ROM Test	269
8.1.30	LowPwrSdramFunTest	269
8.1.31	SyncFlash Memory Test	270
8.1.32	Random Test1	270
8.1.33	CornerCases1	271
8.1.34	CornerCases2	271

8.1.35	CornerCases3	271
8.1.36	Arb2CornerCase4	272
8.1.37	Arb2HMastLockTest	272
8.1.38	Rel1HburstTest	272
8.1.39	Command Delayed Tests	272
8.1.40	IntegrationTest	273
8.1.41	Latency Tests	273
8.1.42	RefreshMissTest	279
8.1.43	DirectDyDtFtchTest	279
8.1.44	MemBGntTest	280
8.1.45	AHBLockIdleInsertTest1	280
8.1.46	AHBLockIdleInsertTest2	280
8.1.47	CornerCases4	281
8.1.48	CornerCases5	281
8.1.49	CornerCases6	281
8.1.50	CornerCases7	281
8.1.51	CornerCases8	281
8.1.52	MultiPortWrProtTest	282
8.2	Verification Test Vector Generation	282
9	INTEGRATION TESTBENCH	283
9.1	VHDL Integration testbench	283
9.1.1	Bus Master Protocol Checker	284
9.1.2	TIC Protocol Checker	285
9.1.3	Generic AHB Slave	285
9.1.4	Test Vectors	285
9.1.5	Running Simulations	285
9.2	Verilog Integration testbench	285
9.2.1	Bus Master Protocol Checker	287
9.2.2	TIC Protocol Checker	287
9.2.3	Generic AHB Slave	288
9.2.4	Test Vectors	288
9.2.5	Running Simulations	288
9.3	Trickbox Description	288
9.3.1	Generic AHB Slave	288
9.3.2	TIC Watcher Module	291
10	INTEGRATION TESTS	293
10.1	TIC Validation Test Cases	293
10.1.1	Transfer tests	293
10.1.2	Response tests	293
10.2	Integration Tests	294
10.2.1	AMBA signal integration test	294
10.2.2	Non-AMBA intra-chip integration test	294
10.2.3	Primary I/O signal integration test	295
10.3	Test Vector Generation in the Integration Test environment	295

11	CUSTOMISATION	296
11.1	Changing the number of AHB memory interface ports	296
11.2	Reducing memory data bus width	296
11.3	Removal of TIC	296
11.4	Removal of Static Memory Controller	297
11.5	Removal of Dynamic Memory Controller	298
11.6	Removal of TIC and Static memory controller	298
11.7	Removal of TIC and Dynamic memory controller	299
11.8	Reducing memory device support	300

1 ABOUT THIS DOCUMENT

1.1 Change history

1.1.1 Current status and anticipated changes

This document has been reviewed and the review comments have been incorporated.

1.1.2 Change history

Issue	Date	Change
A01	25 th October, 2001	First Draft
A02	5 th December, 2001	Review comments incorporated
A	7 th December, 2001	Review comments incorporated
B01	8 th May, 2002	First Draft for Release 2v0
B02	29 th May, 2002	Review Comments incorporated
B03	8 th November 2002	Dynamic State Machine Arbitration Modification added. AHB Priority implementation in buffer added. Extra test cases added in the verification section to verify the above changes, and to calculate latency.
B	4 th May 2004	Extra test cases added in the verification section.

1.2 References

This document refers to the following documents.

Ref.	Doc. No.	Title
1	ARM IHI0011	AMBA Specification Rev2.0
2	ARM DDI 0215B	ARM PrimeCell MultiPort Memory Controller (PL172) Technical Reference Manual
3	PL172 INTM 0000 B	ARM PrimeCell MultiPort Memory Controller (PL172) Integration Manual
4	ARM DUI 0006	AMBA Compliance Test Suite User Guide
5	ARM DDI 0138	Example AMBA System Technical Reference Manual
6	ARM DUI 0092	Example AMBA System User Guide
7	SC056 TRM 0000 A01	AHB Example Bus Master Technical Reference Manual

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
------	---------

AHB	Advanced High performance Bus, an AMBA bus designed for high-performance peripherals.
AHB master	A device that can send out requests over an AHB bus.
AHB slave	A device that fulfils requests provided by an AHB master.
AMBA	Advanced Microcontroller Bus Architecture.
MPMC	MultiPort Memory Controller.
FIFO	First In First Out Buffer
Mux	Multiplexer
Quad-quad-word buffer or QQWord buffer	A 16-word deep data buffer
TSM	Transfer State Machine
DASM	Data Access State Machine
DASM1	Data Access State Machine 1
DASM2	Data Access State Machine 2
FCSM	Flash Command State Machine
MODSM	Mode state machine
EBI	External bus interface
TIC	Test Interface Controller

1.4 Conventions used in this document

1.4.1 Signals

- Signal names are shown in bold font type.
- Active low signals are prefixed by a lower case 'n'. In the case of AHB signals by 'Hn', or postfixed by 'n'.
- AHB signals are prefixed by an upper case 'H'.
- When a signal is described as being 'Asserted', the actual level depends on whether the signal is active HIGH or active LOW. 'Asserted' means HIGH for active high signals and LOW for active low signals.

1.4.2 Bytes, Half-words and Words

- A byte is 8 bits.
- A half-word is two bytes i.e. 16 bits.
- A word is 4 bytes i.e. 32 bits.

1.4.3 Bits, bytes, K and M

- The suffix 'B' (upper case) is used to indicate BYTES.
- The suffix 'b' (lower case) is used to indicate BITS.

- When used to indicate an amount of memory, the suffix 'k' means 1024.
- When used to indicate a frequency, the suffix 'k' means 1000.
- When used to indicate an amount of memory, the suffix 'M' means $1024^2 = 1048576$.
- When used to indicate a frequency, the suffix 'M' means 1,000,000.

2 SCOPE

This document describes the design and implementation of the ARM PrimeCell PL172 MPMC (MultiPort Memory Controller). The design description is split between three sections – Section 4, 5 and 6. The testbenches and the test programs for functional verification and integration testing are described in Sections 7 to 10.

Section 4 presents an overview of the MPMC design, describing the functional partitioning of the MPMC and the signal interconnections. Section 5 presents detailed descriptions on the MPMC design. Section 6 describes the design assumptions. Section 7 describes the testbench and Section 8 describes the test programs used to generate test vectors for functional verification of the MPMC. Section 9 describes the testbenches and Section 10 describes the test programs used to generate test vectors for Integration Testing of the MPMC. Section 11 describes the customisations that are possible in the MPMC PL172.

3 INTRODUCTION

The MPMC is a part of a library of reusable IP blocks, which can be more easily integrated into a typical ASIC design flow.

For further information on this block refer to the ARM MPMC Controller (PL172) Technical Reference Manual [Ref.2]. The ARM PrimeCell MPMC Controller (PL172) Integration Manual [Ref.3] describes how the block may be integrated into a typical industry standard ASIC design flow.

The AMBA Compliance Test Suite User Guide [Ref.4] is a source for further information about AMBA bus compliance testbenches.

The Example AMBA System Technical Reference Manual [Ref.5] and User Guide [Ref.6] describe an example micro-controller based on an ARM processor and the low-power, generic design methodology of AMBA. Further information can also be found on use of TICTalk test vectors for production test.

4 MPMC DESIGN OVERVIEW

The MultiPort Memory Controller (MPMC) is an Advanced Microcontroller Bus Architecture (AMBA) block that connects to the Advanced High-performance Bus (AHB). There is one AHB slave interface for programming the MPMC and 4 AHB slave interfaces for accessing the memory devices that are connected to the MPMC. However, the MPMC (PL172) can support upto 8 AHB slave interfaces with some minor modifications. The MPMC (PL172) controls memory devices connected to 8 chip selects, four of which are asynchronous static memories like ROMs, page mode ROMs, Flash devices and SRAMs. The other four chip selects control dynamic memories like SDRAMs and Micron SyncFlash devices.

4.1 Signal Description

The following tables describe the MultiPort Memory Controller interface signals. **Tables 4.1-1** and **Table 4.1-2** list the AHB slave memory interface and register interface signals, while **Tables 4.1-3** and **Table 4.1-4** list the master interface signals. **Table 4.1-5** lists the clocks in the MPMC. **Tables 4.1-6** and **Table 4.1-7** list the MPMC Pad Interface and Pad Control signals. **Tables 4.1-8** and **Table 4.1-9** list the miscellaneous signals and EBI related signals. **Table 4.1-10** lists the scan test related signals in the MPMC.

In **Table 4.1-1**, the signal names for each port can be found by substituting the port number 0, 1, 2, or 3, for the symbol x.

For more information on the AHB, please see AMBA Specification Rev2.0 [Ref. 1].

Signal Name	Type	Source / Destination	Description
HCLK Bus clock	Input	Clock Source	This clock times all bus transfers. All signal timings are related to the rising edge of HCLK.
HRESETn Reset	Input	Reset controller	The Bus reset signal is active LOW and is used to reset the system and the bus. This is the only active LOW signal.
HADDRx[27:0] Address bus	Input	AHB Master	The AHB address bus to the AHB Memory interface.
HWRITEx Transfer direction	Input	AHB Master	When HIGH this signal indicates a write transfer and when LOW, a read transfer to the memory.
HSIZEx[2:0] Transfer size	Input	AHB Master	Indicates the size of the transfer to the Memory. The MPMC AHB Slave Memory Interface supports only 8-bit, 16-bit and 32-bit accesses '000' represents 8-bit '001' represents 16-bit '010' represents 32-bit. If HSIZEx[2:0] indicates a transfer width larger than 32-bits, the MPMC returns an ERROR response on HRESP[1:0]
HTRANSx[1:0] Transfer type	Input	AHB Master	Indicates whether a data-transaction is to be done for the current transfer to Memory. '00' represents IDLE. '01' represents BUSY. '10' represents NSEQ '11' represents SEQ
HBURSTx[2:0] Burst Type	Input	AHB Master	Indicates if the transfer forms part of a burst. 4, 8 and 16 beat bursts are supported and the burst can be either incrementing or wrapping.
HREADYINx Transfer done	Input	AHB Slave	When HIGH the HREADYIN signal indicates that a transfer has finished on the bus. When LOW, it indicates that a wait state has been introduced by the AHB Slave.
HMASTLOCKx	Input	AHB Arbiter	When HIGH, this signal indicates that the master requires locked access to the SDRAM and no other master must be granted the bus until this signal is LOW.

Signal Name	Type	Source / Destination	Description
HSELMPMCxG Global Slave select	Input	AHB Decoder	Each MPMC AHB slave memory interface has its own global slave select signal. When HIGH, HSELMPMCxG indicates that the current transfer is intended for any one of the memory chips connected to the MPMC. This signal is a combinatorial decode of the address bus.
HSELMPMCxCS[7:0] Chip select-specific select	Input	AHB Decoder	Each MPMC AHB slave memory interface has its own set of chipselect-specific select signals. When HIGH, the relevant bit of HSELMPMCxCS[7:0] indicates that the current transfer is intended for a memory chip connected to a particular chip select output of the MPMC corresponding to that bit. For example, if bit 0 is asserted during an AHB transfer, it indicates that the transfer is intended for chip select 0. This signal is a combinatorial decode of the address bus.
HWDATAx[31:0] Write data bus	Input	AHB Master	The write data bus is used to transfer data from the master to the MPMC AHB Slave Memory Interface during write operations. The MPMC AHB Slave Memory Interface supports only 8-bit, 16-bit and 32-bit accesses.
HRDATAx[31:0] Read data bus	Output	AHB Master	The read data bus is used to read data from the MPMC AHB Slave Memory Interface. The MPMC AHB Slave Memory Interface supports only 8, 16 and 32-bit accesses.
HREADYOUTx Transfer done	Output	AHB Master	When HIGH the HREADYOUTx signal indicates that a transfer targeting the MPMC AHB Slave Memory Interface has finished on the bus. When LOW, it indicates that a wait state has been introduced by the AHB Slave
HRESPx[1:0] Transfer response	Output	AHB Slave	The transfer response provides additional information on the status of a transfer. Defined responses from AHB slave are, OKAY, ERROR, RETRY and SPLIT. The MPMC AHB Slave Memory Interface only drives OKAY or ERROR response.

Table 4.1-1 AHB Slave Memory Interface signals

Signal Name	Type	Source / Destination	Description
HADDRREG[11:2] Address bus	Input	AHB Master	The system address bus to the AHB register interface.
HWRITEREG Transfer direction	Input	AHB Master	When HIGH this signal indicates a write transfer and when LOW, a read transfer to the MPMC registers.
HSIZEREG[2:0] Transfer size	Input	AHB Master	Indicates the size of the transfer. The MPMC supports only 32 bit WORD accesses to all its registers. So all transfers to and from the MPMC controller must be 32-bit (HSIZE[2:0] = '010'). Transfer sizes other than 32-bit generate an ERROR response.

Signal Name	Type	Source / Destination	Description
HTRANSREG Transfer type	Input	AHB Master	Indicates whether a data-transaction is to be done for the current transfer to the MPMC Registers. HTRANS[1] on the AHB bus must be connected to the single-bit-wide HTRANSREG input of the MPMC. '1' represents NSEQ or SEQ and thus a valid data transfer. '0' represents BUSY or IDLE.
HREADYINREG Transfer done	Input	AHB Slave	When HIGH the HREADYINREG signal indicates that a transfer has finished on the bus. When LOW, it indicates that a wait state has been introduced by the AHB Slave
HSELMPMCREG AHB Slave select	Input	AHB Decoder	The MPMC AHB slave register interface has its own slave select signal. This signal indicates that the current transfer is intended for the registers of the MPMC. This signal is a combinatorial decode of the address bus.
HWDATAREG20TO19 [20:19] Write data bus	Input	AHB Master	HWDATA[20:19] on the AHB bus must be connected to the 2-bit signal HWDATA20TO19 signal.
HWDATAREG15TO0 [15:0] Write data bus	Input	AHB Master	HWDATA[15:0] on the AHB bus must be connected to the HWDATA15TO0[15:0] signal.
HRDATAREG[20:0] Read data bus	Output	AHB Master	The read data bus is used to read data from the MPMC AHB Slave Register Interface. The MPMC AHB Slave Register Interface supports only 32-bit accesses. The HRDATAREG[20:0] signal must be connected to the HRDATA[20:0] bits of the AHB bus.
HREADYOUTREG Transfer done	Output	AHB Master	When HIGH the HREADYOUTREG signal indicates that a transfer targeting the MPMC AHB Slave Register Interface has finished on the bus. When LOW, it indicates that a wait state has been introduced by the AHB Slave
HRESPREG[1:0] Transfer response	Output	AHB Slave	The transfer response provides additional information on the status of a transfer. Defined responses from AHB slave are, OKAY, ERROR, RETRY and SPLIT. The MPMC AHB Slave Register Interface only drives OKAY or ERROR response.

Table 4.1-2 AHB Slave Register Interface signals

Signal Name	Type	Source / Destination	Description
HADDR TIC[31:0] Address bus	Output	AHB Slave	The 32-bit system address bus.

Signal Name	Type	Source / Destination	Description
HTRANSTIC[1:0] Transfer type	Output	AHB Slave	Indicates the type of the current transfer, which can be NONSEQ, SEQ, IDLE or BUSY.
HWRITETIC Transfer direction	Output	AHB Slave	When HIGH this signal indicates a write transfer and when LOW a read transfer.
HSIZETIC[2:0] Transfer size	Output	AHB Slave	Indicates the size of the transfer. The Mpmc's TIC allows 8, 16 and 32-bit transfer widths. 8-bit: HSIZE[2:0] = '000' 16-bit: HSIZE[2:0] = '001' 32-bit: HSIZE[2:0] = '010'
HPROTTIC[3:0] Protection control	Output	AHB Slave	Provides information about a bus access.
HLOCKTIC Lock information for locked transfers	Output	AHB Arbiter	This signal is used to indicate to the arbiter that the requesting master is going to perform a number of indivisible transfers and the arbiter must not grant the bus to another bus master once the locked transfer has commenced.
HBURSTTIC[2:0] Burst type	Output	AHB Slave	Indicates the type of burst transfer. <u>Only INCR accesses are supported by the TIC Interface.</u> So HBURSTTIC[2:0] will always have a value of "001" in the MPMC.
HWDATATIC[31:0] Write data bus	Output	AHB Slave	The write data bus is used to transfer data from the master to the bus slaves during write operations.
HRDATATIC[31:0] Read data bus	Input	AHB Slave	The read data bus is used to transfer data from bus slaves to the bus master during read operations.
HREADYINTIC Transfer done	Input	AHB Slave	When HIGH the HREADYINTIC signal indicates that a transfer has finished on the bus. This signal may be driven LOW to extend a transfer, by the AHB slave.
HRESPTIC[1:0] Transfer response	Input	AHB Slave	The transfer response provides additional information on the status of a transfer. Four different responses are supported: OKAY, ERROR, RETRY and SPLIT. To indicate that the transfer has completed successfully, the slave uses the OKAY response. To indicate that an error has occurred, the slave uses the ERROR response. To indicate that the data is not ready, the slave uses the RETRY and SPLIT responses.

Table 4.1-3 AHB Master signals from the TIC

Signal Name	Type	Source / Destination	Description
HBUSREQTIC AHB bus request	Output	AHB arbiter	Bus request signal used by the Test Interface controller to request access to the AHB bus. When HIGH, this signal indicates that the MPMC is requesting the ownership of the bus.
HGRANTTIC AHB bus grant	Input	AHB arbiter	Grant signal generated by the arbiter. This signal indicates that the MPMC is currently the highest priority master requesting the bus. The MPMC gains bus ownership when HGRANTTIC and HREADYINTIC are high on the rising edge of HCLK.

Table 4.1-4 AHB Master bus request signals

Signal Name	Type	Source / Destination	Description
MPMCCLK Memory clock	Input	Clock Source	MPMC Memory Clock. MPMCCLK times all external memory transfers. MPMCCLK must be synchronous to HCLK. MPMCCLK can operate at the following ratios with reference to HCLK: - 1:1 - 1:2. Used for SDRAM and SyncFlash devices.
MPMCCLKDELAY	Input	Clock Source	Delayed version of MPMC Memory Clock. This clock is used to drive out the SDRAM / SyncFlash commands in command delayed mode.
MPMCFBCLKIN3...0 Fed-back clock in	Input	Clock Source	Fed-back clocks from the SDRAM / SyncFlash devices. There are 4 such clocks – one for each byte lane. Each clock is used to register one byte of the 32-bit data returned by the SDRAM device.
MPMCCLKOUT[3:0] SDRAM clock out	Output	Synchronous memory devices	Memory clock output. Each bit of MPMCCLKOUT[3:0] is to be connected to the clock input of SDRAM and SyncFlash devices connected to the 4 dynamic memory chipselects.

Table 4.1-5 MPMC clock signals

Signal Name	Type	Source / Destination	Description
-------------	------	----------------------	-------------

Signal Name	Type	Source / Destination	Description
MPMCCKEOUT[3:0] SDRAM clock enable	Output	Pad	SDRAM clock enables. Active HIGH. The MPMCCKEOUT[3:0] signals are used at the pin of the dynamic memory devices connected to the nMPMCDYCSOUT[3:0]. Used for SDRAM devices.
MPMCDQMOUT[3:0] SDRAM data mask	Output	Pad	Data mask output to SDRAMs. Active HIGH. The signals MPMCDQMOUT[3:0] mask byte lanes [31:24], [23:16], [15:8], [7:0] on the data bus. Used for SDRAM devices.
nMPMCBLSOUT[3:0] Static memory byte late select	Output	Pad	Byte lane select signals. Active LOW. The signals nMPMCBLSOUT[3:0] select byte lanes [31:24], [23:16], [15:8], [7:0] on the data bus. Used for static memories.
nMPMCRASOUT SDRAM row address strobe	Output	Pad	Row address strobe for SDRAM devices and SyncFlash Devices. Active LOW. Used for SDRAM devices.
nMPMCCASOUT SDRAM column address strobe	Output	Pad	Column address strobe for SDRAM devices and SyncFlash Devices. Active LOW. Used for SDRAM devices.
nMPMCOEOUT Static memory output enable	Output	Pad	Output enable for static memories. Active LOW. Used for static memory devices.
nMPMCWEOUT Memory write enable, Test acknowledge	Output	Pad	Write enable for memories. Active LOW. Used for SDRAM and static memories. Note: This pin is re-used in TIC test mode. The test bus acknowledge signal gives external indication that the test bus has been granted and indicates when a test access has completed. When TESTACK is LOW, the current test vector must be extended until TESTACK becomes HIGH.
nMPMCSTCSOUT[3:0] Static memory chip select	Output	Pad	Static memory chip selects. Default active LOW. Used for static memory devices.
nMPMCDYCSOUT[3:0] Dynamic memory chip select	Output	Pad	SDRAM chip selects. Active LOW. Used for SDRAM devices.

Signal Name	Type	Source / Destination	Description
MPMCADDR[27:0] Memory address	Output	Pad	Address output. Used for both Static and SDRAM devices. SDRAM memories use bits [14:0]. Static memories use bits [27:0].
MPMCDATAOUT[31:0] Write data	Output	Pad	Data output to memory. Used for the static memory controller, the synchronous memory controller and the Test Interface Controller (TIC).
MPMCDATAIN[31:0] Read data	Input	Pad	Read data from memory. Used for the static memory controller, the synchronous memory controller and the Test Interface Controller (TIC).
nMPMCRPOUT Reset power down	Output	Pad	Reset power down to SyncFlash memory. Active LOW. Used for the synchronous memory controller.
MPMCTESTREQA Test bus request A in test mode	Input	TicBox	During test, this signal is used to indicate the type of test vector that will be applied in the following cycle. The signal gives indication that the test bus has been granted and completion of a test access. When TESTACK is LOW, the current test vector must be extended until TESTACK becomes HIGH.
MPMCTESTREQB Test bus request B in test mode	Input	TicBox	During test, this signal is used, in combination with TESTREQA, to indicate the type of test vector that will be applied in the following cycle. The signal gives indication that the test bus has been granted and completion of a test access. When TESTACK is LOW, the current test vector must be extended until TESTACK becomes HIGH.
MPMCTESTIN Test Mode	Input	Pad	This signal is used to place the MPMC into TIC test mode. The MPMC is placed in the TIC test mode by asserting MPMCTESTIN HIGH during HRESETn assertion.

Table 4.1-6 Pad Interface signals

Signal Name	Type	Source / Destination	Description
nMPMCDATAEN[3:0] Data enable	Output	Pad control	Tri-State I/O pad enables for the byte lanes of the external memory data bus MPMCDATA[31:0]. Active LOW. Each bit enables the byte lanes [31:24], [23:16], [15:8], [7:0] of the data bus independently. Used for the static memory controller, the synchronous memory controller and the Test Interface Controller (TIC).

Signal Name	Type	Source / Destination	Description
MPMCRPVHHOUT VHH on Reset power down	Output	Pad control	Voltage control for RP output signal. Used to indicate that the reset power down is to be driven to the SyncFlash memory device has to be driven to VHH.
			nMPMCRPO UT MPMCRPVHHOUT RP
			0 0 0V
			0 1 0V
			1 0 3V
			1 1 8V

Table 4.1-7 Pad Control Signals

Signal Name	Type	Source / Destination	Description
nPOR	Input	Reset Controller	Power on reset (active LOW).
MPMCSREFREQ	Input	Power Management Unit	Self-refresh request. When HIGH, it indicates a self-refresh request from the Power Management Unit.
MPMCSREFACK	Output	Power Management Unit	Self-refresh acknowledgement. When HIGH, it indicates that the MPMC has placed the dynamic memories in self-refresh mode.
MPMCBIGENDIAN	Input	Pad	Endian Mode bit. This power on reset value can be over written through the register interface. When HIGH, it indicates Big-endian mode. When LOW, it indicates Little-endian mode.
MPMCSTCS1MW[1:0]	Input	Pad	Memory width of CS1. This power on reset value can be over written through the register interface. "00" indicates 8-bits, "01" indicates 16-bits and "10" indicates 32-bits.
MPMCSTCS0POL	Input	Pad	Polarity of static memory CS0. This power on reset value can be over written through the register interface. When HIGH, it indicates active HIGH chip select and when LOW, it indicates an active LOW chip select.
MPMCSTCS1POL	Input	Pad	Polarity of static memory CS1. This power on reset value can be over written through the register interface. When HIGH, it indicates active HIGH chip select and when LOW, it indicates an active LOW chip select.

Signal Name	Type	Source / Destination	Description
MPMCSTCS2POL	Input	Pad	Polarity of static memory CS2. This power on reset value can be over written through the register interface. When HIGH, it indicates active HIGH chip select and when LOW, it indicates an active LOW chip select.
MPMCSTCS3POL	Input	Pad	Polarity of static memory CS3. This power on reset value can be over written through the register interface. When HIGH, it indicates active HIGH chip select and when LOW, it indicates an active LOW chip select.
MPMCSTCS1PB	Input	Pad	Polarity of byte lane select for static memory CS1. This power on reset value can be over written through the register interface. When HIGH, for reads and writes, the respective active bits of nMPMCBLSOUT[3:0] are LOW. When LOW, for reads, all the bits of nMPMCBLSOUT[3:0] are HIGH and for writes, the respective active bits of nMPMCBLSOUT[3:0] are LOW.
MPMCREL1CONFIG	Input	Pad	Static memory address mode select. When HIGH, it indicates the static memory addressing mode of PL172 release 1v0 in which the static memory address should be connected as follows: - When external memory width is 8-bits, MPMCADDROUT[27:0] should be connected to the A[27:0] pins of the static memory device - When external memory width is 16-bits, MPMCADDROUT[27:1] should be connected to the A[26:0] pins of the static memory device and MPMCADDROUT[0] is not used - When external memory width is 32-bits, MPMCADDROUT[27:2] should be connected to the A[25:0] pins of the static memory device and MPMCADDROUT[1:0] is not used. When LOW, it indicates that the static memory addresses should be connected as follows: - When external memory width is 8-bits or 16-bits or 32-bits, MPMCADDROUT[27:0] should be connected to the A[27:0] pins of the static memory device.

Table 4.1-8 Miscellaneous signals

Name	Type	Source/ Destination	Description
MPMCEBIGNT	Input	EBI	The MPMCEBIGNT signal goes HIGH (active) when the EBI grants the external memory bus to the MPMC.

MPMCEBIBACKOFF	Input	EBI	The MPMCEBIBACKOFF signal goes HIGH (active) when the EBI wants to remove the MPMC from the memory bus so that another memory controller can be granted the memory bus.
MPMCEBIREQ	Output	EBI	The MPMCEBIREQ signal goes HIGH (active) when the MPMC requires the control of the external memory bus.

Table 4.1-9 EBI control signals

Signal Name	Type	Source / Destination	Description
SCANENABLE	Input	Test Controller	Scan Enable signal. Indicates that the circuit is in scan-test mode.
SCANINHCLK	Input	Test Controller	HCLK domain Scan data input for clocking in the test pattern in scan-test mode.
SCANINMPMCCLK	Input	Test Controller	MPMCCLK domain Scan data input for clocking in the test pattern in scan-test mode.
SCANINFBCLKIN0	Input	Test Controller	MPMCFBCLKIN0 domain Scan data input for clocking in the test pattern in scan-test mode.
SCANINFBCLKIN1	Input	Test Controller	MPMCFBCLKIN1 domain Scan data input for clocking in the test pattern in scan-test mode.
SCANINFBCLKIN2	Input	Test Controller	MPMCFBCLKIN2 domain Scan data input for clocking in the test pattern in scan-test mode.
SCANINFBCLKIN3	Input	Test Controller	MPMCFBCLKIN3 domain Scan data input for clocking in the test pattern in scan-test mode.
SCANINCLKDELAY	Input	Test Controller	MPMCCLKDELAY domain Scan data input for clocking in the test pattern in scan-test mode.
SCANOUTHCLK	Output	Test Controller	HCLK domain Scan data output for observing the flip-flop contents in scan test mode.
SCANOUTMPMCCLK	Output	Test Controller	MPMCCLK domain Scan data output for observing the flip-flop contents in scan test mode.
SCANOUTFBCLKIN0	Output	Test Controller	MPMCFBCLKIN0 domain Scan data output for observing the flip-flop contents in scan test mode.
SCANOUTFBCLKIN1	Output	Test Controller	MPMCFBCLKIN1 domain Scan data output for observing the flip-flop contents in scan test mode.
SCANOUTFBCLKIN2	Output	Test Controller	MPMCFBCLKIN2 domain Scan data output for observing the flip-flop contents in scan test mode.
SCANOUTFBCLKIN3	Output	Test Controller	MPMCFBCLKIN3 domain Scan data output for observing the flip-flop contents in scan test mode.
SCANOUTCLKDELAY	Output	Test Controller	MPMCCLKDELAY domain Scan data output for observing the flip-flop contents in scan test mode.

Table 4.1-10 Scan Test related signals

4.2 Programmer's Model

In this section, information about all the registers in the MPMC (PL172) is given.

Address	Type	Width	Reset Value HRESETn	Reset Value nPOR	Register Name	Description
MPMC Base + 0x000	R/W	2-0	0x1	0x3	MPMCControl	This register is used to control the operation of the memory controller.
MPMC Base + 0x004	R	2-0	N/A	0x4	MPMCStatus	This register provides the status of the memory controller.
MPMC Base + 0x008	R/W	9-8, 0	N/A	0x0	MPMCConfig	This register is used to configure the operation of the memory controller.
MPMC Base + 0x020	R/W	15-13, 8-7, 2-0	N/A	0x006	MPMCDyCntl	This register is used to control the operation of the dynamic memory controller.
MPMC Base + 0x024	R/W	10-0	N/A	0x0	MPMCDyRef	This register is used to control the refresh period of the Dynamic memories.
MPMC Base + 0x028	R/W	1-0	N/A	0x00	MPMCDyRdConfig	This register is used to select the command delayed mode or clock delayed mode for accessing dynamic memories.
MPMC Base + 0x030	R/W	3-0	N/A	0xF	MPMCDytRP	This register allows the Dynamic memory precharge command period (t_{RP}) to be programmed.
MPMC Base + 0x034	R/W	3-0	N/A	0xF	MPMCDytRAS	This register allows the Dynamic memory precharge period (t_{RAS}) to be programmed.
MPMC Base + 0x038	R/W	3-0	N/A	0xF	MPMCDytSREX	This register allows the Dynamic memory self refresh exit time (t_{SREX}) to be programmed.
MPMC Base + 0x03C	R/W	3-0	N/A	0xF	MPMCDytAPR	This register allows the Dynamic memory last data-out to Active command in case of autoprecharge read (to same bank) time (t_{APR}) to be programmed.
MPMC Base + 0x040	R/W	3-0	N/A	0xF	MPMCDytDAL	This register allows the Dynamic memory Data-In to Active command time (t_{DAL} or t_{APW}) to be programmed.

Address	Type	Width	Reset Value HRESETn	Reset Value nPOR	Register Name	Description
MPMC Base + 0x044	R/W	3-0	N/A	0xF	MPMCDytWR	This register allows the Dynamic memory write recovery time or the Data-In to precharge time (t_{WR} or t_{DPL} or t_{RWL} or t_{RDL}) to be programmed.
MPMC Base + 0x048	R/W	4-0	N/A	0x1F	MPMCDytRC	This register allows the Dynamic memory AutoRefresh and Active Command period to be programmed (t_{RC}).
MPMC Base + 0x04C	R/W	4-0	N/A	0x1F	MPMCDytRFC	This register allows the Dynamic memory AutoRefresh to Active Command Period (t_{RFC}) to be programmed.
MPMC Base + 0x050	R/W	4-0	N/A	0x1F	MPMCDytXSR	This register allows the Dynamic memory Self refresh exit to Active command time (t_{XSR}) to be programmed.
MPMC Base + 0x054	R/W	4-0	N/A	0x1F	MPMCDytRRD	This register allows the Dynamic memory Active Bank A to Active Bank B time (t_{RRD}) to be programmed.
MPMC Base + 0x058	R/W	4-0	N/A	0x1F	MPMCDytMRD	This register allows the Dynamic memory Load Mode register to Active Command time (t_{MRD}) to be programmed.
MPMC Base + 0x080	R/W	9-0	N/A	0x000	MPMCStExdWt	This register allows the Static memory Extended Wait Access time to be programmed.
MPMC Base + 0x100	R/W	20-19, 14, 12-7, 4-3	N/A	0x000	MPMCDyConfig0	This register allows the configuration information for the SDRAM connected to chip select 4 to be programmed.
MPMC Base + 0x104	R/W	9-8, 1-0	N/A	0x303	MPMCDyRasCas0	This register allows the RAS and CAS latencies for the SDRAM connected to chip select 4 to be programmed.
MPMC Base + 0x120	R/W	20-19, 14, 12-7, 4-3	N/A	0x000	MPMCDyConfig1	This register allows the configuration information for the SDRAM connected to chip select 5 to be programmed.
MPMC Base + 0x124	R/W	9-8, 1-0	N/A	0x303	MPMCDyRasCas1	This register allows the RAS and CAS latencies for the SDRAM connected to chip select 5 to be programmed.

Address	Type	Width	Reset Value HRESETn	Reset Value nPOR	Register Name	Description
MPMC Base + 0x140	R/W	20-19, 14, 12-7, 4-3	N/A	0x000	MPMCDyConfig2	This register allows the configuration information for the SDRAM connected to chip select 6 to be programmed.
MPMC Base + 0x144	R/W	9-8, 1-0	N/A	0x303	MPMCDyRasCas2	This register allows the RAS and CAS latencies for the SDRAM connected to chip select 6 to be programmed.
MPMC Base + 0x160	R/W	20-19, 14, 12-7, 4-3	N/A	0x000	MPMCDyConfig3	This register allows the configuration information for the SDRAM connected to chip select 7 to be programmed.
MPMC Base + 0x164	R/W	9-8, 1-0	N/A	0x303	MPMCDyRasCas3	This register allows the RAS and CAS latencies for the SDRAM connected to chip select 7 to be programmed.
MPMC Base + 0x200	R/W	20-19, 8-6, 3, 1-0	N/A	0x00	MPMCStConfig0	This register allows the configuration information for the static memory connected to chip select 0 to be programmed.
MPMC Base + 0x204	R/W	3-0	N/A	0x0	MPMCStWtWen0	This register allows the delay from the chip select assertion to the write enable assertion to be programmed for the static memory connected to chip select 0.
MPMC Base + 0x208	R/W	3-0	N/A	0x0	MPMCStWtOen0	This register allows the delay from the chip select assertion to the output enable assertion to be programmed for the static memory connected to chip select 0.
MPMC Base + 0x20C	R/W	4-0	N/A	0x1F	MPMCStWtRd0	This register allows the delay from the chip select assertion to the read access to be programmed for the static memory connected to chip select 0.
MPMC Base + 0x210	R/W	4-0	N/A	0x1F	MPMCStWtPg0	This register allows the delay from the chip select assertion to the asynchronous page mode read access to be programmed for the static memory connected to chip select 0.

Address	Type	Width	Reset Value HRESETn	Reset Value nPOR	Register Name	Description
MPMC Base + 0x214	R/W	4-0	N/A	0x1F	MPMCStWtWr0	This register allows the delay from the chip select assertion to the write access to be programmed for the static memory connected to chip select 0.
MPMC Base + 0x218	R/W	3-0	N/A	0xF	MPMCStWtTurn0	This register allows the number of bus turn around cycles to be programmed for the static memory connected to chip select 0.
MPMC Base + 0x220	R/W	20-19, 8-6, 3, 1-0	N/A	0x00	MPMCStConfig1	This register allows the configuration information for the static memory connected to chip select 1 to be programmed.
MPMC Base + 0x224	R/W	3-0	N/A	0x0	MPMCStWtWen1	This register allows the delay from the chip select assertion to the write enable assertion to be programmed for the static memory connected to chip select 1.
MPMC Base + 0x228	R/W	3-0	N/A	0x0	MPMCStWtOen1	This register allows the delay from the chip select assertion to the output enable assertion to be programmed for the static memory connected to chip select 1.
MPMC Base + 0x22C	R/W	4-0	N/A	0x1F	MPMCStWtRd1	This register allows the delay from the chip select assertion to the read access to be programmed for the static memory connected to chip select 1.
MPMC Base + 0x230	R/W	4-0	N/A	0x1F	MPMCStWtPg1	This register allows the delay from the chip select assertion to the asynchronous page mode read access to be programmed for the static memory connected to chip select 1.
MPMC Base + 0x234	R/W	4-0	N/A	0x1F	MPMCStWtWr1	This register allows the delay from the chip select assertion to the write access to be programmed for the static memory connected to chip select 1.

Address	Type	Width	Reset Value HRESETn	Reset Value nPOR	Register Name	Description
MPMC Base + 0x238	R/W	3-0	N/A	0xF	MPMCStWtTurn1	This register allows the number of bus turn around cycles to be programmed for the static memory connected to chip select 1.
MPMC Base + 0x240	R/W	20-19, 8-6, 3, 1-0	N/A	0x00	MPMCStConfig2	This register allows the configuration information for the static memory connected to chip select 2 to be programmed.
MPMC Base + 0x244	R/W	3-0	N/A	0x0	MPMCStWtWen2	This register allows the delay from the chip select assertion to the write enable assertion to be programmed for the static memory connected to chip select 2.
MPMC Base + 0x248	R/W	3-0	N/A	0x0	MPMCStWtOen2	This register allows the delay from the chip select assertion to the output enable assertion to be programmed for the static memory connected to chip select 2.
MPMC Base + 0x24C	R/W	4-0	N/A	0x1F	MPMCStWtRd2	This register allows the delay from the chip select assertion to the read access to be programmed for the static memory connected to chip select 2.
MPMC Base + 0x250	R/W	4-0	N/A	0x1F	MPMCStWtPg2	This register allows the delay from the chip select assertion to the asynchronous page mode read access to be programmed for the static memory connected to chip select 2.
MPMC Base + 0x254	R/W	4-0	N/A	0x1F	MPMCStWtWr2	This register allows the delay from the chip select assertion to the write access to be programmed for the static memory connected to chip select 2.
MPMC Base + 0x258	R/W	3-0	N/A	0xF	MPMCStWtTurn2	This register allows the number of bus turn around cycles to be programmed for the static memory connected to chip select 2.

Address	Type	Width	Reset Value HRESETn	Reset Value nPOR	Register Name	Description
MPMC Base + 0x260	R/W	20-19, 8-6, 3, 1-0	N/A	0x00	MPMCStConfig3	This register allows the configuration information for the static memory connected to chip select 3 to be programmed.
MPMC Base + 0x264	R/W	3-0	N/A	0x0	MPMCStWtWen3	This register allows the delay from the chip select assertion to the write enable assertion to be programmed for the static memory connected to chip select 3.
MPMC Base + 0x268	R/W	3-0	N/A	0x0	MPMCStWtOen3	This register allows the delay from the chip select assertion to the output enable assertion to be programmed for the static memory connected to chip select 3.
MPMC Base + 0x26C	R/W	4-0	N/A	0x1F	MPMCStWtRd3	This register allows the delay from the chip select assertion to the read access to be programmed for the static memory connected to chip select 3.
MPMC Base + 0x270	R/W	4-0	N/A	0x1F	MPMCStWtPg3	This register allows the delay from the chip select assertion to the asynchronous page mode read access to be programmed for the static memory connected to chip select 3.
MPMC Base + 0x274	R/W	4-0	N/A	0x1F	MPMCStWtWr3	This register allows the delay from the chip select assertion to the write access to be programmed for the static memory connected to chip select 3.
MPMC Base + 0x278	R/W	3-0	N/A	0xF	MPMCStWtTurn3	This register allows the number of bus turn around cycles to be programmed for the static memory connected to chip select 3.
MPMC Base + 0xF00	R/W	0 (one-bit register)	0x0	0x0	MPMCITCR	This register is used to select the various test modes.

Address	Type	Width	Reset Value HRESETn	Reset Value nPOR	Register Name	Description
MPMC Base + 0xF20	R/W	9-0	0x0	0x0	MPMCITIP	Control and read the MPMCSTCS1PB, MPMCSTCS0POL, MPMCSTCS1POL, MPMCSTCS2POL, MPMCSTCS3POL, MPMCSTCS1MW, MPMCBIGENDIAN and MPMCSREFREQ input lines.
MPMC Base + 0xF40	R/W	1-0	0x00	0x00	MPMCITOP	Control and read the MPMCRPVHHOUT and MPMCSREFACK output lines.
MPMC Base + 0xFD0	R	7-0	0x33	0x33	MPMCPeriphId4	Peripheral ID register Bits 39:32.
MPMC Base + 0xFD4	R	7-0	0x0	0x0	MPMCPeriphId5	Reserved (Peripheral ID register).
MPMC Base + 0xFD8	R	7-0	0x0	0x0	MPMCPeriphId6	Reserved (Peripheral ID register).
MPMC Base + 0xFDC	R	7-0	0x0	0x0	MPMCPeriphId7	Reserved (Peripheral ID register).
MPMC Base + 0xFE0	R	7-0	0x72	0x72	MPMCPeriphId0	Peripheral ID register Bits 7:0.
MPMC Base + 0xFE4	R	7-0	0x11	0x11	MPMCPeriphId1	Peripheral ID register Bits 15:8.
MPMC Base + 0xFE8	R	7-0	0x04	0x14	MPMCPeriphId2	Peripheral ID register. Bits 23:16.
MPMC Base + 0xFEC	R	7-0	0x07	0x07	MPMCPeriphId3	Peripheral ID register. Bits 31:24.
MPMC Base + 0xFF0	R	7-0	0x0D	0x0D	MPMCPCellId0	PrimeCell ID register. Bits 7:0.
MPMC Base + 0xFF4	R	7-0	0xF0	0xF0	MPMCPCellId1	PrimeCell ID register. Bits 15:8.
MPMC Base + 0xFF8	R	7-0	0x05	0x05	MPMCPCellId2	PrimeCell ID register. Bits 23:16.
MPMC Base + 0xFFC	R	7-0	0xB1	0xB1	MPMCPCellId3	PrimeCell ID register. Bits 31:24.

Table 4.2-1 Memory map for the MPMC registers

NOTE: Writes to a read-only register will not change the contents of the register. However, the MPMC AHB slave register interface will return an OKAY response on the AHB for such accesses.

4.3 Design Hierarchy

The hierarchy in the MPMC is shown in **Figure 4.3-1**. A short description about the functionality of each of the sub-modules is also given within brackets.

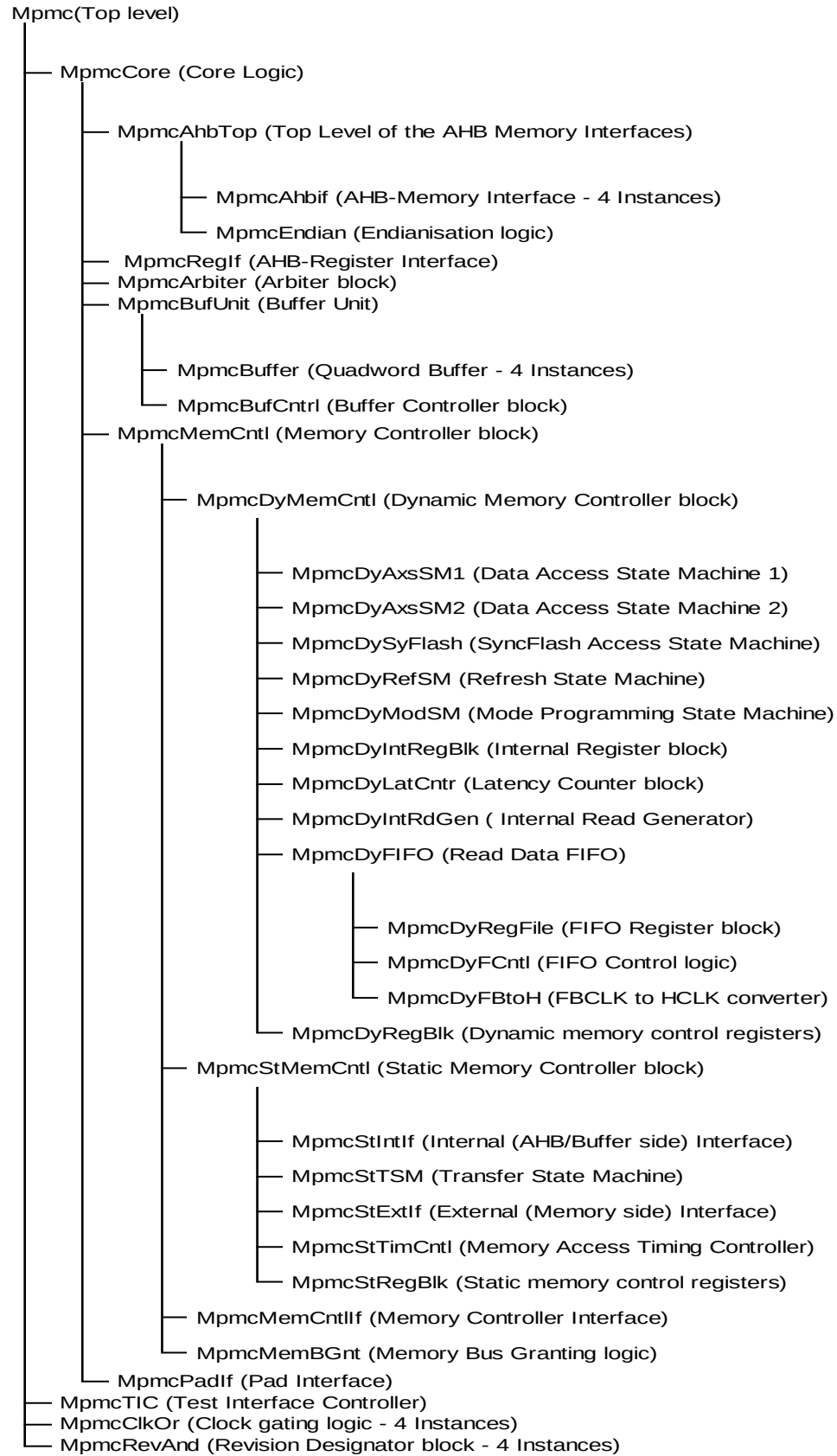


Figure 4.3-1 MPMC Design Hierarchy

4.4 MPMC Block Partitioning

At the topmost level, the MPMC consists of the following blocks:

- MPMC Core Logic (MpmcCore)
- MPMC TIC Interface (MpmcTIC)
- MPMC Clock Gating Logic (MpmcClockOr)
- MPMC Revision Designator Block (MpmcRevAnd)

The MPMC Core logic block contains the main functionality of the MPMC. It contains the memory controller, buffers, arbiter, AHB interfaces and the Pad Interface blocks. The MPMC TIC block contains the test interface controller functionality of the MPMC. The MPMC Core logic and the MPMC TIC Interface share some of the output pins at the topmost level of the MPMC. The logic to combine the outputs from the two blocks is within the Pad Interface block in the MPMC Core Logic. The clock gating logic module of the MPMC (MpmcClockOr) is instantiated four times to generate the four output clocks, MPMCCLKOUT[3:0]. The revision designator block (MpmcRevAnd) is also instantiated four times to generate the four bits of the revision number of the MPMC.

A block schematic of the ARM PrimeCell Multiport Memory Controller (PL172) is shown in **Figure 4.4-1**.

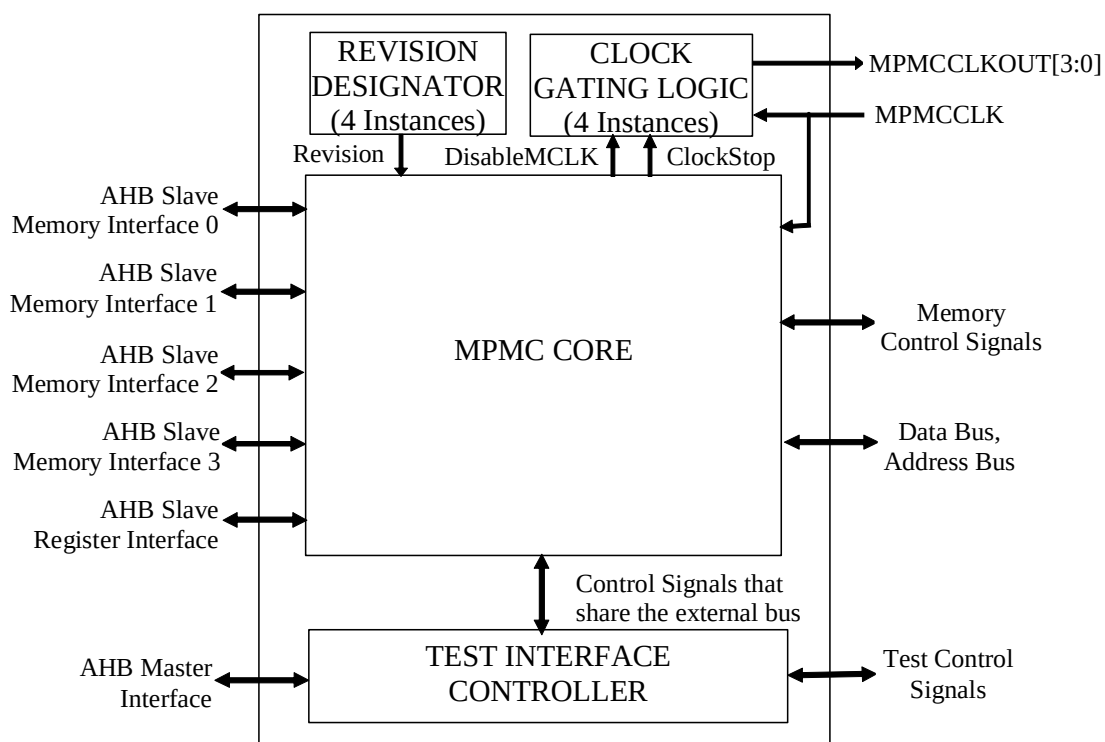


Figure 4.4-1 ARM PrimeCell Mutiport Memory Controller (PL172) block diagram

The MPMC Core logic consists of the following major sub-blocks:

- AHB Memory Interfaces top level block to provide the interface between the MPMC and the AHB ports
- AHB Register Interface to provide the interface between the MPMC registers and the AHB register port
- Arbiter block to arbitrate among requesting AHB ports
- Buffer Unit block containing the 4 quadquadword buffers and their control logic
- Memory controller block containing the Dynamic and Static memory controller logic.

- Pad Interface to combine the memory bus signals from the static and dynamic memory controllers.

A block schematic of the MPMC Core logic is shown in **Figure 4.4-2**. In this figure, the signal names for each AHB port can be found by substituting the port number 0, 1, 2, or 3, for the symbol x.

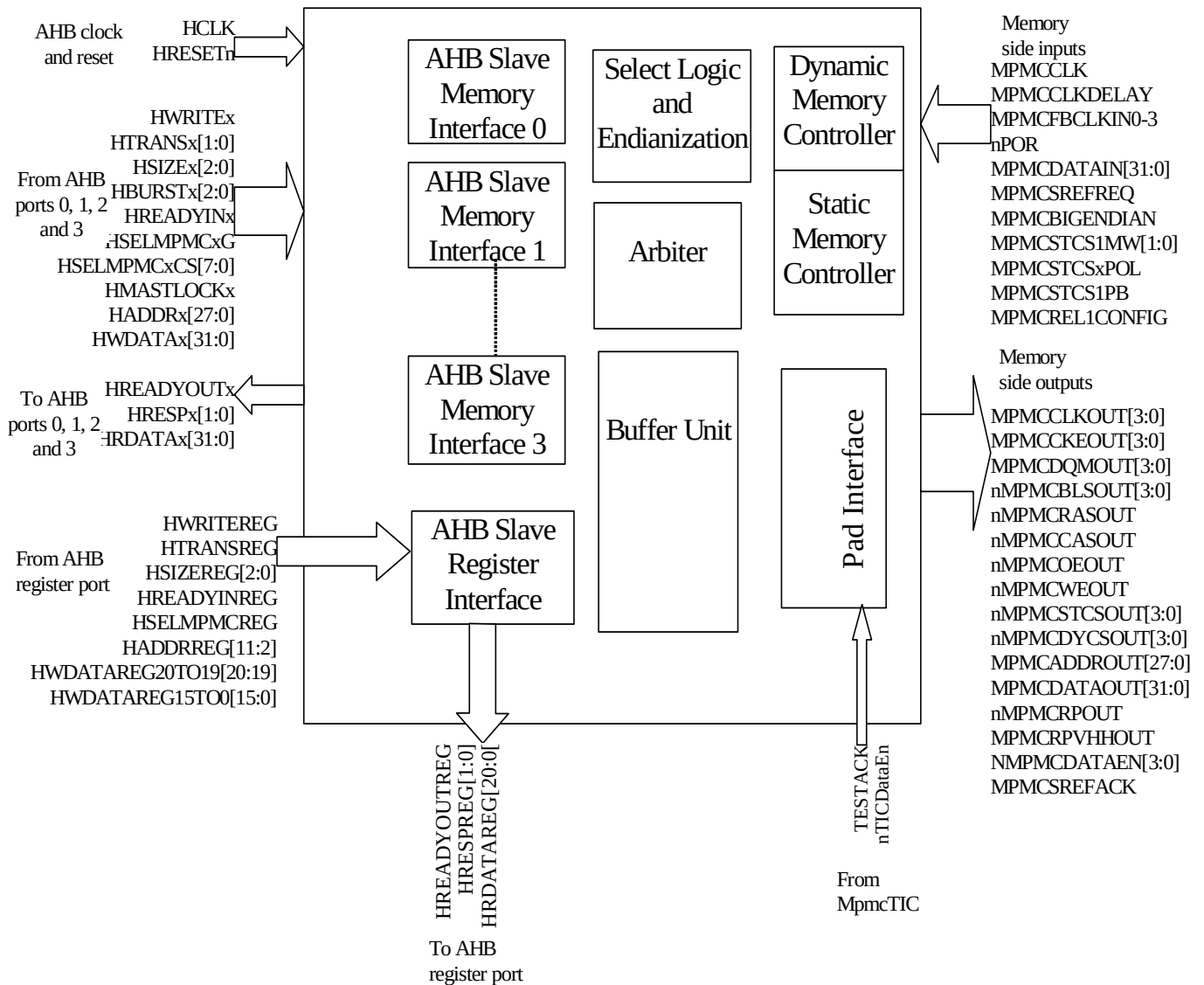


Figure 4.4-2 Block diagram of the sub-blocks in the MPMC Core logic

4.5 AHB Memory Interfaces top level block

The port level connections of the AHB memory interface (MpmcAhbTop) are shown in **Figure 4.5-1**. AHB memory interface top level is a structural block. It contains the following sub-blocks.

- 4 instances of identical AHB interfaces (AHB0, AHB1, AHB2, AHB3)
- Endian block.

Description of each port is given in the sub-sections describing the sub-blocks.

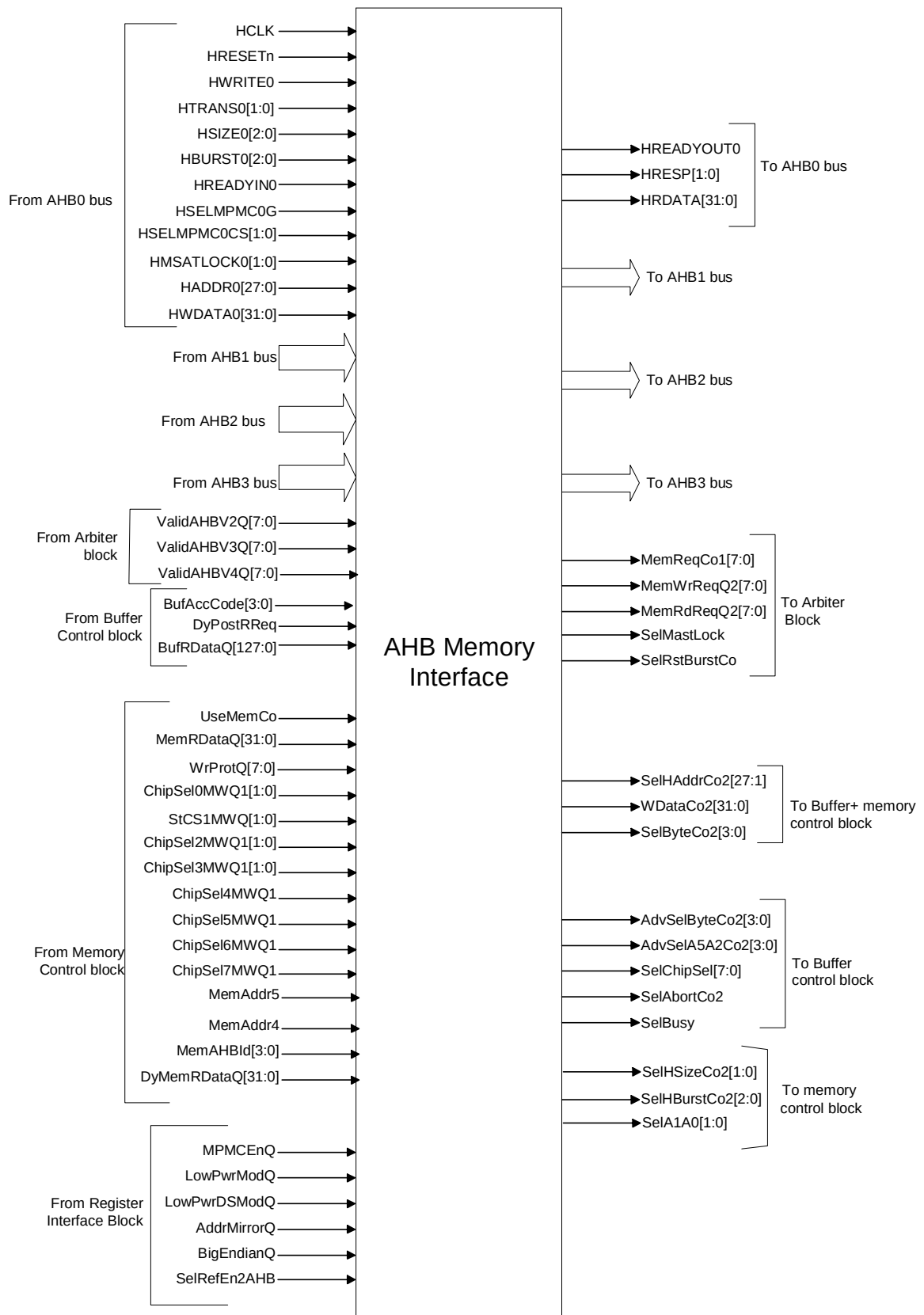


Figure 4.5-1 Block Inputs and Outputs of MpmcAhbTop

4.5.1 AHB Memory Interface

Figure 4.5-2 shows the port level block diagram of one of the AHB interfaces (MpmcAhbif). AHB interface allows the MPMC to connect the AMBA AHB system. Following functionality is implemented in this block

- AHB Address and control signals registering
- Memory transfer request generation
- Predictive address generation for sequential transfers
- Valid byte indicator generation
- Read data selection and packing based on endian mode
- Error monitoring to detect the transfers that result in an error response.
- AHB response generation logic to generate the HREADYOUT and HRESP[1:0] signals.

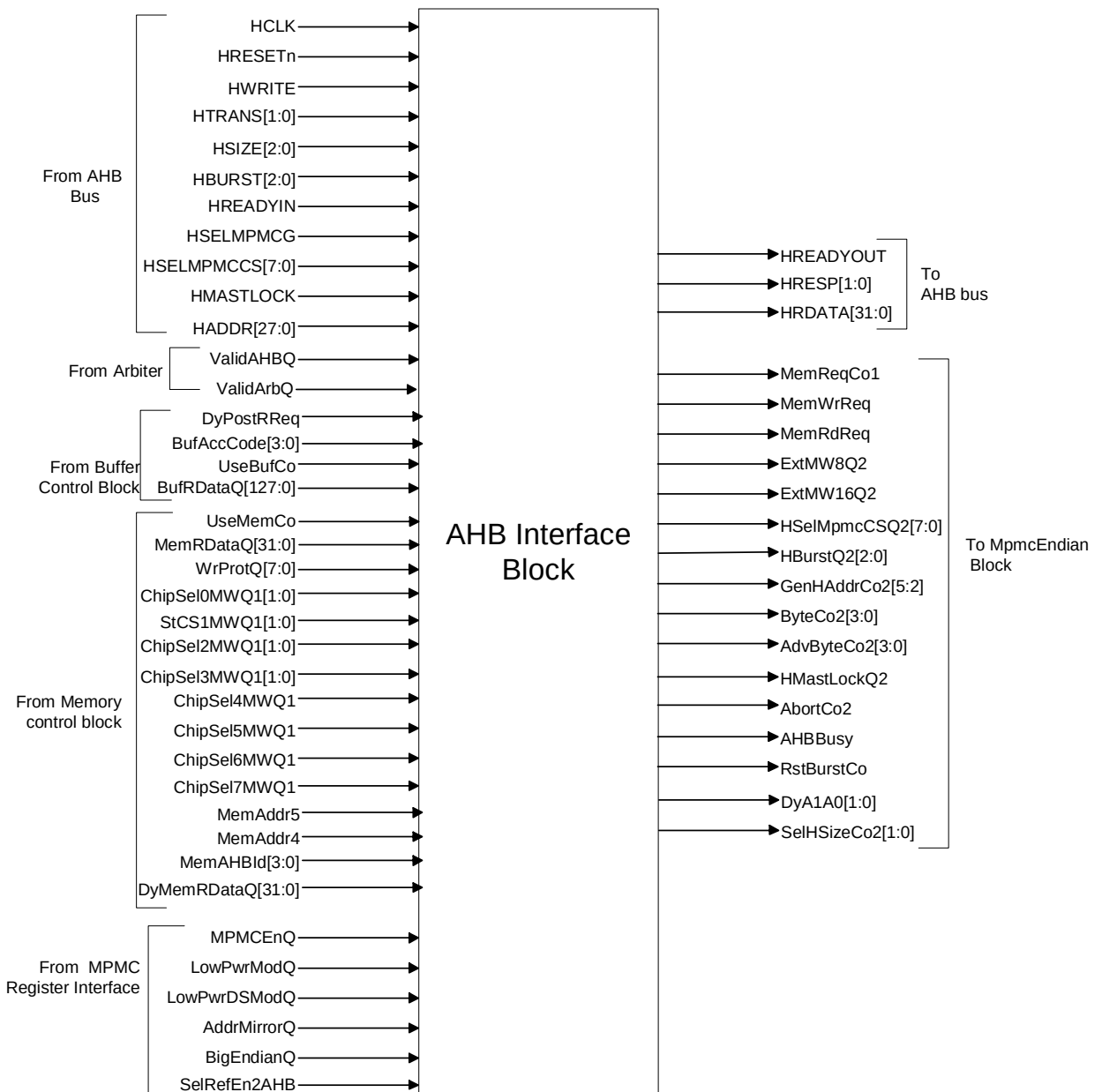


Figure 4.5-2 Block Inputs and Outputs of MpmcAhbIf

The list of inputs to the MpmcAhbIf module is shown in **Table 4.5-1** and the list of outputs from the MpmcAhbIf module is shown in **Table 4.5-2**

Signal Name	Source	Description
HCLK	Clock Generator	AHB Clock
HRESETn	Reset Generator	AHB Reset

Signal Name	Source	Description
HSELMPMC GC	AHB Decoder	Global select signal for the MPMC. High on this signal indicates that, the corresponding AHB interface is selected by the AHB master to perform a read or a write transfer to the memory.
HSELMPMCCS[7:0]	AHB Decoder	This signal indicates the bank number (chip select) to which the master want to perform the data transfer.
HWRITE	AHB Master	When high, this signal indicates that the MPMC is selected for the write transfer.
HTRANS[1:0]	AHB Master	This signal indicates the transfer type on the AHB.
HADDR[27:0]	AHB Master	This signal gives the address to which the transfer is to happen.
HSIZE[2:0]	AHB Master	This signal gives the width of the data transfer.
HREADYIN	AHB Master	This input indicates that AHB Slave (the slave who owned the AHB bus) has completed the data transfer and MPMC can proceed with data transfer if it is selected by any of the master.
HMASTLOCK	AHB Master	Indicates that current master is performing a locked sequence of transfer. Arbiter block inside the MPMC uses this signal.
BufRDataQ[127:0]	Buffer Unit	Read data from the buffer control block. AHB interface should sample this vector only if the acknowledge signal UseBufCo from the buffer block is sampled high.
ValidAHBQ	Arbiter	A HIGH on this signal indicates that arbiter module selected the corresponding AHB for the data transfer.
ValidArbQ	Arbiter	A HIGH on this signal indicates that the current cycle is a valid arbitration cycle
UseBufCo	Buffer Unit	Data transfer acknowledge signal from the buffer control block. AHB interface uses this signal for HREADYOUT generation. In case of read transfer, buffer control block sets the UseBufCo signal, to indicate the AHB interface that, requested data is ready, and it can be sampled from BufRDataQ [127:0]. In case of write transfer, this signal indicates that the data from the AHB master is written into the buffer.
MPMCEnQ	AHB Register Interface	This signal is used by the error monitoring logic. AHB interface generates the error response if the AHB master tries to perform a data transfer when MPMC is not enabled.
LowPwrModQ	AHB Register Interface	This signal is used by the error monitoring logic. AHB interface generates the error response if the AHB master tries to perform a data transfer when low power mode is enabled.

Signal Name	Source	Description
LowPwrDSModQ	AHB Register Interface	This signal is used by the error monitoring logic. AHB interface generates the error response if the AHB master tries to perform a data transfer when low power deep sleep mode is enabled.
AddrMirrorQ	AHB Register Interface	Indicated the normal or reset memory map. Reset value of this signal is high. If AddrMirrorQ is set, CS1 is mirrored to CS0 and CS4.
BigEndianQ	AHB Register Interface	This signal is set to high for big endian mode of operation. It is used by the read data packing logic and Valid byte generation logic.
SelRefEn2AHB	AHB Register interface	Self refresh status signal. This signal is set when a self-refresh request is high. The error-monitoring block uses this signal. AHB memory interface generates the error response, when an AHB master initiate a dynamic memory transfer while MPMC is in self-refresh mode.
MemRDataQ [31:0]	Memory Controller	Read data from the combined memory control block. AHB interface should sample this vector only if the acknowledge signal UseMemCo from the memory control block is sampled high.
DyMemRDataQ[31:0]	Memory Controller	Read data from the dynamic memory control block. AHB interface should sample this vector only if the acknowledge signal DyUseMem from the dynamic memory control block is sampled high. This signal is used when data is returned from the dynamic memory to the AHB interface and the buffer in parallel.
UseMemCo	Memory Controller	Data transfer acknowledge signal from the memory control block. AHB interface uses this signal for HREADYOUT generation. In case of read transfer, memory control block sets the UseMemCo signal, to indicate the AHB interface that, requested data is ready, and it can be sampled from MemRdDataQ [31:0]. In case of write transfer, this signal indicates that the data from the AHB master is written into the memory.
DyUseMem	Memory controller	A HIGH on this signal indicates that requested from the dynamic memory is available at DyMemRDataQ[31:0]
WrProtQ[7:0]	Memory Controller	This vector signal indicates the write protect status of the each memory banks. This signal is used by the error monitoring logic in the AHB interface. AHB interface generates the error response when AHB master tries to perform a write transfer to a write protected memory bank.
ChipSel0MWQ1[1:0]	Memory Controller	This Signal indicates the width of the memory, which is connected to the chip select zero. Chip select zero is used for connecting the static memories. Chip select zero supports the static memories of width 8-bit, 16-bit and 32-bit. Memory width information is used by the read data packing logic.

Signal Name	Source	Description
StCS1MWQ1[1:0]	Memory Controller	This Signal indicates the width of the memory, which is connected to the chip select zero. This signal is derived from the output of the mux, which is used to select between the pin value and the register value of memory width for chip select 1. Chip select one is used for the static memories. Chip select one supports the static memories of width 8-bit, 16-bit and 32-bit. Memory width information is used by the read data packing logic.
ChipSel2MWQ1[1:0]	Memory Controller	This Signal indicates the width of the memory, which is, connected to the chip select, two. Chip select two is used for interfacing the static memories. Chip select two supports the static memories of width 8-bit, 16-bit and 32-bit. Memory width information is used by the read data packing logic.
ChipSel3MWQ1[1:0]	Memory Controller	This Signal indicates the width of the memory, which is, connected to the chip select, three. Chip select three is used for interfacing the static memories. Chip select three supports the static memories of width 8-bit, 16-bit and 32-bit. Memory width information is used by the read data packing logic.
ChipSel4MWQ1	Memory Controller	This Signal indicates the width of the memory, which is, connected to the chip select, four. Chip select four is used for interfacing the dynamic memories. Chip select four supports the dynamic memories of width 16-bit and 32-bit. Memory width information is used for read data packing logic
ChipSel5MWQ1	Memory Controller	This Signal indicates the width of the memory that is connected to the chip select five. Chip select five is used for interfacing the dynamic memories. Chip select five supports the dynamic memories of width 16-bit and 32-bit. Memory width information is used for read data packing logic
ChipSel6MWQ1	Memory Controller	This Signal indicates the width of the memory, which is, connected to the chip select, six. Chip select six is used for interfacing the dynamic memories. Chip select six supports the dynamic memories of width 16-bit and 32-bit. Memory width information is used for read data packing logic
ChipSel7MWQ1	Memory Controller	This Signal indicates the width of the memory, which is, connected to the chip select, seven. Chip select seven is used for interfacing the dynamic memories. Chip select seven supports the dynamic memories of width 16-bit and 32-bit. Memory width information is used for read data packing logic
DyPostRReq	Buffer Unit	Dynamic memory read request. This signal is used in the AHB interface to decide whether to fetch the read data directly from the dynamic memory controller or from the buffer.
BufAccCode[8:0]	Buffer Unit	Buffer access code given by the buffer block along with the DyPostRReq to indicate the AHB interface whose memory read request is posted.

Signal Name	Source	Description
MemAHBId[[3:0]	Memory Controller	AHB ID returned by the memory controller along with the read data. This signal indicates the AHB interface for which the memory controller is returning read data.
MemAddr4	Memory Controller	Address bit4 returned by the memory control block along with the read data. This signal is used along with MemAddr5 to determine the quadword for which data is being returned by the memory controller.
MemAddr5	Memory Controller	Address bit5 returned by the memory control block along with the read data. This signal is used along with MemAddr4 to determine the quadword for which data is being returned by the memory controller.

Table 4.5-1 Block Inputs of MPMCAhbif

Signal Name	Destination	Description
HREADYOUT	AHB Master	This signal goes to the AHB bus from the MPMC Interface, indicating the ready status of the MPMC memory interface. A HIGH on the HREADYOUT indicates that on going transfer was finished. The MPMC drives a LOW on this signal to extend the transfer.
HRESP	AHB Master	This signal along with HREADYOUT signal is used to generate the MPMC response on the AHB bus. The MPMC can generate OKAY and ERROR responses.
HRDATA	AHB Master	AHB read data bus. Data transfer from memory to AHB is done through this bus.
ExtMW8Q2	Endian block	Indicates the width of the memory, which is connected to the selected memory bank. ExtMW8Q2 is set if the selected memory bank width is 8-bit.
ExtMW16Q2	Endian block	Indicates the width of the memory that is connected to the selected memory bank. ExtMW8Q2 is set if the selected memory bank width is 16-bit.
MemReqCo1	Arbiter block	Memory data transfer request signal. A HIGH on this signal indicates that one of the AHB masters connected to this AHB bus wants to perform a data transfer to the location, which is addressed by the MPMC.
MemWrReq	Arbiter block	This signal is relevant only when MemReqCo1 is high. MemWrReq is set for a write request.
MemRdReq	Arbiter block	This signal is relevant only when MemReqCo1 is high. MemRdReq is set for a read request.

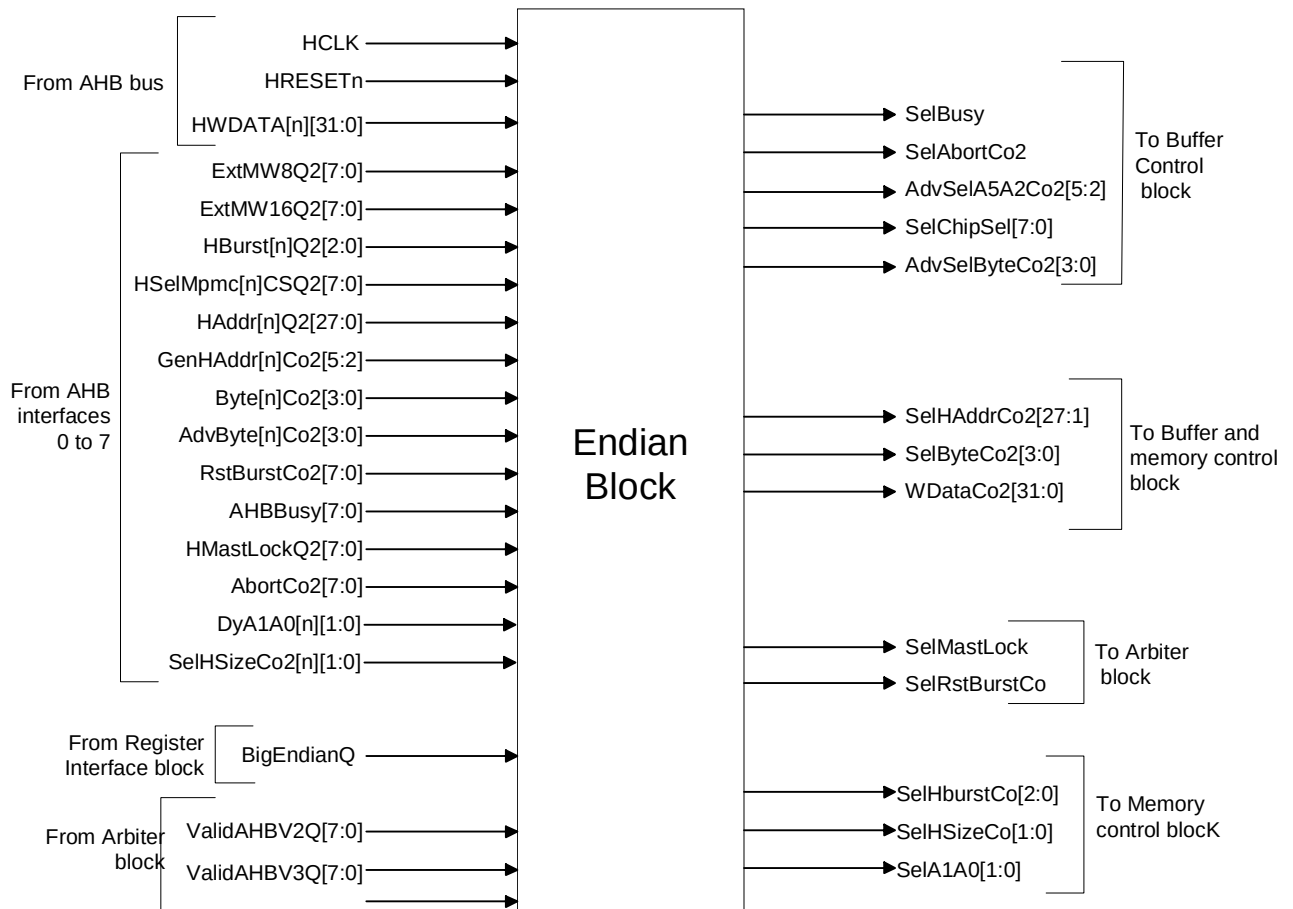
Signal Name	Destination	Description
HSelMpmcCSQ2[7:0]	Endian block	Registered version of HSELMPMCCS[7:0]. HSELMPMCCS[7:0] has been registered only when HREADYIN input is sampled as high. HSelMpmcCSQ2[0] and HSelMpmcCSQ2 [4] are driven zero if the address mirroring is enabled. If address mirroring is enabled, all accesses to the chip select zero and chip select four enables the chip select one.
HBurstQ2[2:0]	Endian block	Registered version of HBURST[2:0]. AHB Control signal, HBURST[2:0] is sampled only when HREADYIN and HSELMPMCG are high.
HSizeQ2[1:0]	Endian block	Registered version of HSIZE[1:0]. AHB Control signals, HSIZE[1:0] is sampled only when HREADYIN and HSELMPMCG are high.
HAddrQ2[29:1]	Endian block	Registered version of HADDR[27:1]. AHB Control signals, HADDR[27:0] is sampled only when HREADYIN and HSELMPMCG are high.
DyA1A0[1:0]	Endian block	Registered version of HADDR[1:0]. AHB Control signals, HADDR[1:0] is sampled only when HREADYIN and HSELMPMCG are high.
GenHAddrCo2[5:2]	Endian block	Internally generated address. Indicates the next possible address going request by the AHB. Predictive address generation logic uses the control signals HSIZE and HBURST for the internal address generation.
ByteCo2[3:0]	Endian block	Byte information signal. AHB data bus width is 32 bit. This 4-bit vector is used for indicating the valid byte, for transfers that are less than a word size. This signal indicates the valid byte in case of a write transfers and requested byte in case of read transfer.
AdvByteCo2[3:0]	Endian block	Advanced byte information signal. This signal is identical to ByteCo2 signal but it conveys the information about the next data transfer. This signal indicates the valid byte in case of a write transfers and requested byte in case of read transfer.
HMAstLockQ2	Endian block	Registered version of HMASTLOCK. HMASTLOCK signal is sampled only when HREADYIN is high. This signal is used to prevent re-arbitration as long HMASTLOCK is sampled HIGH.
AbortCo2	Endian block	A HIGH on this signal indicates that AHB interface is performing an ERROR response. So other blocks should abort the request initiated by the AHB interface in the previous clock cycle.
AHBBusy	Endian block	A HIGH on this signal indicates that AHB interface is responding for a BUSY transfer.
RstBurstCo	Endian block	A HIGH on this signal indicates that end of the on going burst. The Arbiter block uses this signal to re-arbitrate among requesting AHBs.

Table 4.5-2 Block outputs of MpmcAhbif

4.5.2 Endian Block

The Endian block (MpmcEndian) contains the write data packing logic, which endianises the write data to the quadword buffers or memory controller based on the Big/Little endian mode bit. The MPMC (PL172) is designed to connect to up to 8-AHB buses. Depending on the arbitration priority the arbiter grants one AHB to access the buffer and memory. This block uses the signal ValidAHBQ[7:0] from the arbiter to perform the selection of the address, data and control signals from the granted AHB.

Figure 4.5-3 shows the port level block diagram of the endian block.

**Figure 4.5-3 Block Inputs and Outputs of MpmcEndian**

The list of inputs and outputs of the MpmcEndian module are shown in **Table 4.5-3** and **Table 4.5-4**

Signal Name	Source	Description
HCLK	Clock Generator	AHB Clock
HRESETn	Reset Generator	AHB Reset

Signal Name	Source	Description
BigEndianQ	AHB Register Interface	This signal is set to high for big endian mode of operation. It is used for the write data packing.
ValidAHBV2Q[7:0], ValidAHBV3Q[7:0], ValidAHBV4Q[7:0]	Arbiter	AHB select signal from the arbiter block. PL172 is provided with 4 AHB interfaces. Depending on the priority of the requesting AHB, arbiter provides grant to one of the requesting AHB. Arbiter issues the grant through the vector signal ValidAHBV2Q[7:0]. Functionality of ValidAHBV3Q[7:0] and ValidAHBV4Q[7:0] is same as ValidAHBV2Q[7:0]. Purpose of this additional signal is to distribute the load and to avoid the extra buffer insertion to increase the drive during synthesis.
ExtMW8Q2[7:0]	AHB Memory Interface 7 to 0	Indicates the width of the memory, which is connected to the selected memory bank addressed by the AHB master. MPMC endian block can support 8-AHB interfaces; each bit in the 8-bit vector ExtMW8Q2[7:0] is assigned for a particular AHB. For e.g. ExtMW8Q2[0] is used by the AHB0 interface, ExtMW8Q2[1] is used by the AHB1 interface etc. High on the signal ExtMW8Q2[0] indicates that the memory bank, which is selected by the AHB0, is assigned to a memory of width 8-bit.
ExtMW16Q2[7:0]	AHB Memory Interface 7 to 0	Indicates the width of the memory, which is connected to the selected memory bank addressed by the AHB master. MPMC endian block can support 8-AHB interfaces; each bit in the 8-bit vector ExtMW16Q2[7:0] is assigned for a particular AHB. For e.g. ExtMW16Q2[0] is used by the AHB0 interface, ExtMW16Q2[1] is used by the AHB1 interface etc. High on the signal ExtMW16Q2[0] indicates that the memory bank, which is selected by the AHB0, is assigned to a memory of width 16-bit.
HBurst[n]Q2[2:0]	AHB Memory Interface 7 to 0	Registered version of HBURST[2:0] from the AHB interfaces. HBurst0Q2[2:0] is the register version of HBURST0[7:0] from AHB0 interface, HBurst1Q2[2:0] is the register version of HBURST1[7:0] from AHB1 interface, etc. This block can support 8-AHB interfaces, so n can vary from 0 to 7.
HSize[n]Q2[1:0]	AHB Memory Interface 7 to 0	Registered version of HSIZE[1:0] from the AHB interfaces. HSize0Q2[1:0] is the register version of HSIZE0[7:0] from AHB0 interface, HSize1Q2[1:0] is the register version of HSIZE1[7:0] from AHB1 interface, etc. This block can support 8-AHB interfaces, so n can vary from 0 to 7.
HWDATA[n][31:0]	AHB Master	Write data from AHB bus. HWDATA0[31:0] is the write data bus from AHB0 interface, HWDATA1[31:0] is the write data bus from AHB1, etc. The MPMC Endian block can support 8-AHB interfaces, so n can vary from 0 to 7.

Signal Name	Source	Description
HselMpmc[n]CS2[7:0]	AHB Memory Interface 7 to 0	Registered version of AHB select signal. n indicates the AHB port number. HselMpmc0CSQ2[7:0] is select signal from AHB0, HselMpmc1CSQ2[7:0] is the select signal from AHB1, etc. n can vary from 0 to 7.
HAddr[n]Q2[27:1]	AHB Memory Interface 7 to 0	Registered version of AHB address bus. n represents the AHB port number and can vary from 0 to 7.
DyA1A0[n][1:0]	AHB Memory Interface 7 to 0	Registered version of AHB address bit A1 A0. n represents the AHB port number and can vary from 0 to 7.
GenHAddr[n]Co2[5:2]	AHB Memory Interface 7 to 0	Generated address from the AHB interface block. GenHAddr0Co2[3:2] is the generated address from the AHB interface 0, GenHAddr1Co2[3:2] is the generated address from the AHB interface 1, etc. n Represent the AHB port number can varies from 0 to 7.
Byte[n]Co2[3:0]	AHB Memory Interface 7 to 0	Valid byte indicator. Byte0Co2[3:0] is from AHB interface block0, Byte1Co2[3:0] is from AHB interface1, etc. n can vary from 0 to 7.
AdvByte[n]Co2[3:0]	AHB Memory Interface 7 to 0	Advanced valid byte indicator. AdvByte0Co2[3:0] is from AHB interface block0, AdvByte1Co2[3:0] is from AHB interface1, and etc. n can vary from 0 to 7.
RstBurstCo[7:0]	AHB Memory Interface 7 to 0	End of burst indicator. RstBurstCo[0] is from AHB interface block0, RstBurstCo[1] is from AHB interface1, and etc.
AHBBusy[7:0]	AHB Memory Interface 7 to 0	AHB busy status indicator. High on the signal AHBBusy [0] indicates that AHB master, which is accessing the AHB0 port, is performing BUSY operation. Similarly high on the signal AHBBusy[1] indicates that AHB master which is accessing to AHB1 port is performing BUSY operation, and etc.
HMAstLockQ2[7:0]	AHB Memory Interface 7 to 0	Registered version of HMASTLOCK signal. HMAstLockQ2[0] is from AHB0, HMAstLockQ2[1] is from AHB1, and etc.
AbortCo2[7:0]	AHB Memory Interface 7 to 0	Error indicator signal. AbortCo2[0] is set to high by AHB0 if any error condition happens in the AHB0 interface. Similarly AbortCo2[1] is set to high by AHB1 if any error condition happens in the AHB1 interface, and etc.

Table 4.5-3 Block Inputs of MpmcEndian

Signal Name	Destination	Description
SelHBurstCo2[2:0]	Memory Controller	HBURST information of the selected AHB interface. AHB selection is done using the signal ValidAHBV3Q[7:0] from the arbiter block. SelHBurstCo2[2:0] is the output of a mux whose input is Hburst[n]Q2[2:0] and select line is ValidAHBV3Q[7:0]

Signal Name	Destination	Description
SelHSizeCo2[1:0]	Memory Controller	HSIZE information of the selected AHB interface. AHB selection is done using the signal ValidAHBV3Q[7:0] from the arbiter block. SelHSizeCo2[1:0] is the output of a mux whose input is HSize[n]Q2[1:0] and select line is ValidAHBV3Q[7:0]
SelChipSel[7:0]	Buffer Unit	Chip select signal from the selected AHB. AHB selection is done using the signal ValidAHBV4Q [7:0] from the arbiter block. SelHBurstCo3[2:0] is the output of a mux whose input is HSelMpmc[n]CSQ2[2:0] and select line is ValidAHBV4Q[7:0]
SelHAddrCo2[27:1]	Buffer Unit	Address bus from the selected AHB interface. AHB selection is done using the signal ValidAHBV2Q [7:0] from the arbiter block. SelHAddrCo2[27:1] is the output of a mux whose input is HAddr[n]Q2[2:0] and select line is ValidAHBV2Q[7:0]
SelA1A0[1:0]	Memory Controller	Address bit A1 and A0 from the selected AHB interface. AHB selection is done using the signal ValidAHBV3Q[7:0] from the arbiter block. SelA1A0[1:0] is the output of a mux whose input is DyA1A0[n]Q2[1:0] and select line is ValidAHBV3Q[7:0]
AdvSelA5A2Co2[3:0]	Buffer Unit	Advanced address bus from the selected AHB interface. AHB selection is done using the signal ValidAHBV4Q[7:0] from the arbiter block. AdvSelA5A2Co2[3:0] is the output of a mux whose input is GenHAdder[n]Q2[302] and select line is ValidAHBV4Q[7:0]
SelByteCo2[3:0]	Buffer Unit, Memory Controller	Valid byte line indicate signal from the selected AHB. AHB selection is done using the signal ValidAHBV3Q[7:0] from the arbiter block. SelByteCo2[3:0] is the output of a mux whose input is Byte[n]Co2[2:0] and select line is ValidAHBV3Q[7:0]
AdvSelByteCo2[3:0]	Buffer Unit	Advanced valid byte line indicate signal from the selected AHB. AHB selection is done using the signal ValidAHBV3Q[7:0] from the arbiter block. AdvSelByteCo2[3:0] is the output of a mux whose input is AdvByte[n]Co2[7:0] and select line is ValidAHBV3Q[7:0]
WDataCo2[31:0]	Buffer Unit	Selected AHB's write data bus after endianisation. AHB selection is done using the signal ValidAHBV4Q[7:0] from the arbiter block.
SelAbortCo2	Buffer Unit	Transfer abort signal from the selected AHB interface. AHB selection is done using the signal ValidAHBV3Q[7:0] from the arbiter block. SelAbortCo2 is the output of a mux whose input is AbortCo2[7:0] and select line is ValidAHBV3Q[7:0]
SelBusy	Buffer Unit, Memory Controller	AHB busy indicate signal from the selected AHB. AHB selection is done using the signal ValidAHBV3Q[7:0] from the arbiter block. SelBusy is the output of a mux whose input is AHBBusy[7:0] and select line is ValidAHBV3Q[7:0]
SelMastLock	Arbiter	AHB HMASTLOCK signals from the selected AHB interface. AHB selection is done using the signal ValidAHBV4Q[7:0] from the arbiter block. SelMastLock is the output of a mux whose input is HMastLockQ2[7:0] and select line is ValidAHBV4Q[7:0]

Signal Name	Destination	Description
SelRstBurstCo	Arbiter	End of burst indicate signal from the selected AHB interface. AHB selection is done using the signal ValidAHBV4Q[7:0] from the arbiter block. SelRstBurstCo2 is the output of a mux whose input is RstBurstCo2[7:0] and select line is ValidAHBV4Q[7:0]

Table 4.5-4 Block Outputs of MpmcEndian

4.6 AHB Register Interface

The AHB Register Interface provides the interface between the AHB and the registers of the MPMC. This module also contains all the common (non-chip-select-specific) registers, the integration test registers, the peripheral ID registers and the PrimeCell Id registers. This module contains the logic to generate the appropriate responses to the register access transfers on the AHB. It also contains the logic to select the appropriate bank of chip-select-specific registers, which reside in the static and dynamic memory controller blocks.

Figure 4.6-1 shows the port level block diagram of AHB Register interface (MpmcRegIf).

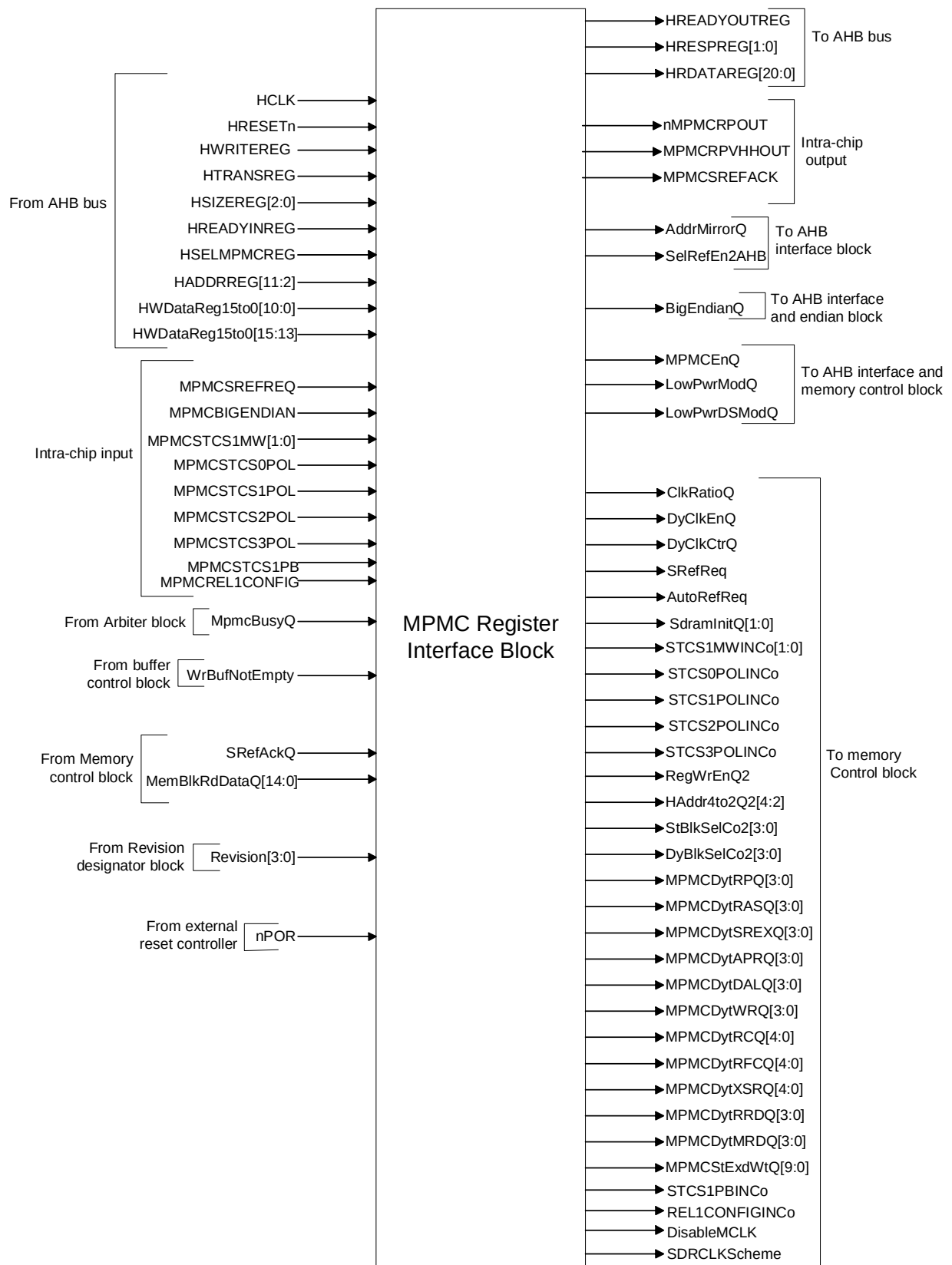


Figure 4.6-1 Block Inputs of MpmcRegIf

The list of inputs and outputs of the MpmcRegIf module are shown in **Table 4.6-1** and **Table 4.6-2**

Signal Name	Source	Description
HCLK	Clock Generator	AHB Clock
HRESETn	Reset Generator	AHB Reset
nPOR	External reset controller	Power on Reset
HSELMPMCREG	AHB Master	AHB select signal. High on this signal indicates that, AHB register interface is selected by the AHB master to perform a read or a write transfer.
HWRITEREG	AHB Master	When high, this signal indicates that the AHB register interface is selected for a write transfer.
HTRANSREG[1]	AHB Master	Indicates the AHB transfer types. HTRANSREG[1] is set to high for SEQ or NSEQ transfer.
HADDRREG[11:2]	AHB Master	Indicates the address of the register to which the transfer is to happen.
HSIZE[2:0]	AHB Master	This signal indicates the width of the data transfer. Register interface supports only 32-bit transfers.
HREADYINREG	AHB Master	This input indicates that AHB Slave (the slave who owned the AHB bus) has completed the data transfer and MPMC register interface can proceed if it is selected by the master
HWDataReg15to0[10:0]	AHB Master	AHB write data bus. Is used to transfer the data from AHB master to the selected register.
HWDataReg15to0[15:13]	AHB Master	AHB write data bus. Is used to transfer the data from AHB master to the selected register.
MemBlkRdDataQ[14:0]	Memory Controller	Register read data bus from the memory control block. Registers that are specific to dynamic and static chip selects are defined in the corresponding block. MemBlkRdDataQ [14:0] is used to transfer the content of these registers to the AHB register interface.
MPMCSREFREQ	Power Management Unit	Self-refresh request input signal from the power management unit.
MPMCBIGENDIAN	Reset Controller	Endian selects input. If high, big-endian mode is selected. This hardware input determines endian mode of the MPMC on power on reset. It can be overwritten by the software through the endian mode bit in the MPMCCConfig register.
MPMCSTCS1MW [1:0]	Reset Controller	Chip select 1 memory width. This hardware input indicates the external memory width of the chip select 1 on power on reset. It can overwrite by the software through the MW bits in the MPMCStaticConfig1 register.
MPMCSTCS0POL	Reset Controller	Chipselect 0 polarity select. This hardware input indicates the polarity of the chip select 0, on power on reset. It can overwrite by the software through the PC bit in the MPMCStaticConfig0 register.
MPMCSTCS1POL	Reset Controller	Chipselect 1 polarity select. This hardware input indicates the polarity of the chip select 1, on power on reset. It can overwrite by the software through the PC bit in the MPMCStaticConfig1 register.

Signal Name	Source	Description
MPMCSTCS2POL	Reset Controller	Chipselect 2 polarity select. This hardware input indicates the polarity of the chip select 2, on power on reset. It can overwrite by the software through the PC bit in the MPMCStaticConfig2 register.
MPMCSTCS3POL	Reset Controller	Chipselect 3 polarity select. This hardware input indicates the polarity of the chip select 3, on power on reset. It can overwrite by the software through the PC bit in the MPMCStaticConfig3 register.
MPMCSTCS1PB	Reset Controller	Polarity of byte lane select for static memory CS1. This power on reset value can be over written through the register interface. When HIGH, for reads and writes, the respective active bits of nMPMCBLSOUT[3:0] are LOW. When LOW, for reads, all the bits of nMPMCBLSOUT[3:0] are HIGH and for writes, the respective active bits of nMPMCBLSOUT[3:0] are LOW
MPMCREL1CONFIG	Reset Controller	Static memory address mode select. When HIGH, it indicates the static memory addressing mode of PL172 release 1v0 in which the static memory address should be connected as follows: - When external memory width is 8-bits, MPMCADDROUT[27:0] should be connected to the A[27:0] pins of the static memory device - When external memory width is 16-bits, MPMCADDROUT[27:1] should be connected to the A[26:0] pins of the static memory device and MPMCADDROUT[0] is not used - When external memory width is 32-bits, MPMCADDROUT[27:2] should be connected to the A[25:0] pins of the static memory device and MPMCADDROUT[1:0] is not used. When LOW, it indicates that the static memory addresses should be connected as follows: When external memory width is 8-bits or 16-bits or 32-bits, MPMCADDROUT[27:0] should be connected to the A[27:0] pins of the static memory device.
Revision	Revision Designator	Indicates the peripheral revision number. This vector signal can be read by the AHB master through the MPMCPeriphID3 register.
MpmcBusy	Arbiter	High on this signal indicates that MPMC is busy with a transaction. This status signal can be read by the AHB master through the MPMCStatus register.
WrBufNotEmpty	Buffer Unit	High on this signal indicates that write buffer is not empty. This status signal can be read by the AHB master through the MPMCStatus register.
SrefAckQ	Memory Controller	Self-refresh acknowledge signal from memory control block. High on this signal indicates that memory has been entered into the self-refresh mode. This status signal can be read by the AHB master through the MPMCStatus register.

Table 4.6-1 Block Inputs of MpmcRegIf

Signal Name	Destination	Description
HREADYOUTREG	AHB Master	High on this signal indicates that a transfer has finished on the bus. This signal can be driven low to extend the transfer. MPMC register interface contains zero wait State and single wait State register.
HRESPREG [1:0]	AHB Master	Indicates the transfer response. AHB register interface generates OKAY and ERROR transfer responses.
HRDATAREG [31:0]	AHB Master	AHB Read data bus is used to transfer the data from MPMC register interface to the AHB master.
MPMCEnQ	AHB memory interface, Memory Controller	High on this signal Indicates that MPMC is enabled. This control signal can be accessed by the bit-0 of the MPMC Control register.
AddrMirrorQ	AHB memory interface	Indicates the normal or reset memory map. Reset value of this signal is high. If set, CS1 (chip select 1) is mirrored to CS0 (Chip select 1) and CS4 (Chip select 4). This signal can be modified by the bit-1 of the MPMC Control register.
LowPwrModQ	AHB memory interface, Memory Controller	This signal indicates the normal or low power mode. This signal can be controlled by the bit-2 of the MPMC Control register. The LowPwrModQ signal is set for low power mode of operation.
BigEndianQ	AHB memory interface, Endian block, Memory Controller	This signal is set to high for big endian mode of operation. Software can access this signal through the bit-0 of MPMCCConfig register. Reset value of this signal is determined by the input pin MPMCBIGENDIAN
LowPwrDSModQ	AHB memory interface, Memory Controller	Low power deep sleep mode enable signal. This signal can be set through the bit-13 of the MPMCDynamicControl register.
STCS1MWINCo [1:0]	Memory Controller	Indicates the memory width of the external memory connected to the bank one. The input pin MPMCSTCS1MW determines reset value of this signal. Reset value can be over written by software through the bits 1 and 0 of the MPMCStaticConfig1 register.
STCS0POLINCo	Memory Controller	Indicates the chip select polarity of the external memory connected to the bank zero. The input pin MPMCSTCS0POL determines reset value of this signal. Reset value can be over written by software through the bit 6 of the MPMCStaticConfig0 register.
STCS1POLINCo	Memory Controller	Indicates the chip select polarity of the external memory connected to the bank one. The input pin MPMCSTCS1POL determines reset value of this signal. Reset value can be over written by software through the bit 6 of the MPMCStaticConfig1 register.
STCS2POLINCo	Memory Controller	Indicates the chip select polarity of the external memory connected to the bank two. The input pin MPMCSTCS2POL determines reset value of this signal. Reset value can be over written by software through the bit 6 of the MPMCStaticConfig2 register.
STCS3POLINCo	Memory Controller	Indicates the chip select polarity of the external memory connected to the bank three. The input pin MPMCSTCS3POL determines reset value of this signal. Reset value can be over written by software through the bit 6 of the MPMCStaticConfig3 register.

Signal Name	Destination	Description
STCS1PBINCo	Memory Controller	Indicates the byte lane state of the external memory connected to chip select one. The input pin MPMCSTCS1PB determined the reset value of this signal. Reset value can be overwritten by software through the bit 5 of the MPMCStaticConfig1 register.
REL1CONFIGINCo	Memory Controller	Indicates the static memory addressing mode to be followed for all static memory chipselects. The input pin MPMCREL1CONFIG determines the value of this signal.
MPMCRPVHHOUT	Memory Controller	SyncFlash reset/power down voltage signal. This signal can be controlled by the software through the bit-15 of the MPMCDynamicControl register.
ClkRatioQ	Memory Controller	Indicates HCLK : MPMCCLKOUT ratio. MPMCCLKOUT can be programmed to equal or two times faster than the HCLK. This signal is controlled by the bit 7 of the MPMCCConfig register.
DyClkEnQ	Memory Controller	Dynamic memory clock control signal. This signal can be accessed through the bit-1 of the MPMCDynamicControl register.
DyClkCtrQ	Memory Controller	Dynamic memory clock enable signal. This signal can be accessed through the bit-0 of the MPMCDynamicControl register.
SrefReq	Memory Controller	Self refresh request signal. This signal can be controlled by both hardware and software. Software can access this pin through bit-2 of MPMCDynamicControl register. Also this signal can be controlled by the input pin MPMCSREFREQ. SrefReq is the OR-ed version of software and hardware request.
AutoRefReq	Memory Controller	This signal is used by the memory control block for initiating the refresh cycle. AutoRefReq is a one HCLK width signal, set to high when ever the internal free running counter, refresh counter, counts one. So frequency of this signal is determined by the reload value of the free running counter. Reload value can be programmed through the MPMCDynamicRefresh register.
SdramInitQ	Memory Controller	SDRAM initialisation commands are given to the memory control block through this signal. Software can write to these signals through the bit 8 and bit 7 of the MPMCDynamicControl register.
DisableMCLK	Clock Gating Logic	Disable MPMCCLKOUT signal. This signal is used to disable the MPMCCLKOUT[3:0] signals through software control. Software can control this signal though the bit 5 of the MPMCDynamicControl register.
SDRClkScheme[1:0]	Pad Interface Block	SDRAM Clock scheme. This signal is used to determine the clocking scheme of the commands to the dynamic memory. The MPMC follows the command delayed mode or the clock delayed mode depending on the values programmed in bits 0 and 1 of the MPMCDynamicReadConfig register.
SelfRefEn2AHB	Memory Controller	Self refresh status signal to the AHB interface. This signal is set to high when a self-refresh request is high. This signal is used in the AHB memory interface to generate the error response, when an AHB master initiate a dynamic memory transfer while MPMC is in self refresh mode.
RegWrEnQ2[4:2]	Memory Controller	A HIGH on this signal indicates that register is selected for a write transfer.

Signal Name	Destination	Description
StBlkSelCo2 [3:0]	Memory Controller	Static memory register block select signal. Registers that are specific to each static memory chip select are defined in the respective part of static memory control block. MPMC supports four static memory chips selects. Each signal of this four-bit vector is used as the selects signal for one of the static register bank.
DyBlkSelCo2[3:0]	Memory Controller	Dynamic memory register block select signal. Registers that are specific to each dynamic memory chip select are defined in the respective part of the dynamic memory control block. MPMC supports four dynamic memory chips selects. Each signal of this four-bit vector is used as the selects signal for one of the dynamic register bank.
HAddr4to2Q2 [4:2]	Memory Controller	This vector is used by the memory control block to select the specific registers in the selected memory bank. Bank selection is done by the StBlkSelCo2 and DyBlkSelCo2 signals.
MPMCDytRPQ [3:0]	Memory Controller	Programmed value of the MPMCDynamicRP register is available at this vector.
MPMCDytRAS [3:0]	Memory Controller	Programmed value of the MPMCDynamicRAS register is available at this vector.
MPMCDytSREXQ [3:0]	Memory Controller	Programmed value of the MPMCDynamicSREX register is available at this vector.
MPMCDytAPRQ [3:0]	Memory Controller	Programmed value of the MPMCDynamicAPR register is available at this vector.
MPMCDytDALQ [3:0]	Memory Controller	Programmed value of the MPMCDynamicDAL register is available at this vector.
MPMCDytWRQ [3:0]	Memory Controller	Programmed value of the MPMCDynamicWR register is available at this vector.
MPMCDytRCQ [4:0]	Memory Controller	Programmed value of the MPMCDynamicRC register is available at this vector.
MPMCDytRFCQ [4:0]	Memory Controller	Programmed value of the MPMCDynamicRFC register is available at this vector.
MPMCDytXSRQ [4:0]	Memory Controller	Programmed value of the MPMCDynamicXSR register is available at this vector.
MPMCDytRRDQ [3:0]	Memory Controller	Programmed value of the MPMCDynamicRRD register is available at this vector.
MPMCDytMRDQ [3:0]	Memory Controller	Programmed value of the MPMCDynamicMRD register is available at this vector.
MPMCStExdWtQ [9:0]	Memory Controller	Programmed value of the MPMCStaticExtendedWait register is available at this vector.
nMPMCRPOUT	Pad	SyncFlash reset/power down signal. This signal can be controlled by the software through the bit-14 of the MPMCDynamic control register.
MPMCSREFACK	Power Management Unit	MPMC self-refresh acknowledgement signal. High on this signal indicates that SDRAM is in self-refresh mode. This signal is available the external world through the pin MPMCSREFACK

Table 4.6-2 Block Outputs of MpmcRegIf

4.7 Arbiter

The Arbiter (MpmcArbiter) performs the following functions:

- Routing the AHB memory transfer requests from MpmcAHBTop to the MpmcBufUnit block.
- Prioritisation of AHB Requests and granting control of Buffer or Memory interface to the highest priority AHB requesting for the transaction.
- Ensures Lock transactions from AHB once granted would continue to be in control of the memory interface until the transaction is totally completed.

The Arbiter in MPMC (PL172) supports 8 AHB ports. However, only 4 ports are used and the others are tied off and can be used for future expansion.

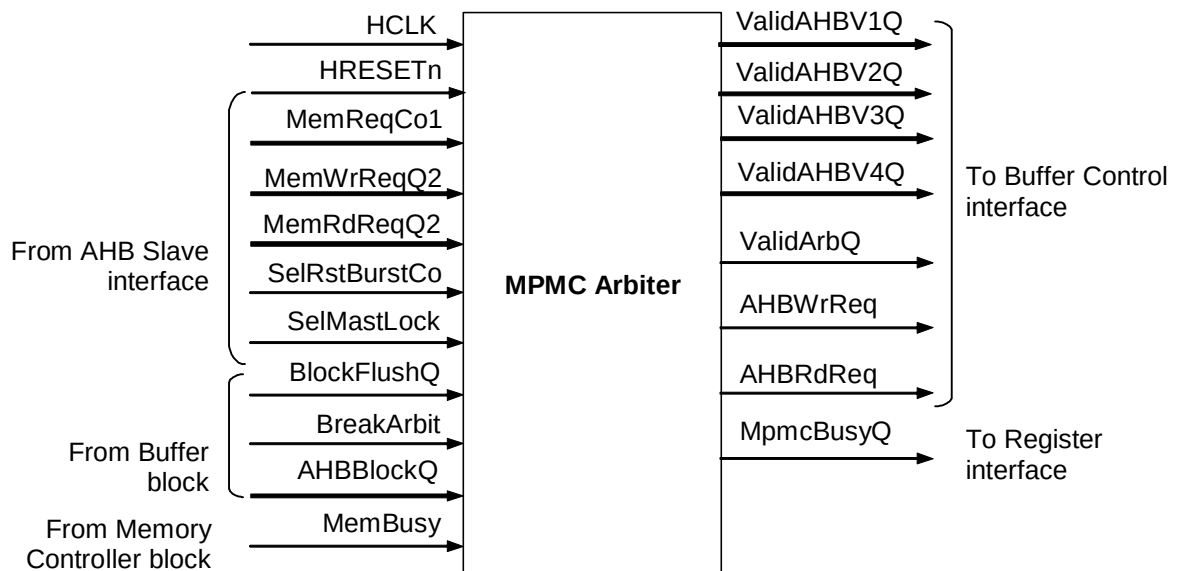


Figure 4.7-1 Block Inputs and Outputs of MpmcArbiter

A brief description of the interface signals of the arbiter block is given in **Table 4.7-1** and **Table 4.7-2**:

Signal Name	Source	Description
HCLK	Clock Generator	AHB Clock
HRESETn	Reset Generator	AHB Reset
MemReqCo1[7:0]	AHB memory interface	Requests lines, one from each of the eight AHB slaves to the Arbiter, demanding access to the buffers or the memory. These signals will become active in the very same clock in which the AHB comes out with a cycle in the Mpmc address space and will be sustained as long as the requested transaction is not completed by the Mpmc. When request from the AHB slave is high, the Arbiter grants the AHB based on its priority over other requesting AHBs. However, rearbitration is not done if a locked transaction is in progress.
MemWrReqQ2[7:0]	AHB memory interface	Indicates if the requested Mpmc transaction is a write transaction. It is used in the Arbiter in the clock subsequent to Arbitration selection.

Signal Name	Source	Description
MemRdReqQ2[7:0]	AHB memory interface	Indicates if the requested Mpmc transaction is a read transaction. It is used in the Arbiter in the clock subsequent to Arbitration selection.
SelRstBurstCo	AHB memory interface	When high, indicates that the currently granted AHB has completed all of its transactions, or a QWORD burst boundary has been reached or an error event has occurred on the granted AHB or the AHB has transitioned to a new NSEQ transaction. Based on the signal, the arbiter starts a new arbitration cycle and if other AHBs are requesting, a new AHB is granted the control of the buffers and the memory.
SelMastLock	AHB memory interface	Indicates that the currently granted AHB is initiating a LOCK transfer respectively and it should continue to remain the granted AHB until the entire LOCK transfer is completed in totality.
BlockFlushQ	Buffer Unit	Protective signal from Buffer to ensure that AHB0 once backedoff will be the default arbiter granted AHB until it is successful in initiating its read transaction. This is to ensure controllable latencies on AHB0 for reads
BreakArbit	Buffer Unit	When high, indicates that for the granted transaction the buffer logic has detected a condition, which can allow re-arbitration. This would be under circumstances such as, if the granted AHB were to need access to a buffer and there are no free buffers and memory interface is busy preventing a buffer to be freed up or the access resulted in the need to post a read request to Dynamic or Static memory space and that particular memory controller was busy handling a previous transaction. This is done to allow other AHBs, which might not need the above requirements to gain access and use bandwidth effectively.
AHBBlockQ[7:0]	Buffer Unit	Is used to block AHBs, which resulted in the BreakArbit signal to be activated to be blocked from subsequent arbitration until the BreakArbit causing condition is negated in the buffers block.
MemBusy	Memory controller	When high, indicates that the external memory interface is busy with some previous read or write transaction.

Table 4.7-1 Block Inputs of MpmcArbiter

Signal Name	Destination	Description
ValidAHBV1Q[7:0], ValidAHBV2Q[7:0], ValidAHBV3Q[7:0], ValidAHBV4Q[7:0]	Buffer Unit	Indicate the AHB, which is the master of Buffers and Memory Interface in the current clock. Total of four versions of the same signal are generated to ensure adequate drive to meet synthesis timings.
ValidArbQ	Buffer Unit, Memory Controller	When high indicates that the current cycle is a properly arbitrated cycle. It is generated in response to MemReqCo1[0:7] being active.

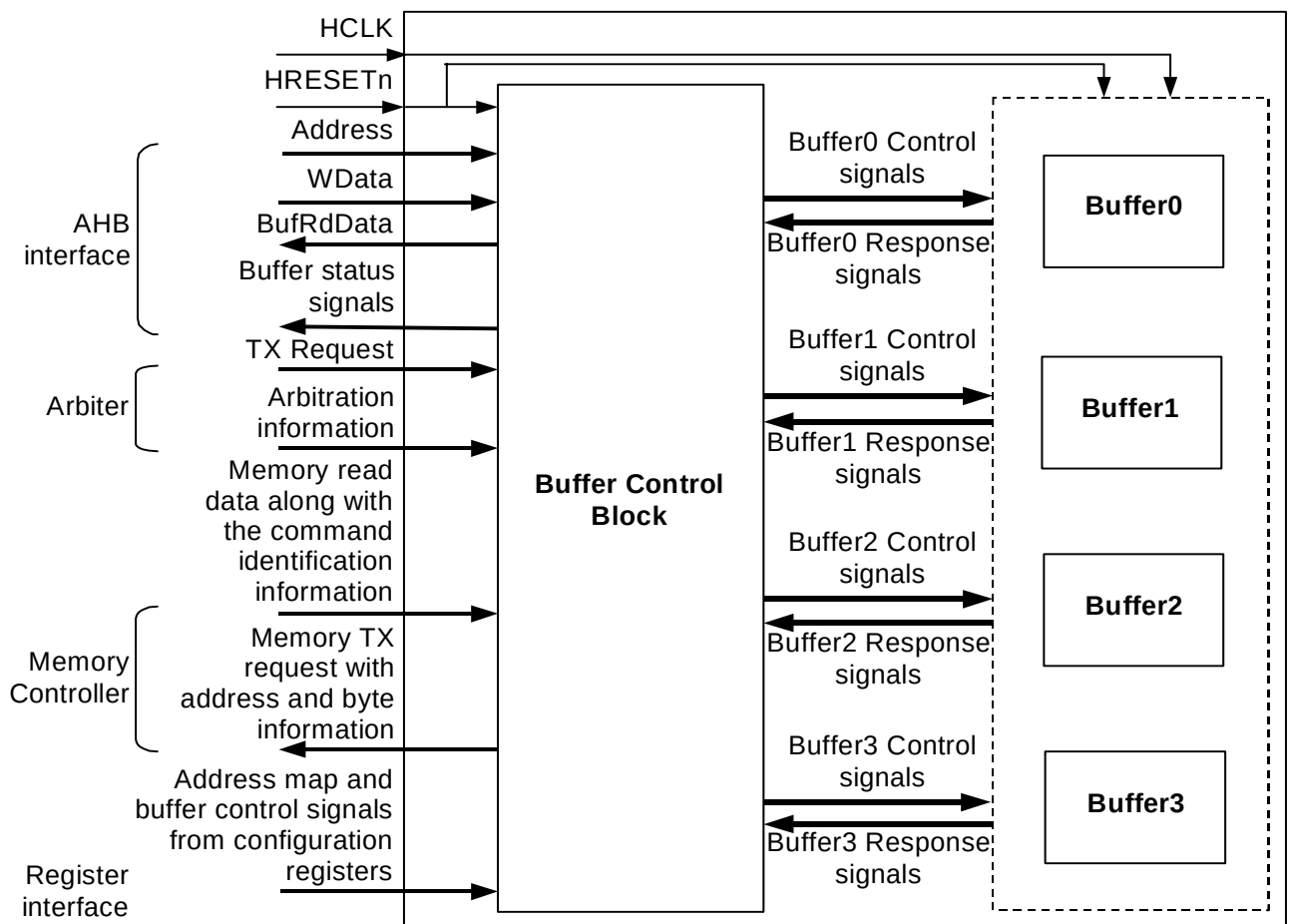
Signal Name	Destination	Description
AHBWrReq	Buffer Unit	Generated if the granted AHB requested a write transaction.
AHBRdReq	Buffer Unit	Generated if the granted AHB is requesting for a read transaction.
MpmcBusyQ	AHB register interface	When high indicates that either there is a valid Arbitration cycle in progress due to AHB requests or the external memory interface is busy with a past posted transaction. It is to be used to determine if the Mpmc is busy or idle.

Table 4.7-2 Block Outputs of MpmcArbiter

4.8 Buffer Unit

The buffer unit acts as the channel through which the physical memory is accessed from the AHB. The Buffer Unit (MpmcBufUnit) contains the following 5 sub-blocks:

- The buffer control logic (MpmcBufCntrl)
- Four instances of identical Quad-quad-word buffers (MpmcBuffer).

**Figure 4.8-1 Block Diagram of the Buffer Unit**

A brief description of the interface signals of the MpmcBufUnit block is given in **Table 4.8-1** and **Table 4.8-2** below:

Signal Name	Source	Description
HCLK	Clock Generator	AHB Clock
HRESETn	Reset Generator	AHB Reset
SelAddr[27:2]	AHB memory interface	Address lines driven by the AHB granted by the Arbiter
SelByte[3:0]	AHB memory interface	Valid Byte lanes driven by the AHB granted by the Arbiter.
SelChipSel[7:0]	AHB memory interface	External memory chip selects driven by the AHB granted by the Arbiter.
AdvSelA3A2[1:0]	AHB memory interface	Advance A3 and A2 address lines driven by the granted AHB.
AdvSelA5A4[1:0]	AHB memory Interface	Advance A5 and A4 address lines driven by the granted AHB.
AdvSelByte[3:0]	AHB memory interface	Advance Byte lane information driven by the granted AHB
WData[31:0]	AHB memory interface	Write Data driven by the granted AHB
SelAbort	AHB memory interface	Driven by the granted AHB if the current transaction has to be aborted due to width of transaction error or write to Write Protect memory.
SelBusy	AHB memory interface	Driven by the granted AHB if the current transaction results in a Busy response on the AHB interface.
ValidAHBQ[7:0]	Arbiter	Granted AHB Id
ValidArbQ	Arbiter	When high indicates a Valid Arbitration cycle and hence a valid transaction to be operated by the buffer logic.
AHBWrReq	Arbiter	When high indicates that the granted AHB is attempting a Write transaction
AHBRdReq	Arbiter	When high indicates that the granted AHB is attempting a Read transaction
AddrMap0Q[6:0]	Memory Controller	Address re-mapping to be used for external chip select4 (Dynamic Memory Chip Select 0)
AddrMap1Q[6:0]	Memory Controller	Address re-mapping to be used for external chip select5 (Dynamic Memory Chip Select 1)
AddrMap2Q[6:0]	Memory Controller	Address re-mapping to be used for external chip select6 (Dynamic Memory Chip Select 2)
AddrMap3Q[6:0]	Memory Controller	Address re-mapping to be used for external chip select7 (Dynamic Memory Chip Select 3)
ChipSel4MWQ	Memory Controller	External memory width of chip select 4
ChipSel5MWQ	Memory Controller	External memory width of chip select 5
ChipSel6MWQ	Memory Controller	External memory width of chip select 6
ChipSel7MWQ	Memory Controller	External memory width of chip select 7
BufEnQ[7:0]	Memory Controller	When high indicates on a chip select wide basis if the transaction to the given chip select should be buffer enabled.
MemAHBId[8:0]	Memory Controller	Handle returned by the memory controller for a transaction posted by the buffer unit to the memory controller block. All zeros will be returned for un-buffered access and the original requesting AHB Id will be returned for Buffered read transactions.
MemA3A2[1:0]	Memory Controller	Address A3 and A2 returned by the memory controller to indicate the appropriate word position in the buffer which needs to be updated with the returned data.
MemAddr5	Memory Controller	Address A5 returned by the memory controller to indicate the appropriate QWORD position in the buffer which needs to be updated with the returned data.

Signal Name	Source	Description
MemAddr4	Memory Controller	Address A4 returned by the memory controller to indicate the appropriate QWORD position in the buffer which needs to be updated with the returned data.
MemByte[3:0]	Memory Controller	Valid Byte information for the data returned by the memory controller to the buffer to update the appropriate byte locations.
MemRData[31:0]	Memory Controller	Data returned by the memory controller in response to a read request.
DyUseMem	Memory Controller	When high, indicates that the dynamic memory controller is returning data
StUseMem	Memory Controller	When high, indicates that the static memory controller is returning data
DyBufFullQ	Memory Controller	When high, indicates that the dynamic memory controller cannot accept any more transactions to dynamic memory space
StBufFullQ	Memory Controller	When high, indicates that the static memory controller cannot accept any more transactions to the static memory space
DyEndOfBurst	Memory Controller	When high, indicates that the dynamic memory controller has returned the last data for a requested burst.
StEndOfBurst	Memory Controller	When high, indicates that the static memory controller has returned the last data for a requested burst.
StWaitingWrQ	Memory Controller	When high, indicates that no Flushing of buffers to static memory space should take place as the static memory is actually expecting a continued Write Burst from a Un-buffered access.

Table 4.8-1 Block Inputs of MpmcBufUnit

Signal	Destination	Description
UseBuf	AHB memory interface	Indicates that the buffers have honoured the current AHB transaction and the AHB can issue HREADYOUT in the subsequent clock cycle.
BufRDataQ[127:0]	AHB memory interface	The full contents of buffer are made available to the AHB interface for buffer enabled reads and the AHB interface is expected to pick the appropriate 32 bit wide data for endianisation operation.
UseMemCo	AHB memory interface	Asserted by the buffer block to indicate to AHB that the current un-buffered transaction can be completed. Is used for generation of HREADYOUT in the very same clock as it become high.
WrBufNotEmpty	AHB register interface	Status bit indicating if the buffers are full or empty. When high at least one buffer has write data.

Signal	Destination	Description
BreakArbit	Arbiter	When high indicates that for the granted transaction the buffer logic has detected a condition, which can allow re-arbitration. This would be under circumstance such as, if the granted AHB were to need access to a buffer and there are no free buffers and memory interface is busy preventing a buffer to be freed up or the access resulted in the need to post a read request to Dynamic or Static memory space and that particular memory controller was busy handling a previous transaction. This is done to allow other AHB's which might not need the above requirements to gain access and use bandwidth effectively.
AHBBlockQ[7:0]	Arbiter	Is used to block AHBs which resulted in the BreakArbit signal to be activated to be blocked from subsequent arbitration until the BreakArbit causing condition is negated in the buffers block.
StMemAddr[27:4]	Memory Controller	Linear address from AHB for accessing memory devices in the Static memory space.
DyMemAddr[29:0]	Memory Controller	AHB address lines remapped as per the address re-mapping table in the PL172 TRM [Ref. 2]. It consists of Ras and Cas addresses and bank select lines needed to access memory devices in the Dynamic memory space.
BufDriveByte[15:0]	Memory Controller	Valid Byte lane information to the memory controller for the given access. The signal also has the AHB address line A3 and A2 information embedded.
BufDataToSt[127:0]	Memory Controller	Expanded Data bus to Memory controller. Done to enable single clock flushes. For direct AHB access data will be valid only in one of the four 32 bit words. Used for passing the data to the static memory controller
BufDataToDy[127:0]	Memory Controller	Expanded Data bus to Memory controller. Done to enable single clock flushes. For direct AHB access data will be valid only in one of the four 32 bit words. Used for passing the data to the dynamic memory controller.
StChipSel[3:0]	Memory Controller	This information is provided to decode the individual four banks in the Static memory space.
DyChipSel[3:0]	Memory Controller	This information is provided to decode the individual four banks in the Static memory space.
DyA5A2[1:0]	Memory Controller	AHB address A5, A4, A3 and A2 information provided to the Dynamic memory controller to start the burst of four word access from the correct address.
UnBufMemAcc	Memory Controller	Indicates that the current access to the memory controller is a direct access from the AHB and will not be operated by the buffer block.
BufAccCode[8:0]	Memory Controller	This handle from the buffer block to the Memory controller block indicates the source and type of access. The Memory controller will have to return the same with UseMem for the AHB & Buffer to interpret and direct to appropriate requestor.
StPostWReq	Memory Controller	Indicates Write request posted to static memory controller.
StPostRReq	Memory Controller	Indicates read request posted to static memory controller.
DyPostWReq	Memory Controller	Indicates Write request posted to dynamic memory controller.
DyPostRReq	Memory Controller	Indicates read request posted to dynamic memory controller.

Table 4.8-2 Block Outputs of MpmcBufUnit

4.8.1 Buffer

The Quad-Quad-word buffer (MpmcBuffer) performs the following functions:

- Acts as the storage mechanism for all buffer-enabled access and is Quad-QWORD deep in size.
- Updates Data, Address, Chip Select and Bytelane information for access from AHB and updation of Data from memory interface.
- Indicates if the current AHB transaction is a HIT to the buffer.
- Indicates if the current AHB read transaction has all the necessary Bytelanes available in the buffer.
- Address translation for Dynamic memory access as per the address-re-mapping table in the MPMC (PL172) TRM [Ref. 2].
- Keeps track of buffer flushes in QWORD granularity to the memory.

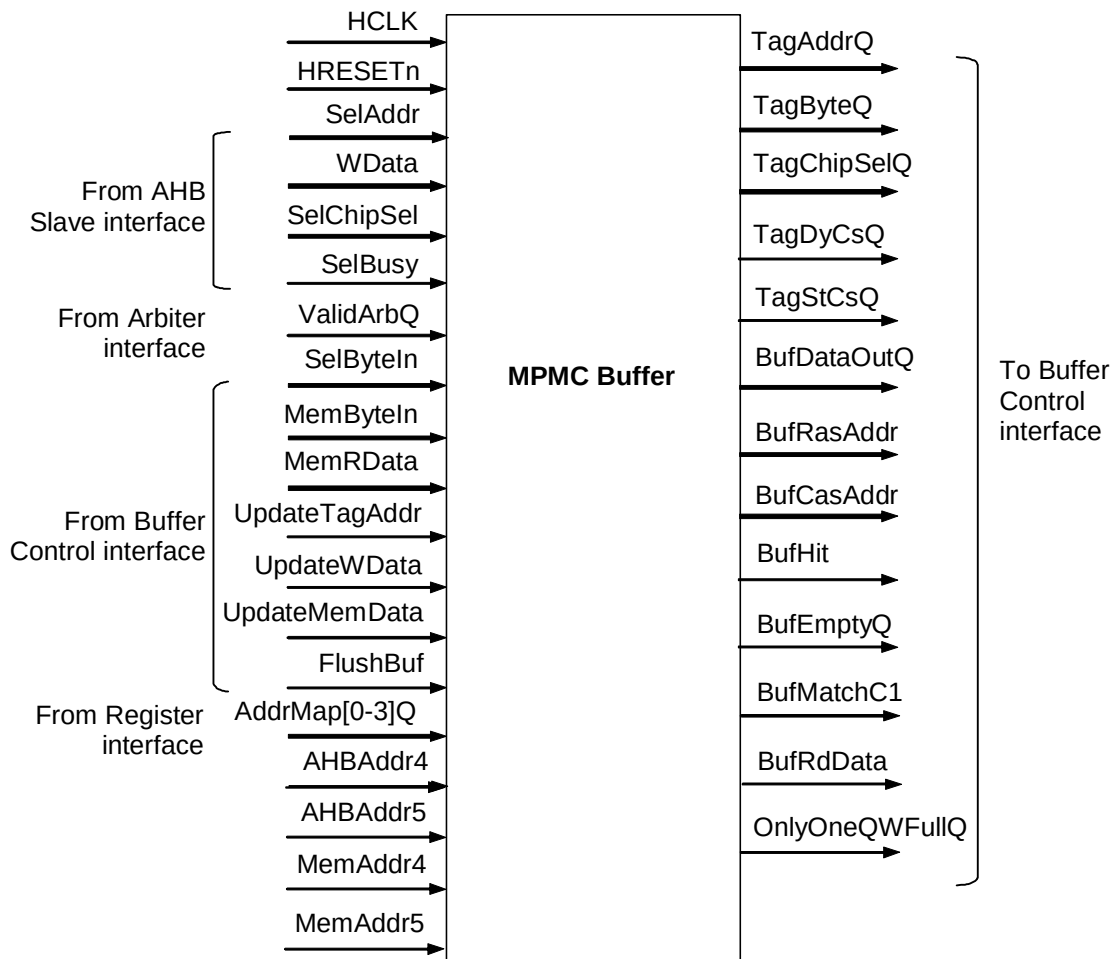


Figure 4.8-2 Block Inputs and Outputs for MpmcBuffer

A brief description of the interface signals of the MpmcBuffer block is given in **Table 4.8-3** and **Table 4.8-4** below:

Signal	Source	Description
HCLK	Clock Generator	AHB Clock
HRESETn	Reset Generator	AHB Reset
SelAddr[27:6]	AHB memory interface	This signal carries the address information from the currently granted AHB for the current transaction.

Signal	Source	Description
WData[31:1]	AHB memory interface	This signal carries the data information from the currently granted AHB for a given Write transaction. SelBusy, when high indicates that the current transaction from the AHB has entered into Busy State and hence no writes into the buffer should take place until the AHB exits the State.
SelChipSel[7:0]	AHB memory interface	When high, selects the appropriate external memory bank to be accessed. The lower 4 bits correspond to static memory space and the upper 4 for the dynamic memory space. All of the chip selects are orthogonal.
SelBusy	AHB memory interface	Driven by the granted AHB if the current transaction results in a Busy response on the AHB interface.
ValidArbQ	Arbiter	When high, indicates that there is a valid AHB transaction that needs to be honoured by the buffer block. When it is low the buffer block is free to do housekeeping functions like flushing the buffers to effectively use the external memory bandwidth.
MemByteIn[15:0]	Memory Controller	Byte lane selects, to select the correct 8-bit storage in data returned by memory controller respectively.
MemRData[31:0]	Memory Controller	This signal carries the data returned by the memory controller for bufferable posted read access
MemAddr5	Memory Controller	In conjunction with MemAddr4 determines the appropriate QWORD that needs to be operated upon for memory side transactions.
MemAddr4	Memory Controller	In conjunction with MemAddr5 determines the appropriate QWORD that needs to be operated upon for memory side transactions.
AddrMap[3..0]Q[6:0]	Memory Controller	This signal carries the information for address translation to be carried out for access to dynamic memory chip selects [3..0].
SelByteIn[15:0]	Buffer Controller	Byte lane selects, to select the correct 8-bit storage in the 128-bit width organised buffer for access from AHB.
UpdateTagAddr	Buffer Controller	When high indicates that the buffer should update itself with a new address
UpdateWData	Buffer Controller	When high indicates that the buffer should update itself with data from AHB side
UpdateMemData	Buffer Controller	When high indicates that the buffer should update itself with data returned by the memory controller
FlushBuf	Buffer Controller	When high, indicates that the buffer should empty its current contents to the memory controller if it has valid data available.
AHBAddr5	Buffer Controller	In conjunction with AHBAddr4 determines the appropriate QWORD that needs to be operated upon for AHB transactions.
AHBAddr4	Buffer Controller	In conjunction with AHBAddr5 determines the appropriate QWORD that needs to be operated upon for AHB transactions.
MemAddr5	Buffer Controller	In conjunction with AHBAddr4 determines the appropriate QWORD that needs to be operated upon for memory side transactions.
MemAddr4	Buffer Controller	In conjunction with AHBAddr5 determines the appropriate QWORD that needs to be operated upon for memory side transactions.

Table 4.8-3 Block inputs of MpmcBuffer

Signal Name	Destination	Description
TagAddrQ[27:4]	Buffer Controller	Address information stored in the buffer
TagByteQ[15:0]	Buffer Controller	Chip select information stored in the buffer
TagChipSelQ[7:0]	Buffer Controller	Valid byte lane information stored in the buffer
BufDataOutQ[127:0]	Buffer Controller	Data stored in the buffer
BufRdData[127:0]	Buffer Controller	Read data to AHB
TagDyCsQ	Buffer Controller	When high, indicates that the buffer contains dynamic memory data
TagStCsQ	Buffer Controller	When high, indicates that the buffer contains static memory data
BufRasAddr[14:0]	Buffer Controller	Remapped RAS address for the linear address stored in the buffer.
BufCasAddr[14:0]	Buffer Controller	Remapped CAS address for the linear address stored in the buffer.
BufHit	Buffer Controller	When high, indicates that the current AHB access has its address and chip select information matching with the contents of the buffer.
BufEmptyQ	Buffer Controller	When high, indicates that the buffer is empty because there are no contents written from AHB or the buffer has been flushed or it contains only data read from memory side.
OnlyOneQWFullQ	Buffer Controller	Buffer has write data only in one QWORD and hence is allocatable simultaneously with the flush cycle related to that.
ByteMatchC1	Buffer Controller	When high indicates that for the given read access that resulted in a BufHit also has all of the requested byte lane information locally in the buffer logic.

Table 4.8-4 Block Outputs of MpmcBuffer

4.8.2 Buffer Controller

This block is responsible for generation of all control signals needed for the individual buffers and also for managing the top level buffer input, output ports and the necessary handshake with the MpmcAHBTop, MpmcArbiter and MpmcMemCntl blocks.

The port level connections of the MpmcBufCntrl Interface are shown in **Figure 4.8-3**.

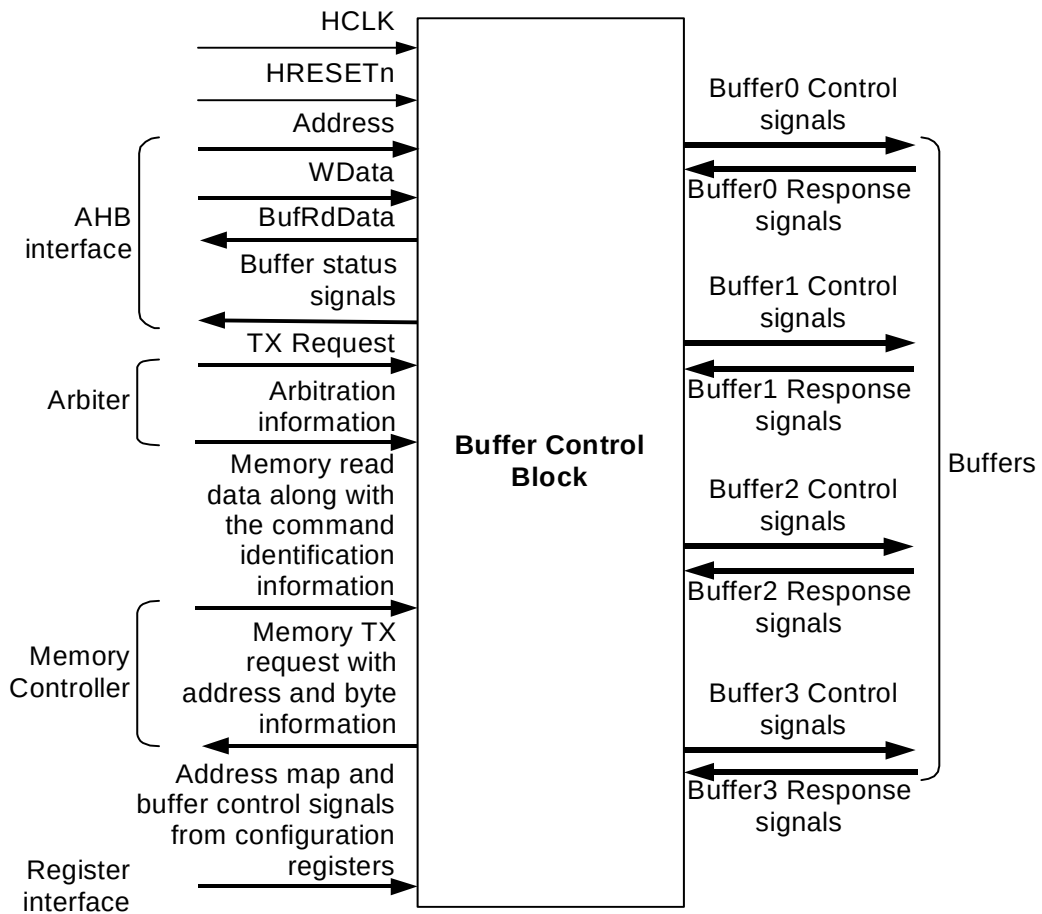


Figure 4.8-3 Block Inputs and Outputs for MpmcBufCntl

The various interface signals related to this block are as described in **Table 4.8-5** and **Table 4.8-6**:

Signal Name	Source	Description
HCLK	Clock Generator	AHB Clock
HRESETn	Reset Generator	AHB Reset
SelAddr[27:2]	AHB memory interface	Address information from the currently granted AHB for the current transaction.
Wdata[31:0]	AHB memory interface	Data from the currently granted AHB for a given Write transaction.
SelChipSel[7:0]	AHB memory interface	When high, selects the appropriate external memory bank to be selected. The lower 4 bits correspond to static memory space and the upper 4 for the dynamic memory space. All of the chip selects are orthogonal.
SelByte[3:0]	AHB memory interface	When high, indicates that the current transaction from the AHB has, carries valid data in the byte lanes for writes and is requesting for data on the byte lanes for reads.
AdvSelA3A2[1:0]	AHB memory interface	Synthetically generated next likely word address transaction (address lines 3 downto 2) by the granted AHB.
AdvSelA5A4[1:0]	AHB memory interface	Synthetically generated next likely Quad-Quadword address transaction (address lines 5 downto 4) by the granted AHB.

Signal Name	Source	Description
AdvSelByte[3:0]	AHB memory interface	Synthetically generated next likely byte address transaction (address lines 1 downto 0) by the granted AHB
SelAbort	AHB memory interface	When high, indicates that the granted AHB, is requesting the initiated transaction to be aborted. It is generated for transactions like writes to write protected area, width of transaction errors.
ValidAHBQ[7:0]	Arbiter	Granted AHB's ID. One bit of this signal will be asserted at a time indicating the AHB port which has been granted.
ValidArbQ	Arbiter	When high, indicates a Valid Arbitration cycle and hence a valid transaction to be operated by the buffer logic.
AHBWrReq	Arbiter	When high, indicates that the granted AHB is attempting a Write transaction.
AHBRdReq	Arbiter	When high, indicates that the granted AHB is attempting a Read transaction
ModeDone	Memory Controller	Mode Programming status indication.
MemAHBId[8:0]	Memory Controller	Returned by the memory controller block along with requested read data. MemAHBId is used to route the data to requesting quadword buffer or AHB port.
MemA3A2[1:0]	Memory Controller	Returned by the memory controller block along with requested read data. MemA3A2 is used to update the correct word of the QWORD buffers appropriately.
MemByte[3:0]	Memory Controller	Returned by the memory controller block along with requested read data. MemByte is used to update the correct bytes in the QWORD buffers appropriately
DyUseMem	Memory Controller	When high, indicates that current data being returned by the memory controller is for an access to the dynamic memory space.
StUseMem	Memory Controller	When high, indicates that current data being returned by the memory controller is for an access to the static memory space.
DyBufFullQ	Memory Controller	When high, indicates that the dynamic memory controller is unable to accept any more new requests.
StBufFullQ	Memory Controller	When high, indicates that the static memory controller is unable to accept any more new requests.
DyEndOfBurstQ	Memory Controller	When high, indicates that the current cycle is the last cycle of a given requested burst from the Dynamic memory controller.
StEndOfBurstQ	Memory Controller	When high, indicates that the current cycle is the last cycle of a given requested burst from the Static memory controller.
StWaitingWrQ	Memory Controller	When high indicates that no flush write should take place as the static memory is actually expecting a continued write burst for unbuffered access.
AddrMap[3..0]Q[6:0]	Memory Controller	Address map information for address translation to be carried out for accesses to the dynamic memory chip selects [3..0].
BufEnQ[7:0]	Memory Controller	Each bit, when high, indicates that any access from any AHB to the given chip select range is a buffer enabled access.
TagAddr[3..0]Q[27:4]	Buffers 3 to 0	Stored address information in the four QWORD buffers.
TagByte[3..0]Q[15:0]	Buffers 3 to 0	Stored valid byte lane information for the data stored in the four QWORD deep buffers.

Signal Name	Source	Description
TagChipSel[3..0]Q[7:0]	Buffers 3 to 0	Stored chip select information in the four buffers.
TagDyCs[3..0]Q	Buffers 3 to 0	Each bit, when high, indicates that the corresponding buffer has data related to dynamic memory space.
TagStCs[3..0]Q	Buffers 3 to 0	Each bit, when high, indicates that the corresponding buffer has data related to static memory space.
BufDataOut[3..0]Q[127:0]	Buffers 3 to 0	Data store in each of the four buffers and driven out to the memory controller for flushes
BufRdData[3..0]Q[127:0]	Buffers 3 to 0	Data store in each of the four buffers and driven out to AHB interface for buffer hit read access.
BufHit[3..0]	Buffers 3 to 0	Each bit, when high, indicates that the given AHB transaction is a matching with the address stored in the corresponding buffer.
BufEmpty[3..0]Q	Buffers 3 to 0	Each bit, when high indicate that the given buffer is empty.
ByteMatchC1[3..0]	Buffers 3 to 0	Each bit, when high, indicates that there is a buffer hit and also for the given read transaction all of the byte information requested by the AHB can be serviced from the buffer in itself.
OnlyOneQWFullQ[3..0]Q	Buffers 3 to 0	Each bit, when high indicate that the given buffer has only one more QWORD to be flushed and indicates that the buffer can be reallocated along with the flush.
BufRasAddr[3..0][14..0]	Buffers 3 to 0	Converted (mapped) RAS address for the stored AHB address from each of the buffers
BufCasAddr[3..0][14..0]	Buffers 3 to 0	Converted (mapped) CAS address for the stored AHB address from each of the buffers

Table 4.8-5 Block Inputs of MpmcBufCntrl

Signal Name	Destination	Description
UseBuf	AHB memory interface	When high, indicates that the buffers are honouring the granted AHB and the AHB is free to generate HREADYOUT in the subsequent HCLK edge. If it were a write transaction, it allows the AHB to proceed with the next transaction. If it were a read, the signal indicates that the granted AHB can also sample read data from the buffers, BufRdDataQ[127:0].
BufRdDataQ[127:0]	AHB memory interface	Buffer read data returned with UseBuf assertion.
UseMemCo	AHB memory interface	When high, indicates that it is response to an Unbuffered memory access and the memory controller is ready to complete the transaction in the very same clock.
WrBufNotEmpty	AHB register interface	When high, indicates that at least one of the four buffers has un-flushed write data
SelByteIn[15:0]	Buffer 3 to 0	Each bit, when high, indicates which of the sixteen byte lanes in the buffer are being accessed and need to be updated for writes from the AHB side.
MemByteIn[15:0]	Buffer 3 to 0	Each bit, when high, indicates which of the sixteen byte lanes in the buffer are being accessed and need to be updated for writes from the memory side.

Signal Name	Destination	Description
AHBAddr5	Buffer 3 to 0	Address 5 driven for the given AHB side transaction to the buffers
AHBAdd4	Buffer 3 to 0	Address 4 driven for the given AHB side transaction to the buffers
UpdateTagAddr[3:0]	Buffers 3 to 0	Each bit, when high, indicates that the selected buffer needs to update itself with a new address driven by the granted AHB. Only one of the bits will go high at any given time.
UpdateWData[3:0]	Buffers 3 to 0	Each bit, when high, indicates that the selected buffer needs to update itself with new write data driven by the granted AHB. Only one of the bits will go high at any given time.
FlushBuf[3:0]	Buffers 3 to 0	Each bit, when driven high, indicates that the selected buffer has to flush its contents to the memory interface. Only one of the bits will go high at any given time.
UpdateMemData[3:0]	Buffers 3 to 0	Each bit, when high, indicates that the data returned by the memory controller needs to be updated into the selected buffer. UpdateMemData is orthogonal with UpdateWData and only one of the bits will go high at any given time
BreakArbit	Arbiter	When high, indicates that for the granted transaction, the buffer logic has detected a condition, which can allow re-arbitration. This would be under circumstance such as, if the granted AHB were to need access to a buffer and there are no free buffers and memory interface is busy preventing a buffer to be freed up or the access resulted in the need to post a read request to Dynamic or Static memory space and that particular memory controller was busy handling a previous transaction. This is done to allow other AHB's which might not need the above requirements to gain access and use bandwidth effectively.
AHBBlockQ[7:0]	Arbiter	Is used to block AHB's which resulted in the BreakArbit signal to be activated to be blocked from subsequent arbitration until the BreakArbit causing condition is negated in the buffers block.
BlockFlushQ	Arbiter	Protective signal from Buffer to ensure that AHB0 once backedoff will be the default arbiter granted AHB until it is successful in initiating its read transaction. This is to ensure controllable latencies on AHB0 for reads. Internally to the buffer controller the signal also acts to mask unwanted flushes than that would be needed to make a buffer available for AHB0 to be allocated.
StMemAddr[27:4]	Memory Controller	Address lines driven for accessing static memory
DyMemAddr[29:0]	Memory Controller	Address lines driven for accessing dynamic memory. Upper half of address bus carries RAS and bank information and the lower half the CAS address and bank information.

Signal Name	Destination	Description
BufDriveByte[15:0]	Memory Controller	Valid byte information for the transaction passed to the memory controller interface
BufDataToSt[127:0]	Memory Controller	Data for write transaction to static memory
BufDataToDy[127:0]	Memory Controller	Data for write transaction to dynamic memory
StChipSel[3:0]	Memory Controller	Chip select information for the external static memory to be accessed
DyChipSel[3:0]	Memory Controller	Chip select information for the external dynamic memory to be accessed.
DyA5A2[1:0]	Memory Controller	Address bits to be used by the dynamic memory controller to start the burst at the appropriate position for doing a burst of four word transactions.
UnBufMemAcc	Memory Controller	When high, indicates that the current transaction initiated by the AHB is directly to the memory controller bypassing the buffers.
BufAccCode[8:0]	Memory Controller	Indicates the requestor source (AHB ID or Buffer number) and also the type of access (buffered or unbuffered) to the memory controller. The Memory controller will have to return the same with UseMem for the AHB & Buffer to interpret and direct to appropriate requestor.
StPostWReq	Memory Controller	When high, indicates that the transaction initiated to static memory space is a write.
StPostRReq	Memory Controller	When high, indicates that the transaction initiated to static memory space is a read.
DyPostWReq	Memory Controller	When high, indicates that the transaction initiated to dynamic memory space is a write.
DyPostRReq	Memory Controller	When high, indicates that the transaction initiated to dynamic memory space is a read.

Table 4.8-6 Block Outputs of MpmcBufCtrl

4.9 Memory Controller

This block (MpmcMemCntl) is the top-level module of the memory control logic of the MPMC. It receives dynamic or static memory access requests from the buffer control logic and converts these requests into memory access commands on the external memory bus.

This module contains the following blocks:

- The dynamic memory controller block, which controls all the dynamic memory accesses of the MPMC.
- The static memory controller block, which controls all the static memory accesses of the MPMC.
- The memory bus granting logic, which controls the memory bus granting and degranting logic for the two memory controller blocks

- The memory controller interface, which contains the interface logic between the signals exchanged between the memory controllers and the buffer and AHB interfaces.

A block schematic of the sub-blocks in the Memory Controller block is shown in **Figure 4.9-1**. The ports of MpmcMemCntl are described in the sub-sections on the modules in the dynamic memory controller and static memory controller blocks.

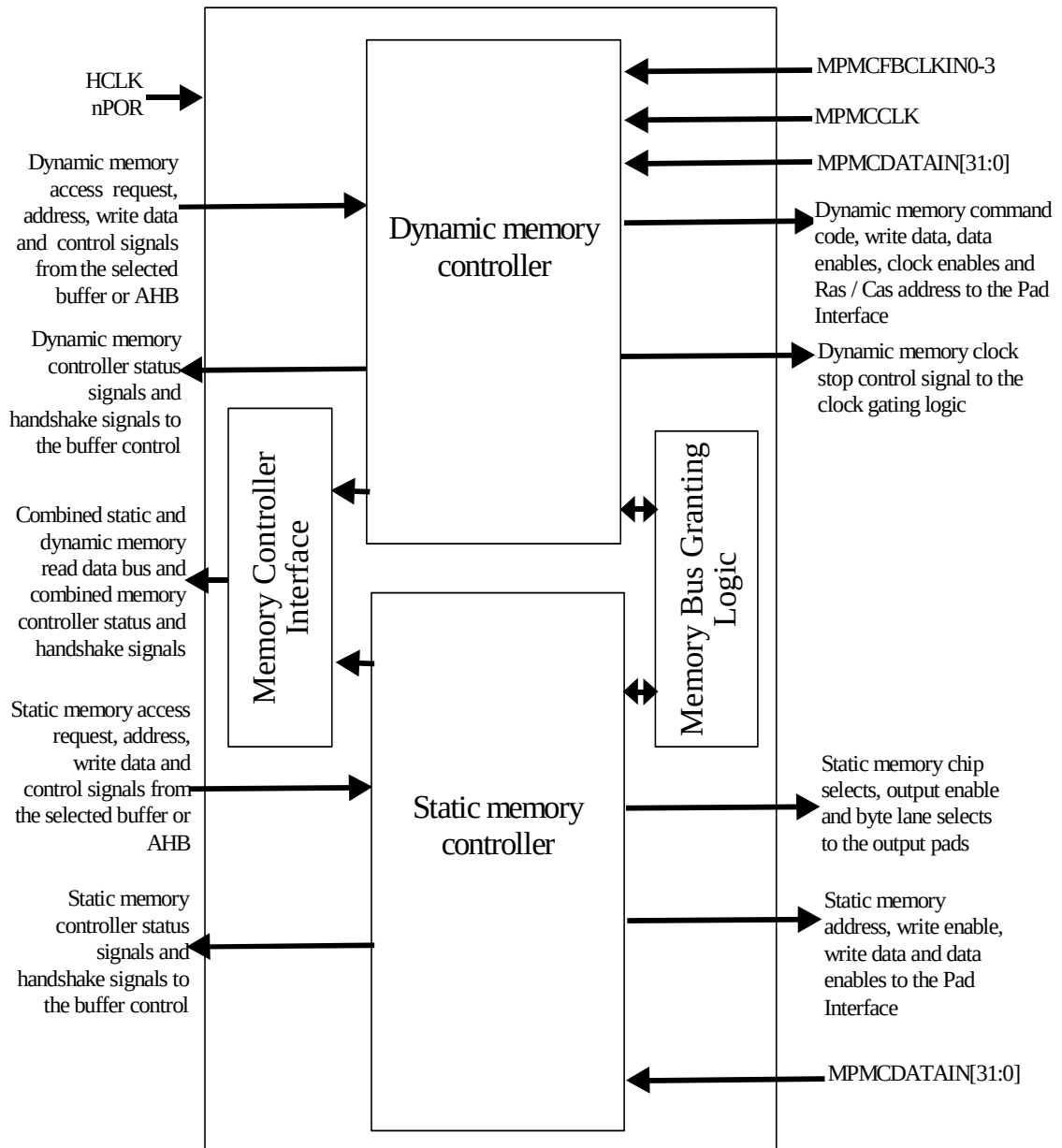


Figure 4.9-1 Block Diagram of the Memory Controller

4.10 Dynamic Memory Controller

The dynamic memory controller block (MpmcDyMemCntl) receives the dynamic memory access requests along with the address, data and byte lane information from the Buffer Unit. The requests are then processed and the dynamic

memory read or write commands are issued to the target memory device according to the timings programmed in the dynamic memory control registers. The dynamic memory controller contains the following sub-blocks:

- The dynamic memory register block, which is instantiated 4 times. Each block contains the dynamic memory configuration registers to control the dynamic memory device connected to one chip select.
- The dynamic memory read request generation block, which internally generates the read request to the data access state machine depending on the data requirement of the requested AHB burst.
- The dynamic memory data access State Machine blocks (DASM1 and DASM2), that describe the two State Machines, which performs data-accesses on the SDRAM chip-selects.
- The dynamic memory mode State Machine, which controls the mode register programming of the SDRAMs.
- The dynamic memory refresh State Machine, which controls the issue of auto-refresh and self-refresh commands to the SDRAMs.
- The dynamic memory SyncFlash State Machine (FCSM), which controls the issue of commands to the Micron SyncFlash device.
- The dynamic memory read data FIFO block which instantiates the dynamic memory FIFO control logic and the FIFO registers.
- The dynamic memory internal register block, which stores the internal registers and flags to be used by the State Machines.
- The dynamic memory latency control block, which contains various counters, which track the latencies for SDRAM accesses.

A block schematic of the sub-blocks in the dynamic memory controller block is shown in **Figure 4.10-1**

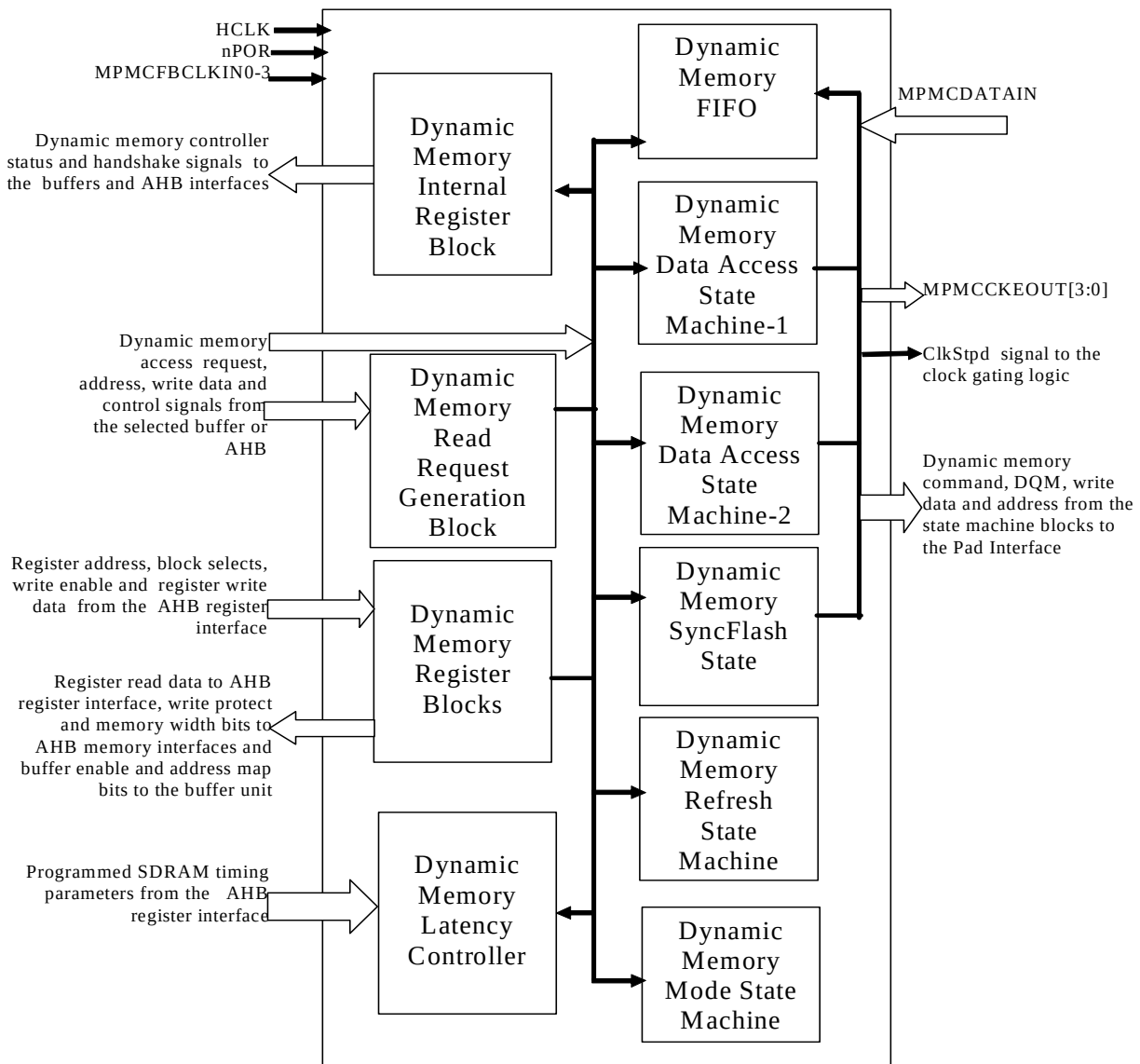


Figure 4.10-1 Block diagram of the Dynamic memory controller

4.10.1 Dynamic memory register block

This block (MpmcDyRegBlk) implements the dynamic memory control registers for one chip select. This block is instantiated four times to realise the control registers for the four dynamic memory chip selects. The AHB register interface block (MpmcRegIf) asserts the DyBlkSelCo2 signal corresponding to one dynamic memory chip select when a register used to control that chip select's memory is selected. This block generates the write enables for the registers in the selected dynamic memory register block on writes from the AHB to one of the registers in this block. It drives the selected register data on the DyBlkRdData bus to the AHB register interface block. When not selected, the DyBlkRdData bits are driven to zero to enable the use of OR logic for combining read data from the different register blocks.

Figure 4.10-2 describes the block inputs and outputs of the dynamic memory register block.

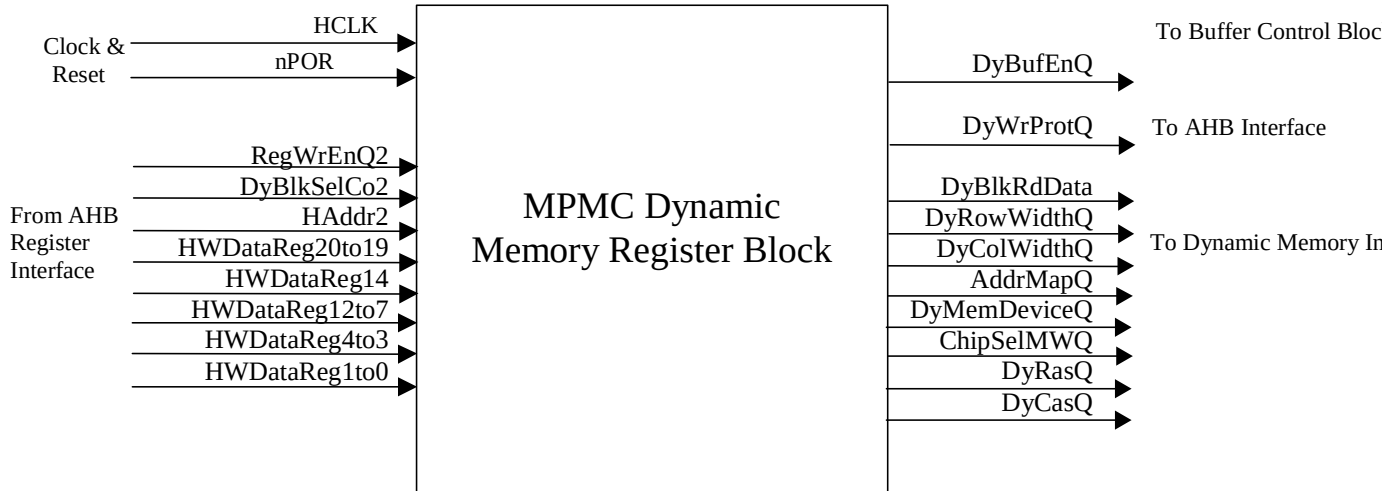


Figure 4.10-2 Block Inputs and Outputs for MpmcDyRegBlk

The various interface signals related to this block are as described in **Table 4.10-1** and **Table 4.10-2**:

Signal Name	Source	Description
HCLK	Clock Generator	AHB Clock.
NPOR	Reset Generator	Power on reset.
RegWrEnQ2	AHB Register Interface	Registered register write request from AHB slave register interface.
DyBlkSelCo2	AHB Register Interface	Dynamic register block select.
Haddr2	AHB Register Interface	Registered HADDRREG bit 2.
HWDDataReg20to19	AHB Register Port	Register write data bits 20 to 19.
HWDDataReg14	AHB Register Port	Register write data bit 14.
HWDDataReg12to7	AHB Register Port	Register write data bits 12 to 7.
HWDDataReg4to3	AHB Register Port	Register write data bits 4 to 3.
HWDDataReg1to0	AHB Register Port	Register write data bits 1 to 0.

Table 4.10-1 Block Inputs of MpmcDyRegBlk

Signal Name	Destination	Description
DyBlkRdData	AHB register interface	Register read data.
DyBufEnQ	Buffer unit block	Buffer enable
AddrMapQ	Buffer unit block	Dynamic memory address mapping

Signal Name	Destination	Description
DyWrProtQ	AHB memory interface	Dynamic memory write protect.
ChipSelMWQ	AHB memory interface, Dynamic memory internal register block, Dynamic read request generation block	Dynamic memory width.
DyRowWidthQ	Dynamic memory internal register block	Dynamic memory row width.
DyColWidthQ	Dynamic memory internal register block	Dynamic memory column width.
DyMemDeviceQ	Dynamic memory internal register block	Dynamic memory device.
DyRasQ	Dynamic memory internal register block	Ras delay value from MpmcDyRasCas register.
DyCasQ	Dynamic memory internal register block	Cas delay value from MpmcDyRasCas register.

Table 4.10-2 Block Outputs of MpmcDyRegBlk

4.10.2 Dynamic Memory Read Request Generation Block

This block generates multiple read data request if the AHB master needs more than one quad-word of data within a single AHB burst. Dynamic data access state machines return 4 words of data for each read request. By using the HSIZE and HBURST information, this block calculates the data requirement of the requesting burst and internally generates the read request to the data access state machine. **Figure 4.10-3** describes the block inputs and outputs of the dynamic read request generation block

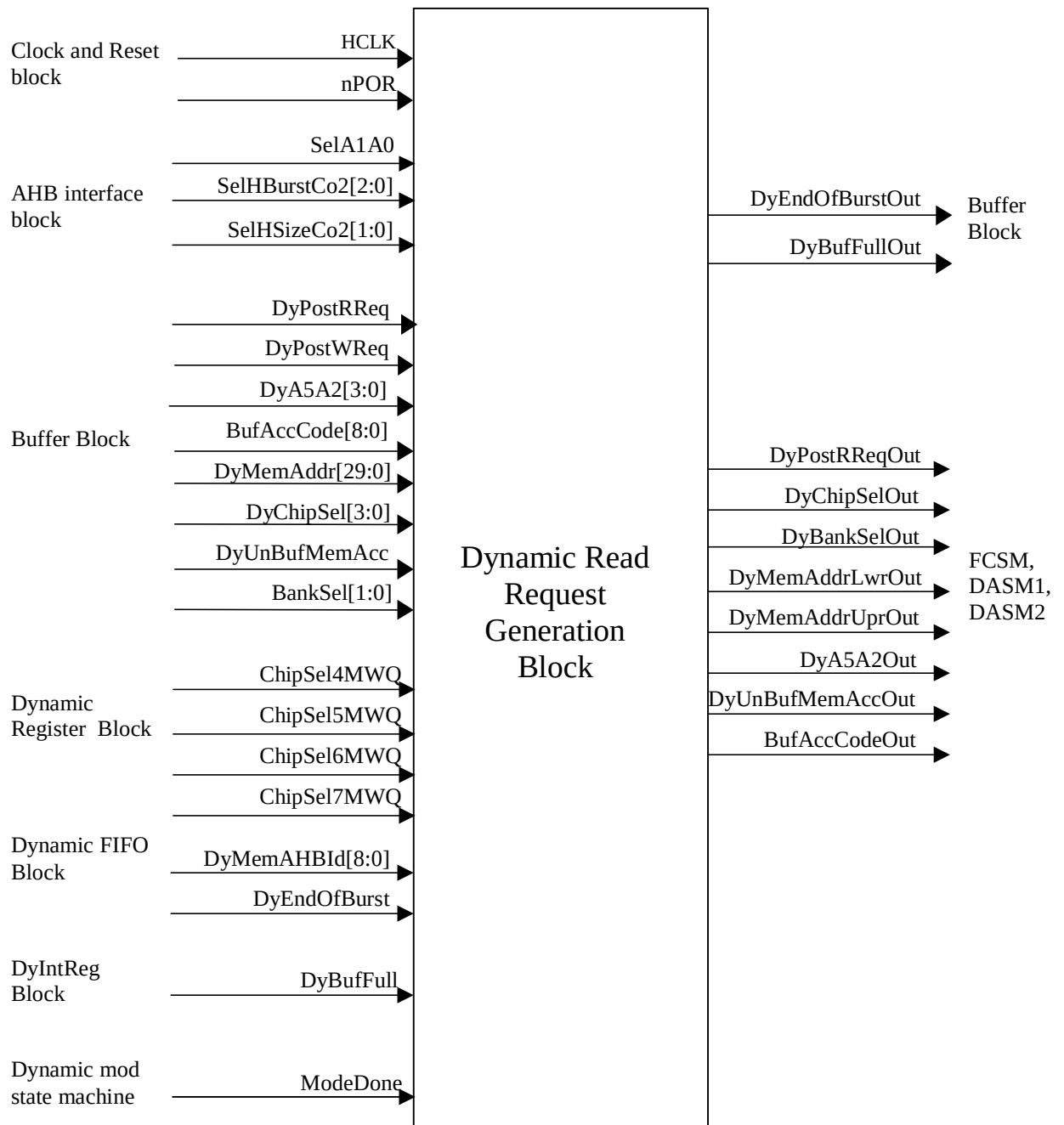


Figure 4.10-3 Block Inputs and Outputs for MpmcDyIntRdGen

The various interface signals related to this block are as described in **Table 4.10-3** and Table 4.10-4:

Signal Name	Source	Description
HCLK	Clock Generator	AHB Clock.
nPOR	Reset Generator	Power on reset.

Signal Name	Source	Description
SelA1A0	AHB memory interface	Address bits A1, A0 of the selected AHB
SelHBurstCo2[2:0]	AHB memory interface	HBURST information of the selected AHB interface
SelHSizeCo2[1:0]	AHB memory interface	HSIZE information of the selected AHB interface
DyPostRRReq	Buffer Unit	Read data request to the dynamic memory controller
DyPostWReq	Buffer Unit	Write data request to the dynamic memory controller
DyA5A2[3:0]	Buffer Unit	AHB Address bits A5 to A2 of the AHB for which the read request has been posed by the buffer
BufAccCode[8:0]	Buffer Unit	Buffer access code from buffer block. This information has to be returned to the buffer along with the read data
DyMemAddr[29:0]	Buffer Unit	Translated memory address. AHB Address has to be translated to a format of - Bank, Ras, Cas - which is used by the dynamic memory controller
DyChipSel[3:0]	Buffer Unit	Dynamic chip select
DyUnBufMemAcc	Buffer Unit	When high, indicates an unbuffered access to dynamic memory.
BankSel[1:0]	Buffer Unit	Dynamic memory bank select
ChipSel4MWQ	Dynamic memory register Block	External memory width of chip select 4
ChipSel5MWQ	Dynamic memory register Block	External memory width of chip select 5
ChipSel6MWQ	Dynamic memory register Block	External memory width of chip select 6
ChipSel7MWQ	Dynamic memory register Block	External memory width of chip select 7
DyMemAHBId[8:0]	Dynamic memory read data FIFO	AHB ID to be returned along with the read data to indicate the source of the dynamic memory access request.
DyEndOfBurst	Dynamic memory read data FIFO	When high, indicates the last read data of the burst
DyBufFull	Dynamic memory internal register block	When high, indicates that the dynamic control logic is busy
ModeDone	Dynamic memory mode state machine	A Low on this signal indicates that mode initialisation is in progress.

Table 4.10-3 Block Inputs of MpmcDyIntRdGen

Signal Name	Destination	Description
DyPostRReqOut	DASM1, DASM2, FCSM, MODSM	Dynamic post read request. If the AHB burst size is greater than a quad word, multiple read requests are generated for a single read request from the buffer.
DyChipSelOut[3:0]	DASM1, DASM2, FCSM, MODSM	Registered version of DyChipSel with DyPostRReq/DyPostWReq.
DyBankSelOut[1:0]	DASM1, DASM2	Registered version of DyBankSel with DyPostRReq/DyPostWReq.
DyMemAddrLwrOut[9:0]	DASM1, DASM2, FCSM	Registered version of DyMemAddr[9:0] with DyPostRReq/DyPostWReq. In case of internally generated requests, new DyMemAddrLwrOut is generated by adding the burst size value to the old DyMemAddrLwrOut.
DyMemAddrUprOut[29:10]	DASM1, DASM2, FCSM	Registered version of DyMemAddr[29:10] with DyPostRReq/DyPostWReq.
DyA5A2Out[3:0]	DASM1, DASM2	Registered version of DyA5A2 with DyPostRReq/DyPostWReq.
DyUnBufMemAccOut	DASM1, DASM2, FCSM	Registered version of DyUnBufMemAcc with DyPostRReq/DyPostWReq.
BufAccCodeOut[8:0]	DASM1, DASM2, FCSM, MODSM	Registered version of BufAccCode with DyPostRReq/DyPostWReq.
DyEndOfBurstOut	Buffer Unit	End of burst for the current AHB burst. This signal is used for AHB re-arbitration.
DyBufFullOut	Buffer Unit	A high on this signal indicates that Dynamic control logic is busy. The buffer unit checks the status of this signal before posting a read or write request to the dynamic control logic.

Table 4.10-4 Block Outputs of MpmcDyIntRdGen

4.10.3 Dynamic memory data-access-state-machine

MPMC Dynamic data-access-state-machine-1 (MpmcDyAxsSM1) is a block that issues normal sdram-array-commands to the dynamic memory chip selects. This block describes the state machine, which interprets the read and write request from the buffer-control-logic and translates them into sdram command sequences.

Figure 4.10-4 describes the block input and outputs of the data access state machine 1 block.

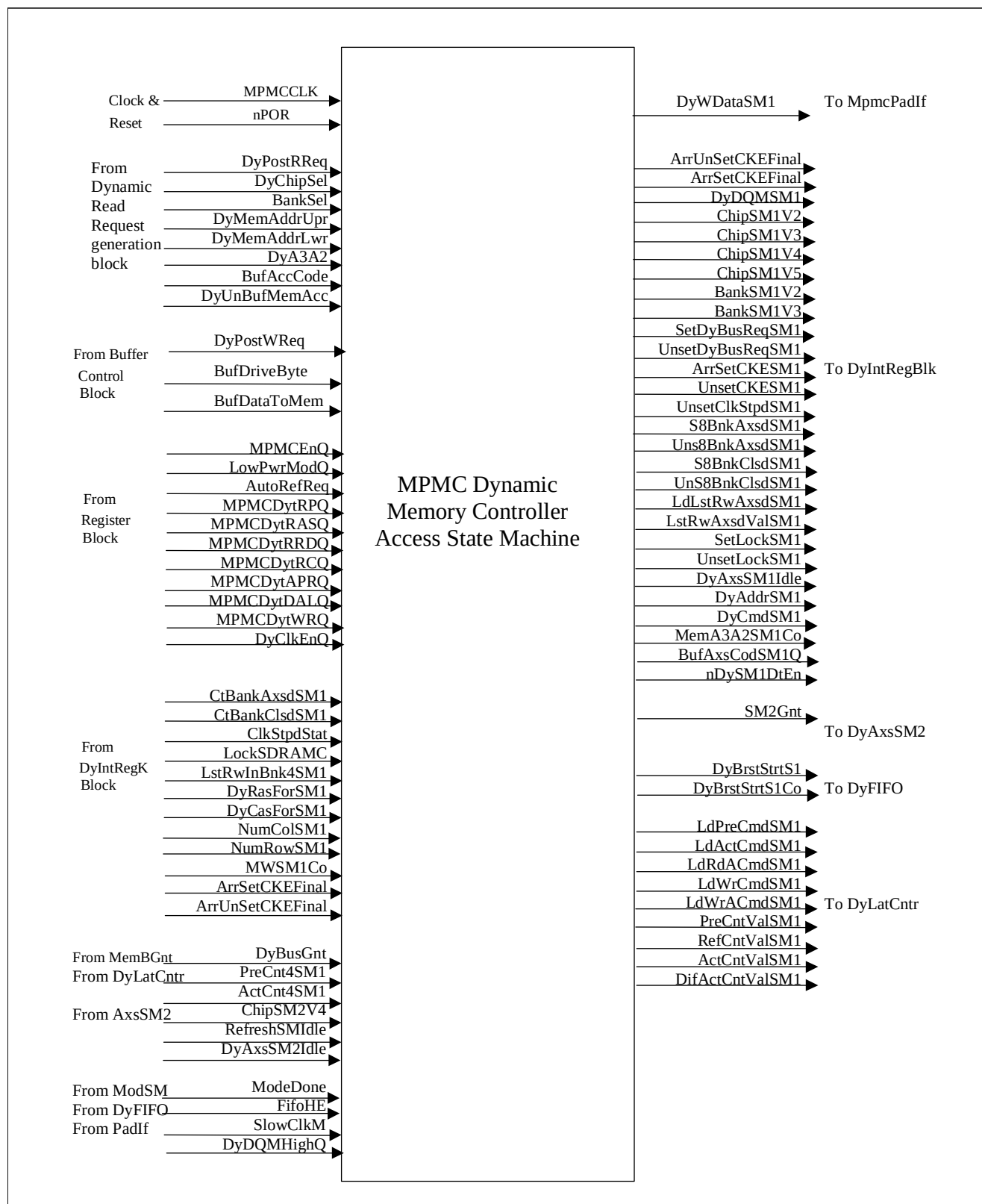


Figure 4.10-4 Block Inputs and Outputs for MpmcDyAxsSM1

The input ports of the data access state machine 1(DASM1) are shown in **Table 4.10-5** and the output ports are shown in **Table 4.10-6** below:

Signal Name	Source	Description
MPMCCLK	Clock Generator	MPMC clock.
nPOR	Reset Generator	Power on reset.
DyPostRReq	Dynamic memory read request generation block	The Buffer unit block raises the read requests to DASM1 by the assertion of this signal.
DyPostWReq	Buffer unit block	The Buffer unit block raises the write requests to DASM1 by the assertion of this signal. The buffer unit block provides the detailed information of the request along with DyPost(R/W)Req signals. The detailed-information is given through DyChipSel, BankSel, DyMemAddr, DyA5A2 and BufAccCode. In case of write requests, the BufDataToMem and BufDriveByte are also driven along with the request assertion.
DyChipSel[1:0]	Dynamic memory read request generation block	Indicates which dynamic chip select is targeted.
BankSel[1:0]	Dynamic memory read request generation block	Indicates the particular bank targeted within the chip-select.
DyMemAddr[29:0]	Dynamic memory read request generation block	Indicates the row and column address to be applied on memories' address lines.
DyA5A2[3:0]	Dynamic memory read request generation block	Indicates the word address for the data transfer that is requested.
BufAccCode[8:0]	Dynamic memory read request generation block	Indicates the requesting AHB or buffer ID.
BufDataToMem[127:0]	Buffer unit block	Write data to memory, which DASM1 uses to drive the data lines of the dynamic memory devices.
BufDriveByte[15:0]	Buffer unit block	Valid byte information, which DASM1 uses to mask the data to the dynamic memory devices.
DyUnBufMemAcc	Dynamic memory read request generation block	Indicates whether the write-request is a buffered request or not. It is to be noted that in case of unbuffered write requests, the BufDataToMem bus is driven one clock after the DyPostWReq pulse.
CtBankAxsdSM1	Dynamic memory internal register block	Indicates whether the bank assigned to DASM1 is already under access by DASM2
CtBankClsdSM1	Dynamic memory internal register block	Indicates whether the bank is precharged/closed.
LstRwInBnk4SM1[12:0]	Dynamic memory internal register block	In case of an open-bank, it indicates the row that was last accessed.
DyRasForSM1[1:0]	Dynamic memory internal register block	Indicates the ACT-to-RD/WR delay for the chip allocated to DASM1.
DyCasForSM1[1:0]	Dynamic memory internal register block	Indicates the RD to data-arrival latency for the chip allocated to DASM1.

Signal Name	Source	Description
NumColSM1[2:0]	Dynamic memory internal register block	Number of columns in the chip allocated to DASM1
NumRowSM1[1:0]	Dynamic memory internal register block	Number of rows in the chip allocated to DASM1
MWSM1	Dynamic memory internal register block	Indicates whether the memory-data-width is 16 or 32 bits for the chip allocated to DASM1.
ClkStpdStat	Dynamic memory internal register block	Indicates whether the clocks supplied to the memories are shut or not
LockSDRAMC	Dynamic memory internal register block	DASM1 can issue commands only if DASM2 is not active on the command lines. This mutual exclusion is achieved with LockSDRAMC, which when high indicates that the targeted bank is currently under DASM2's control
MPMCDytRPQ[3:0]	AHB register interface block	Dynamic memory precharge command period (t_{RP}).
MPMCDytRASQ[3:0]	AHB register interface block	Dynamic memory precharge period (t_{RAS}).
MPMCDytRRDQ[3:0]	AHB register interface block	Dynamic memory Active Bank A to Active Bank B time (t_{RRD}).
MPMCDytRCQ[4:0]	AHB register interface block	Dynamic memory AutoRefresh and Active Command period (t_{RC}).
MPMCDytAPRQ[3:0]	AHB register interface block	Dynamic memory last data-out to Active command in case of autoprecharge read (to same bank) time (t_{APR}).
MPMCDytDALQ[3:0]	AHB register interface block	Dynamic memory Data-In to Active command time (t_{DAL} or t_{APW}).
MPMCDytWRQ[3:0]	AHB register interface block	Dynamic memory write recovery time or the Data-In to precharge time (t_{WR} or t_{DPL} or t_{RWL} or t_{RDL}).
MpmcEnQ	AHB register interface block	MPMC enable. When HIGH, the DASM1 initiates accesses to the dynamic memory device. When LOW, DASM1 does not initiate any accesses to the dynamic memory device.
LowPwrModQ	AHB register interface block	MPMC low power mode enable. When LOW, the DASM1 initiates accesses to the dynamic memory device. When HIGH, DASM1 does not initiate any accesses to the dynamic memory device.
AutoRefReq	AHB register interface block	A pulse generated by a free running counter loaded with refresh-timer.
DyClkEnQ	AHB register interface block	Indicates whether the CKE line is to be turned off by DASM1 when there are no access pending to the chip select.
DyBusGnt	Memory Bus Grant logic block	A HIGH on this line indicates the memory bus grant to the dynamic memory controller block.
PreCnt4SM1[3:0]	Dynamic memory latency control block	Precharge count for DASM1. Indicates that PRE command can be given to the targeted bank.

Signal Name	Source	Description
ActCnt4SM1[4:0]	Dynamic memory latency control block	Active count for DASM1. Indicates that ACT command can be given to the targeted bank.
ChipSM2[3:0]	Dynamic data access state machine 2	Indicates the chip-select on which DASM2 is working..
DyAxsSM2Idle	Dynamic data access state machine 2	Indicates whether DASM2 is in idle State. ChipSM2 and DyAxsSM2Idle are together used for ascertaining whether the CKE can be pulled low by DASM1
ModeDone	Dynamic mode state machine	Indicates that the mode programming sequence is over and DASM1 can interpret the subsequent read-accesses as requests for normal sdram memory array accesses.
FifoHE	Dynamic memory read data FIFO	FIFO half empty flag used by DASM1 to stall read accesses so that the FIFO is not 'overwhelmed' by read accesses following each other in pipeline.
FifoPtrEqual	Dynamic memory read data FIFO	FIFO read and write pointers are equal and Fill level is less than four
SlowClkM	Pad interface block	MPMC has got two clock domains (HCLK and MPMCCLK) with one being half of the other. For proper synchronization of data crossing domains, it is necessary that the faster clock samples the signals driven at slower clock only once and also the faster clock should not drive information at two consecutive clock edges. SlowClkM is used for this purpose.
DyDQMHighQ	Pad interface block	Indicates whether the DQM was high in the previous clock. This helps in saving one clock in case of unity clock latency devices.
ArrSetCKEFinal	Dynamic memory internal register block	OR-ed version of set clock enable from all the state machine
ArrUnSetCKEFinal	Dynamic memory internal register block	OR-ed version of unset clock enable from all the state machine

Table 4.10-5 Block inputs of MpmcDyAxsSM1

Signal Name	Destination	Description
DyWDataSM1[31:0]	Pad interface block	Write-data information given to the memory-devices during write operations.
DyDQMSM1[3:0]	Pad interface block	DQM information given to the memory-devices during write operations.
ChipSM1V[2..5][3:0]	Dynamic memory internal register block	Targeted chip-select information which DASM1 registers when it receives the read or write request. Multiple versions of this signal have been generated to have better synthesis timings.

Signal Name	Destination	Description
BankSM1V[2..3][1:0]	Dynamic memory internal register block	Targeted chip-select information which DASM1 registers when it receives the read or write request. Multiple versions of this signal have been generated to have better synthesis timings.
ArrSetCKESM1[3:0]	Dynamic memory internal register block	4-bit-vector (with each bit corresponding to each dynamic-memory-chip select), whose bits are set when the DASM1 starts-off an access corresponding to a particular chip-select.
UnsetCKESM1	Dynamic memory internal register block	Single bit signal which is pulled high when DASM1 finishes access to a targeted chip and 'sees' that DASM2 is also not going to access the same-chip.
UnsetClkStpdSM1	Dynamic memory internal register block	Indicates the memory clocks to start off. Goes high when DASM1 is about to start an access.
S8BnkAxsdSM1	Dynamic memory internal register block	Set bank accessed by DASM1. Asserted when access to a bank (there can be 4 banks per chip select and thus there can be 16 'banks' at maximum) is started.
Uns8BnkAxsdSM1	Dynamic memory internal register block	Asserted when DASM1 moves to idle State after finishing the access.
S8BnkClsdSM1	Dynamic memory internal register block	Set Bank Closed by DASM. Asserted when a precharge-command is issued by DASM1.
UnS8BnkClsdSM1	Dynamic memory internal register block	Unset Bank closed by DASM1
LdLstRwAxsdSM1	Dynamic memory internal register block	Whenever DASM1 issues an active command it records the row-accessed-information. This is required to know which particular row is currently open in a bank. LdLstRwAxsdSM1 signal is a strobe to load this information.
LstRwAxsdValSM1[12:0]	Dynamic memory internal register block	Indicates the Row-address on the address lines, whenever an ACT command is issued by DASM1.
SetLockSM1	Dynamic memory internal register block	Asserted to set the mutually shared lock variable between DASM1 and DASM2 which is used to prevent mutual exclusion between DASM1 and DASM2 transitions.
UnsetLockSM1	Dynamic memory internal register block	Asserted to clear the mutually shared lock variable between DASM1 and DASM2 which is used to prevent mutual exclusion between DASM1 and DASM2 transitions.
DyAddrSM1[14:0]	Dynamic memory internal register block	Row / Column Address driven when DASM1 transitions between 'command-issuing' states
DyCmdSM1[3:0]	Dynamic memory internal register block	Command code asserted when DASM1 transitions between 'command-issuing' states

Signal Name	Destination	Description
MemA5A2SM1Co[1:0]	Dynamic memory internal register block	This signal is generated by incrementing the DyA5A2 value posted with the read request (according to memory-data-bus width) and is transmitted out with the read data only during read data accesses.
BufAxsCodSM1Q	Dynamic memory internal register block	This signal is the AHB or Buffer requestor code, which is stored by DASM1 during start of the data access. BufAxsCodSM1Q is used as a tag to be returned along with read data.
NDySM1DtEn	Dynamic memory internal register block	This signal is asserted (active low) when DASM1 attempts write accesses.
SM2Gnt	Dynamic memory data access state machine 2	This signal is de-asserted whenever DASM1 is attempting to issue a command on the controller command lines. This prevents both the DASMs from issuing command at the same time.
DyBrstStrtS1	Dynamic memory read data FIFO	Asserted during the dynamic memory read accesses. This signal is timed so that it is asserted when the first-data from the memory devices are to be written into the receive-FIFO and de-asserted after the last data of the read burst is received.
DyBrstStrtS1Co	Dynamic memory read data FIFO	D-input of DyBrstStrtS1
LdPreCmdSM1	Dynamic memory latency control block	Latency counter loading strobe generated when PRE command is issued.
LdActCmdSM1	Dynamic memory latency control block	Latency counter loading strobe generated when ACT command is issued.
LdRdACmdSM1	Dynamic memory latency control block	Latency counter loading strobe generated when RD command is issued.
LdWrCmdSM1	Dynamic memory latency control block	Latency counter loading strobe generated when WR command is issued.
LdWrACmdSM1	Dynamic memory latency control block	Latency counter loading strobe generated when WRA command is issued.
PreCntValSM1[3:0]	Dynamic memory latency control block	Precharge count value, which is driven to tRAS when an ACT command is issued, is driven to tWR when WR command is issued and is driven to tRP when a PRE command is issued.
ActCntValSM1[4:0]	Dynamic memory latency control block	Act count value, which is driven to tRP when a PRE command is issued, is driven to tRC when another ACT command is issued to the same bank, is driven to (tAPR + cas-latency + burstlength) when RDA is issued and is driven to (tDAL + burstlength) in case of WRA issual.
DifActCntValSM1[4:0]	Dynamic memory latency control block	This signal is driven to tRRD when ACT is applied to other banks.

Signal Name	Destination	Description
RefCntValSM1[4:0]	Dynamic memory latency control block	This signal is driven similar to ActCntValSM1 except that tRC is not loaded during active command.

Table 4.10-6 Block outputs of MpmcDyAxsSM1

4.10.4 Dynamic memory mode State Machine

The Dynamic memory mode State Machine (MpmcDyModSM) executes the mode initialisation sequence for SDRAMs and SyncFlash devices. **Figure 4.10-5** shows the block inputs and outputs of this block.

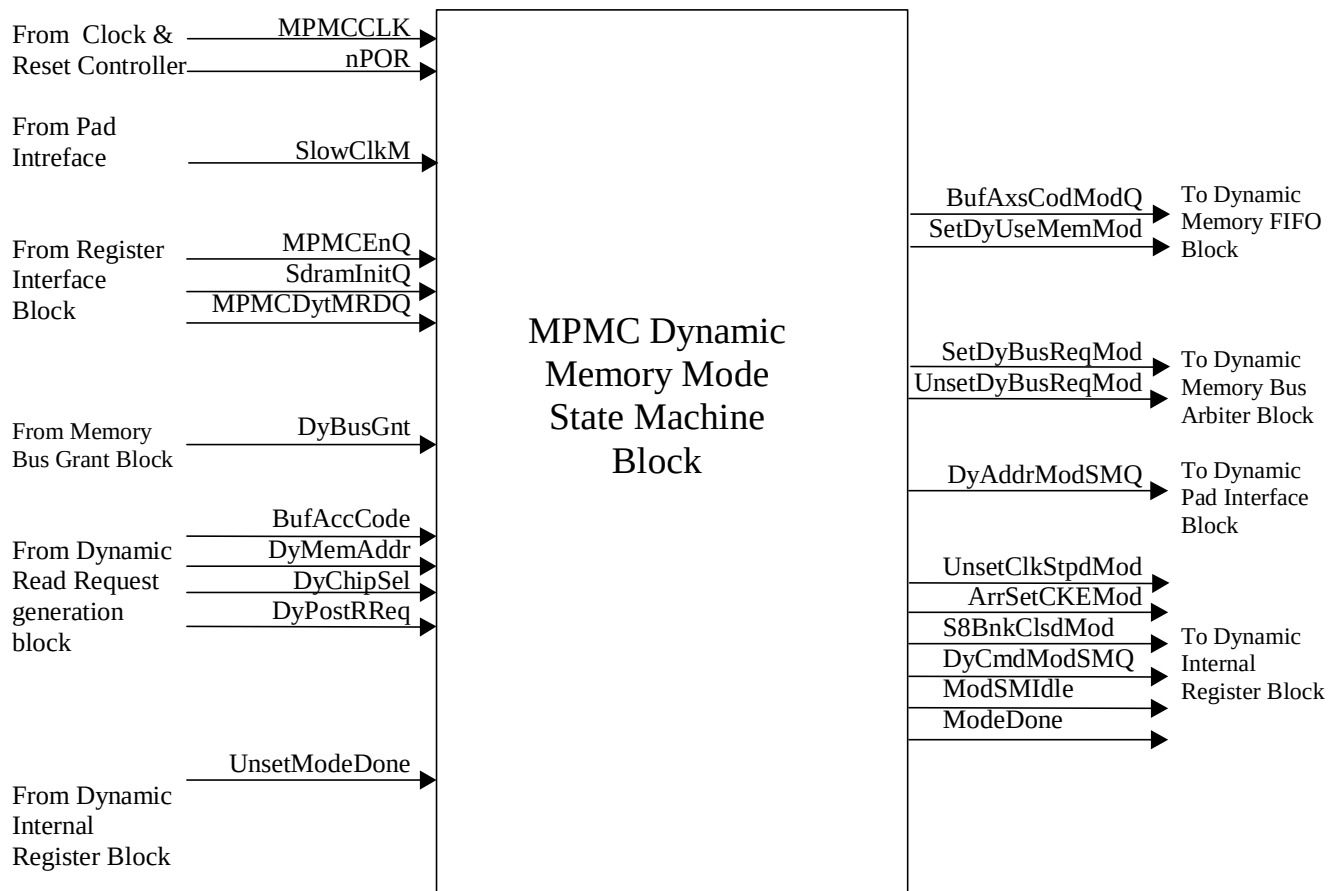
**Figure 4.10-5 Block Inputs and Outputs for MpmcDyModSM**

Table 4.10-7 and **Table 4.10-8** describe the input and output ports of MpmcDyModSM.

Signal Name	Source	Description
MPMCCLK	Clock Generator	MPMC clock.

Signal Name	Source	Description
nPOR	Reset Generator	Power on reset.
SlowClkM	Pad interface block	Slow clock of MPMCCLK and HCLK.
MPMCEnQ	AHB register interface	Bit 0 of MPMCControl register, the port being a global MPMC disable.
SdramInitQ[1:0]	AHB register interface	SDRAM initialisation control bits from MPMCDyControl register.
MPMCDytMRDQ[3:0]	AHB register interface	Dynamic memory load mode register to active command time tMRD.
DyPostRRReq	Dynamic memory read request generation block	Posted read request to dynamic memory controller
DyBusGnt	Memory Bus Grant logic block	Memory bus grant to synchronous memory controller.
DyMemAddr[14:0]	Dynamic memory read request generation block	Memory address from buffer.
BufAccCode[8:0]	Dynamic memory read request generation block	Access requestor AHB or buffer code for mode state machine.
DyChipSel[3:0]	Dynamic memory read request generation block	Chip select for current access.
UnsetModeDone	Dynamic memory internal register block	Resets the ModeDone bit due to deep-sleep-entry.
BufAxsCodModQ[8:0]	Dynamic memory internal register block	Access requestor code returned from mode state machine.

Table 4.10-7 Block inputs of MpmcDyModSM

Signal Name	Destination	Description
SetDyUseMemMod	Dynamic memory read data FIFO block	Indicates that the dynamic memory controller is ready to transfer read-data to the buffer for mode register accesses.
SetDyBusReqMod	Dynamic memory internal register block	Signal to assert the bus request to the memory bus grant logic.
UnsetDyBusReqMod	Dynamic memory internal register block	Signal to clear the bus request to the memory bus grant logic.
DyAddrModSMQ[14:0]	Pad interface block	Address issued by Mode state machine.
UnsetClkStpdMod	Dynamic memory internal register block	Indicates to enable the clock for mode programming.
ArrSetCKEMod[3:0]	Dynamic memory internal register block	Pulls the CKE of all the Dynamic Memory Controller chip selects high.
S8BnkClsdMod	Dynamic memory internal register block	Indicates that Mode programming is completed.

Signal Name	Destination	Description
DyCmdModSMQ[6:0]	Dynamic memory internal register block	Command issued by Mode State Machine to the memory
ModSMIdle	Dynamic memory internal register block	When high, indicates that mode State-machine is idle.
ModeDone	Dynamic memory internal register block	Low on this signal indicates that the dynamic memory State Machines is performing mode programming

Table 4.10-8 Block outputs from MpmcDyModSM

4.10.5 Dynamic memory refresh State Machine

The Dynamic memory refresh State Machine block (MpmcDyRefSM) performs Auto Refresh when AutoRefReq signal is asserted and Self Refresh when SRefReq is asserted. **Figure 4.10-6** shows the block inputs and outputs of this block.

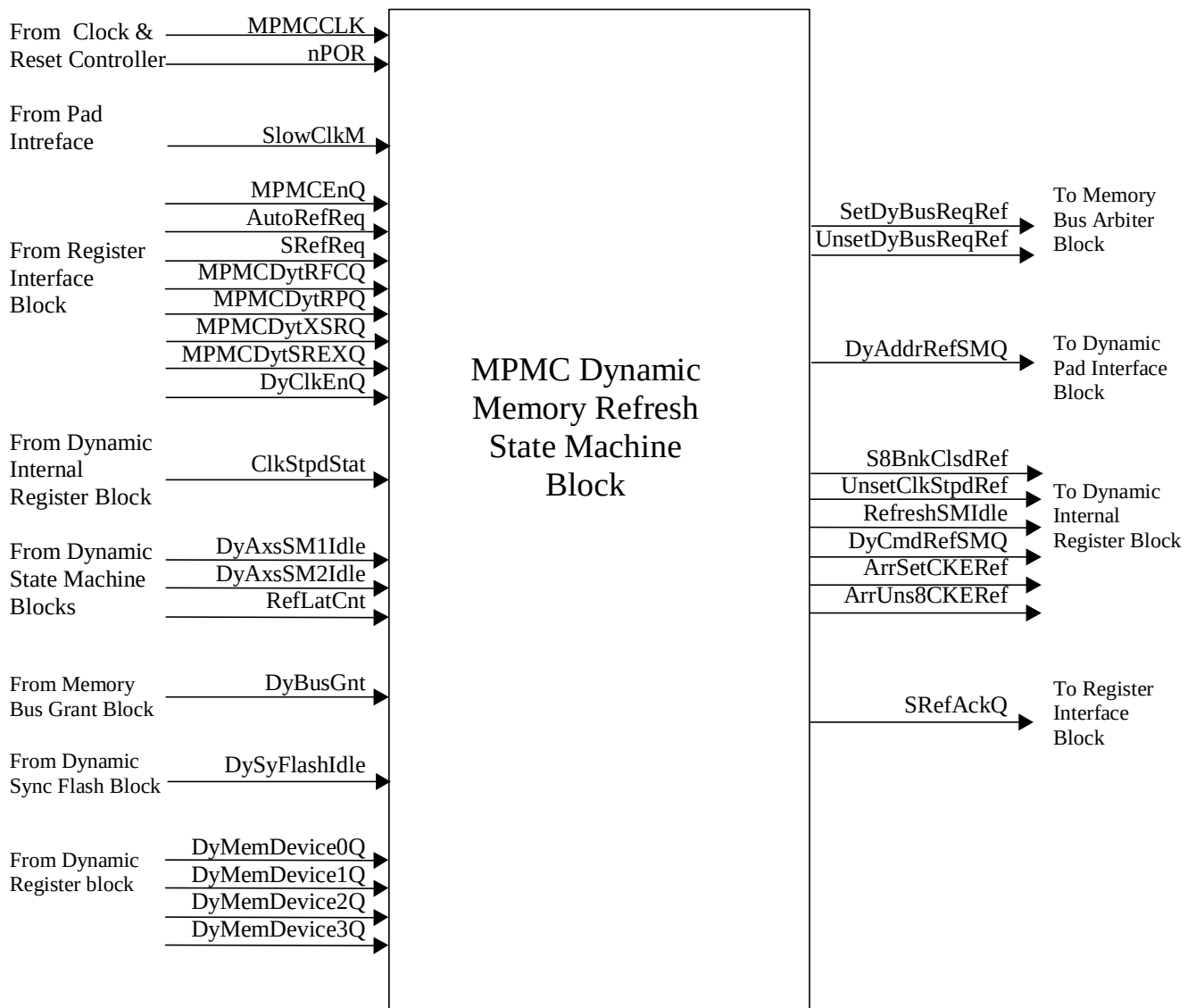


Figure 4.10-6 Block Inputs and Outputs for MpmcDyRefSM

The input and output ports of MpmcDyRefSM are described in **Table 4.10-9** and **Table 4.10-10**

Signal Name	Source	Description
MPMCCLK	Clock Generator	MPMC Clock.
nPOR	Reset Generator	Power on reset
SlowClkM	Pad interface block	Slow clock of MPMCCLK and HCLK
MPMCEnQ	AHB register interface	Bit 0 of MPMCControl register, the port being a global MPMC disable.
AutoRefReq	AHB register interface	Auto refresh request. Signal goes active for a single cycle.
SRefReq	AHB register interface	Self refresh request

Signal Name	Source	Description
MPMCDytRFCQ[4:0]	AHB register interface	Dynamic memory auto refresh to active command period tRFC.
MPMCDytRPQ[3:0]	AHB register interface	Dynamic memory precharge command period tRP
MPMCDytXSRQ[4:0]	AHB register interface	Dynamic memory self refresh exit to active command time tXSR.
MPMCDytSREXQ[3:0]	AHB register interface	Dynamic memory self refresh exit time tSREX.
DyClkEnQ	AHB register interface	Dynamic memory clock enable value when no commands are being issued.
ClkStpdStat	Dynamic memory internal register block	SDRAM clock gating status.
DyAxsSM1Idle	Dynamic memory state machine 1 block.	State machine 1 idle.
DyAxsSM2Idle	Dynamic memory state machine 2 block.	State machine 2 idle.
RefLatCnt[4:0]	Dynamic memory latency control block	Refresh latency count.
DyBusGnt	Memory Bus Grant logic block	Dynamic memory controller bus grant.
DySyFlashIdle	Dynamic memory SyncFlash state machine	SyncFlash state machine idle.
DyMemDevice[0..3]Q[1:0]	Dynamic memory register block	Chip select 4 to 7 memory type. DyMemDevice0Q is for chip select 4, DyMemDevice1Q is for chip select 5 and so on.

Table 4.10-9 Block inputs of MpmcRefSM

Signal Name	Destination	Description
SetDyBusReqRef	Dynamic memory internal register block	Signal to set the bus request to the memory bus granting logic.
UnsetDyBusReqRef.	Dynamic memory internal register block	Signal to clear the bus request to the memory bus granting logic
DyAddrRefSMQ	Pad interface block	Address issued by Refresh State Machine to Memory.
S8BnkClsdRef	Dynamic memory internal register block	Indicates that Refresh is completed.
UnsetClkStpdRef	Dynamic memory internal register block	Indicates to start the clock.
RefreshSMIdle	Dynamic memory internal register block	Indicates that Refresh State Machine is Idle.

Signal Name	Destination	Description
DyCmdRefSMQ[6:0]	Dynamic memory internal register block	Command issued by Refresh State Machine to the Memory.
ArrSetCKERef[3:0]	Dynamic memory internal register block	Indicates to set CKE to the Memory.
ArrUns8CKERef[3:0]	Dynamic memory internal register block	Indicates to unset the CKE to Memory.
SRefAckQ	Dynamic memory internal register block	Self refresh acknowledge. Indicates that self-refresh has been performed.

Table 4.10-10 Block outputs of MpmcDyRefSM

4.10.6 Dynamic memory SyncFlash State Machine

The dynamic memory SyncFlash State Machine block (MpmcDySyFlash) describes the flash-command-set State Machine, which is used to issue the SyncFlash commands. **Figure 4.10-7** shows the block inputs and outputs of this block.

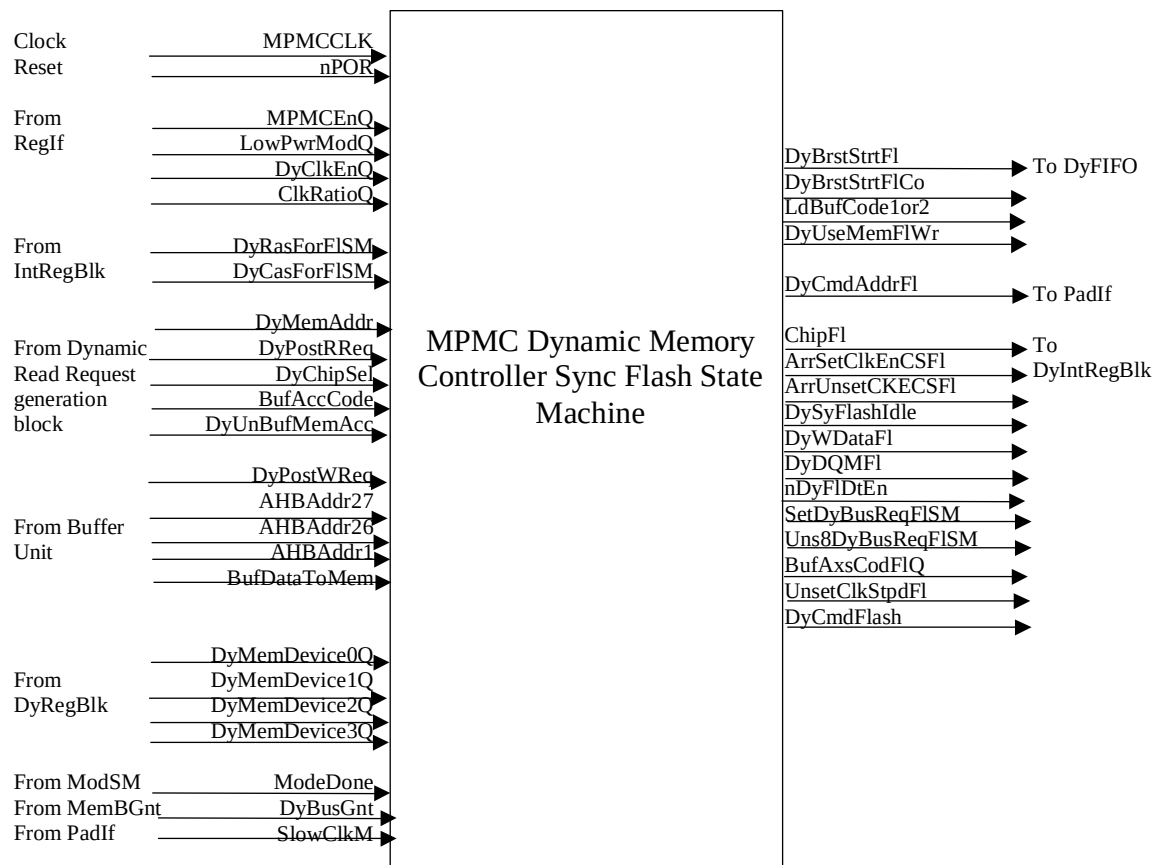


Figure 4.10-7 Block Inputs and Outputs for MpmcDySyFlash

The input and output ports of MpmcDySyFlash are described in **Table 4.10-11** and **Table 4.10-12**.

Signal Name	Source	Description
MPMCCLK	Clock Generator	MPMC clock.
nPOR	Reset Generator	Power on reset.
MPMCEnQ	AHB register interface	Bit 0 of MPMCControl register, the bit being a global MPMC enable.
LowPwrModQ	AHB register interface	Bit 2 of MPMCControl register, reduces memory-controller power consumption.
DyClkEnQ	AHB register interface	Clock-enable control
ClkRatioQ	AHB register interface	HCLK to MCCLK ratio
DyRasForFISM[1:0]	Dynamic memory internal register block	RAS to CAS delay for the chip accessed by Flash-State-machine.
DyCasForFISM[1:0]	Dynamic memory internal register block	CAS delay for the chip accessed by Flash-State-machine.
DyMemAddr[29:1]	Dynamic memory read request generation block	Multiplexed RAS and CAS address.
DyPostWReq	Buffer unit	Posted write request.
DyPostRReq	Dynamic memory read request generation block	Posted read request.
DyChipSel[3:0]	Dynamic memory read request generation block	Indicates one out of the four SDRAM chip selects.
AHBAddr27	Buffer unit	27th bit of HADDR. For certain operations this bit is used to select which SyncFlash command to issue.
AHBAddr26	Buffer unit	26th bit of HADDR. For certain operations this bit is used to select a SyncFlash command
AHBAddr1	Buffer unit	1st bit of HADDR. For certain operations this bit is used to select a SyncFlash command.
DyUnBufMemAcc	Dynamic memory read request generation block	Indicates current posting is a direct unbuffered access from AHB.
BufDataToMem[15:0]	Buffer unit	Data to memory controller to be written to physical memory.
BufAccCode[8:0]	Dynamic memory read request generation block	Access requestor AHB or buffer Id information provided to memory controller with the request
DyMemDevice[3..0]Q[1:0]	Dynamic memory register block	Determines the memory type of chip-selects 7 to 4. DyMemDevice0Q determines the memory type of chip-select 4. DyMemDevice1Q determines the memory type of chip-select 5 and so on.
ModeDone	Dynamic memory mode state machine	A low on this signal indicates that the dynamic memory State Machines is performing mode programming.

Signal Name	Source	Description
DyBusGnt	Memory bus granting logic block	Bus grant to dynamic memory controller.
SlowClkM	Pad interface	Slow clock indicator to MPMCCLK side.

Table 4.10-11 Block inputs of MpmcDySyFlash

Signal Name	Destination	Description
DyBrstStrtFI	Dynamic memory read data FIFO	Indicates the FIFO to start clocking in read-data from external memory data bus for SyncFlash State Machine.
DyBrstStrtFICo	Dynamic memory read data FIFO	D-input of DyBrstStrtFI.
LdBufCode1or2	Dynamic memory read data FIFO	Indicates whether to load requestor code for DASM1 or DASM2.
DyUseMemFIWr	Dynamic memory read data FIFO	Indicates that the unbuffered write for programming flash has completed on AHB.
DyCmdAddrFI[14:0]	Pad interface block	Memory address given out with flash command issue.
ChipFI[3:0]	Dynamic memory internal register block	The chip select which the flash State Machine is currently acting on.
ArrSetClkEnCSFI[3:0]	Dynamic memory internal register block	Sets the CKE pin of the chip-select that is being accessed.
ArrUnsetCKECSFI[3:0]	Dynamic memory internal register block	Unsets the CKE pin of the chip-select that is being accessed.
DySyFlashIdle	Dynamic memory internal register block	Indicates that Flash command state machine is Idle.
DyWDataFI[15:0]	Dynamic memory internal register block	Write data from flash-command-execution-state-machine(FCSM).
DyDQMFI[3:0]	Dynamic memory internal register block	Data mask information from FCSM.
nDyFIDtEn	Dynamic memory internal register block	Asserts/de-asserts the data-enable lines during write/read.
SetDyBusReqFISM	Dynamic memory internal register block	Sets the external-bus-grant request.
Uns8DyBusReqFISM	Dynamic memory internal register block	Unsets the external-bus-grant request.
BufAxsCodFIQ[8:0]	Dynamic memory internal register block	BufAccCode given along with the PostR/Wreq, which is recorded into BufAxsCodFI.
UnsetClkStpdFI	Dynamic memory internal register block	Switches on the SDRAM clocks.

Signal Name	Destination	Description
DyCmdFlash[3:0]	Dynamic memory internal register block	Flash-command issued by the State-machine

Table 4.10-12 Block outputs of MpmcDySyFlash

4.10.7 Dynamic memory read data FIFO block

The dynamic memory read data FIFO (MpmcDyFIFO) is used to store the read data from the dynamic memories. The dynamic memory read data FIFO block has the following sub-blocks:

- The FIFO register file (MpmcDyRegFile)
- The FIFO control logic (MpmcDyFCntl)
- The MPMCFBCLKIN[3..0] to HCLK domain conversion file (MpmcDyFBtoH)

Figure 4.10-8 shows the block diagram of the dynamic FIFO block along with the inputs and outputs of this block

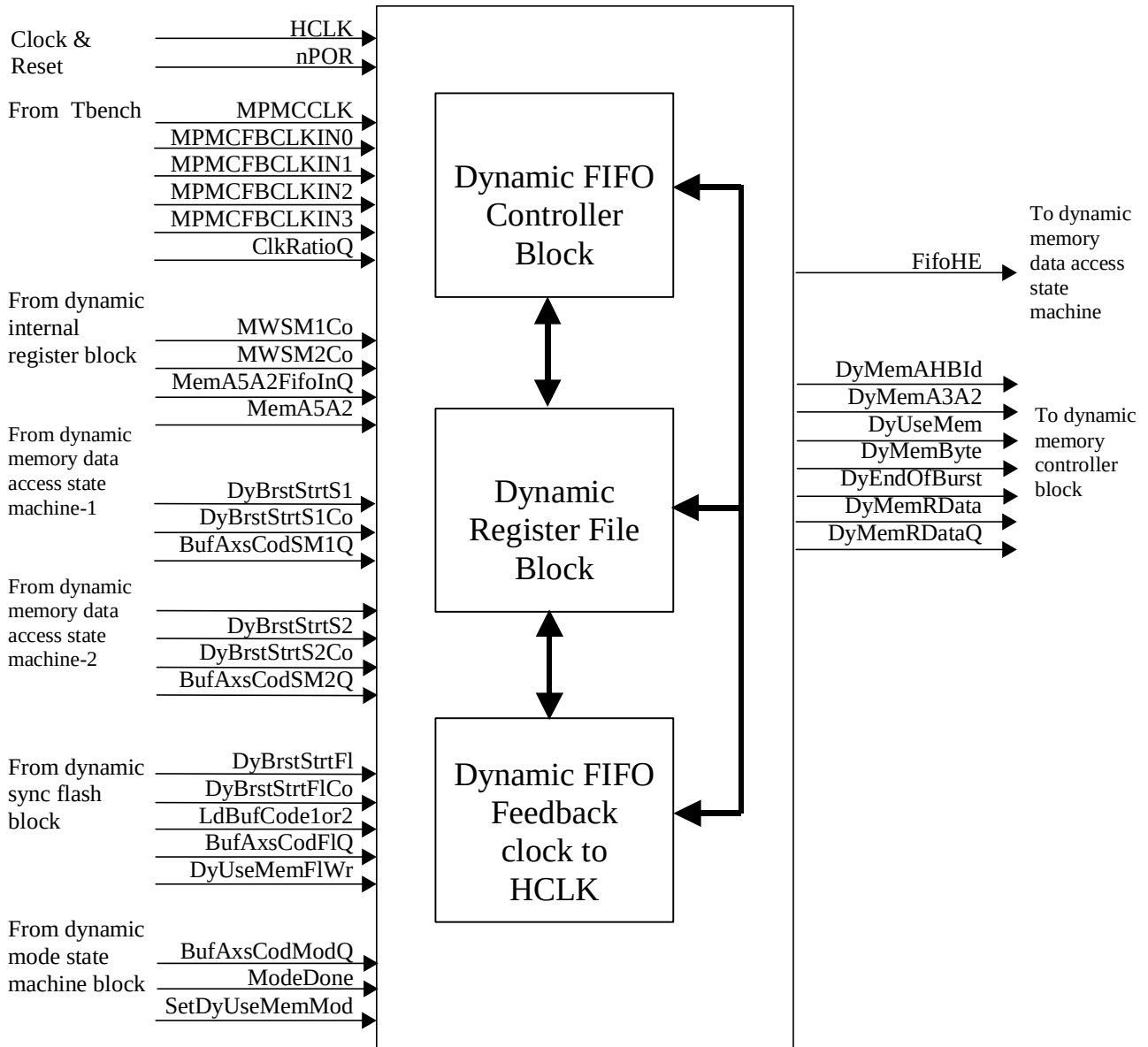


Figure 4.10-8 Block Inputs and Outputs for MpmcDyFIFO

Table 4.10-13 and Table 4.10-14 describe the input and output ports of MpmcDyFIFO.

Signal Name	Source	Description
HCLK	Clock Generator	AHB clock.
MPMCCLK	Clock Generator	MPMC clock.
MPMCFBCLKIN[3..0]	Clock Generator	Separate MPMC feedback clock to register memory read data for bytes 0, 1, 2, and 3
NPOR	Reset Generator	Power on reset.
ClkRatioQ	AHB register interface	HCLK to MPMCCLK ratio.

Signal Name	Source	Description
MWSM1Co	Dynamic memory internal register block	Memory width of device being accessed by DASM1.
MWSM2Co	Dynamic memory internal register block	Memory width of device being accessed by DASM2.
MemA5A2FifoInQ[3:0]	Dynamic memory internal register block	Address bits 5 to 2 to be clocked into the FIFO along with respective data.
DyBrstStrtS1	Dynamic memory data access state machine 1	Indicates the FIFO to start clocking in read-data from external memory data bus for DASM1.
DyBrstStrtS1Co	Dynamic memory data access state machine 1	D-input of DyBrstStrtS1.
DyBrstStrtS2	Dynamic memory data access state machine 2	Indicates the FIFO to start clocking in read-data from external memory data bus for DASM2.
DyBrstStrtS2Co	Dynamic memory data access state machine 2	D-input of DyBrstStrtS2.
DyBrstStrtFI	Dynamic memory SyncFlash state machine 2	Indicates the FIFO to start clocking in read-data from external memory data bus for SyncFlash State Machine.
DyBrstStrtFICo	Dynamic memory SyncFlash state machine 2	D-input of DyBrstStrtFI.
LdBufCode1or2	Dynamic memory internal register block	Indicates whether to load requestor code for DASM1 or DASM2. DyFWrDataCo is FIFO write data.
BufAxsCodSM1Q[8:0]	Dynamic memory data access state machine 1	Access requestor code for SM1.
BufAxsCodSM2Q[8:0]	Dynamic memory data access state machine 2	Access requestor code for SM2.
BufAxsCodFIQ[8:0]	Dynamic memory SyncFlash State machine	Access requestor code for SyncFlash State Machine.
BufAxsCodModQ[8:0]	Dynamic memory Mode State machine	Access requestor code for mode State Machine.
ModeDone	Dynamic memory Mode State machine	Mode state machine status. A low on this signal indicates that the dynamic memory controller is performing mode programming
DyFWrDataCo[31:0]	Pads	Memory read data, MPMCDATAIN signal connected as the write data bus to the FIFO.
SetDyUseMemMod	Dynamic memory Mode State machine	Indicates that the dynamic memory controller is ready to transfer read-data to the buffer for mode register accesses.
DyUseMemFIWr	Dynamic memory SyncFlash State machine	Indicates that the unbuffered write for programming flash has completed on AHB.
FifoHE	Dynamic memory SyncFlash State Machine	FIFO half empty flag

Signal Name	Source	Description
FifoPtrEqual	DASM1, DASM2	FIFO read and write pointers are equal and Fill-level is less than four (half-full).

Table 4.10-13 Block inputs of MpmcDyFIFO

Signal Name	Destination	Description
DyMemAHBId[8:0]	Memory controller interface block	Requestor code returned by the dynamic memory controller with the DyUseMem assertion
DyMemA3A2[1:0]	Memory controller interface block	Bits 3 and 2 of memory address
DyMemRData[31:0]	Memory controller interface block	FIFO read data. This is the read data returned on reads from dynamic memory
DyMemRDataQ[31:0]	Memory controller interface block	Registered version of DyMemRData
DyMemByte[3:0]	Memory controller interface block	Memory byte lanes driven by dynamic controller
DyUseMem	Buffer unit, AHB memory Interface	FIFO not empty flag to be used to indicate the availability of read data to the buffer or AHB
DyEndOfBurst	Dynamic memory read request generation block	Indicates that the data returned is the last of particular SDRAM burst

Table 4.10-14 Block outputs of MpmcDyFIFO

4.10.8 Dynamic memory internal register block

The dynamic memory internal register block (MpmcDyIntRegBlk) mainly describes the different shared flags and memory-bank-status of 16 different banks. **Figure 4.10-9** describes the block input and output ports of this block.

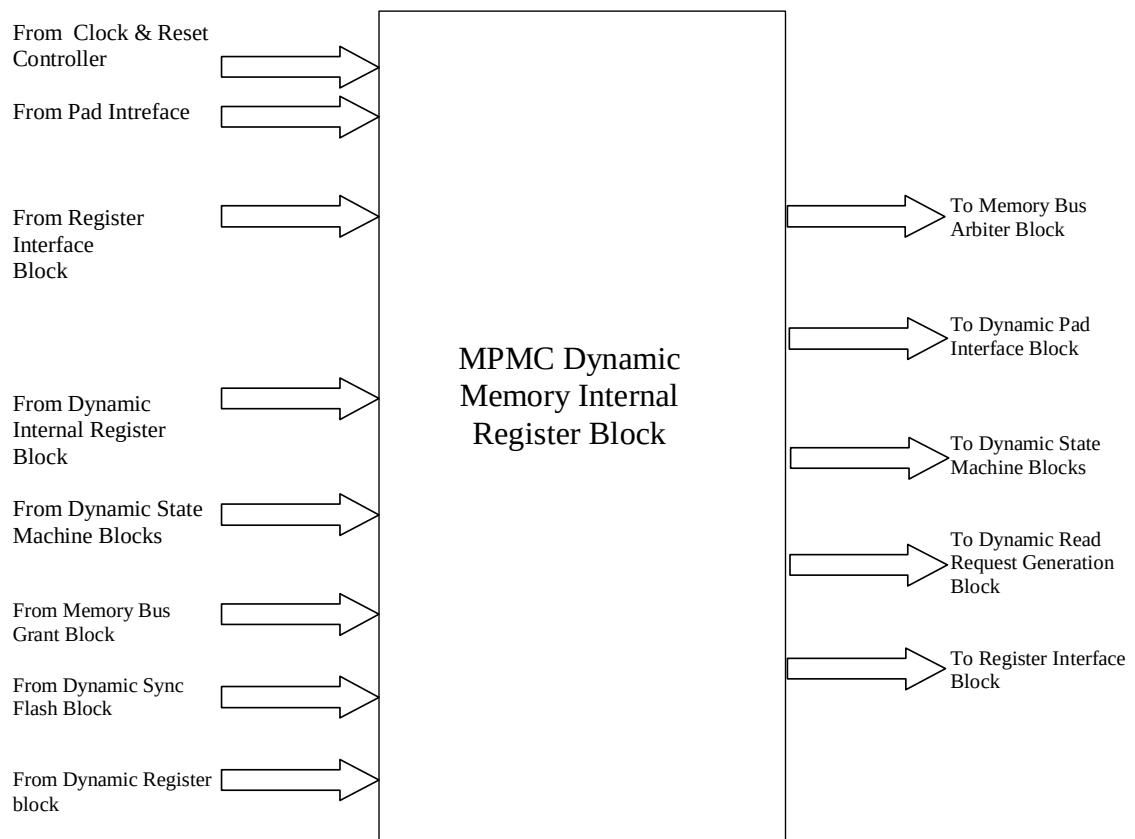


Figure 4.10-9 Block Inputs and Outputs for MpmcDyIntRegBlk

The input and output ports of MpmcDyIntRegBlk are described in **Table 4.10-15** and **Table 4.10-16**.

Signal Name	Source	Description
MPMCCLK	Clock Generator	MPMC clock.
HCLK	Clock Generator	AHB clock
nPOR	Reset Generator	Power on reset
AutoRefReq	AHB register interface	Auto refresh request. This signal goes active for a single cycle.
SRefReq	AHB register interface	Self-refresh request.
DyClkCtrQ	AHB register interface	Dynamic memory clock control.
LowPwrDSModQ	AHB register interface	Low power deep sleep mode.
ClkRatioQ	AHB register interface	HCLK to MCCLK ratio.
DyCmdSM1	Dynamic memory data access state machine 1	Command to be issued from SM1.

Signal Name	Source	Description
ChipSM1V[2..3]	Dynamic memory data access state machine 1	Chip-select on which State Machine 1 is acting. Multiple versions of this signal are used for better synthesis results.
BankSM1V2	Dynamic memory data access state machine 1	Bank-select on which State Machine 1 is acting.
SetDyBusReqSM1	Dynamic memory data access state machine 1	Raises the request to bus-grant-logic.
UnsetDyBusReqSM1	Dynamic memory data access state machine 1	De-asserts the request to bus-grant-logic.
ArrSetCKESM1	Dynamic memory data access state machine 1	Sets the CKE pin of the chip-select that is being accessed.
UnsetCKESM1	Dynamic memory data access state machine 1	De-asserts the CKE pin of the chip-select that is being accessed.
UnsetClkStpdSM1	Dynamic memory data access state machine 1	Switches on the SDRAM clocks.
S8BnkAxsdSM1	Dynamic memory data access state machine 1	Updates the bank status to indicate that it is accessed by one DASM.
Uns8BnkAxsdSM1	Dynamic memory data access state machine 1	Updates the bank status to indicate that it is no more accessed by the DASM.
S8BnkClsdSM1	Dynamic memory data access state machine 1	Indicates that the bank accessed by DASM1 is updated as 'closed' when high.
Uns8BnkClsdSM1	Dynamic memory data access state machine 1	Indicates that the bank accessed by DASM1 is updated as 'opened' when high.
LdLstRwAxsdSM1	Dynamic memory data access state machine 1	Loads the value of last-row accessed in the status-information of the bank accessed by DASM1.
LstRwAxsdValSM1	Dynamic memory data access state machine 1	Last-row-accessed value for the bank accessed by DASM1.
SetLockSM1	Dynamic memory data access state machine 1	Sets the 'shared lock' used by DASM1 and DASM2 for mutual exclusion.
UnsetLockSM1	Dynamic memory data access state machine 1	Unsets the 'shared lock' used by DASM1 and DASM2 for mutual exclusion.
DyAxsSM1Idle	Dynamic memory data access state machine 1	Indicates whether DASM1 is idle.
nDySM1DtEn	Dynamic memory data access state machine 1	Asserts/de-asserts the data-enable lines during write/read.
MemA3A2SM1Co	Dynamic memory data access state machine 1	Address bit 3 and 2 tag from DASM1.
SetDyBusReqSM2	Dynamic memory data access state machine 2	Raises the request to bus-grant-logic.

Signal Name	Source	Description
UnsetDyBusReqSM2	Dynamic memory data access state machine 2	De-asserts the request to bus-grant-logic
DyCmdSM2	Dynamic memory data access state machine 2	Command to be issued from SM2.
ChipSM2V[2..3]	Dynamic memory data access state machine 2	Chip-select on which State Machine 2 is acting.
BankSM2V2	Dynamic memory data access state machine 2	Bank-select on which State Machine 2 is acting.
ArrSetCKESM2	Dynamic memory data access state machine 2	Sets the CKE pin of the chip-select that is being accessed.
UnsetCKESM2	Dynamic memory data access state machine 2	De-asserts the CKE pin of the chip-select that is being accessed.
S8BnkAxsdSM2	Dynamic memory data access state machine 2	Updates the bank status to indicate that it is accessed by one DASM.
Uns8BnkAxsdSM2	Dynamic memory data access state machine 2	Updates the bank status to indicate that it is no more accessed by the DASM.
S8BnkClsdSM2	Dynamic memory data access state machine 2	Indicates that the bank accessed by DASM2 is updated as 'closed' when high.
Uns8BnkClsdSM2	Dynamic memory data access state machine 2	Indicates the bank accessed by DASM2 is updated as 'opened' when high.
UnsetClkStpdSM2	Dynamic memory data access state machine 2	Switches on the SDRAM clocks.
LdLstRwAxsdSM2	Dynamic memory data access state machine 2	Loads the value of last-row accessed in the status-information of the bank accessed by DASM2.
LstRwAxsdValSM2	Dynamic memory data access state machine 2	Last-row-accessed value for the bank accessed by DASM2.
SetLockSM2	Dynamic memory data access state machine 2	Sets the 'shared lock' used by DASM2 and DASM1 for mutual exclusion.
UnsetLockSM2	Dynamic memory data access state machine 2	Unsets the 'shared lock' used by DASM2 and DASM1 for mutual exclusion. DyAxsSM2Idle indicates whether DASM2 is idle.
NDySM2DtEn	Dynamic memory data access state machine 2	Asserts/de-asserts the data-enable lines during write/read.
MemA3A2SM2Co[1:0]	Dynamic memory data access state machine 2	Address bits 3 and 2 tag from DASM2.
DySyFlashIdle	Dynamic memory SyncFlash state machine	Indicates Flash State Machine is Idle.
ChipFI	Dynamic memory SyncFlash state machine	Chip-select on which State flash machine is acting

Signal Name	Source	Description
DyCmdFlash	Dynamic memory SyncFlash state machine	Command from Flash State Machine.
UnsetClkStpdFI	Dynamic memory SyncFlash state machine	Switches on the SDRAM clocks.
ArrSetClkEnCSFI	Dynamic memory SyncFlash state machine	Sets the CKE pin of the chip-select that is being accessed.
ArrUnsetCKECSFI	Dynamic memory SyncFlash state machine	Unsets the CKE pin of the chip-select that is being accessed.
SetDyBusReqFISM	Dynamic memory SyncFlash state machine	Sets the request for external bus.
Uns8DyBusReqFISM	Dynamic memory SyncFlash state machine	Unsets the request for external bus
NDyFIDtEn	Dynamic memory SyncFlash state machine	Asserts/de-asserts the data-enable lines during write/read.
DyCmdRefSMQ	Dynamic memory refresh state machine	Command to be issued from Refresh State Machine.
S8BnkClsdRef	Dynamic memory refresh state machine	Indicates that all the banks are to be closed.
ArrSetCKERef	Dynamic memory refresh state machine	Sets the CKE pin of the respective chip-select.
ArrUns8CKERef	Dynamic memory refresh state machine	De-asserts the CKE pin of the respective chip-select
SetDyBusReqRef	Dynamic memory refresh state machine	Raises the request to bus-grant-logic.
UnsetDyBusReqRef	Dynamic memory refresh state machine	Raises the request to bus-grant-logic
UnsetClkStpdRef	Dynamic memory refresh state machine	Switches on the SDRAM clocks.
RefreshSMIdle	Dynamic memory refresh state machine	Switches on the SDRAM clocks
DyCmdModSMQ	Dynamic memory mode state machine	Command to be issued from Mode State Machine.
S8BnkClsdMod	Dynamic memory mode state machine	Closes all the banks when PALL is given by Mode State Machine.
SetDyBusReqMod	Dynamic memory mode state machine	Raises the request to bus-grant-logic.
UnsetDyBusReqMod	Dynamic memory mode state machine	Raises the request to bus-grant-logic.
ModSMIdle	Dynamic memory mode state machine	Indicates that mode State-machine is idle, when high.

Signal Name	Source	Description
UnsetClkStpdMod	Dynamic memory mode state machine	Switches on the clock for mode programming.
ArrSetCKEMod	Dynamic memory mode state machine	Pulls the CKE of all the DyChip selects high.
DyRas[3..0]Q	Dynamic memory register block	Ras delay value for each dynamic memory chip select. For example, DyRas0Q is the delay value for chip select 4, DyRas1Q is Ras delay value for chip select 5 and so on.
DyCas[3..0]Q	Dynamic memory register block	Cas delay value for each dynamic memory chip select. For example, DyCas0Q is for chip select 4; DyCas1Q is Cas delay value for chip select 5.
DyColWidth[3..0]Q	Dynamic memory register block	Dynamic memory column width for each chip select. For example, DyColWidth0Q is for chip select 4; DyColWidth1Q is dynamic memory column width for chip select 5 and so on.
DyRowWidth[3..0]Q	Dynamic memory register block	Dynamic memory row width for each chip select. For example, DyRowWidth0Q is for chip select 4; DyRowWidth1Q is dynamic memory row width for chip select 5 and so on. .
ChipSel[7..4]MWQ	Dynamic memory register block	Indicates memory width (16-bit or 32 bit) for chip selects 4 to 7.
DyMemDevice[3..0]Q	Dynamic memory register block	Determines the memory type of each dynamic memory chip select. For example, DyMemDevice0Q determines the memory type of chip select 4; DyMemDevice1Q determines the memory type of chip-select 5; and so on.
DyBlkRdData[3..0]	Dynamic memory register block	Read data from register blocks corresponding to each chip select.

Table 4.10-15 Block inputs of MpmcDyIntRegBlk

Signal Name	Destination	Description
MPMCCKEOUT	Pads	Clock enable control to be connected to the CKE pin of memory devices.
DyMemBusy	Arbiter	Indicates that SDRAM control State Machines are busy.
nDyDataEn	Pad interface	Data-enable control for dynamic memory commands.
DyCmd	Pad interface	Command output to the pad interface.
DyBlkRdData	AHB register interface	Register read data
MemA3A2FifoInQ	Dynamic memory read data FIFO	MemA3A2 value to be clocked into the FIFO along with respective data.
DyBusReq	Memory Bus Granting logic	request for the memory-bus from dynamic-memory-controller.

Signal Name	Destination	Description
LockSDRAMC	Dynamic memory data access state machine 1	Flag, indicating whether the other data-access-State Machine is using the external memory bus
CtBankAxsdSM[2..1]	Dynamic memory data access state machine 1 and 2	Indicates that the bank is accessed by the other data-access-State-machine.
CtBankClsdSM1[2..1]	Dynamic memory data access state machine 1 and 2	Indicates that the banks accessed by DASM2 or DASM1 are closed.
LstRwInBnk4SM[2..1]	Dynamic memory data access state machine 1 and 2	Indicates last row opened in the bank being accessed by DASM2 and DASM1.
DyRasForSM[2..1]	Dynamic memory data access state machine 1 and 2	RAS to CAS delay value of the chip-select accessed by DASM2 and DASM1.
DyCasForSM[2..1]	Dynamic memory data access state machine 1 and 2	CAS delay value of the chip-select accessed by DASM2 and DASM1.
NumColSM[2..1]	Dynamic memory data access state machine 1 and 2	Indicates number of bits in Column address for the chip select accessed by DASM2 and DASM1.
NumRowSM[2..1]	Dynamic memory data access state machine 1 and 2	Indicates number of bits in row-address for the chip select accessed by DASM2 and DASM1.
MWSM[2..1]Co	Dynamic memory data access state machine 1 and 2	Indicates the memory width of the chip select accessed by the two data access state machines. MWSM2Co indicates whether the chip select accessed by DASM2 is 16/32 bit wide; and MWSM1Co indicates whether the chip select accessed by DASM1 is 16/32 bit wide.
DyBufFull	Dynamic memory Read Request Generation Block	Indicates that no more PostRdReq or PostWReq to the synchronous memory should be asserted.
UnsetModeDone	Dynamic memory mode state machine	Resets the ModeDone bit due to deep-sleep-entry.
ArrSetCKEFinal	DASM1, DASM2	OR-ed version of set clock enable from all the state machine
ArrUnSetCKEFinal	DASM1, DASM2	OR-ed version of unset clock enable from all the state machine

Table 4.10-16 Block outputs of MpmcDyIntRegBlk

4.10.9 Dynamic memory latency control block

The dynamic memory latency control block (MpmcDyLatCntr) has a set of counters which track the latency between two command issues to a bank. One bank can be accessed by two DASM s, but the second DASM should honour the latencies which arise due to the issue of commands by first DASM. Thus these counters are bank-specific. When a command is issued, the related counters are loaded with required-latency values. The DASM checks the expiry-status of the counters to issue subsequent commands.

Figure 4.10-10 describes the block input and outputs ports of the dynamic memory latency control block

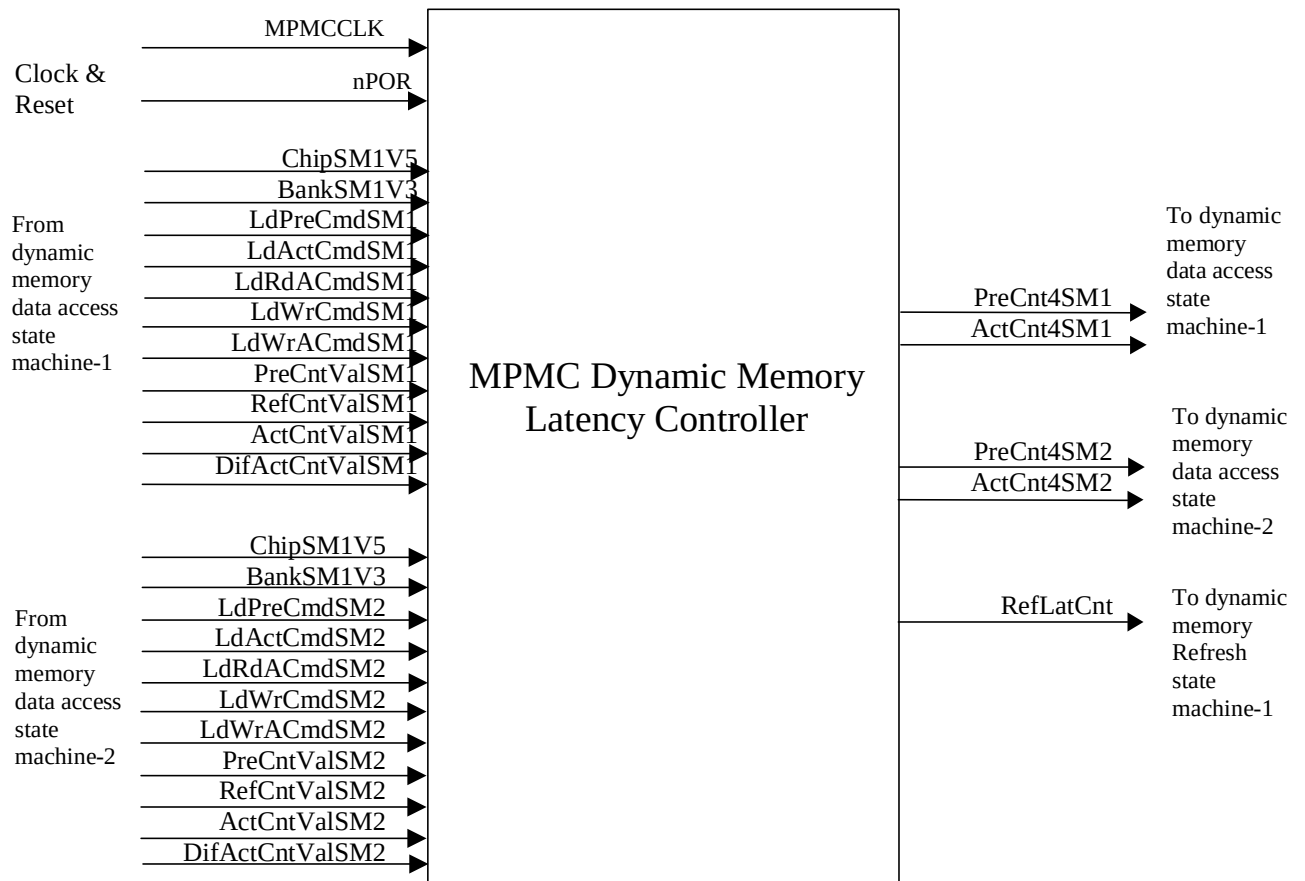


Figure 4.10-10 Block Inputs and Outputs for MpmcDyLatCntr

The input and output ports of MpmcDyLatCntr are described in **Table 4.10-17** and **Table 4.10-18**.

Signal Name	Source	Description
PreCntValSM[2..1]	Dynamic memory data access state machine 1 and 2	Counter value to be loaded for precharge latency counters of the bank-selects being accessed by DASM2 and DASM1.
ActCntValSM[2..1]	Dynamic memory data access state machine 1 and 2	Counter value to be loaded for activate latency counter of the bank-select being accessed by DASM2 and DASM1.
DifActCntValSM[2..1]	Dynamic memory data access state machine 1 and 2	Value to be loaded into the active latency counter of other banks in the chip for DASM2 and DASM1.
RefCntValSM[2..1]	Dynamic memory data access state machine 1 and 2	Value to be loaded into the refresh latency counter for DASM2 and DASM1.
LdPreCmdSM[2..1]	Dynamic memory data access state machine 1 and 2	Load strobe for Precharge latency counters for DASM2 and DASM1.
LdActCmdSM[2..1]	Dynamic memory data access state machine 1 and 2	Load strobe for Activate latency counters for DASM2 and DASM1.

Signal Name	Source	Description
LdRdACmdSM[2..1]	Dynamic memory data access state machine 1 and 2	Load strobe for Activate latency counters for DASM2 and DASM1.
LdWrCmdSM[2..1]	Dynamic memory data access state machine 1 and 2	Load strobe for write access latency counters for DASM2 and DASM1.
LdWrACmdSM[2..1]	Dynamic memory data access state machine 1 and 2	Load strobe for write access with autoprecharge latency counters for DASM2 and DASM1.

Table 4.10-17 Block inputs of MpmcDyLatCntr

Signal Name	Destination	Description
PreCnt4SM[2..1]	Dynamic memory data access state machine 1 and 2	Count value for precharge command issue to the bank accessed by DASM2 and DASM1.
ActCnt4SM[2..1]	Dynamic memory data access state machine 1 and 2	Count value for active command issue to the bank accessed by State Machine 1.
RefLatCnt	Dynamic memory refresh state machine	Latency counter value for refresh command issue.

Table 4.10-18 Block outputs of MpmcDyLatCntr

4.11 Static Memory Controller

The static memory controller block (MpmcStMemCntrl) receives the static memory access requests along with the address, data and byte lane information from the Buffer Unit. The requests are then processed and the static memory read or write commands are issued to the target memory device according to the timings programmed in the static memory control registers. The static memory controller contains the following sub-blocks:

- The static memory transfer State Machine block, which contains the main transfer State Machine to control all static memory transactions.
- The static memory register block, this is instantiated four times. Each block contains the static memory control registers to control the static memory device connected to one chip select.
- The static memory timing control block, which controls the static memory transfer timings.
- The static memory controller external interface block, which provides the interface to the external memory bus.
- The static memory controller internal interface block, which provides the interface with the buffer and AHB interface blocks.

A block schematic of the sub-blocks in the static memory controller block is shown in **Figure 4.11-1**

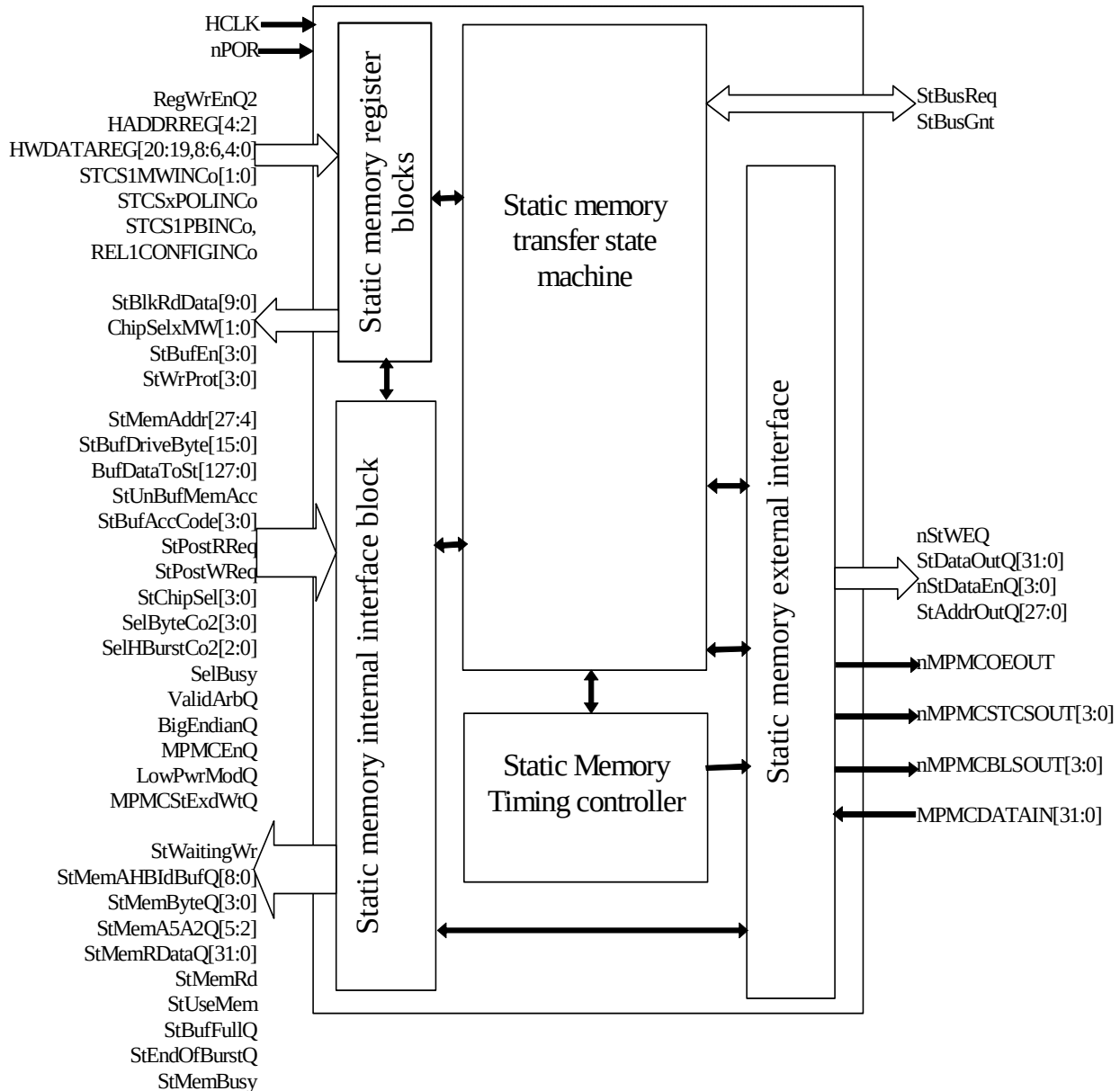


Figure 4.11-1 Block diagram of the Static Memory Controller

4.11.1 Static memory register block

This block (MpmcStRegBlk) implements the static memory control registers for one chip select. This block is instantiated four times to realise the control registers for the four static memory chip selects. The AHB register interface block (MpmcRegIf) asserts the StBlkSelCo2 signal corresponding to one static memory chip select when a register used to control that chip select's memory is selected. This block generates the write enables for the registers in the selected static memory register block on writes from the AHB to one of the registers in this block. It drives the selected register data on the StBlkRdData bus to the AHB register interface block. When not selected, the StBlkRdData bits are driven to zero to enable the use of OR logic for combining read data from the different register blocks.

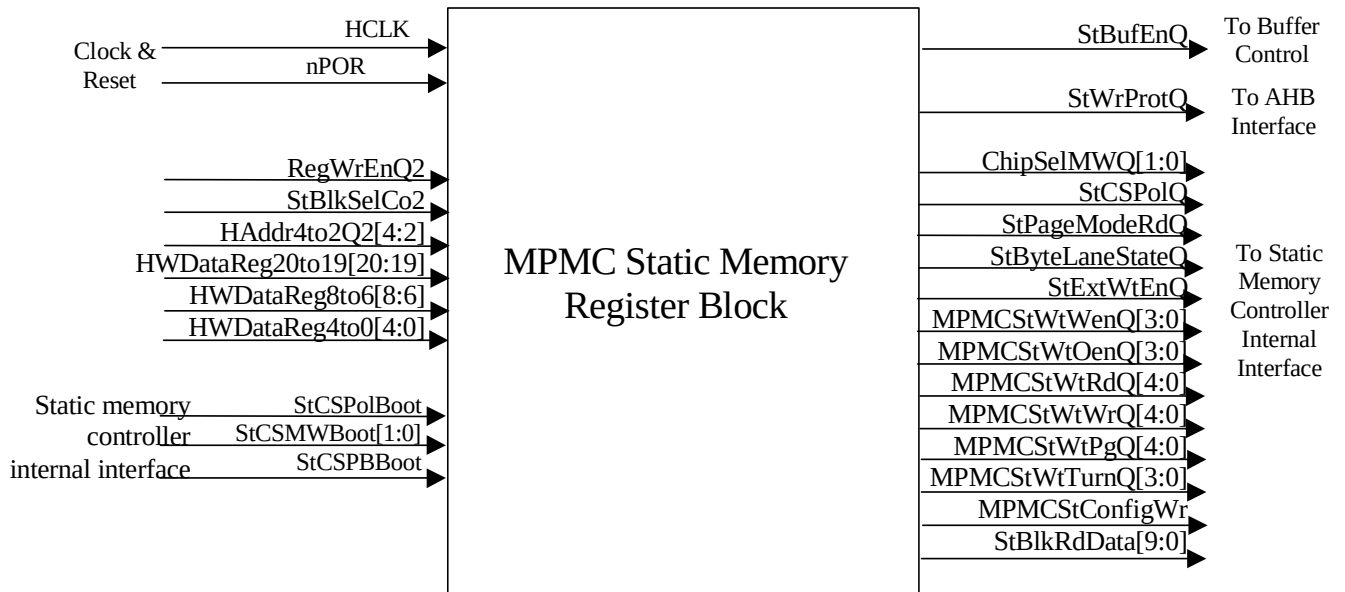


Figure 4.11-2 Block Inputs and Outputs for MpmcStRegBlk

The input and output ports of MpmcStRegBlk are described in **Table 4.11-1** and **Table 4.11-2**.

Signal Name	Source	Description
HCLK	Clock Generator	AHB clock
nPOR	Reset Generator	Power on reset
RegWrEnQ2	AHB register interface	Indicates the registered write enable signal for the registers.
StBlkSelCo2	AHB register interface	Indicates that the static memory register block is selected for a read or write to one of its registers.
HAddr4to2Q2[4:2]	AHB register interface	Registered HADDRREG[4:2] signal used to select the target static memory control register.
HWDDataReg20to19[20:19]	AHB register interface	Bit-slice of the HWDATAREG write databus that is required to write into the static memory configuration registers.
HWDDataReg8to6[8:6]	AHB register interface	Bit-slice of the HWDATAREG write databus that is required to write into the static memory configuration registers.
HWDDataReg4to0[4:0]	AHB register interface	Bit-slice of the HWDATAREG write databus that is required to write into the static memory configuration registers.
StCSPolBoot	Static memory controller internal interface	Boot status of the chip select polarity to be returned on reads to the MPMCStConfig register. This value indicates the pin value on reset and the register value after the first write to the MPMCStConfig register.

Signal Name	Source	Description
StCSMWBoot[1:0]	Static memory controller internal interface	Boot status of the memory width to be returned on reads to the MPMCStConfig register. This value indicates the pin value on reset and the register value after the first write to the MPMCStConfig register.
StCSPBBoot	Static memory controller internal interface	Boot status of the byte lane state to be returned on reads to the MPMCStConfig register. This value indicates the pin value on reset and the register value after the first write to the MPMCStConfig register.

Table 4.11-1 Block inputs of MpmcStRegBlk

Signal Name	Destination	Description
ChipSelMWQ	Static memory controller internal interface block	Indicates the memory width programmed into the static memory configuration register (MPMCStConfig).
StCSPolQ	Static memory controller internal interface block	Indicates the chip select polarity of the 4 static memory chip selects as programmed into the MPMCStConfig registers.
StPageModeRdQ	Static memory controller internal interface block	Page mode read enable of the static memory device associated with the register block.
ByteLaneStateQ	Static memory controller internal interface block	Byte lane state enable of the static memory device associated with the register block.
StExtWtEnQ	Static memory controller internal interface block	Extended wait enable of the static memory device associated with the register block.
MPMCStWtWenQ[3:0]	Static memory controller internal interface block	Indicates the chip select assertion to write enable assertion delay of the static memory device associated with the register block.
MPMCStWtOenQ[3:0]	Static memory controller internal interface block	Indicates the chip select assertion to output enable assertion delay of the static memory device associated with the register block.
MPMCStWtWrQ[4:0]	Static memory controller internal interface block	Indicate the write access time of the static memory device associated with the register block.
MPMCStWtRdQ[4:0]	Static memory controller internal interface block	Read access time of the static memory device associated with the register block.
MPMCStWtPgQ[4:0]	Static memory controller internal interface block	Page mode read access time of the static memory device associated with the register block.
MPMCStWrTurnQ[3:0]	Static memory controller internal interface block	Indicates the turnaround time of the chip select associated with the register block.
StBlkRdData[9:0]	Static memory controller internal interface block	Indicates the register read data bus from the static memory register block.
StBufEnQ	Buffer unit	Indicates whether the accesses to static memory chip select that is associated with this static memory register block are buffered or unbuffered.

Signal Name	Destination	Description
StWrProtQ	AHB memory interface	Indicates whether static memory chip select that is associated with this static memory register block is write protected or not.

Table 4.11-2 Block outputs of MpmcStRegBlk

4.11.2 Static memory controller internal interface block

The static memory controller internal interface block (MpmcStIntIf) provides the interface between the static memory controller and the AHB interface and buffer control blocks. This module generates all the control signals, address and data information required at the start of a static memory access. It uses the information driven by the buffer control block and the acknowledge signals from the static memory transfer State Machine and the static memory external interface to generate these signals. This module also generates some of the signals to be returned to the buffer control block at the end of memory accesses.

This module selects the memory configuration information from the register bits for the memory access request which is currently being serviced. It generates the valid byte information for the current access from the byte valid information driven from the buffer control logic and selects the relevant byte / halfword / word of data from the 128 bit buffer flush for writes. It generates the address for the current access by combining the address driven by the buffer control block and the byte valid information.

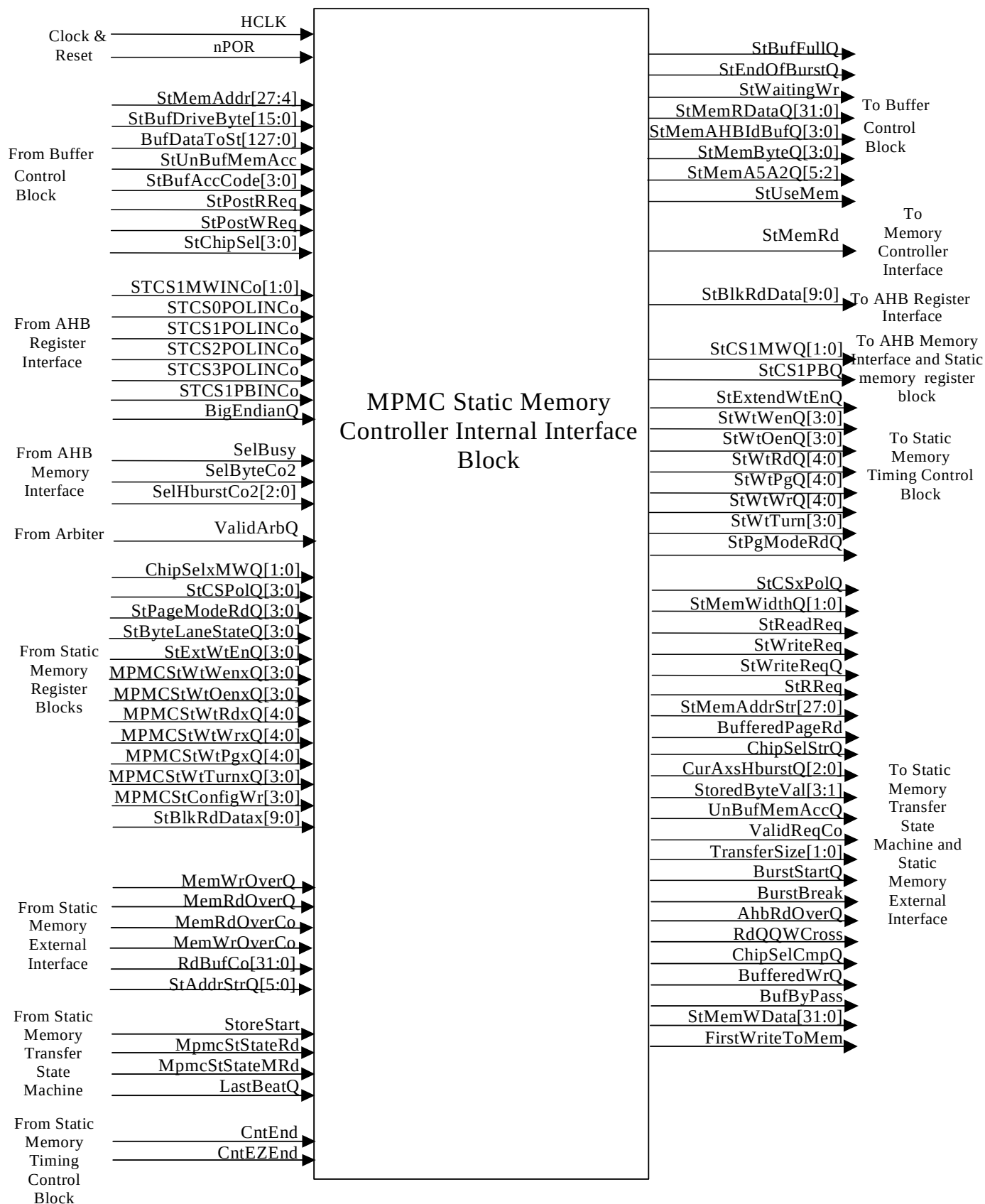


Figure 4.11-3 Block Inputs and Outputs for MpmcStntlf

The input and output ports of MpmcStIntIf are described in **Table 4.11-3 Table 4.11-4**.

Signal Name	Source	Description
HCLK	Clock Generator	AHB clock
nPOR	Reset Generator	Power on reset
StPostRReq	Buffer unit	Indicates a static memory read. A high on this signal indicates that an AHB or a buffer is requesting a static memory read access.
StPostWReq	Buffer unit	Indicates a static memory write. A high on this signal indicates that an AHB or a buffer is requesting a static memory write access.
StUnBufMemAcc	Buffer unit	Indicates whether the current memory access request is a buffered access or an unbuffered access. Sampled when a static memory read or write request (StPostRReq or StPostWReq) is asserted.
StBufAccCode[3:0]	Buffer unit	Indicates the AHB or the buffer that is the requestor of the current access. Sampled when a static memory read or write request (StPostRReq or StPostWReq) is asserted.
BufDataToSt[127:0]	Buffer unit	Indicates the write data to be written by the quad-word buffer or the AHB to the memory. Sampled one clock after the write request is asserted.
StBufDriveByte[15:0]	Buffer unit	Indicates the valid bytes in the 128-bit data that was flushed from the quad-word buffer or the selected AHB. Sampled when a static memory read or write request (StPostRReq or StPostWReq) is asserted.
StMemAddr[27:4]	Buffer unit	Indicates the quadword address of the memory access. Sampled when a static memory read or write request (StPostRReq or StPostWReq) is asserted.
StChipSel[3:0]	Buffer unit	Indicates the static memory chip select that is the target chip select for the current memory access request. Sampled when a static memory read or write request (StPostRReq or StPostWReq) is asserted.
STCS1MWINCo	AHB register interface	Indicates the memory width information that is the output of the Integration test mux for the MPMCSTCS1MW[1:0] pins of the MPMC. The memory width indicated by these signals is followed until the first write into the corresponding static memory configuration register (MPMCStConfig[3..0]). After the first write to the register, the static memory controller uses the value programmed in the register.

Signal Name	Source	Description
STCS[3..0]POLINCo	AHB register interface	Indicates the polarity of the 4 static memory chip selects. These are also the outputs of the integration test mux for the MPMCSTCSxPOL input pins of the MPMC. The chip select polarity indicated by these signals is followed until the first write into the corresponding static memory configuration register (MPMCStConfig[3..0]). After the first write to the register, the static memory controller uses the value programmed in the register.
STCS1PBINCo	AHB register interface	Indicates the byte lane state of chip select 1. This is the output of the integration test mux for MPMCSTCS1PB input pin of the MPMC. The byte lane state indicated by this signal is followed until the first write to the static memory configuration register (MPMCStConfig1). After the first write to the register, the static memory controller uses the value programmed in the register.
BigEndianQ	AHB register interface	Indicates the endian mode of the MPMC. A low on BigEndianQ indicates a little endian mode and a high indicates a big endian mode. This signal is used to increment or decrement the static memory address during AHB burst reads.
SelByteCo2[3:0]	AHB memory interface	Indicates the valid bytes in the current static memory access request.
SelBusy	AHB memory interface	Indicates that there is a BUSY transfer on selected AHB that initiated the current memory access request.
SelHBurstCo2	AHB memory interface	Indicates the burst type for the current static memory access request.
ValidArbQ	Arbiter block	Indicates the valid arbitration window for an AHB. A low on ValidArbQ indicates that a re-arbitration is in progress.
ChipSel[3..0]MWQ	Static memory register blocks for chip selects 3, 2, 1 and 0	indicates the memory width programmed into the static memory configuration register (MPMCStConfig[3..0]).
StCSPolQ[3:0]	Static memory register blocks for chip selects 3, 2, 1 and 0	indicates the chip select polarity of the 4 static memory chip selects as programmed into the MPMCStConfig[3..0] registers
StPageModeRdQ[3:0]	Static memory register blocks for chip selects 3, 2, 1 and 0	Indicates the page mode read enable of the 4 static memory chip selects as programmed into the MPMCStConfig[3..0] registers
StByteLaneStateQ[3:0]	Static memory register blocks for chip selects 3, 2, 1 and 0	Byte lane state enable of the 4 static memory chip selects as programmed into the MPMCStConfig[3..0] registers
StExtWtEnQ[3:0]	Static memory register blocks for chip selects 3, 2, 1 and 0	Extended wait enable of the 4 static memory chip selects as programmed into the MPMCStConfig[3..0] registers.

Signal Name	Source	Description
MPMCStWtWen[3..0]Q[3:0]	Static memory register blocks for chip selects 3, 2, 1 and 0	Indicates the chip select assertion to write enable assertion delay for chip select 3, 2, 1 and 0.
MPMCStWtOen[3..0]Q[3:0]	Static memory register blocks for chip selects 3, 2, 1 and 0	Indicates the chip select assertion to output enable assertion delay for chip select 3, 2, 1 and 0.
MPMCStWtWr[3..0]Q[4:0]	Static memory register blocks for chip selects 3, 2, 1 and 0	Indicate the write access time for chip select 3, 2, 1 and 0.
MPMCStWtRd[3..0]Q[4:0]	Static memory register blocks for chip selects 3, 2, 1 and 0	Indicate the read access time for chip select 3, 2, 1 and 0.
MPMCStWtPg[3..0]Q[4:0]	Static memory register blocks for chip selects 3, 2, 1 and 0	Indicate the page mode read access time respectively for chip select 3, 2, 1 or 0.
MPMCStWrTurn[3..0]Q[3:0]	Static memory register blocks for chip selects 3, 2, 1 and 0	Indicates the turnaround time for chip select 3, 2, 1 and 0.
StBlkRdData[3..0][9:0]	Static memory register blocks for chip selects 3, 2, 1 and 0	Indicates the register read data bus from the static memory register block for chip selects 3, 2, 1 and 0. The read data buses from the four static memory register blocks are logically Ored to generate a single register read data bus from the static memory controller.
MemWrOverQ	Static memory controller external interface block	Indicates the end of a write access to the memory.
MemRdOverQ	Static memory controller external interface block	Indicates the end of a write access to the memory.
MemWrOverCo	Static memory controller external interface block	D-input of MemWrOverQ.
MemRdOverCo	Static memory controller external interface block	D-input of MemRdOverQ.
RdBufCo[31:0]	Static memory controller external interface block	Combinational read data from static memory. This data is obtained by rearranging the data received on the MPMCDATAIN[31:0] during a static memory read access depending on the memory width and transfer size.
StAddrStrQ[5:0]	Static memory controller external interface block	Least significant bits of the stored static memory address. This signal is used to determine next address during read prefetch burst.
StoreStart	Static memory transfer state machine	Indicates that the transfer State Machine is in a State where the chip select has been de-asserted. This signal is used to sustain BurstStartQ as long as the transfer State Machine has not gone into an active State.

Signal Name	Source	Description
MpmcStStateRd	Static memory transfer state machine	Indicates that a static memory read access is currently in progress. When high, it indicates that a read access to memory has been initiated. This signal is used to detect the extra prefetch reads that had already been initiated when the AHB burst was terminated.
MpmcStStateMRd	Static memory transfer state machine	Indicates that a static memory read access is currently in progress with the output enable active. When high, it indicates that a read access to memory has been initiated and the static memory transfer state machine is in the ST_TSM_MEMRD state. This signal is used to detect the extra prefetch reads that had already been initiated when the AHB burst was terminated.
LastBeatQ	Static memory transfer state machine	Indicates the last beat in a read prefetch burst. This signal is used to reset StBufFullQ.
CntEnd	Static memory timing control block	Indicates that the timing counter has expired.
CntEZEnd	Static memory timing control block	Indicates that the timing counter load value is zero.

Table 4.11-3 Block inputs of MpmcStIntf

Signal Name	Destination	Description
StBufFullQ	Buffer unit	Indicates the full status of the static memory command buffer. If the static memory controller is already servicing one static memory access request, this signal is set high. It remains high till the end of the read or write request. StWaitingWr is asserted when the static memory controller is waiting for sequential writes to the memory. A high on this signal is used by the buffer control logic to prevent any buffer flushed to the static memory.
StMemRDataQ[31:0]	Buffer unit	The read data from a static memory is driven on this signal at the end of a static memory read access.
StEndOfBurstQ	Buffer unit	Indicates the end of read access burst initiated by a single read access request. For unbuffered, StEndOfBurstQ is asserted for one clock cycle at the end of every valid memory read access. For buffered page mode enabled read StEndOfBurstQ is asserted for one clock cycle only at the end of the last read from the static memory that will fill the quadword buffer.
StMemAHBIdBufQ[3:0]	Buffer unit	Indicates the buffer that was the requestor for the current access.
StMemByteQ[3:0]	Buffer unit	Indicates the valid bytes of data returned by the static memory.
StMemA5A2Q[5:2]	Buffer unit	Indicates the quadword and word address of the static memory read data returned to the buffer.

Signal Name	Destination	Description
StUseMem	Buffer unit	Asserted to indicate the end of a read or write data transfer to the static memory controller. In case of unbuffered writes, it is asserted for one clock cycle after the StPostWReq is sampled high. In case of buffered and unbuffered reads, it is asserted when the read data from the static memory device is sampled by the MPMC. StMemRData[31:0], StMemAHBIdBufQ[3:0], StMemByteQ[3:0] and StMemA5A2[5: 2] are sampled by the buffer control block or AHB only when they sample a high on StUseMem.
StMemRd	Memory controller interface	Asserted at the end of a static memory read access. This indicates that StMemRData[31:0] contains valid data.
StBlkRdData[9:0]	AHB register interface	Combined static memory registers read data bus. Driven with the logical OR of the register read data buses from the four static memory register blocks.
StCS1MWQ[1:0]	AHB memory interface	Driven with the memory width of the static memory device connected to chip select 1. Follows the value on the MPMCSTCS1MW[1:0] pin (in normal mode) or MPMCI TIP[3:2] (in integration test mode) till the first write to the MPMCStConfig1 register. After the first write, the value programmed into the MpmcStConfig1 register is used.
StCS1PBQ	Static memory controller register interface	Driven with the byte lane state of the static memory device connected to chip select 1. Follows the value on the MPMCSTCS1PB pin (in normal mode) or MPMCI TIP[8] (in integration test mode) till the first write to the MPMCStConfig1 register. After the first write, the value programmed into the MpmcStConfig1 register is used.
StPgModeRdQ	Static memory timing control block	A high on StPgModeRdQ indicates that the current static memory access is to a page mode enabled static memory device.
StExtendWtEnQ	Static memory timing control block	A high on StExtendWtEnQ indicates that the current static memory access is to an extended wait enabled static memory device.
StWtWenQ[3:0],	Static memory timing control block	Indicate the chip select to write enable delay for the static memory device being accessed.
StWtOenQ[3:0]	Static memory timing control block	Indicate the chip select to output enable delay for the static memory device being accessed.
StWtWrQ[4:0]	Static memory timing control block	Indicates the write access time for the static memory device being accessed.
StWtRd[4:0].	Static memory timing control block	Indicates the normal mode read access times for the static memory device being accessed
StWtPgQ[4:0]	Static memory timing control block	Indicates the page mode read access times for the static memory device being accessed
StWtTurn[3:0]	Static memory timing control block	Indicates the number of turnaround cycles to be inserted between reads to different chip selects or between a read and a write.
StMemWidthQ[1:0]	Static memory transfer state machine and static memory external interface	Indicates the memory width of the static memory device being accessed. A value of "00" on this signal indicates an 8-bit memory width, "01" indicates 16-bit memory width and "10" indicates 32-bit memory width.
StBLState	Static memory external interface	Indicates the byte lane State of the static memory device being accessed. A low on this signal indicates that the nMPMCBL SOUT[3:0] will be connected to the write enable pins of the static memory device.

Signal Name	Destination	Description
StCS[3..0]PolQ	Static memory external interface	Indicates the chip select polarity for the memory devices connected to the static memory chip selects 3, 2, 1 or 0.
StReadReq	Static memory transfer state machine	Indicates that a valid static memory read request from the AHB or buffer is being serviced. This signal is asserted high during the read access to the memory if it is the first read in an AHB burst read or if it is a buffered read. During prefetch reads it is not kept asserted.
StWriteReq	Static memory transfer state machine and static memory external interface	Indicates that a valid static memory write request from the AHB or a buffer is being serviced by the static memory controller.
StWriteReqQ	Static memory transfer state machine	Registered version of StWriteReq.
StRReq	Static memory transfer state machine and static memory external interface	A high on StRReq indicates the first clock cycle of a valid read request from an AHB or a buffer.
StMemAddrStr[27:0]	Static memory transfer state machine, Static memory external interface	Indicates the memory address for a valid memory access request.
BufferedPageRd	Static memory transfer state machine, Static memory external interface	A high on BufferedPageRd indicates that the static memory read access that is being serviced is a buffered access to a page mode enable device.
ChipSelStrQ[3:0]	Static memory external interface	Indicates the chip select that is being accessed.
CurAxsHburstQ[2:0]	Static memory external interface	Burst type for current access. This signal indicates the HBURST type for unbuffered accesses to memory.
StoredByteVal[3:1]	Static memory external interface	Indicates the valid bytes in the current static memory write access.
UnBufMemAccQ	Static memory transfer state machine, Static memory external interface	A high on UnBufMemAccQ indicates that the current access is an unbuffered access.
ValidReqCo	Static memory external interface	Indicates a valid read or write access request.
TransferSize[1:0]	Static memory transfer state machine, Static memory external interface	Indicates the size of the data that is written into or read from the static memory device in the current access.
BurstStartQ	Static memory transfer state machine	A high on BurstStartQ indicates the start of a AHB burst.
BurstBreak	Static memory transfer state machine	A high on BurstBreak indicates the end of an AHB burst.
AhbRdOverQ	Static memory transfer state machine, Static memory external interface	A high on AhbRdOverQ indicates that the AHB has finished reading all the data that was fetched from the memory during the previous read access to the memory.
RdQQWCross	Static memory transfer state machine	Indicates a quad-quad-word cross for the current read prefetch burst.
ChipSelCmpQ	Static memory transfer state machine	A high on ChipSelCmp indicates that the current static memory access to the same chip select as the previous one
BufferedWrQ	Static memory transfer state machine, Static memory external interface.	A high on BufferedWrQ indicates that the 32-bit buffer has enough data to write into the memory.
BufByPass	Static memory transfer state machine, Static memory external interface.	BufByPass is asserted when TransferSize is greater than memory width for the current access.

Signal Name	Destination	Description
StMemWData[31:0]	Static memory external interface	Write data for the current write to the static memory. This 32-bit data is selected from the 128-bit buffer data depending on the valid bytes.
FirstWriteToMem	Static memory external interface	Indicates the first write transfer to the memory in a buffer flush write. In case of unbuffered writes, this signal is asserted for every write transfer.

Table 4.11-4 Block outputs of MpmcStIntf

4.11.3 Static memory transfer State Machine

The Static memory transfer State Machine (MpmcStTSM) block controls all the transactions of the static memory controller block to the external memory device. It generates the bus request signal for the control of the external data bus lines. The load signals for the read and write access time counters are generated depending on the type of transfer and the device characteristics. Load signals for the chip select assertion to nMPMCWEOUT and nMPMCOEOUT assertion delay counters are also generated in this block. External bus turnaround cycles are initiated appropriately after a read transfer completion.

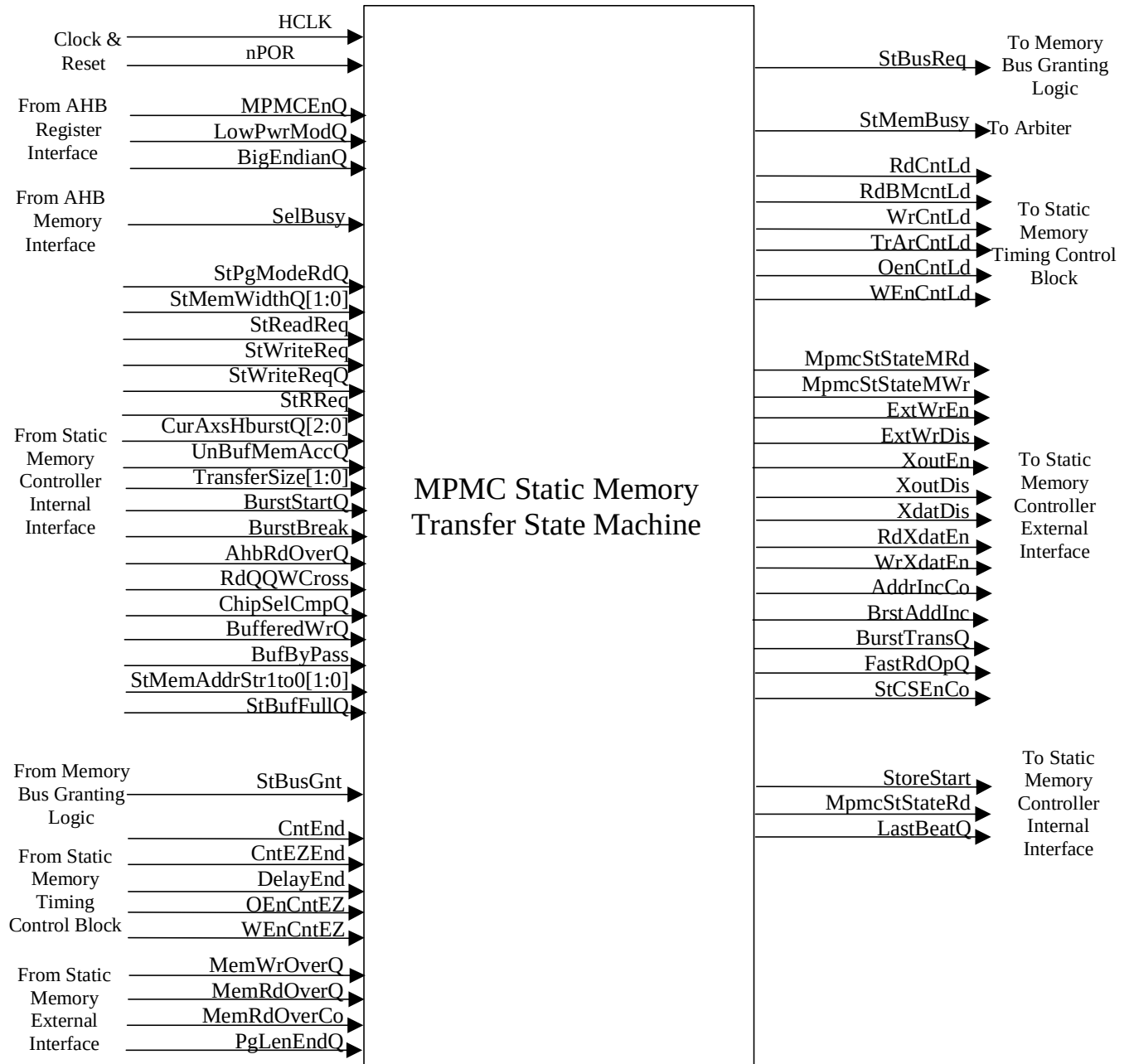


Figure 4.11-4 Block Inputs and Outputs for MpmcStTSM

The input and output ports of MpmcStTSM are described in **Table 4.11-5** and **Table 4.11-6**.

Signal Name	Source	Description
HCLK	Clock Generator	AHB clock
nPOR	Reset Generator	Power on reset

Signal Name	Source	Description
MPMCEnQ	AHB register interface	Global disable for the MPMC. When this signal is asserted high, the MPMC is enabled. When this signal is de-asserted, the TSM does not move out of the idle state. If the TSM was in any non-idle State when the MPMC is disabled, the initiated transfer is completed and the state machine returns to the idle state.
LowPwrModQ	AHB register interface	Indicates that the MPMC has to be put in low power mode, that is no static memory accesses are initiated when the LowPwrModQ signal is asserted high and the TSM stays in the idle State.
BigEndianQ	AHB register interface	Indicates the endian mode of the MPMC. A low on BigEndianQ indicates a little endian mode and a high indicates a big endian mode. This signal is used to detect the number of beats in a AHB read burst for prefetching read data before the AHB interface initiates a read request during AHB burst reads
SelBusy	AHB memory interface	Indicates that there is a BUSY transfer on selected AHB that initiated the current memory access request. This signal is used to stop prefetching read data for AHB burst reads.
StReadReq	Static memory controller internal interface	Indicates that a valid static memory read request from the AHB or buffer is being serviced. This signal is asserted high during the read access to the memory if it is the first read in an AHB burst read or if it is a buffered read. During prefetch reads for AHB bursts, it is not asserted.
StWriteReq	Static memory controller internal interface	Indicates that a valid static memory write request from an AHB or a buffer is being serviced by the static memory controller.
StWriteReqQ	Static memory controller internal interface	Registered version of StWriteReq.
StRReq	Static memory controller internal interface	A high on StRReq indicates the first clock cycle of a valid read request from an AHB or a buffer.
StBufFullQ	Static memory controller internal interface	If the static memory controller is already servicing one static memory access request, StBufFullQ is set high in the static memory controller internal interface. It remains high till the end of the read or write request. This signal is used to generate the static memory controller busy indicator, StMemBusy.
StPgModeRdQ	Static memory controller internal interface	A high on StPgModeRdQ indicates that the current static memory access is to a page mode enabled static memory device. This signal is used in the TSM to issue page mode reads to the static memory device being accessed.
StMemWidthQ[1:0]	Static memory controller internal interface	indicates the memory width of the static memory device being accessed. The TSM issues single or multiple reads and writes to the static memory device depending on the value of StMemWidthQ[1:0] and TransferSize[1:0].

Signal Name	Source	Description
CurAxsHburstQ[2:0]	Static memory controller internal interface	HBURST type for unbuffered accesses. The number read prefetches that are required for the AHB that has been granted priority by the arbiter block is determined from the value of CurAxsHburstQ[2:0], SelMemWidth[1:0] and TransferSize[1:0].
UnBufMemAccQ	Static memory controller internal interface	A high on UnBufMemAccQ indicates that the current access is an unbuffered access.
BurstStartQ	Static memory controller internal interface	A high on BurstStartQ indicates the start of a AHB burst.
BurstBreak	Static memory controller internal interface	A high on BurstBreak indicates the end of an AHB burst. BurstStartQ and BurstBreak are used to start and stop prefetch reads to the static memory.
AhbRdOverQ	Static memory controller internal interface	A high on AhbRdOverQ indicates that the AHB has finished reading all the data that was fetched from the memory during the previous read access to the memory so that the TSM can continue with the read data prefetching.
RdQQWCross	Static memory controller internal interface	Indicates a quad-quad-word cross to stop the current read prefetch burst.
ChipSelCmpQ	Static memory controller internal interface	A high on ChipSelCmpQ indicates that the current static memory access to the same chip select as the previous one and no turnaround cycles are required between the two accesses.
BufferedWrQ	Static memory controller internal interface	A high on BufferedWrQ indicates that the 32-bit buffer has enough data to write into the memory. The TSM waits for this signal to be asserted to issue a static memory write command.
BufByPass	Static memory controller internal interface	Asserted when TransferSize is greater than memory width for the current access. The TSM directly moves into the write state from the idle state if a static memory write request is detected with BufByPass asserted high.
StMemAddrStr[1:0]	Static memory controller internal interface	Indicates the starting address of an unbuffered AHB burst read. This is used to calculate the number of beats in the current AHB burst.
StBusGnt	Memory bus granting logic block	A high on StBusGnt indicates that the static memory controller has been granted control of the external memory bus. The TSM issues a read or write request to a static memory only if StBusGnt is asserted.
CntEnd	Static memory timing control block	Indicates completion of the read or write access time or turnaround time.
CntEZEnd	Static memory timing control block	Timer counter expiry signal to be used by the TSM when the access time or turnaround count values are zero.

Signal Name	Source	Description
DelayEnd	Static memory timing control block	A high on DelayEnd indicates the completion of the chip select assertion to enable delay for nMPMCWEOUT and nMPMCOEOUT. This signal is used to move from the output enable State of the TSM to the memory read State and from the write enable State to the memory write State.
OEnCntEZ	Static memory timing control block	Indicates that the chip select assertion to output enable delay value is equal to zero. OenCntEZ is used to bypass the output enable state in the TSM.
WenCntEZ	Static memory timing control block	Indicates that the chip select assertion to write enable delay value is equal to zero. WenCntEZ is used to bypass the write enable state in the TSM.
MemWrOverQ	Static memory controller external interface	Indicate the end of a write access to the memory. This signal is used by the TSM to move into the appropriate state at the end of a write access to the static memory.
MemRdOverQ	Static memory controller external interface	Indicate the end of a read access to the memory. This signal is used by the TSM to move into the appropriate state at the end of a read access to the static memory.
MemRdOverCo	Static memory controller external interface	D-input of MemRdOverQ.
PgLenEndQ	Static memory controller external interface	Indicates the end of a page in the page mode enabled static memory device that is being accessed. This signal is used to time the read accesses during burst reads to page mode enabled devices.

Table 4.11-5 Block inputs of MpmcStTSM

Signal Name	Destination	Description
StBusReq	Memory bus granting logic block	Asserted by the TSM when it is about to initiate a static memory read or write. This signal indicates a request from the static memory controller for control of the external memory bus.
StMemBusy	Arbiter block	Asserted by the static memory controller that it is currently servicing a static memory access request. This signal is cleared when the TSM is in the idle State and the static memory command buffer is empty.
RdCntLd	Static memory timing control block	Asserted by the TSM to indicate that the timer counter must be loaded with the count value for a normal read access to static memory.
RdBMcntLd	Static memory timing control block	Asserted to indicate that the timer counter must be loaded with the count value for a page mode read access.
WrCntLd	Static memory timing control block	Asserted to indicate that the timer counter must be loaded with the count value for a write access.
TrArCntLd	Static memory timing control block	Asserted to load the timer counter with the count value for the number of turnaround cycles that must be inserted after a read access to a static memory.
OenCntLd	Static memory timing control block	Asserted by the TSM to indicate that the delay counter must be loaded with the output enable assertion delay value.

Signal Name	Destination	Description
WenCntLd	Static memory timing control block	Asserted by the TSM to indicate that the delay counter must be loaded with the write enable assertion delay value.
StCSEnCo	Static memory external interface	Used to assert and deassert the chip select to the static memory device being accessed.
ExtWrEn	Static memory external interface	Enable signal for generation of write enable to the static memory devices.
ExtWrDis	Static memory external interface	Disable signal for generation of write enable to the static memory devices.
XoutEn	Static memory external interface	Enable signal for the assertion of nMPMCOEOUT.
XoutDis	Static memory external interface	Disable signal for the de-assertion of nMPMCOEOUT.
XdatDis	Static memory external interface	Used by the static memory controller external interface to de-assert the nMPMCDATAEN[3:0] output lines.
RdXdatEn	Static memory external interface	Static memory read timings are indicated by asserting RdXdatEn. During reads, only the valid byte lanes are disabled while the other bits of nMPMCDATAEN[3:0] are enabled.
WrXdatEn	Static memory external interface	Static memory write timings are indicated by asserting WrXdatEn. During writes, all the bits of nMPMCDATAEN[3:0] are asserted.
AddrIncCo	Static memory external interface	Asserted to indicate the static memory address should be incremented by a value corresponding to the memory width of the accessed device. This signal is asserted during reads or writes with transfer size greater than memory width.
BrstAddInc	Static memory external interface	Asserted to increment the static memory address in advance during AHB burst reads.
BurstTransQ	Static memory external interface	Asserted to indicate that the current transfer status is a part of a sequence of AHB burst reads.
FastRdOpQ	Static memory external interface	Indicates the transfer size is less than the memory width for the current read access. In such a case, the read data for multiple AHB reads are returned with a single read to the static memory device.
MpmcStStateMRd	Static memory external interface, Static memory controller internal interface	A high on MpmcStStateMRd indicates that the transfer state machine's current state value is ST_TSM_MEMRD.
MpmcStStateMWr	Static memory external interface	A high on MpmcStStateMWr indicates that the transfer state machine's current state value is ST_TSM_MEMWR.
StoreStart	Static memory controller internal interface	Indicates that the transfer State Machine is in a State where the chip select has been de-asserted.
MpmcStStateRd	Static memory controller internal interface	Indicates that a read access to memory has been initiated. The transfer state machine's current state value is ST_TSM_MEMRD or ST_TSM_OENCNT
LastBeatQ	Static memory controller internal interface	Indicates the last beat in a read prefetch burst.

Table 4.11-6 Block outputs of MpmcStTSM

4.11.4 Static memory timing control block

The static memory timing control block (MpmcStTimCntl) is used for the purpose of tracking the static memory read access time completion, write access time completion and access timings for extended wait enabled devices. The chip select assertion to write enable assertion and the chip select assertion to output enable assertion delay is also

tracked by counters in the static memory timing control block. The turnaround cycles to be inserted after a read access are also tracked in this module.

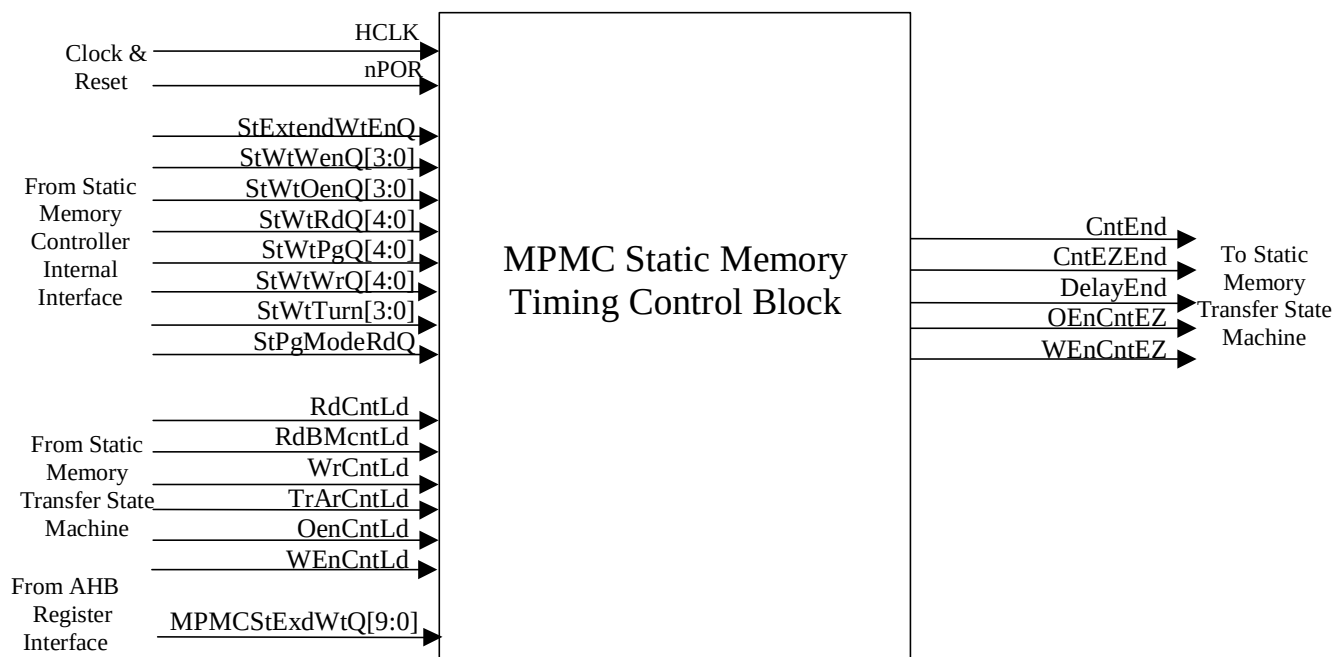


Figure 4.11-5 Block Inputs and Outputs for MpmcStTimCntl

The input and output ports of MpmcStTimCntl are described in **Table 4.11-7** and **Table 4.11-8**.

Signal Name	Source	Description
HCLK	Clock Generator	AHB clock
nPOR	Reset Generator	Power on reset
MPMCSStExdWtQ[9:0]	AHB register interface	Value in the register in the AHB register interface that contains the access time count for extended wait enabled devices.
StPgModeRdQ	Static memory controller internal interface	A high on StPgModeRdQ indicates that the current static memory access is to a page mode enabled static memory device.
StExtendWtEnQ	Static memory controller internal interface	A high on StExtendWtEnQ indicates that the current static memory access is to an extended wait enabled static memory device.
StWtWenQ[3:0]	Static memory controller internal interface	Indicates the chip select to write enable delay for the static memory device being accessed.
StWtOenQ[3:0]	Static memory controller internal interface	Indicates the chip select to output enable delay for the static memory device being accessed.
StWtWrQ[4:0]	Static memory controller internal interface	Indicates the write access time for the static memory device being accessed.
StWtRd[4:0]	Static memory controller internal interface	Indicates the normal mode read access time for the static memory device being accessed.

Signal Name	Source	Description
StWtPgQ[4:0]	Static memory controller internal interface	Indicates the page mode read access time for the static memory device being accessed.
StWtTurn[3:0]	Static memory controller internal interface	Indicates the number of turnaround cycles to be inserted between reads to different chip selects or between a read and a write.
RdCntLd	Static memory transfer state machine	When RdCntLd is asserted, the timer counter is loaded with the count value for a normal read access to static memory.
RdBMcntLd	Static memory transfer state machine	When RdBMcntLd is asserted, the timer counter is loaded with the count value for a page mode read access.
WrCntLd	Static memory transfer state machine	When WrCntLd are asserted, the timer counter is loaded with the count value for a write access.
TrArCntLd	Static memory transfer state machine	When TrArCntLd is asserted, the timer counter is loaded with the count value for the number of turnaround cycles that must be inserted after a read access to a static memory.
OenCntLd	Static memory transfer state machine	When OenCntLd is asserted, the delay counter is loaded with the output enable assertion delay value.
WenCntLdis	Static memory transfer state machine	When WenCntLdis is asserted, the delay counter is loaded with the write enable assertion delay value.

Table 4.11-7 Block inputs of MpmcStTimCntl

Signal Name	Destination	Description
CntEnd	Static memory transfer state machine, Static memory external interface	Indicates completion of the read or write access time or turnaround time. The expiry of the timer counter is used in the TSM to detect the end of a memory access.
CntEZEnd	Static memory transfer state machine, Static memory external interface	Timer counter expiry signal to be used by the TSM when the access time or turnaround count values are zero.
DelayEnd	Static memory transfer state machine	A high on DelayEnd indicates the completion of the chip select assertion to enable delay for nMPMCWEOUT and nMPMCOEOUT. The expiry of the delay counter is used in the TSM to detect the end of a output enable or write enable delay.
OEnCntEZ	Static memory transfer state machine	Indicates that the chip select assertion to output enable delay value is equal to zero.
WenCntEZ	Static memory transfer state machine	Indicates that the chip select assertion to write enable delay value is equal to zero.

Table 4.11-8 Block outputs of MpmcStTimCntl

4.11.5 Static memory controller external interface

The static memory controller external interface (MpmcStExtIf) provides the interface with the external static memory devices for facilitating the read and write transactions. The device control signals, the address and data paths are generated based on the signals from the transfer state machine. When the external memory bus is granted to the static memory controller, the signals of the external memory bus which are shared by the static and dynamic memory controllers are driven to their inactive State.

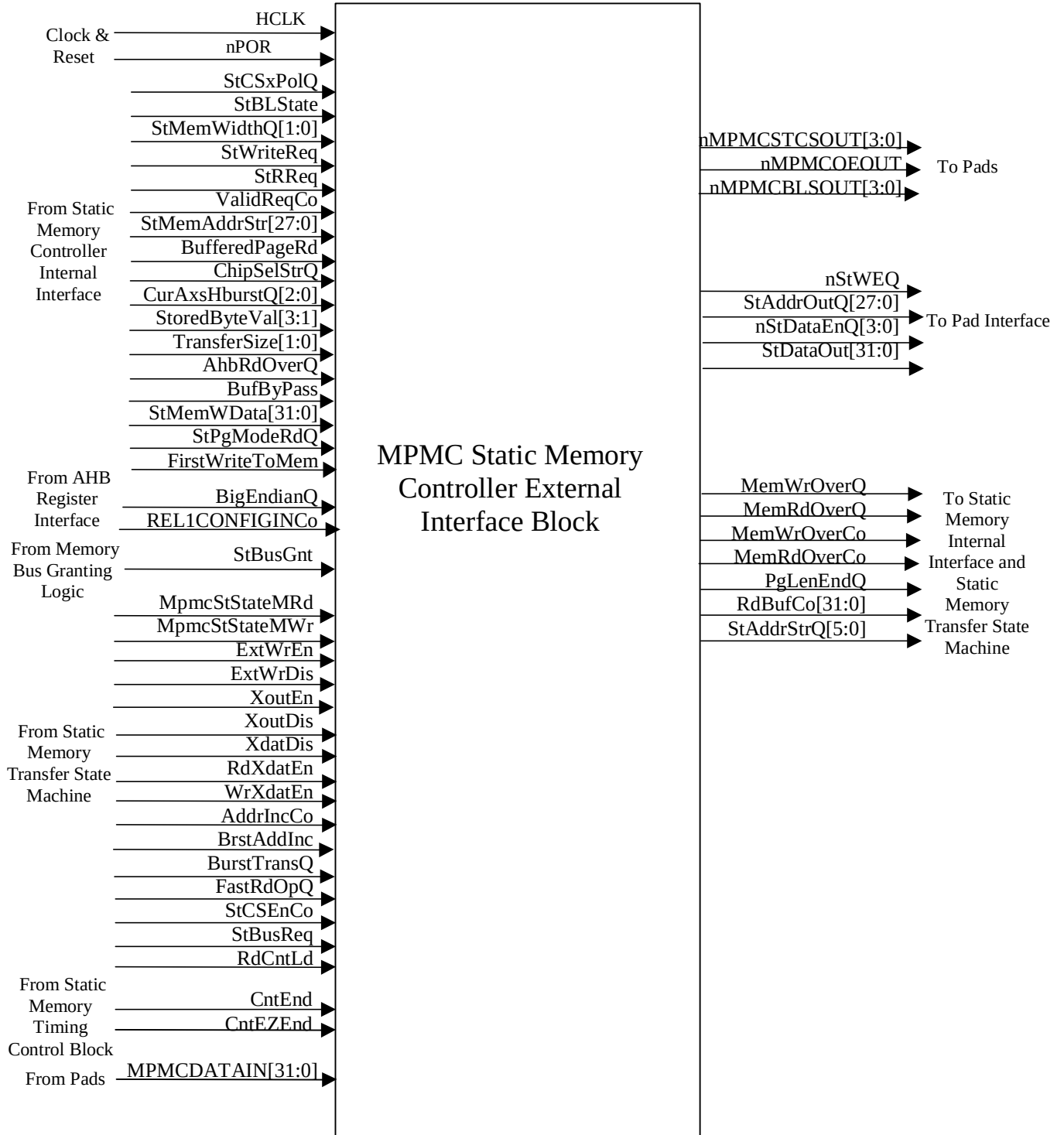


Figure 4.11-6 Block Inputs and Outputs for MpmcStExtIf

The input and output ports of MpmcStExtIf are described in *Table 4.11-9* and *Table 4.11-10*.

Signal Name	Source	Description
HCLK	Clock Generator	AHB clock
nPOR	Reset Generator	Power on reset
ValidReqCo	Static memory controller internal interface	Indicates a valid read or write access request.
StWriteReq	Static memory controller internal interface	Indicates that a valid static memory write request from the AHB or a buffer is being serviced by the static memory controller.
StRReq	Static memory controller internal interface	A high on StRReq indicates the first clock cycle of a valid read request from an AHB or a buffer.
StBLState	Static memory controller internal interface	Indicates the byte lane state of the static memory device being accessed. A low on this signal indicates that the nMPMCBLsOUT[3:0] will be connected to the write enable pins of the static memory device. In such a case, nMPMCBLsOUT[3:0] is driven with the write enable timings.
ChipSelStrQ[3:0]	Static memory controller internal interface	Indicates the chip select that is being accessed. The bit corresponding to the static memory device that is being accessed is set and all the other bits of ChipSelStrQ[3:0] are cleared.
StCS[3:0]PolQ	Static memory controller internal interface	Indicates the chip select polarity for the memory devices connected to the static memory chip selects 3, 2, 1 or 0.
StMemAddrStr[27:0]	Static memory controller internal interface	Indicates the memory address for a valid memory access request.
BufferedPageRd	Static memory controller internal interface	A high on BufferedPageRd indicates that the static memory read access that is being serviced is a buffered access to a page mode enable device.
CurAxsHburstQ[2:0]	Static memory controller internal interface	HBURST type for unbuffered accesses. This signal is used along with BigEndianQ, StMemWidth[1:0] and TransferSize[1:0] to generate the address of the next memory access in advance during a AHB burst of reads.
StoredByteVal[3:1]	Static memory controller internal interface	Indicates the valid bytes in the current static memory write access. It is used to enable or disable the byte lanes (nMPMCBLsOUT[3:0]) during writes to the static memory device.
AhbRdOverQ	Static memory controller internal interface	A high on AhbRdOverQ indicates that the AHB has finished reading all the data that was fetched from the memory during the previous read access to the memory.
BufferedWrQ	Static memory controller internal interface	A high on BufferedWrQ indicates that the 32-bit buffer has enough data to write into the memory.
BufByPass	Static memory controller internal interface	Asserted when TransferSize is greater than memory width for the current access.
StPageModeRdQ	Static memory controller internal interface	Used to differentiate between normal reads and page mode reads.

Signal Name	Source	Description
StMemWData[31:0]	Static memory controller internal interface	Write data for the current static memory write access.
FirstWriteToMem	Static memory controller internal interface	Indicates the first write in a write burst.
BigEndianQ	AHB register interface	The value on BigEndianQ indicates the endian mode of the MPMC. This signal is used for advance generation of the next address in an AHB burst of reads.
REL1CONFIGINCo	AHB register interface	Indicates the static memory addressing mode. When high, the static memory address is not LSB aligned for 16-bit and 32-bit memory width static memory devices. When low, it is LSB aligned irrespective of the static memory width.
StBusGnt	Memory bus granting logic block	Used in the static memory controller external interface to drive inactive values on the shared external memory bus signals when the static memory controller is not granted the control of the external memory bus.
StCSEnCo	Static memory transfer state machine	The relevant bit of nMPMCSTCSOUT[3:0] is asserted when StCSEnCo is sampled high. It is kept asserted when as long as StCSEnCo is asserted.
ExtWrEn	Static memory transfer state machine	The static memory write enable, nStWEQ is asserted when ExtWrEn is asserted.
ExtWrDis	Static memory transfer state machine	The static memory write enable, nStWEQ is de-asserted when ExtWrDis is asserted.
XoutEn	Static memory transfer state machine	The output enable, nMPMCOEOUT is asserted when XoutEn is asserted.
XoutDis	Static memory transfer state machine	The output enable, nMPMCOEOUT is de-asserted when XoutDis is asserted.
XdatDis	Static memory transfer state machine	Used by the static memory controller external interface to de-assert the nMPMCDATAEN[3:0] output lines. During reads, only the valid byte lanes are disabled while the other bits of nMPMCDATAEN[3:0] are enabled. During writes, all the bits of nMPMCDATAEN[3:0] are asserted.
RdXdatEn	Static memory transfer state machine	Static memory read access timings are indicated by asserting RdXdatEn.
WrXdatEn	Static memory transfer state machine	Static memory write timings are indicated by asserting WrXdatEn.
AddrIncCo	Static memory transfer state machine	When AddrIncCo is sampled high, the static memory address is incremented by a value corresponding to the memory width of the accessed device.
BrstAddInc	Static memory transfer state machine	When BrstAddInc is sampled high, the static memory address is incremented in advance during AHB burst reads.
BurstTransQ	Static memory transfer state machine	This advance address is driven on the static memory address bus at the end of a read only if BurstTransQ is asserted high indicating an AHB burst read.

Signal Name	Source	Description
FastRdOpQ	Static memory transfer state machine	If FastRdOpQ is asserted high, the transfer size is less than the memory width and the read data for multiple AHB reads are returned with a single read to the static memory device. So the incremented address for AHB burst reads is driven on the static memory address bus only after the read data from the previous static memory read access has been read by the AHB.
MpmcStStateMRd	Static memory transfer state machine	A high on MpmcStStateMRd indicates that the transfer State Machine's current State value is ST_TSM_MEMRD. This signal is used to identify the read access duration.
MpmcStStateMWr	Static memory transfer state machine	A high on MpmcStStateMWr indicates that the transfer State Machine's current State value is ST_TSM_MEMWR. This signal is used to identify the write access duration.
StBusReq	Static memory transfer state machine	StBusReq is used along with StBusGnt to detect whether a valid static memory access is in progress.
StBusGnt	Static memory transfer state machine	StBusGnt is used along with StBusReq to detect whether a valid static memory access is in progress.
RdCntLd	Static memory transfer state machine	When RdCntLd is asserted, the timer counter is loaded with the count value for a normal read access to static memory. This signal is used in the static memory external interface block to clear PgLenEndQ.
CntEnd	Static memory timing control block	Used to detect the end of the access to the static memory.
CntEZEnd	Static memory timing control block	Used to detect the end of the access to the static memory when the count value is access timer count load value is zero.
MPMCDATAIN[31:0]	Pads	Contains the read data from the memories. The valid data from the memory is rearranged on the appropriate byte lanes depending on the memory width and driven to the static memory controller internal interface as RdBufCo[31:0]

Table 4.11-9 Block inputs of MpmcStExtIf

Signal Name	Destination	Description
nMPMCSTCSOUT[3:0]	Pads	Chip select of the static memory devices connected to chip selects 3, 2, 1 and 0.
nMPMCOEOUT	Pads	Output enable of the static memory devices and
nMPMCBLSOUT[3:0]	Pads	Byte lane select of the static memory devices
StAddrQ[27:0]	Pad interface block	Static memory address output registered out on HCLK.
StWEQ	Pad interface block	Static memory write enable
nStDataEn[3:0]	Pad interface block	Static memory data enable

Signal Name	Destination	Description
StDataOut[31:0]	Pad interface block	Static memory write data output
MemWrOverQ	Static memory transfer state machine, static memory controller internal interface	Indicates the end of a write access to the memory. This signal is used by the TSM to move into the appropriate state at the end of a write access to the static memory.
MemRdOverQ	Static memory transfer state machine, static memory controller internal interface	Indicates the end of a read access to the memory. This signal is used by the TSM to move into the appropriate state at the end of a read access to the static memory.
MemRdOverCo	Static memory transfer state machine, static memory controller internal interface	D-input of MemRdOverQ.
PgLenEndQ	Static memory transfer state machine	Indicates the end of a page in the page mode enabled static memory device that is being accessed. This signal is used to time the read accesses during burst reads to page mode enabled devices.
MemWrOverCo	Static memory controller internal interface	D-input of MemWrOverQ.
RdBufCo[31:0]	Static memory controller internal interface	Read data from the static memory. It is generated combinatorially from MPMCDATAIN[31:0].
StAddrStrQ[1:0]	Static memory controller internal interface	Used to indicate the byte being accessed by the current static memory access.

Table 4.11-10 Block outputs of MpmcStExtIf

4.12 Memory Controller Interface

This block (MpmcMemCntlIf) provides the interface between the dynamic and static memory controllers and the AHB interface and buffers. This block contains the logic for combining the response signals from the dynamic and static memory controllers to provide a single set of signals to the AHB interface and buffer blocks.

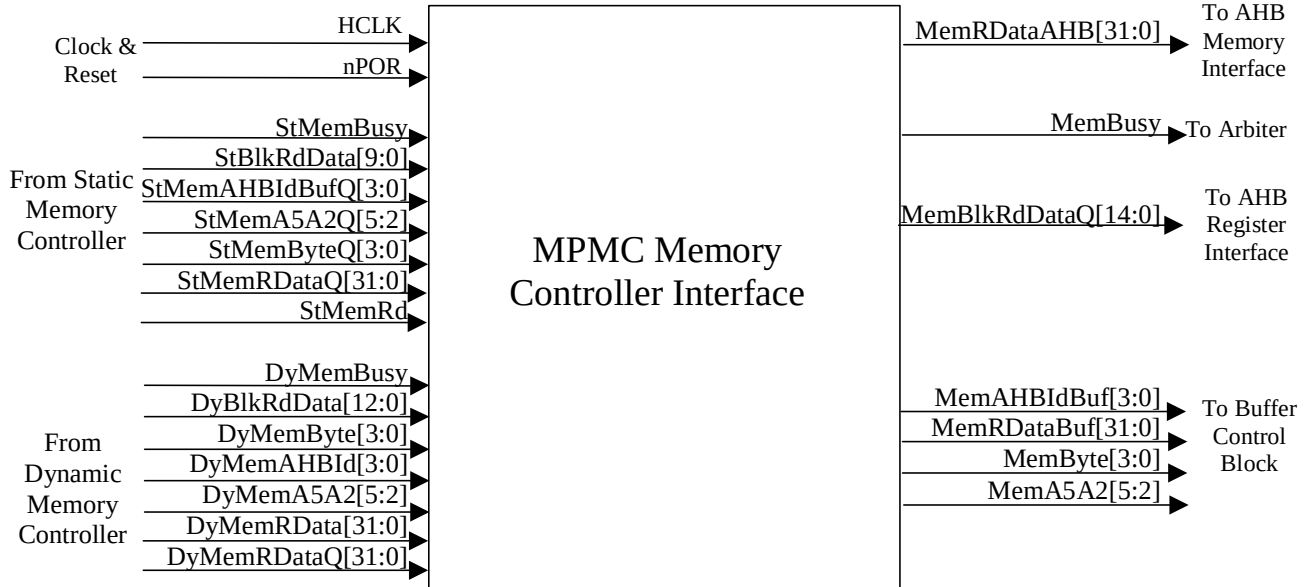


Figure 4.12-1 Block Inputs and Outputs for MpmcMemCntlIf

The input and output ports of MpmcMemCntlIf are described in **Table 4.12-1** and **Table 4.12-2**.

Signal Name	Source	Description
HCLK	Clock Generator	AHB clock
nPOR	Reset Generator	Power on reset
StMemBusy	Static memory controller	Asserted high to indicate that the static memory controller is busy servicing a static memory access request.
StBlkRdData[9:0]	Static memory controller	Combined read data bus for the four static memory register blocks.
StMemAHBIdBufQ[3:0]	Static memory controller	Indicates the buffer that was the requestor for the current static memory access.
StMemA5A2Q[5:2]	Static memory controller	Indicates the word address of the current static memory access.
StMemByteQ	Static memory controller	Indicates the valid bytes of data returned on reads from static memories.
StMemRDataQ[31:0]	Static memory controller	Read data to be returned to the buffers and AHB interfaces from the static memories.
StMemRd	Static memory controller	Mux select for the mux used to drive either the static memory read data or the dynamic memory read data on the combined read data bus to the buffers and AHB interfaces.
DyMemBusy	Dynamic memory controller	Asserted high to indicate that the dynamic memory controller is busy servicing a dynamic memory access request.

Signal Name	Source	Description
DyBlkRdData[12:0]	Dynamic memory controller	Combined read data bus for the four dynamic memory register blocks.
DyMemAHBId[3:0]	Dynamic memory controller	Indicates the buffer or AHB interface that was the requestor for the current static memory access.
DyMemA5A2[5:2]	Dynamic memory controller	Indicates the word address of the current dynamic memory access.
DyMemByteQ[3:0]	Dynamic memory controller	Indicates the valid bytes of data returned on reads from dynamic memories.
DyMemRData[31:0]	Dynamic memory controller	Read data to be returned to the buffers from the dynamic memories.
DyMemRDataQ[31:0]	Dynamic memory controller	Registered version of DyMemRData[31:0], which is used in the AHB interfaces to generate HRDATA[31:0].

Table 4.12-1 Block inputs of MpmcMemCntlIf

Signal Name	Destination	Description
MemBlkRdDataQ[14:0]	AHB register interface	Combined read data bus from the static and dynamic register blocks.
MemBusy	Arbiter block	Combined memory controller busy indicator. It is used in the arbiter to generate the MpmcBusy global status bit.
MemRDataAHB[31:0]	AHB memory interface	Combined memory read data bus to the AHB memory interfaces.
MemAHBIdBuf[3:0]	Buffer unit	Indicates the buffer that was the requestor for the memory access for which StUseMem or DyUseMem has been asserted.
MemRDataBuf[31:0]	Buffer unit	Combined read data bus to the buffers.
MemByte[3:0]	Buffer unit	Indicates the valid bytes in the combined memory read data.
MemA5A2[5:2]	Buffer unit	Indicates the word address of the read data being returned from the memory to the buffer.

Table 4.12-2 Block outputs of MpmcMemCntlIf

4.13 Memory Bus Granting Logic

This block (MpmcMemBGnt) receives the bus requests from the static and dynamic memory controllers and generates the bus grants depending on whether the memory bus is being used by the other controller.

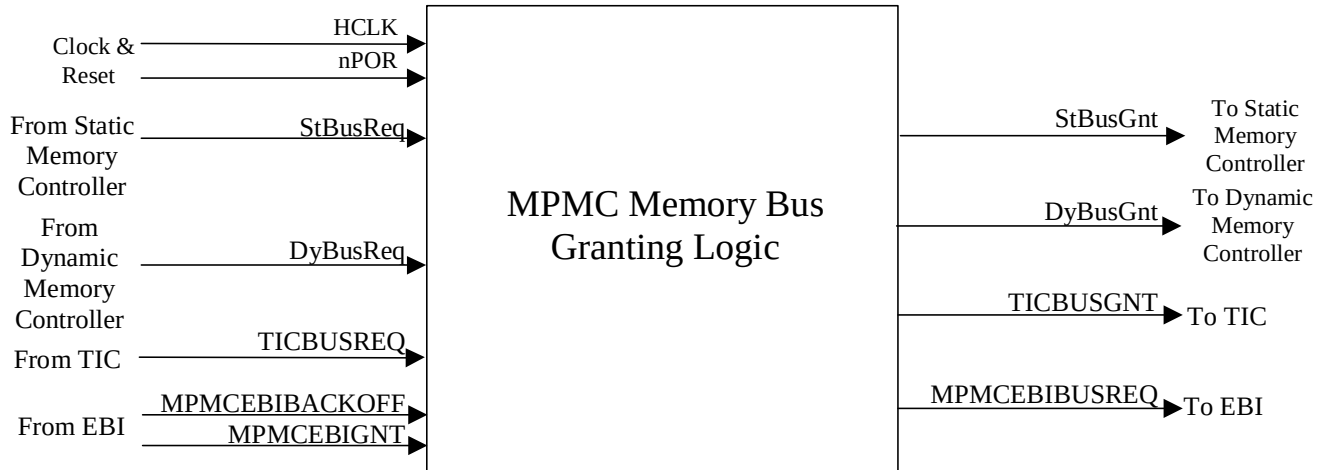


Figure 4.13-1 Block Inputs and Outputs for MpmcMemBGnt

The input and output ports of MpmcMemBGnt are described in **Table 4.13-1** and **Table 4.13-2**.

Signal Name	Source	Description
HCLK	Clock Generator	AHB clock
nPOR	Reset Generator	Power on reset
StBusReq	Static memory controller	Asserted by the static memory controller when it requires control of the external memory bus so that it can issue a static memory access.
DyBusReq	Dynamic memory controller	Asserted by the dynamic memory controller when it requires control of the external memory bus so that it can issue a dynamic memory access.
TICBUSREQ	MPMC Test Interface Controller	Asserted by the test interface controller when it require the control of the external memory bus.
MPMCEBIGNT	External Bus Interface	Asserted by the EBI when the MPMC is granted the control of the external memory bus.
MPMCEBIBACKOFF	External Bus Interface	Asserted by the EBI when some other master requires control of the external memory bus. When this signal is sampled high, the MPMC should relinquish the bus as soon as possible.

Table 4.13-1 Block inputs of MpmcMemBGnt

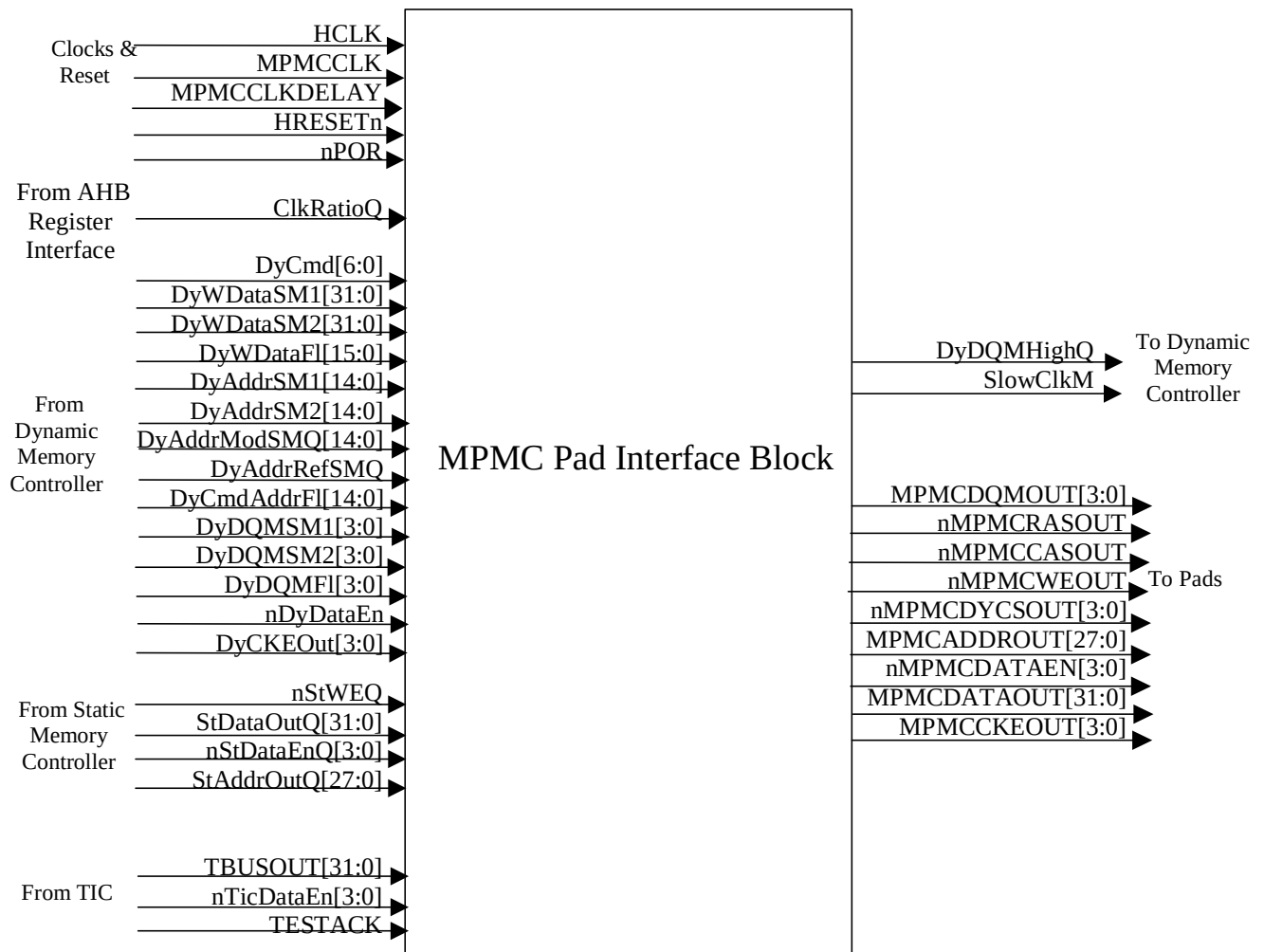
Signal Name	Destination	Description
StBusGnt	Static memory controller	Asserted to indicate that the static memory controller has control of the external memory bus.

Signal Name	Destination	Description
DyBusGnt	Dynamic memory controller	Asserted to indicate that the dynamic memory controller has control of the external memory bus.
TICBUSGNT	MPMC Test Interface Controller	Asserted to indicate that the TIC has control of the external memory bus.
MPMCEBIBUSREQ	External Bus Interface	Asserted to indicate that the MPMC requires control of the external memory bus.

Table 4.13-2 Block outputs of MpmcMemBGnt

4.14 Pad Interface

The pad interface block (MpmcPadIf) takes signals from the static memory controller, the dynamic memory controller and the TIC and drives them to the output pads of the MPMC. This block also generates SlowClkM signal corresponding to the slower of HCLK and MPMCCLK.

**Figure 4.14-1 Block Inputs and Outputs for MpmcPadIf**

The input and output ports of MpmcPadIf are described in **Table 4.14-1** and **Table 4.14-2**.

Signal Name	Source	Description
HCLK	Clock Generator	AHB clock
MPMCCLK	Clock Generator	Memory clock
MPMCCLKDELAY	Clock Generator	Delayed version of MPMCCLK
nPOR	Reset Generator	Power on reset
HRESETn	Reset Generator	AHB reset
ClkRatioQ	AHB register interface	Indicates the ratio of HCLK frequency to MPMCCLK frequency. This signal is used to generate SlowClkM.
SDRClkScheme[1:0]	AHB register interface	SDRAM clock scheme. When SDRClkScheme has a value "00", the MPMC follows a clock delayed mode of accessing the Dynamic memories.
nStWEQ	Static memory controller	Memory write enable signal for accesses to static memory devices.
StDataOutQ[31:0]	Static memory controller	Memory write data for writes to static memory devices.
nStDataEnQ[3:0]	Static memory controller	Data enable for static memory accesses
StAddrQ[27:0]	Static memory controller	Memory address for static memory accesses.
DyCmd[6:0]	Dynamic memory controller	Command code for dynamic memory accesses. The Pad interface decodes this signal into commands on the nMPMCDYCSOUT[3:0], nMPMCRASOUT, nMPMCCASOUT and nMPMCWEOUT.
DyWDataSM[2..1][31:0]	Dynamic memory controller	Memory write data buses from the DASM2, DASM1
DyWDataFI[15:0]	Dynamic memory controller	Memory write data bus from the SyncFlash State machine
DyAddrSM[2..1][14:0]	Dynamic memory controller	Memory address from DASM2 and DASM1
DyAddrModSMQ[14:0]	Dynamic memory controller	Memory address from mode state machine
DyAddrRefSMQ	Dynamic memory controller	Memory address from refresh state machine.
DyCmdAddrFI[14:0]	Dynamic memory controller	Memory address from SyncFlash state machine.
DyDQMSM[2..1][3:0]	Dynamic memory controller	DQM values from the DASM1, DASM2
DyDQMFI[3:0]	Dynamic memory controller	DQM values from the SyncFlash state machine
DyCKEOut[3:0]	Dynamic memory controller	CKE values for dynamic memory accesses.
nDyDataEn	Dynamic memory controller	Signal indicating whether nMPMCDATAEN[3:0] should be asserted or de-asserted during dynamic memory accesses.
TicDataEn[3:0]	MPMC Test Interface Controller	Tri-State I/O pad enable for the output data bus for TIC accesses.
TESTACK	MPMC Test Interface Controller	Test acknowledge from TIC. This signal is to be driven on nMPMCWEOUT
TBUSOUT	MPMC Test Interface Controller	Test bus output from TIC. This signal will be driven on MPMCDATAOUT[31:0]

Table 4.14-1 Block inputs of MpmcPdif

Signal Name	Destination	Description
SlowClkM	Dynamic memory controller	Indicates the slower of HCLK and MPMCCLK. This is used by the dynamic memory controller to ensure that the dynamic memory commands are launched on the correct phase of the clock.
DyDQMHIGHQ	Dynamic memory controller	Asserted to indicate that at least one bit of MPMCDQMOUT[3:0] has been asserted.
MPMCCKEOUT[3:0]	Pads	CKE inputs to the four dynamic memory chip selects.
MPMCDQMOUT[3:0]	Pads	DQM input to the dynamic memories controlled by the MPMC.
nMPMCRASOUT	Pads	Row address strobe of the dynamic memory.
nMPMCCASOUT	Pads	Column address strobe of the dynamic memory.
nMPMCDYCSOUT[3:0]	Pads	Dynamic memory chip selects for the dynamic memories controlled by the MPMC.
nMPMCWEOUT	Pads	Write enable to the static and dynamic memories. For the TIC interface, it is the TESTACK signal.
MPMCADDRROUT[27:0]	Pads	Address lines of the static and dynamic memories connected to the MPMC.
MPMCDATAOUT[31:0]	Pads	Write data for the static and dynamic memories. For TIC, it is the TBUSOUT[31:0] signal.
nMPMCDATAEN[3:0]	Pads	Tri-State I/O pad enable for the write data bus, MPMCDATAOUT[31:0]. It is shared by the static memory controller, the dynamic memory controller and the TIC

Table 4.14-2 Block outputs of MpmcPdif

4.15 Clock gating logic

This module (MpmcClockOr) contains a single OR gate to be used for clock-gating to control the enabling of the clock output to the synchronous memories. The clocks to the memory devices are pulled high when they are not being accessed in order to save power. The ClkStpd signal from the dynamic memory controller is used to control the output clocks. This module is instantiated four times, one for each bit of MPMCCLKOUT[3:0]

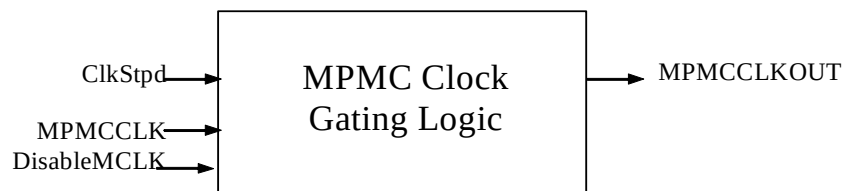


Figure 4.15-1 Block Inputs and Outputs for MpmcClockOr

The input and output ports of MpmcClockOr are described in **Table 4.15-1** and **Table 4.15-2**.

Signal Name	Source	Description
MPMCCLK	Clock Generator	MPMC Clock
ClkStpd	MPMC Core logic	Signal to disable the clock to the dynamic memories when the dynamic memory is not being accessed.
DisableMCLK	MPMC Core logic	Software control bit to disable the clock to dynamic memories.

Table 4.15-1 Block inputs of MpmcClockOr

Signal Name	Destination	Description
MPMCCLKOUT	Pads	Clock output to dynamic memories connected to the MPMC. The clock output of instantiation 0 is to be connected to the dynamic memory device connected to chip select 4, instantiation 1 is for chip select 5, instantiation 2 is for chip select 6 and instantiation 4 is for chip select 7

Table 4.15-2 Block outputs of MpmcClockOr

4.16 Revision Designator Block

This module (MpmcRevAnd) contains a single AND gate to be used as a place-holder cell to mark the Revision of the controller. The 2 input pins are tied-off at the top level of the hierarchy. These "TieOffs" can be identified during layout and re-wired to "VDD" or "VSS" if needed.

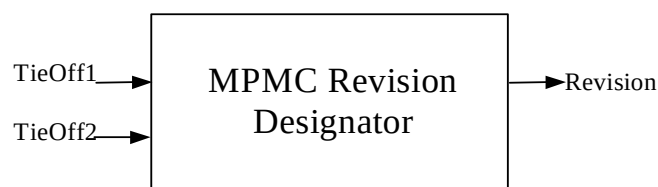


Figure 4.16-1 Block Inputs and Outputs for MpmcRevAnd

The input and output ports of MpmcRevAnd are described in **Table 4.16-1** and **Table 4.16-2**.

Signal Name	Source	Description
TieOff[2..1]	MPMC top-level block	Signals to indicate the values to be used to generate the revision number

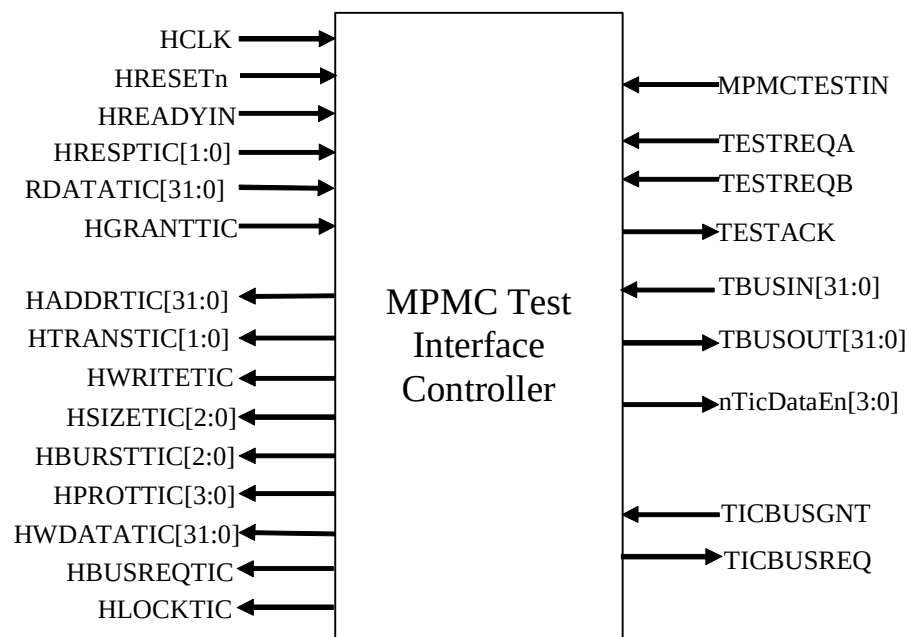
Table 4.16-1 Block inputs of MpmcRevAnd

Signal Name	Source	Description
Revision	MPMC Core	Revision number information to be used in the PrimeCell identification registers.

Table 4.16-2 Block outputs of MpmcRevAnd

4.17 Test Interface Controller

The MPMC TIC bus master module (MpmcTIC) helps to allow externally applied test vectors to be converted into internal AHB transfers with the appropriate sequence. The TIC controls the External Test Bus and AHB data buses HRDATATIC and HWDATATIC during the application of Test Interface Format (TIF) test vectors

**Figure 4.17-1 Block Inputs and Outputs of MpmcTIC**

The input ports of MpmcTIC are described in **Table 4.17-1** and the outputs are described in **Table 4.17-2**.

Signal Name	Source	Description
HCLK	Clock Generator	AHB clock
HRESETn	Reset Generator	AHB reset

Signal Name	Source	Description
HRESPTIC[1:0]	AHB slave	The transfer response provides additional information on the status of a transfer. Four different responses are supported: OKAY, ERROR, RETRY and SPLIT. To indicate that the transfer has completed successfully, the slave uses the OKAY response. To indicate that an error has occurred, the slave uses the ERROR response. To indicate that the data is not ready, the slave uses the RETRY and SPLIT responses.
HREADYINTIC	AHB slave	When HIGH the HREADYINTIC signal indicates that a transfer has finished on the bus. This signal may be driven LOW to extend a transfer, by the AHB slave.
HRDATATIC[31:0]	AHB slave	The read data bus is used to transfer data from bus slaves to the bus master during read operations
HGRANTTIC	AHB arbiter	Grant signal generated by the arbiter. This signal indicates that the MPMC is currently the highest priority master requesting the bus. The MPMC gains bus ownership when HGRANTTIC and HREADYINTIC are high on the rising edge of HCLK.
TESTREQA	Ticbox	During test, this signal is used to indicate the type of test vector that will be applied in the following cycle. The signal gives indication that the test bus has been granted and completion of a test access. When TESTACK is LOW, the current test vector must be extended until TESTACK becomes HIGH.
TESTREQB	Ticbox	During test, this signal is used, in combination with TESTREQA, to indicate the type of test vector that will be applied in the following cycle. The signal gives indication that the test bus has been granted and completion of a test access. When TESTACK is LOW, the current test vector must be extended until TESTACK becomes HIGH.
TBUSIN	Ticbox	This signal is used to place the MPMC into TIC test mode. The MPMC is placed in the TIC test mode by asserting MPMCTESTIN HIGH during HRESETn assertion.
MPMCTESTIN	Pads	This signal is used to enter TIC test mode. When MPMCTESTIN is asserted high during a reset, the MPMC enters TIC test mode.
TICBUSGNT	MPMC Core logic	This signal indicates that the TIC has control of the external memory bus. This signal is generated from MPMCEBIGNT.

Table 4.17-1 Block Inputs of MpmcTIC

Signal Name	Destination	Description
HADDRTIC[31:0]	AHB Slave, Decoder	Indicates the AHB address for the transfer initiated by the AHB interface MPMC's TIC.
HTRANSTIC[1:0]	AHB Slave	Indicates the transfer type for the transfer initiated by the MPMC's TIC module.

Signal Name	Destination	Description
HWRITETIC	AHB Slave	Indicates whether the transfer initiated on the AHB is a read or write transfer.
HSIZETIC[2:0]	AHB Slave	Indicate the size of the data for the AHB transfer.
HBURSTTIC[1:0]	AHB Slave	Indicates the burst type for the AHB transfer
HPROTTIC[3:0]	AHB Slave	Indicates the protection control for the AHB transfer.
HWDATATIC[31:0]	AHB Slave	Write data bus for the AHB master interface.
HBUSREQTIC	AHB Arbiter	AHB bus request signal to the AHB arbiter.
HLOCKTIC	AHB Arbiter	If HLOCKTIC is asserted, it indicates that the MPMC's TIC should not be degraded by the AHB arbiter
TBUSOUT[31:0]	MPMC Core	Output data bus.
nTicDataEn[3:0]	MPMC Core	Data enable for the data on TBUSOUT[31:0].
TESTACK	MPMC Core	Test acknowledge signal in test mode. This signal shares the nMPMCWEOUT pin with the static and dynamic memory controllers.
TICBUSREQ	MPMC Core	Indicates that TIC requires control of the external memory bus. This signal is used to generate MPMCEBIREQ

Table 4.17-2 Block outputs of MpmcTIC

5 MPMC IMPLEMENTATION DETAILS

In this section, the implementation details of the blocks listed in section 4 are provided.

5.1 MPMC Top Level Module (Mpmc)

The top-level module is a structural block which instantiates and interconnects the two main functional blocks in the MPMC – the MPMC Core logic and the MPMC TIC interface. It also contains the revision designator block and the clock gating logic for the output clock to the synchronous memories. All the functional blocks in the MPMC are described in the following sections.

5.2 MPMC Core (MpmcCore)

The MPMC Core logic block is a structural block containing the blocks that facilitate the MPMC's memory controller functionality. It contains the AHB slave interfaces for memory accesses and the AHB slave interface for accesses to the MPMC registers. An arbiter block arbitrates between the AHB slave memory interfaces. Four quad-word buffers and their control logic are provided to enhance the performance of the MPMC. The memory controller block containing the static memory controller and the dynamic memory controller issues the proper sequence of commands to read and write into the memories. The pad interface block registers the commands to the dynamic memory controller and combines the output pins that are shared by the static memory controller, the dynamic memory controller and the TIC. The following sections 5.4 to section 5.13 contain a detailed description of all the blocks in MPMCCore.

5.3 MPMC AHB Memory Interfaces Top Level (MpmcAhbTop)

AHB top level provides an interface between MPMC and AMBA AHB buses. A Simplified block diagram of AHB top level is shown in **Figure 5.3-1 AHB top level block diagram**. The AHB top level block can support 8 AHB interfaces. MPMC PL172 contains only 4 AHB interfaces. So the unused signals corresponding to the remaining AHB interfaces are tied to their inactive values at the AHB top level block. The AHB top level is a structure module containing the following sub modules

- 4 instances of AHB interface block, one for each AHB port. Each of these AHB interface blocks provides the interface between one AHB port and the MPMC.
- Endian block. The endian block is used to endianise the write data to the memory and to generate the valid byte information to access the quadword buffers or the memory devices. This block also selects the current access information from the AHB that has been selected by the Arbiter block.

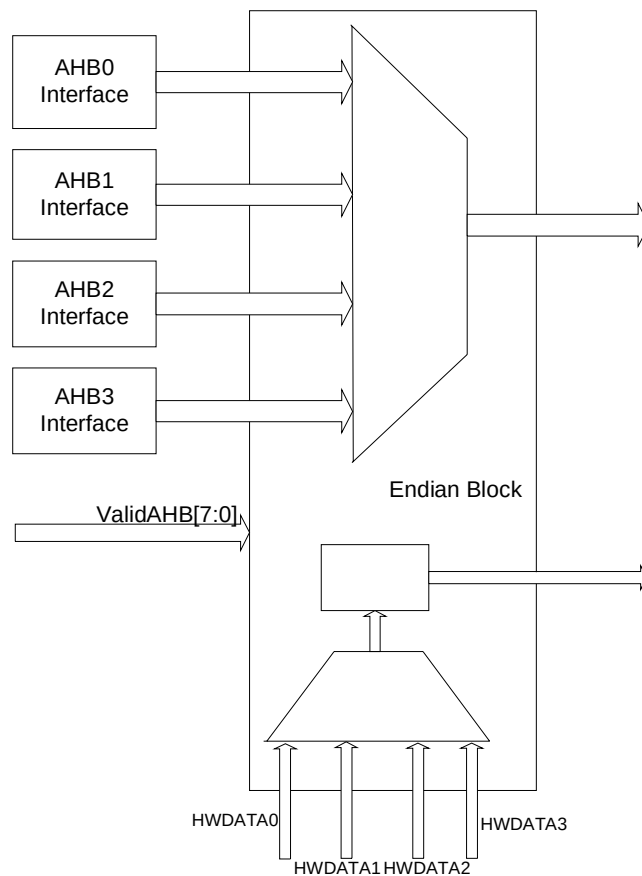


Figure 5.3-1 AHB top level block diagram

The AHB top level block has direct interaction with the following sub-modules in the MPMC: AHB Register interface, Arbiter block, Buffer Unit and Memory control blocks.

A general data flow can be explained as follows. Software has to program the MPMC registers in the required configuration before using it for memory data transfer. Each AHB interface instance can read the relevant register information programmed through the AHB register interface block. Whenever a valid AHB master request is sampled on an AHB, the relevant AHB interface asserts a memory transfer request to the arbiter block. The Arbiter block acknowledges the request with a grant signal. The Arbiter uses the priority logic for the grant generation. MPMC can support only one AHB transfer at a time. Once the arbiter block selects an AHB interface, that AHB has the control to stream data through the buffers or the memory controller blocks. The AHB interface inserts wait states in the transfer if the MPMC is not ready to perform the requested data transfer. AHB interfaces can perform data transfer with the buffer unit or the memory controller block. Once the buffer unit or the memory controller are ready to perform the data transfer, it provides an acknowledge signal to the AHB interface. The AHB interface generates the OKAY response using these acknowledge signals and asserts the HREADYOUT signal to end the transfer on the AHB side.

5.3.1 Ahb Memory Interface Block (MpmcAhbif)

The AHB memory interface block provides the interface between one AHB port and the quadword buffers or the memory controller block. A detailed block diagram of AHB interface is shown in **Figure 5.3-2 AHB interface block diagram**.

Following functionality is implemented in this block:

- Address and control signal registering: The AHB address and control signals are registered on the rising edge of HCLK in this block and the registered signals are used for generating the memory access requests.

- Memory access request generation: The memory access requests from the AHB are processed and the valid requests are passed on to the quadword buffer or memory controller.
- Predictive address generation: The next address to be accessed is generated in order to be able to write sequential data into the buffer or read sequentially from the buffer without any wait states.
- Valid byte indicator logic: The valid bytes for a read or write access to the quadword buffers or memory are generated based on the endian mode of the MPMC.
- Read data selection and packing: The read data is selected and packed depending on the endian mode.
- Error monitoring: The error generating conditions are checked and errors are flagged to the AHB master
- Response generation: The AHB response generation on the HRESP[1:0] lines are done in this block.

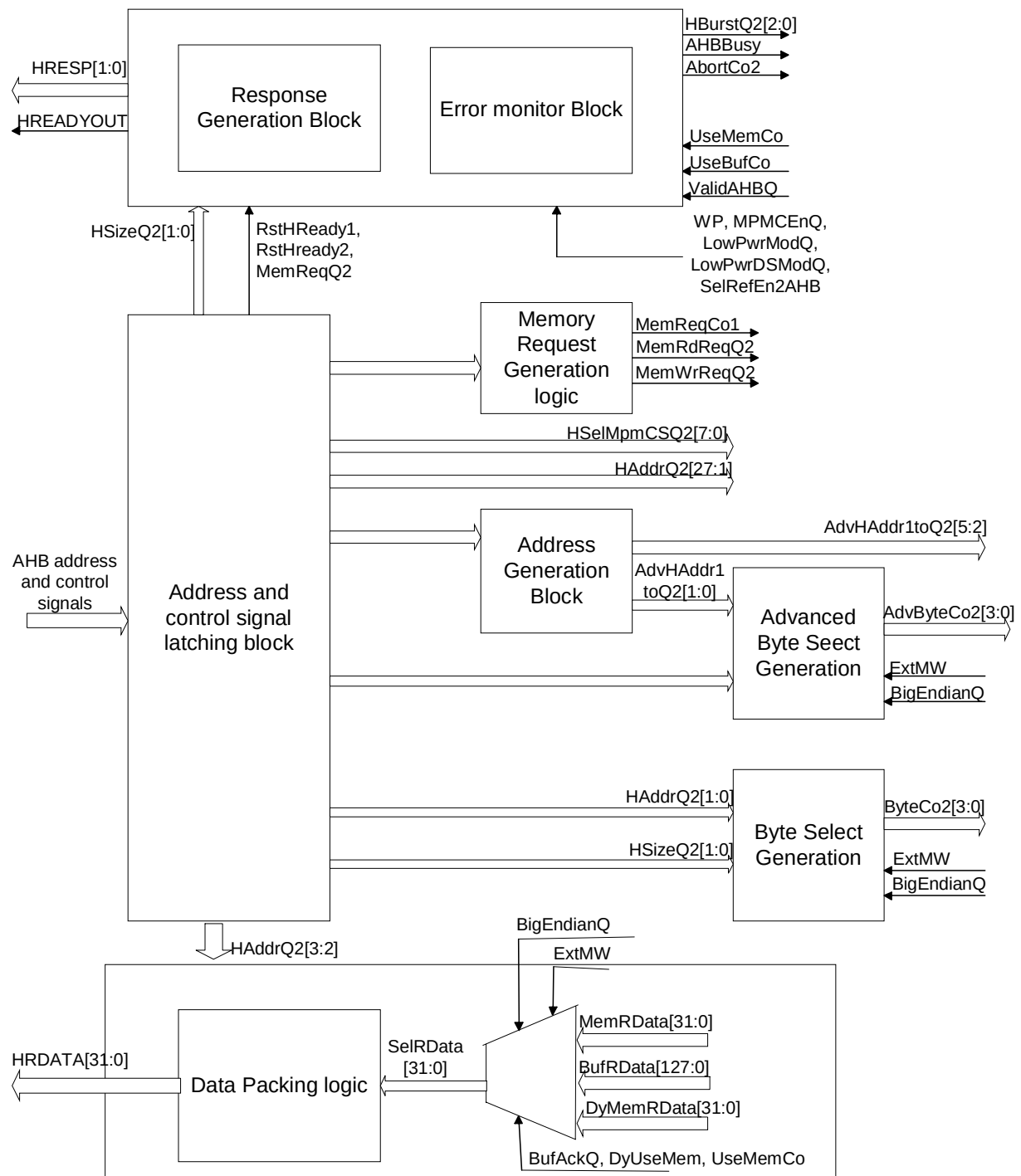


Figure 5.3-2 AHB interface block diagram

Address and control signals latching

This block samples the address and control signals and gives to the other sub modules inside the MPMC. Following signals - HAddrQ2, HSizeQ2, HBurstQ2, HSelMpmcCSQ2, HMAstLockQ2 - are generated in this block. The AHB address and control signals are sampled only when HREADYIN and HSELMPMC are high. HAddrQ2 is the registered

version of AHB address bus. This signal is generated by sampling HADDR when HREADYIN and HSELMPMC are high. HSizeQ2, HBurstQ2, and HMAstLockQ2 are generated similarly by sampling the AHB signals HSIZE, HBURST and HMASTLOCK when HREADYIN and HSELMPMCG are high.

HSELmpmcCSQ2 [7:0] is generated by sampling HSELMPMCCS [7:0] when HREADYIN is high. HSELMPMCCS sampling is implemented in two stages. In the first stage HSELMPMCCS[7:0] is sampled when HREADYOUT is high and an intermediate signal HSELmpmcCSCo1[7:0] is generated. In the second stage, HSELmpmcCSCo1[7:0] is sampled with HREADYIN and the combinational signal NxtHSELmpmcCS [7:0] is generated, which is then clocked with HCLK to create HSELmpmcCSQ2[7:0]. The combinational signal, HSELmpmcCSCo1[7:0] is used by the error generation logic. This two level sampling is done to meet synthesis timing for the write error generation logic. Note that the internally generated signal HREADYOUT is available much earlier than the HREADYIN signal from the AHB bus.

Address mirroring is also done at this level. If the Address mirror input from the AHB register interface is high, chip select 1 is mirrored onto chip select 0 and chip select 4. Software cannot access memory bank0 and memory bank4 if address mirror bit is high. All accesses to bank zero and bank four enables the chip select one.

Memory transfer request generation

This block asserts the memory data transfer requests whenever a valid data transfer request appears on the AHB. MemReqCo1, MemRdReqQ2, MemWrReqQ2 and RstBurstCo are the signals generated from this block.

A High on MemReqCo1 signal indicates that the AHB transfer is in a pending state or in an ongoing state. HREADYIN = '1', HSELMPMCG = '1' and HTRANS = NSEQ indicate the beginning of a new AHB burst. MemReqCo1 is set under this condition. Re-setting of MemReqCo1 is done under any of the following conditions

- HREADYOUT = '1' and HSELMPMCG = '0' indicating the end of data transfer.
- HREADYOUT = '1' and HTRANS = IDLE indicating that the AHB master wants to break the data transfer

The Arbiter block uses the MemReqCo1 to generate the AHB grant.

Memory read data transfer request, MemRdReqQ2, is set when

- HSELMPMCG = 1 and HREADYIN = 1 and HWRITE = 0 and HTRANS = not (IDLE) indicating that the AHB interface is selected for a read operation

MemRdReqQ2 is re-set under any of the following three conditions.

- HSELMPMCG = 1 and HREADYIN = 1 and HWRITE = 1 and HTRANS = not (IDLE) indicating that the AHB interface is selected for a write operation
- HSELMPMCG = 0 and HREADYIN = 1 indicating that the AHB interface is deselected after AHB master has finished the data transfer.
- HSELMPMCG = 1 and HREADYIN = 1 and HTRANS = IDLE indicating that the AHB master wants to break the data transfer.

Note that in case of direct memory read (case where an aligned increment read request is asserted by the AHB and the requested data is not present in the buffer) read request is de-asserted once the read request is posted to the memory control state machine.

Memory write data transfer request, MemWrReqCo1, is set when

- HSELMPMC = 1 and HREADYIN = 1 and HWRITE = 1 and HTRANS = not (IDLE) indicating that the AHB interface is selected for a write operation.

MemWrReqCo1 is re-set under any of the following three conditions.

- HSELMPMC = 1 and HREADYIN = 1 and HWRITE = 0 and HTRANS = not (IDLE) indicating that the AHB interface is selected for a read operation.

- HSELMPMC = 0 and HREADYIN = 1 indicating that the AHB interface is deselected after AHB master has finished the data transfer.
- HSELMPMC = 1 and HREADYIN = 1 and HTRANS = IDLE indicating that the AHB master wants to break the data transfer burst.

The signal RstBurstCo is used to indicate the end of a burst. The Arbiter block uses this pulse signal for re-arbitrating the AHB grant. This signal is set to high under the following conditions

- IDLE ("00") is sampled on the HTRANS lines when HREADY is high. This indicates that master want to break the data transfer burst.
- The select line (HSELMPMCG) is sampled as low when HREADY is high indicating the end of data transfer.
- NSEQ is sampled on the HTRANS line indicating that the AHB master has initiated a new AHB transfer burst. But in case of a fixed length burst - INCR4, INCR8, INCR16, WRAP4, WRAP8, WRAP16 – It is possible to predict the end of burst condition before NSEQ sampled on the HTRANS line.
- Address line is about to cross the QWord (16 word) boundary.
- An error condition has occurred during the data transfer

Predictive address generation

The MPMC supports zero wait state response to sequential memory accesses. By design, first memory access of a burst has a minimum of one wait state and the following sequential accesses can be zero wait state or delayed responses depending on the availability of the requested data and the AHB arbitration. In order to support a zero wait state response, the AHB interface block has to drive the response signals (HREADYOUT and HRESP) and the read data (HRDATA [31:0]) on the very next clock cycle after the AHB requested the data. According to AHB specification (Ref 1) HREADYOUT has to be stable within 40% of the clock period after the rising edge of the HCLK. Since this time may not be sufficient for the entire logic, advanced address generation is used.

Start of an AHB burst is indicated by the presence of NSEQ (non-sequential) on the HTRANS[1:0] lines. According to AHB specification, HSIZE and HBURST information will not change within an AHB burst. In addition, the remaining transfers in a burst are SEQ (sequential) and the address will be related to the previous address. This information is used by the address generation logic. The next address is equal to the present address plus the data transfer size (HSIZE in byte). Also, note that in case of WRAP type of bursts, the transfer address wraps at the wrapping boundary. Wrapping boundary is equal to the size (HSIZE in byte) multiplied by the number of beats in the transfer. This wrapping address is taken into account for calculating the next address in advance.

Valid byte indicator

ByteCo2 and AdvByteCo2 are generated in this block. MPMC read and write data buses are 32-bit wide. However, the MPMC supports 32-bit, 16-bit and 8-bit data transfers. Internally, the buffer unit and the memory controller blocks use 32-bit data buses. ByteCo2[3:0] is used to indicate the valid bytes in the 32-bit data bus. In case of a 32-bit AHB transfer, ByteCo2 [3:0] = "1111"; in the case of a 16-bit data transfer, ByteCo2 [3:0] can be "0011" or "1100"; and in the case of an 8-bit data transfer, ByteCo2 [3:0] can be "0001" or "0010" or "0100" or "1000". Generation of ByteCo2 [3:0] depends on the values of HSizeQ2[2:0], HAddr1to0Q2[1:0], the programmed memory width and the endian mode. Please refer to the MPMC PL172 TRM [Ref. 2] for more details on the byte selection logic based on memory width, endianness and HSIZE.

Generation of AdvByteCo2[3:0] is similar to that of ByteCo2 [3:0]. But AdvByteCo2[3:0] indicates the valid byte information of the next access. Generation of the AdvByteCo2 [3:0] depends on the HSIZE [1:0], AdvHAddr1to0Q2, the programmed memory width and the endian mode. This logic uses the next address generated in advance.

Read data selection and packing

Read data to the AHB interface can be served by the buffer unit or by the memory control unit. Buffer control logic sets the acknowledgement signal UseBufCo, if the requested data is available in the buffer.

If the requested data is not available in the buffer, buffer control logic post a read request to the memory control block. Memory control block fetches the requested data and gives it to the buffer. If the requested AHB burst is an aligned

fixed INCR burst, it is possible to improve the latency by providing a parallel read data path to the AHB interface while the data is being filled into the buffer. In all other cases data flow is from memory control logic to the buffer block and from buffer to the AHB interface. Direct memory fetch is performed for the following cases

1. HSIZE = WORD, HBURST= INCR4, HADDR [3:2] = 00
2. HSIZE = WORD, HBURST= INCR8, HADDR [4:2] = 000
3. HSIZE = WORD, HBURST= INCR4, HADDR [5:2] = 0000

In case of direct memory fetch DyUseMem signal is used as the acknowledgement. In case of unbuffered access UseMemCo is used as the acknowledgement

If the acknowledgement signal UseBufCo is high then the data is read from the vector BufRData [127:0]. AHB address line HAddr [3:2] is used for the selection of 32-bit read data from the 128-bit read data bus of the buffer unit. If the acknowledgement signal DyUseMem is high, AHB interface select the read data from vector DyMemRDataQ [31:0] or if the acknowledgement signal UseMemCo is high read data is selected from the vector MemRData [31:0]. An intermediate read data signal SelRData [31:0] is generated after this muxing between memory read data bus and the buffer read data bus.

HRDATA is generated from SelRData after performing the endianisation. Following logic is used for the endianisation of read data bus

- In little endian mode SelRData can be directly assigned to HRDATA irrespective of the memory width
- In big endian mode, if the programmed memory width is the same as AHB data width (32-bit) SelRData can be directly assigned to HRDATA
- In big endian mode, if the programmed memory width is 16-bit , data bus has to be swapped at 16-bit boundary i.e. HRDATA [31:0] <= SelRData[15:0] & SelRData[31:16]
- In big endian mode, if the programmed memory width is 8-bit , data bus has to be swapped at 8-bit boundaries i.e. HRDATA [31:0] <= SelRData[7:0] & SelRData[15:8] & SelRData[23:16] & SelRData[31:24]

Data re-circulation logic is implemented in the HRDATA path to reduce toggling of the read data bus.

Error monitoring

Error response is generated under the following conditions

- AHB Master request a transfer of HSIZE greater than 32
- Memory transfer is requested when the MPMC enable bit is '0'
- Memory transfer is requested when MPMC is in power down mode
- Memory transfer is requested when MPMC is in deep sleep mode
- Memory transfer is requested to the dynamic memory when MPMC is self refresh mode
- Master initiated a write transfer to a write protected area

Write protect checking and HSIZE checking are performed only in the first clock of the HBURST. Following assumptions are made at this stage

- HSIZE will not change within a burst
- AHB cannot cross from one memory bank to another bank within a single burst. (1K address boundary crossover restriction by the AMBA specification)

If any of the above mentioned error conditions happen during the data transfer, the error monitor block sets SetErrResp signal to high. Response generation block generates the ERROR response on AHB bus when a high is sampled on the SetErrResp signal.

Response generation

The MPMC AHB slave supports only two kinds of responses - ERROR and OKAY. OKAY response is the default response of the MPMC AHB slave if it is not selected. If the MPMC AHB slave port is selected, the slave can insert wait states if it is not ready for the data transfer. MPMC inserts WAIT under the following conditions.

- Initial response to any new read/write burst. A new burst is identified by the NSEQ on the HTRANS line.
- AHB interface is waiting for the acknowledgement signal from the buffer unit or the memory controller block. The buffer unit and the memory controller set input signals UseBufCo and UseMemCo when they are ready to perform the data transfer.
- When the AHB burst crosses the quad-word boundary. Signal QWCross is set when the AHB address crosses a quad word boundary.

This block generates an ERROR response on the HRESP lines when the signal SetErrResp from the error monitor block is sampled as high.

OKAY response is generated by setting HREADYOUT = '1' and HRESP [1:0] = "00".

WAIT states can be inserted by setting HREADYOUT = '0' and HRESP [1:0] = "00". In MPMC, wait state is terminated by ERROR response or OKAY response.

ERROR response is a two-cycle response. It is generated by

- Setting HREADYOUT = '0' and HRESP = "01" in the first HCLK cycle and
- Setting HREADYOUT = '1' and HRESP = "01" in the second HCLK cycle.

5.3.2 Endian Block (MpmcEndian)

The endian block is used to endianise the write data as well as to provide the common link between the multiple AHB interfaces and the buffer unit and arbiter blocks.

This block performs the following functionality

- AHB interface muxing: This is the logic used to select the signals from the AHB interface for the port that has been selected by the Arbiter.
- Write data endianisation: This logic endianises the write data to the quadword buffers or the memory controller based on the endian mode of the MPMC.

AHB interface muxing

MPMC endian block can support 8 AHB interfaces. Currently, the PL172 contains only 4 AHB interfaces. Unused pins are tied to zero at the AHB interface top level. Whenever a valid transfer to the MPMC is identified at the AHB bus, the AHB interface asserts the transfer request to the arbiter module. Depending on the priority of the requesting AHB, the arbiter allows the grant to one of the requesting AHBs. It is indicated by the 8-bit vector signals ValidAHBV1Q[7:0] and ValidAHBV2Q[7:0]. Functionality of these signals is the same. The MPMC arbiter generates two signals vectors to reduce the load on the signal and to avoid the delay caused by the load. MPMCEndian block uses these signals to multiplex the selected AHB signal from among the 8 AHB interfaces. This multiplexing is done using AND-OR structure to get better timing performance. It is assumed that the arbiter module can select only one AHB at a same time. This assumption supports the usage of AND-OR structure.

Write data endianisation

MPMC can be configured for both little endian and big endian mode of operation. Endianisation of write data is done at this block. Please refer the section 5.7 of the MPMC PL172 TRM [Ref. 2] for more details on the endianisation logic. Data width of the AHB interface is 32-bit. At the memory side MPMC can interface to 8-bit, 16-bit and 32-bit memories. The vector signal SelWDataQ2 is generated from the AHB interface muxing block. SelWDataQ2 is the

registered version of selected AHB write data bus. Endian logic uses SelWDataQ2 as the input and generates the output WDataCo2. The following logic is used for the endianisation of write data bus

- In little endian mode SelWDataQ2 can be directly assigned to WDataCo2 irrespective of the memory width
- In big endian mode, if the programmed memory width is same as AHB data width (32-bit) SelWDataQ2 can be directly assigned to WDataCo2
- In big endian mode, if the programmed memory width is 16-bit, the data bus is swapped at 16-bit boundary i.e. $WDataCo2[31:0] \leftarrow SelWDataQ2[15:0] \& SelWDataQ2[31:16]$
- In big endian mode, if the programmed memory width is 8-bit, the data bus is swapped at 8-bit boundaries i.e. $WDataCo2[31:0] \leftarrow SelWDataQ2[7:0] \& SelWDataQ2[15:8] \& SelWDataQ2[23:16] \& SelWDataQ2[31:24]$

Note that the signal ByteCo [3:0] indicates valid bytes in the endianised write data bus, WDataCo2 [31:0]. Endianisation of the ByteCo [3:0] is done at AHB interface module. Signal SelExtMW8Q3 is asserted if the programmed memory width of the selected bank is 8-bit. Signal SelExtMW16Q3 is asserted if the programmed memory width of the selected bank is 16-bit.

5.4 AHB Register Interface (MpmcRegIf)

The AHB Register Interface block provides the interface between the AHB and the registers that are used to configure and control the MPMC (PL172).

The following functionality is implemented in this module

- Response generation: This block contains the logic to generate the appropriate AHB response for each transfer on the AHB.
- All common registers are implemented (Address offset 0x000 to 0x080) in the AHB register interface block.
- Integration test registers and PrimeCell identification registers are also implemented in this block.
- First level address decoding for the static and dynamic block register. The actual registers reside in the respective register blocks (MpmcStRegBlk – 4 instances and MpmcDyRegBlk – 4 instances).
- Integration input and output muxes for primary inputs and outputs.
- Refresh counter for generating the refresh request at equal intervals according to the programmed value.

Response generation block

The AHB register interface is designed to provide OKAY and ERROR responses. MPMC contains zero wait state and single wait state registers. MPMC contains 68 registers. Among them, 3 registers - MPMCControl, MPMCStatus and MPMCConfig - are zero wait state registers and the remaining 65 are one wait state registers. Please refer to the TRM [Ref. 2] for the functional description of each register.

The MPMC Register interface supports only 32-bit accesses. The MPMC generates ERROR response when the AHB master performs an access with HSIZE other than 32.

The AHB Register interface is selected when HSELMPMCREG, HREADYIN and HTRANSREG are high. The register interface does not have to distinguish between the BUSY or IDLE transfers or between NSEQ and SEQ transfers from the AHB master. It responds to only NSEQ and SEQ transfers. So HTRANS[0] bit in the AHB is not required in the MPMC register interface. HTRANSREG indicate the status of HTRANS [1] in the AHB bus, this bit is set to high when the AHB master performs a SEQ or NSEQ transfer. The address decoder in the AHB system bus generates HSELMPMCREG. HSELMPMCREG is set when AHB master accesses an address location, which is mapped to a MPMC register. A high on HREADYINREG signal indicates that a transfer has finished on the AHB bus and the AHB bus is free. Signal RegBlkSelCo1 in the AHB register interface is generated by logically AND-ing HSELMPMCREG,

HREADYIN and HTRANSREG. So a high on RegBlkSelCo1 signal indicates that the register interface is selected for a transfer.

In an AHB system, whenever a slave is accessed, it must provide a response, which indicates the status of the transfer. Register interface uses HREADYOUTREG and HRESPREG [1:0] to generate the transfer response. OKAY response is generated by driving "00" on HRESPREG [1:0] and '1' on HREADYOUTREG signal. Wait states can be inserted by driving '0' on the HREADYOUTREG signal. ERROR response is a two cycle response. This response is generated by driving '0' on the HREADYOUTREG and "01" on HRESPREG [1:0] signals in the first clock cycle and driving '1' on the HREADYOUTREG and "01" on HRESPREG [1:0] signals in the second AHB clock cycle. Combinational signal NxtHSizeErr in the AHB register interface is set when AHB the master initiates a transfer of HSIZE other than 32. Signal HsizeErr is the registered version of NxtHSizeErr. Response generation logic generates the error response when HsizeErr is sampled high by performing the following sequence of operation in the next two clocks.

- Drive '0' to the HREADYOUTREG, and "01" to the HRESPREG [1:0]. Also set the combinational signal NxtErrRsp2. Signal ErrRsp2 is the registered version of NxtErrRsp2. (First phase of the error response).
- Drive '1' to the HREADYOUTREG, and "01" to the HRESPREG [1:0]. Also re-set the combinational signal NxtErrRsp2. (Second phase of the error response). Signal ErrRsp2 is used to differentiate between the phase 1 and phase 2 of the error response. ErrRsp2 is set to high only in phase 2 of the error response.

According to AMBA specifications [Ref. 1], default response of AHB slave is OKAY response. MPMC register interface drives OKAY response when it is not selected.

If the AHB master has selected a zero wait state register in the AHB register interface, signal ZeroWait is set to high. The signal ZeroWait is generated by decoding the address of the zero wait state registers. All accesses to the register interface with ZeroWait low is considered as a single wait state register access. So the response generation block inserts one wait state for all the register accesses if the address decode signal ZeroWait is sampled low.

Register read/write request

Read and write enable signals for all the registers are generated from this module. The register read request is asserted by setting RegRdEn to high and the register write request is asserted by setting LatRegWrReq to high.

RegRdEn is set to high when

- HSELMPMCREG = 1, HREADYIN REG= 1, HWRITEREG = 0, HTRANSREG = 1 (Register interface is selected for read operation)

RegRdEn is set to low when

- HSELMPMCREG = 1, HREADYINREG = 1, HWRITEREG =1, HTRANSREG = 1 (Register interface is selected for write operation)
- HSELMPMCREG = 0, HREADYINREG = 1. (Register interface is deselected and the register interface has finished the data transfer on AHB)

RegWrEn is set to high when

- HSELMPMCREG = 1, HREADYINREG = 1, HWRITEREG = 1, HTRANSREG = 1. (Register interface is selected for write operation)

LatRegWrReq is set to low when

- HSELMPMCREG = 1, HREADYINREG = 1, HWRITEREG = 0, HTRANSREG = 1 (Register interface is selected for read operation)
- HSELMPMCREG = 0, HREADYINREG = 1. (Register interface is deselected and the register interface has finished the data transfer on AHB)

Register select line generation

The first level of decode of the address lines, which is required for the register access is done in this module. MPMC contains 68 registers. All these registers are located in such a way that the MPMC has to decode the address lines HADDR [11:2] for selecting a particular register. This module performs two level of address decoding. First level of address decoding is done using the HADDR [11:2] and second level of decoding is done using the registered version of HADDR [11:2].

Depending on the functionality, MPMC registers can be divided into four groups. Address given to each register depends on this group. The first level decode generates following outputs

- CommRegSel = 1, when HADDR [11:8] = "0000": Common registers, which are used for configuring the MPMC, are under this group. There are 17 registers under this category – MPMCControl, MPMCStatus, MPMCConfig, MPMCDynamicControl, MPMCDynamicRefresh, MPMCDynamicRP, MPMCDynamicRAS, MPMCDynamicSREX, MPMCDynamicAPR, MPMCDynamicDAL, MPMCDynamicWR, MPMCDynamicRC, MPMCDynamicRFC, MPMCDynamicXSR, MPMCDynamicRRD, MPMCDynamicMRD and MPMCStaticExtendenWait. All this registers and selects line for these registers are implemented in this block.
- TstIdSel = 1 when HADDR [11:8] = "1111": Registers which are used for integration testing and identification purposes are under this group. There are 15 registers under this category – MPMCITCR, MPMCITIP, MPMCI TOP, MPMCPeriphId [1,2,3,4,5,6,7,8], and MPMCPCellId [0,1,2,3]. All these registers and the select lines for these registers are implemented in this block.
- DyBlkSel = 1 when HADDR [11:8] = "0001": Registers which are specific to dynamic memory chip select are under this group. There are 8 registers under this group – MPMCDynamicConfig [0,1,2,3], MPMCDynamicRasCas [0,1,2,3]. All these registers are implemented in dynamic memory controller block.
- StBlkSel = 1 when HADDR [11:8] = "0010": Registers which are specific to static memory chip selects are under this group. There are 28 registers under this group – MPMCStaticConfig [0,1,2,3], MPMCStaticWaitWen [0,1,2,3], MPMCStaticWaitOen [0,1,2,3], MPMCStaticWaitRd [0,1,2,3], MPMCStaticWaitPage [0,1,2,3], MPMCStaticWaitWr [0,1,2,3], and MPMCStaticWaitTurn [0,1,2,3]. All these registers are implemented in static memory controller block.

Second level address decodes required for all static and dynamic memory registers are done in this block. MPMC contains 8 banks, 4 for static and 4 for dynamic memories. This decode generates separate select lines for each memory bank's registers. This second level of decode uses the internal registered version of the AHB address, iHAddr7to2

StBlkSelCo2 (0) = 1 when StBlkSel = 1 and iHAddr7to2 [7:5] = "000"

StBlkSelCo2 (1) = 1 when StBlkSel = 1 and iHAddr7to2 [7:5] = "001"

StBlkSelCo2 (2) = 1 when StBlkSel = 1 and iHAddr7to2 [7:5] = "010"

StBlkSelCo2 (3) = 1 when StBlkSel = 1 and iHAddr7to2 [7:5] = "011"

DyBlkSelCo2 (0) = 1 when DyBlkSel = 1 and iHAddr7to2 [7:5] = "000"

DyBlkSelCo2 (1) = 1 when DyBlkSel = 1 and iHAddr7to2 [7:5] = "001"

DyBlkSelCo2 (2) = 1 when DyBlkSel = 1 and iHAddr7to2 [7:5] = "010"

DyBlkSelCo2 (3) = 1 when DyBlkSel = 1 and iHAddr7to2 [7:5] = "011"

Register Write Logic

All MPMC common registers are implemented in AHB register interface. 17 registers which are located in the address 0x000 to 0x080 and 15 registers which are located in the address range 0xF00 to 0xFFFF are implemented in this block. A MPMC register is written with the content of HWDATA, when the write enable for that register is sampled high on the rising edge of the clock. MPMCControl and MPMCDynamicControl are zero wait state registers. So HWDATA lines are directly used for these registers. All one wait state registers use the registered version of HWDATA bus. This reduces the load on HWDATA bus and gives better synthesis results.

Separate register write enables are generated for all the common registers. For example, MPMCControlWr signal is used as the write enable signal for the zero wait State register MPMCControl. MPMCControl is set when ComRegSel = '1', iRegWrEn = '1', and iHAddr7to2 [7:2] = "00000". **Figure 5.4-1 Register Write block** explains the register write logic.

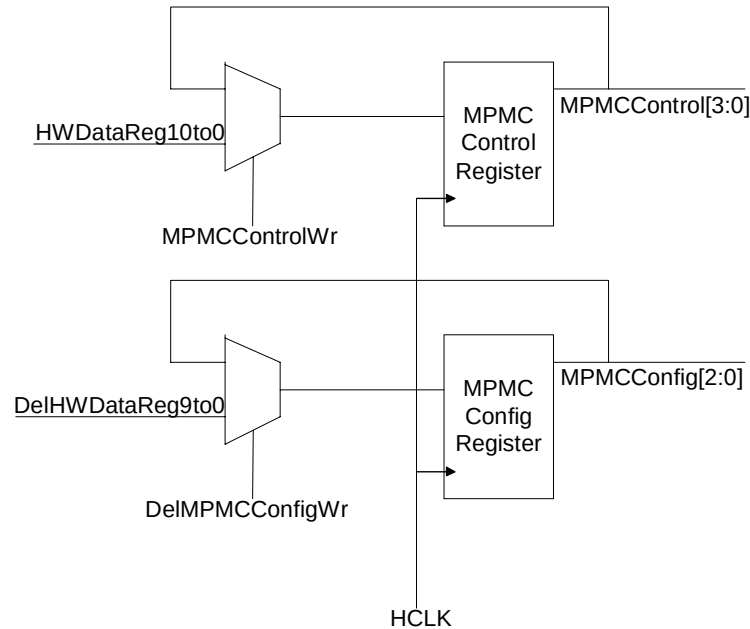


Figure 5.4-1 Register Write block

MPMC contains two reset domains nPOR and HRESETn. Bits 0,2 and 3 of the MPMCControl register are reset by HRESETn and bit 1 of the MPMCControl register is reset by nPOR. Integration test registers – MPMCTCR, MPMCTIP and MPMCTOP - are in HRESETn domains. All other registers are reset by nPOR. All registers in the MPMC are clocked by HCLK.

Register Read Logic

The AHB master can read the registers through the read data bus HRDATAREG[20:0]. An output mux required for the register read is implemented in this block. HRDATAREG is driven to the AHB as a combinational signal. According to the AMBA specification (Ref 1), HRDATAREG requires a set-up time of 60% with respect to HCLK. So maximum allowable time in these paths is 40% of HCLK time. This may not be sufficient for an output mux with 68 registers. So the output read mux is implemented in two stages. All one-wait state register blocks are implemented in parallel and muxed. Output of this mux is registered and fed to the final read data OR-logic. In case of zero-wait-state registers, output of the mux is directly connected to the read data OR-logic. **Figure 5.4-2 Register Read block** explains the output read data muxing

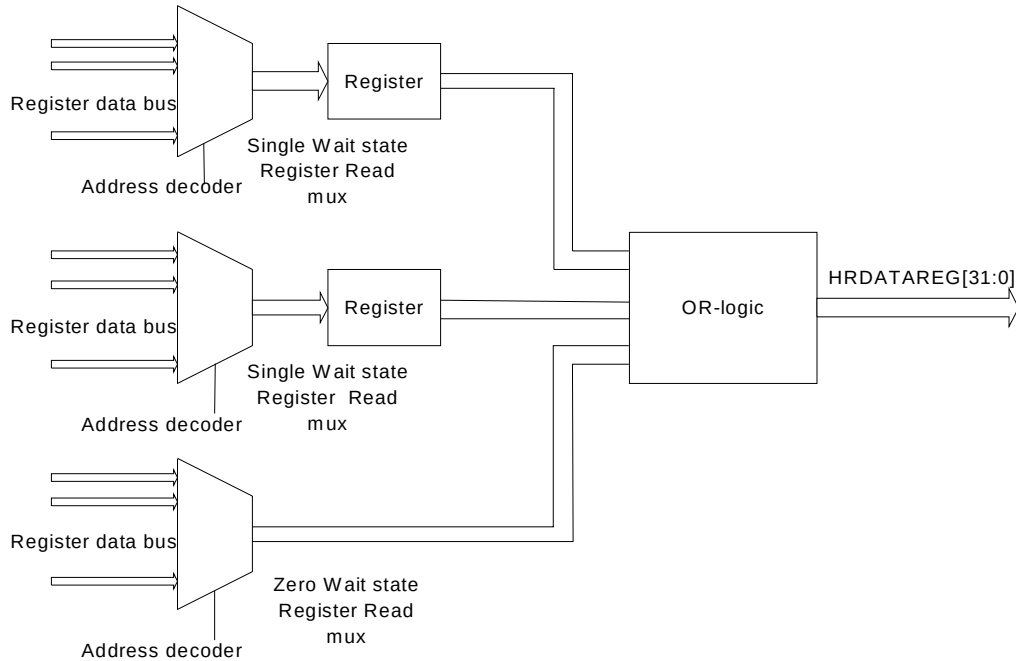


Figure 5.4-2 Register Read block

Refresh request logic

Dynamic memory refresh request is generated from this block. The MPMC supports both auto refresh and self refresh. The signal `SRefReq` in the MPMC register interface is set to indicate the self-refresh request to the memory control block. Software can set the `SrefReq` signal by writing '1' to bit-2 of the `MPMCDynamicControl` register. Self-refresh can also set through the hardware by setting the input pin `MPMCSREFREQ`.

Register interface block sets the signal `AutoRefReq` to indicate the auto-refresh request. `AutoRefReq` is a one-HCLK-wide signal that repeats at regular intervals. Duration of this interval can be programmed by software, using the `MPMCDynamicRefresh` register, `MPMCDyRefReQ [10:0]`. `AutoRefReq` generation logic uses two free running counters, `SmallCnt` and `RefCnt`. `SmallCnt` is a 4-bit free running down counter, re-loaded with "1111" whenever the counter gets expired. `SmallCnt` performs the down counts only if the enable signal `RefreshEn` is high. `RefreshEn` is set high when the `MPMCDynamic` refresh register is written with a value greater than zero. `SmallCnt0` is set high whenever the 4-bit counter, `SmallCnt`, counts down to "0000". `RefCnt` is an 11-bit down counter running on HCLK. Signal `SmallCnt0` is used as the enable signal for `RefCnt`. `RefCnt` is re-loaded with the `MPMCDyRefQ` register value whenever the counter expires or when a new value is written into the `MPMCDyRefQ` register. `AutoRefReq` is set high when `RefCnt` counts down to zero.

So for example, assume that a value 0x003 has been written into the `MPMCDyRefQ` register. This means that the `AutoRefresh` should be issued to the dynamic memories once in 48 HCLK cycles. The refresh request generation logic sets the `RefreshEn` signal high and the 4-bit down counter, `SmallCnt`, starts counting down from "1111". `SmallCnt` is re-loaded with "1111" every 16 HCLKs. So the signal `SmallCnt0` is set to high for one HCLK on every 16 HCLK period. `RefCnt` decrements its value whenever it samples a high on the signal `SmallCnt0`. So `AutoRefReq` signal is set to high for one HCLK, once in an every 48 HCLK (i.e. 3x16)

Software can disable the Auto-Refresh by writing 0x00 to `MPMCRRefReqQ` register. Auto-refresh request generation logic uses nPOR for reset. Hence auto-refresh requests are issued to the dynamic memory in the presence of a soft reset (`HRESETn` assertion).

Integration Test Registers

The MPMC integration test registers allows the integrator to verify that the MPMC has been connected into the target system correctly. The `ITEN` bit in the `MPMCTCR` register is used for configuring the MPMC in integration test mode.

ITEN is low for normal operation. When ITEN is set high by software, multiplexes on the inputs and outputs are switched to use the test mode values.

Figure 5.4-3 Integration Intrachip Input Register explains the implementation details of the input integration test harness. The MCIITIP is an 8-bit register. The ITEN bit is used as the control bit for the multiplexer, which is used in the read path of the intra-chip inputs MPMCSREFREQ, MPMCBIGENDIAN, MPMCSTCS1MW, MPMCSTCS0POL, MPMCSTCS1POL, MPMCSTCS2POL, MPMCSTCS3POL, MPMCSTCS1PB and MPMCREL1CONFIG. If the ITEN control bit is de-asserted, intra-chip inputs are routed to the core logic; else the stored register value is driven on the internal line.

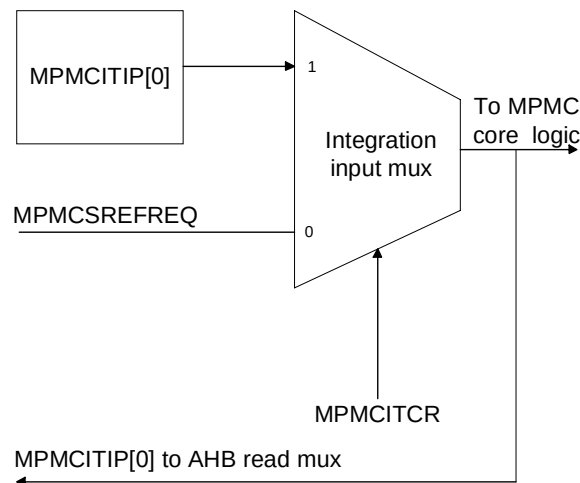


Figure 5.4-3 Integration Intrachip Input Register

Figure 5.4-4 Integration Intrachip Output Register illustrates the implementation details of the output integration test harness in the case of intra-chip outputs. MPMCITOP is a 2-bit register. If the ITEN bit is set, MPMCITOP [1:0] is connected with the output pins MPMCSREFACK and MPMCRPVHHOUT else these pins are driven by core logic.

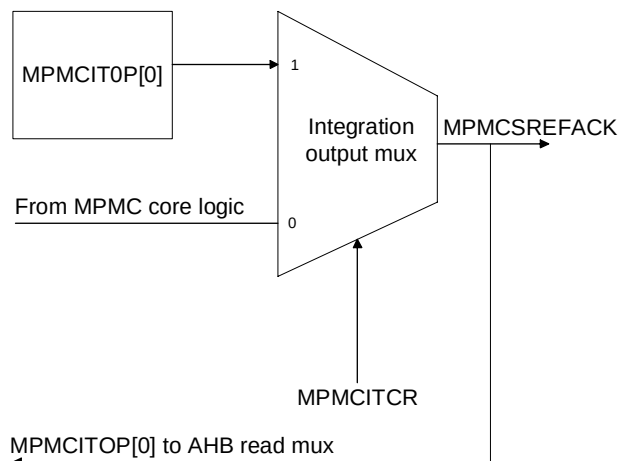


Figure 5.4-4 Integration Intrachip Output Register

5.5 Arbiter (MpmcArbiter)

The function of the arbiter is to arbitrate between requests from the eight AHB's and indicate the same to the buffer unit to further process the granted AHB. Arbitration is determined by three important parameters: the priority of the requesting AHB; the locked status of the request; and the backoff masking.

The arbiter essentially generates two important signals.

- ValidArbQ when high indicates that the arbiter has granted one of the AHB's access to the buffer and from thereof to the memory.
- ValidAHBQ indicates the ID of the granted AHB for the given arbitration cycle.

The functions are described in detail below:

Activation of a New Arbitration Cycle

A new Arbitration cycle is initiated when there is no valid arbitration cycle in progress as well as the absence of BlockFlushQ signal from the buffer controller and one of the eight AHBs raises transaction request by activating its respective MemReqCo1 line. In the subsequent raising edge of the clock, the Arbiter will indicate a new arbitration cycle by activating ValidArbQ as high. The BlockFlushQ protection is intended to guarantee definitive latencies for AHB0 reads.

De-Activation of a New Arbitration Cycle

An active arbitration cycle will be terminated under either of the following events occurring:

- a) BreakArbit goes active from the buffer block, indicating that the requested transaction cannot be completed due to lack of resources at the buffer or memory blocks. These include conditions such as no buffers available and the memory is busy and hence a buffer cannot be vacated, transaction is to a locked buffer waiting for data from the memory.
- b) SelRstBurstCo goes active from the AHB side indicating that the granted AHB is done with its bursted transaction or the address on the granted AHB has traversed a Quad-QWORD boundary.

Activation of a New AHB Id

A new AHB id is determined only when there is no valid arbitration in progress. A simple priority-encoding scheme is used to arbitrate multiple AHB's requesting for the resources at the same time. AHB0 has the highest priority and AHB7 having the lowest priority. To determine a new granted AHB the following pre-processing is done prior to applying the priority encoding logic. The raw request from the AHB's is initially masked with the Backoff information from the buffer to eliminate AHB's which would not be serviceable in the current context. This is done to allow better usage of the resources. The masked value is used to determine the new AHB that would be granted based on the priority-encoding scheme.

De-Activation of a New AHB Id

A granted AHBId is sustained as long as the arbitration cycle is valid and a new arbitration takes place only when ValidArbQ is sensed low. Even under that circumstance if the granted AHB has initiated a Locked transaction as indicated by SelMastLock being true, then the same AHBId is maintained even if there is a higher priority request. Therefore in a Lock transaction crossing a Quad-QWORD boundary, when the ValidArbQ would deactivate the same AHB can be assured of being granted the memory resources without any break in ownership.

Other signals generated from the Arbiter are AHBWrReq and AHBrdReq either of which goes high based on whether the granted AHB is intending a Write or a Read transaction respectively. The MpmcBusyQ, a status bit goes high, if either the Arbiter is active, going to be active in the subsequent clock or if the external memory interface is busy due to a past posted transaction.

5.6 Buffer Unit (MpmcBufUnit)

In this section, the sub-blocks within the Buffer Interface logic are explained in more details.

5.6.1 Buffer (MpmcBuffer)

The buffer block acts as the intermediate storage for Write transactions from AHB to memory and as a small Quad-QWORD deep cache for read transactions from AHB to memory. All the necessary control signals are mostly generated from the MpmcBufCntrl block.

The following are the functions carried out inside of the buffer block.

- Storage of address and chip select information
- Storage of data and byte lane information
- Generation of Buffer Hit to determine address match
- Generation of Buffer Empty Status to determine a free buffer
- Generation of remapped address for dynamic memories
- Maintaining the appropriate QWORD to be flushed

Storage of Address and Chip select Information

Whenever the UpdateTagAddr signal from the MpmcBufCntrl block is sensed high inside the buffer block the following actions are carried out else the old register values are maintained:

The address from the AHB occupying the buffer, the corresponding chip select and the corresponding type of access i.e. Static or Dynamic are registered and maintained until the buffer is granted to a different address.

Storage of Data and Byte Lane Information

The buffer block has two sources of data and valid byte lane information: the AHB interface and the Memory Controller.

Whenever UpdateWData is high, it indicates to the buffer that the data from the granted AHB needs to be updated into the buffer. Which bytes in the QWORD location will be updated is determined by decoding the 16-bit SelByteIn information. The MSB of SelByteIn indicates that the upper most of the QWORD needs to be updated. At the time of data Updation from AHB side, information on validity of all the Byte Lanes is also registered. This is achieved by logically OR-ing the current Valid Byte Lane information (iTagByteQ) along with the current valid byte lane information (SelByteIn). The OR-ing is done to maintain the validity of the contents of location, which are not updated with data in the current clock. Which among the four QWORDS would be operated is determined by AHBAAddr5 and AHBAAddr4.

Whenever UpdateMemData is high, it indicates to the buffer that the data from the memory needs to be updated into the buffer and this data is in response to a read, which was posted to the memory controller earlier. Which bytes in the QWORD location will be updated is determined by decoding the 16-bit MemByteIn information and also AND'ing the current valid bytelane information. The reason for the logical AND-ing is because the buffer might have part data in the buffer, which is yet to be updated into the memory, and hence those byte locations should not be updated with data from the memory side. The MSB of MemByteIn indicates that the upped most of the QWORD needs to be updated. At the time of data update from memory side, information on the validity of all Byte Lanes is also registered. This is achieved by logical OR-ing the current Valid Byte Lane information (iTagByteQ) along with the current valid byte lane information (MemByteIn). The OR'ing is done to maintain the validity of the contents of location, which are not updated with data in the current clock. Which among the four QWORDS would be operated is determined by MemAddr5 and MemAddr4.

Also whenever UpdateTagAddr goes high, it indicates that the buffer is being vacated and is being occupied with a new address. At this point the valid byte lane information of the buffer (iTagByteQ) is reset to all invalid State.

Under all other conditions the current data contents and the valid byte lane information of the buffer is maintained.

Generation of Buffer Hit

On all active arbitration clocks the buffer compares the stored address and stored chip selects with the currently driven address from the AHB. If there is a match with both the parameters then it is indicated that the current transaction is to this buffer by activating BufHit signal as high.

For all read transactions from the AHB, additionally the byte lane requested by the read is compared with the stored valid byte lane information. This is done to check if the entire requested read data can be supplied from the buffer itself or some part or all of it needs to be fetched from the memory. ByteMatchC1 going high indicates a Byte Match in addition to a buffer hit match.

Generation of Buffer Empty Status

The buffer is treated as not empty (Occupied) whenever there is an update of data from AHB interface. Whenever the buffer contents are flushed to memory or when there is an update of data from memory side to an Empty buffer the buffer is treated as empty. BufEmptyQ being driven high indicates the fact that the buffer is empty.

Generation of Re-Mapped Address

The address stored in the buffer is linear. For dynamic memories the linear address needs to be translated to Ras and Cas addresses. Based on the device connected to the chip select a different conversion mechanism needs to be followed. More details of the remapping can be found in the TRM [Ref. 2]. The design has been optimised for the various likely combinations and appropriate Ras, Cas and Bank Selects are generated using the stored linear address and the stored chip select. The remapping configuration to be used is determined by the ADDRMAP bits of the MpmcSyncConfig registers.

Maintaining the Appropriate QWORD to be flushed

The memory interface can handle only an QWORD deep flush at any point in time. The buffer block maintains empty flags on an QWORD granularity and also maintains a signal OnlyOneQWFull which is high whenever there is only one QWORD which is not empty. Using the two flags, a counter rolls over starting from QWORD 0 to QWORD 3. The logic ensures that the data and the address that need to be driven to the buffer control block take care of the changes to address lines 5 and 4 during these rollovers.

5.6.2 Buffer Controller (MpmcBufCntrl)

The buffer control block is responsible for generation of all control signals needed for the individual buffers and also for managing the top level buffer input, output ports and the necessary handshake with the MpmcAhbTop, MpmcArbiter, MpmcMemCntl blocks.

The following key functions are carried out inside the MpmcBufCntrl block:

- Buffer natural flushing algorithm for unforced buffer flush
- Access type determination
- Buffer Selection
- Buffer Control Signal generation
- LRU Management
- Forced Flushing Management
- Backoff Management

- UnBlocking Management
- Memory Controller Interfacing
- AHB Interface Management

Buffer Natural Flushing Algorithm

The four Quad-QWORD deep buffers, having their contents vacated to memory interface are treated as a buffer flush operation. There are two possible flushing events:

- Natural flushing and
- Forced flushing.

A buffer flush is termed as a natural flush if the flushing happened when there are no pending active AHB transactions or the current transaction is a write transaction to the buffer. A 2-bit counter which increments and points to the next buffer once a flush has happened determines the buffer, which is to be flushed. A natural flush can be activated by two conditions:

- Buffers are not empty and there is no active arbitration cycle in progress.
- Buffers are not empty and it is a Write buffer enabled transaction to a buffer

When the above two conditions are satisfied and if no AHB requests have been rejected and backedoff due to either absence of empty buffers or memory controller not being in a position to service the request and the pointed buffer meets the following conditions a Natural Buffer Flush event will occur and the buffer contents will be transferred to memory interface. If a buffer is filled totally for all the Quad-QWORDS then four such cycles would be needed for a buffer to be totally emptied. The sub conditions, all of which are necessary for one bit of GenFlushReq[3:0] to go high indicating a natural flush cycle are:

- The buffer contains at least one byte of data written from the AHB interface which has not yet been posted to memory
- The type of memory content stored in the buffer (Dynamic or Static) can be transmitted to the respective memory controllers and
- The buffer is not locked and waiting for data to be received from the memory controller due a read command posted in the past.

Also addressed in this section of the design is a corner case related to the following event: If during a arbitration window the buffer into which a AHB port is transacting is likely to be the next flushing buffer, then move the flushing pointer. Since if the transaction were a write all of the contents would not have flowed into the buffer before the flushing due to the pipeline architecture of AHB.

Access Type Determination

All of the logic in the MpmcBufCntl block function based on this fundamental operation. When an AHB transaction reaches the block, firstly it is determined if the transaction is a buffer enabled or disabled operation and subsequent events are based on this determination. A operation from the AHB is treated as a buffer enabled operation when the active chip select being accessed by the AHB has its BufEnQ bit set to High in the AHB config register space. Else the operation is treated as unbuffered and the iUnBufMemAcc signal goes high. For dynamic memory transactions until the respective mode done bits are set, all transactions to the particular chip select are treated as unbuffered.

Buffer Management Algorithms

If the transaction requested from the AHB is a buffered transaction the sequence of events followed is as follows:

- Buffer Selection
- Buffer Control Signal generation
- LRU Management

- Forced Flushing Management
- Backoff Management
- UnBlocking Management
- Memory Controller Interfacing
- AHB Interface Management

BufferSelection Algorithm

The buffer selection algorithm is implemented in a very straightforward manner. Any buffer, which indicates an address match with the AHB driven address by asserting its BufHit signal high is the, default-selected buffer. If the address is not matched with any of the four buffers then LRU (Least Recently Used) algorithm is used to select the buffer that will service the currently granted AHB transaction. The selected buffer is identified by one of the four bits of CurrentBufId being driven high.

Buffer Control Signal Generation

Once a buffer is selected, it will result in one or two of the following control signal generation.

TagAddress Updation:

The control signal is generated to the buffer to update the current AHB address information as the address, which should be maintained, by the buffer. The current access should be valid as indicated by ValidArbQ and it should not be a Hit to any of the buffers as indicated by AddrHit signal being zero for all of the buffers and it should match the LRU ID and also it should be a Buffered Memory access and if the buffer is empty. If the buffer is not empty the Logic is modified such a way that UpdateTagAddr will be driven high only if a flush is possible and the buffer has only one QWORD to be flushed, as indicated by OnlyOneQWFull being high. This has the elegance of removing dependency on Read or Write for generation of the signal. The other common signals which prevent the generation of UpdateTagAddr are the presence of SelAbort indicating that the current transaction will be aborted by the AHB or if the selected buffer is waiting on read data from memory interface for a previous posted read transaction.

DataUpdation:

The buffers to update the current AHB data and byte lane information use this control signal. UpdateWData is generated as an anding of registered UpdateWDataQ and UpdateWDataQ to maintain the AHB data pipeline. It is to be noted that AHB side update into the Buffers will take place only for Writes. The UpdateWData will be generated for a new transaction into an empty buffer one clock after the UpdateTagAddr generation and sustained as long AHBWrReq is high for the given burst. For Write transaction to a Buffer Hit selected buffer, the generation will start from the very 1st clock in which AHBWrReq is sampled to be high.

Byte Lane Expansion:

The four-bit byte lane information driven by the AHB is expanded to a 16-bit information for the buffers to process. This expansion is achieved by mapping the incoming four-bit SelByte or AdvSelByte signals to the appropriate 4-bit position inside the sixteen-bit wide iSelByte generated signal. The mapping is based on the address lines A3 and A2 and if the transaction is a read or a write. For the very first transaction always the SelByte information is routed and for subsequent transaction the AHB interface generated advanced byte information (AdvSelByte) is used to achieve a 1-0-0-0 wait State performance. UseBufQ, which is used to indicate to the AHB to generate HREADYOUT for the current transaction, is used to switch between SelByte and AdvSelByte. In addition the final byte information driven to the buffers namely the SelByteIn information either uses the raw iSelByte or the clocked iSelByte information based on if the transaction were a read or write from the AHB respectively. This is implemented keeping in mind the AHB pipeline between address and data phases.

LRU Management

The LRU algorithm is implemented in the design for effective management of the buffers. Based on the buffer selected the LRU logic is managed. At reset, buffer 0 is treated as LRU and mapped at Stage0. Buffer 3 is treated as

the most recently used and mapped to stage3. Depending on the LRU stage position at which the current buffer selected is, it will move up by one level as more recently used and the buffer pointed by the one level up stage occupies the current position. Thus a buffer, which gets pushed to the stage0, becomes the LRU. The whole logic is implemented as a four stage to cater for the buffer count. LRU algorithm does not differentiate if the access is a read or a write. When a normal flush occurs, the flushed buffer is treated as LRU and all the other 3 stages move up one level as more recently used.

Forced Flushing Management

In addition to the natural flushes described, the design also incorporates forced flushes to effectively use the buffer. Forced Flushing comes into play, when in a valid arbitration window, a non-empty and non-address match buffer is allocated to the granted AHB. Under such circumstance the current contents need to be flushed to memory and the buffer loaded with the new AHB address information. The design ensures that both the operations will occur at the very same clock edge enhancing the performance. This forced flushing will be performed if the type of memory access currently stored in the buffer (Static or Dynamic) can be posted to the appropriate memory controller interface and also the buffer is not waiting on a previous posted read request. If the necessary conditions are not valid, then the AHB is backed off from the arbitration. The event of either a natural or forced flush cycle is indicated by one of the four iFlushBuf bits driven high and FlushCycle driven high.

Backoff Management

Backoff algorithm is implemented in the buffer to better handle multiple AHB requests. Backoff algorithm is implemented only for buffer enabled transactions. The logic behind the backoff algorithm can be simplistically stated as follows: "Any AHB which initiates a transaction, not transactable in the current clock will be backed off until the negating condition which prevented the transaction in the first place is inactivated". The requesting AHB is asked to backoff and it is duty of the arbiter to honour the request by not allowing backed off AHB's to enter arbitration even if they are high priority requests until the backoff condition is removed. The whole concept of backing off and arbitration break is to allow other AHBs a chance to get to available resources in the buffer instead of blocking all transactions until the memory responds. This is done with the confidence that there is at least 3 plus clocks before any memory will respond with data and attempt is made to utilise the free clocks. In fact it is expected to be much larger approx. 4-5 clocks for Dynamic memory and as delayed as the wait states programmed for the static memories. None of the backoff will result in an AHB change if the current AHB is transacting a locked transaction. This guarantee is from the arbiter. BreakArbit being driven high to the arbiter indicates a backoff cycle.

The following four break functions determine the various blocking and arbiter breaking logic involved:

- BreakByRdPost going high is for access to buffers, which resulted in a read command being posted to memory, and have to wait until the memory returns the requested data. Each buffer has an eight-bit register, which keeps track of the initiating AHB id. The registers are InitiatorBuf0Q to InitiatorBuf3Q
- BreakByLkdBufAcc going high is for access to buffers when, a different AHB is accessing a buffer for which it was not the original initiator and the burst transaction is not completed as requested by the initiating AHB, since all of the data requested from the memory by the original AHB are not yet updated into the buffer. Each buffer has a eight bit register which cumulatively keeps track of all AHB's that tried to access a locked buffer before the memory returned the last of the initiator requested data. The registers are AHBRdBlock0Q to AHBRdBlock3Q.
- BreakByNoStBuf going high when the current transaction results in a buffer, which has static memory contents having to be vacated, but the Static memory controller is unable to honour the request. An eight-bit register keeps track of all AHBs, which resulted in a request for a static memory transfer when the static memory controller was unable to honour the request. The register is iAHBStBufBlockQ.
- BreakByNoDyBuf going high when the current transaction results in a buffer, which has dynamic memory contents having to be vacated, but the dynamic memory controller is unable to honour the request. An eight-bit register keeps track of all of the AHBs, which resulted in a request for a dynamic memory transfer when the dynamic memory controller was unable to honour the request. The register is iAHBDyBufBlockQ.

When any of the above four events occur, the MpmcBufCntrl block stores the AHB id in conjunction with the event, which triggered it. This is done to help the arbiter to block such AHBs from arbitration until the backoff condition is removed. The AHBs to be masked is indicated by driving the AHBBlockQ (Which is a cumulative content of backed-

off and unblocked AHBs). Also if the backoff was generated from an AHB0 read, an additional signal BlockFlushQ which has the following dual functionality is generated.

- Prevent Natural flushes from distorting buffer latencies. Only the exact number of flushes needed to allocate a buffer to AHB0 will be allowed.
- Ensure that at the arbiter all other AHBs will be masked out of the arbitration until AHB0 is able to post its read command from the buffers.

Both of the above functionalities are implemented to ensure predictable latencies on AHB0 buffer enabled read transactions.

UnBlocking Management

Once an event capable of resulting in an AHB backoff gets negated, it is the responsibility of the MpmcBufCntrl block to unblock the AHBs and allow them to rejoin the arbitration process. iAHBStBufBlockQ and iAHBDyBufBlockQ are reset to zero as and when StBufFullQ and DyBufFullQ become zero respectively. This indicates that the respective memory controllers are ok to accept new commands from the buffer block. All of the AHBs stored in iAHBStBufBlockQ and iAHBDyBufBlockQ xor'd from iAHBBlockQ and these AHBs are allowed back into arbitration. When a buffer receives the first of the data requested by a posted read command, the respective InitiatorBufxQ's are reset and released similarly. This occurs when the first UseMem is sensed high in conjunction with the MemAHBId returned by the memory controller interface. When the last UseMem is returned then the respective AHB RdBlockxQ's are also reset and released. StEndOfBurstQ or DyEndOfBurstQ being driven high indicates this event.

Memory Controller Interfacing:

The MpmcBufCntrl interfaces to the two memory controllers Static and Dynamic whenever the need arises for posting either a read or write request. Either the buffer contents are transferred or the raw AHB information is transferred to the memory controllers. This determination is based on the simple logic of either the current cycle is a flushcycle or otherwise. Whenever FlushCycle goes high indicating a flush operation the buffer contents are transferred, else the AHB information is transferred.

The information transferred to the memory controller are Linear Address, Remapped Address, Byte Lane Information, 128 bits of Data and a read or a write command. In addition a control code called the BufAccCode is driven along with the command. If it is a flush operation or a buffer enabled read post, the BufAccCode carries the buffer id, else it carries the AHB Id. The memory controller is expected to return this code whenever it returns with the data so that appropriately it will be consumed by the buffer or by the AHB directly. MSB of BufAccCode if high indicates that AHB needs to consume the data and 0 indicates that the buffer will consume the data.

The memory controller throttles the acceptance of new commands through the StBufFullQ and DyBufFullQ lines respectively. If high a new command to the respective controllers cannot be posted. The Memory controller also informs the availability of Read data and ready generation for unbuffered transactions by driving UseMem high and also driving the address lines A5, A4, A3 and A2 on MemA5A2 to indicate the appropriate Quad-QWORD position of the data. The BufAccCode received by the memory controller along with the command is returned with the UseMem for appropriate switching. The information is returned on MemAHBId.

AHB Interface Management:

Two sets of information are returned by the buffer interface to AHB interface. UseBuf indicating that the currently granted AHB can generate HREADYOUT in the subsequent clock and BufRDataQ if the transaction were a read transaction. UseBuf is generated so as to achieve 1-0-0-0 wait State transaction on the AHB interface.

5.7 Memory Controller (MpmcMemCntl)

The MPMC memory controller block is a structural block containing the static memory controller, the dynamic memory controller, the memory controller interface and the memory bus granting logic. The following **sections 5.9** to **section 5.12** contain a detailed description of all the blocks in the MPMC memory controller block.

5.8 Dynamic Memory Controller (MpmcDyMemCntl)

In this section, the sub-blocks within the Dynamic Memory Controller are described in detail.

5.8.1 Dynamic Memory Controller Register Block (MpmcDyRegBlk)

Details of the MPMC dynamic memory register block functionality are described in this sub-section. The dynamic memory register block is instantiated four times in the dynamic memory controller to realise the register blocks associated with each static memory chip select. The dynamic memory register block select (DyBlkSelCo2[3:0]) signal is used to select one dynamic memory register block among the four.

Each dynamic memory controller register block implements the following functionality:

- Dynamic memory registers
- Register write logic
- Register read logic
- Row width and Column width extraction logic from the address map information.

Registers implemented in the dynamic memory register block

- The dynamic memory configuration register, MPMCDyConfig. This register is used to program the configuration of the dynamic memory connected to the chip select associated with the dynamic memory register block. This register contains the write protect bit (DyWrProtQ), the buffer enable bit (DyBufEnQ), the address mapping bits (AddrMapQ[5:0]), the memory width bit (ChipSelMWQ), the memory device type bit (DyMemDeviceQ).
- The dynamic memory Ras and Cas register, MpmcDyRasCasQ. This register is used to program the RAS and CAS delays of the dynamic memory connected to the chip select associated with the dynamic memory register block.

Register write logic

The register write logic for the dynamic memory register block is as in **Figure 5.8-1**. A dynamic memory register is written with the data on HWDATAREG when the write enable for that register is sampled high on the rising edge of HCLK. The register to be written is selected by decoding the sampled registered address (HAddrReg4to2Q2[4:0]) and the dynamic memory register block select (DyBlkSelCo2) from the AHB register interface block. The write enable signals are generated by combining the address decode with the sampled register write enable signal (RegWrEnQ2) from the AHB register interface block. Registered versions of the data on HWDATAREG is written into the registers decoded by using registered HaddrReg4to2Q2[4:2], DyBlkSelCo2 and RegWrEnQ2 to meet synthesis timings. For all writes to the dynamic memory registers, one wait state is added.

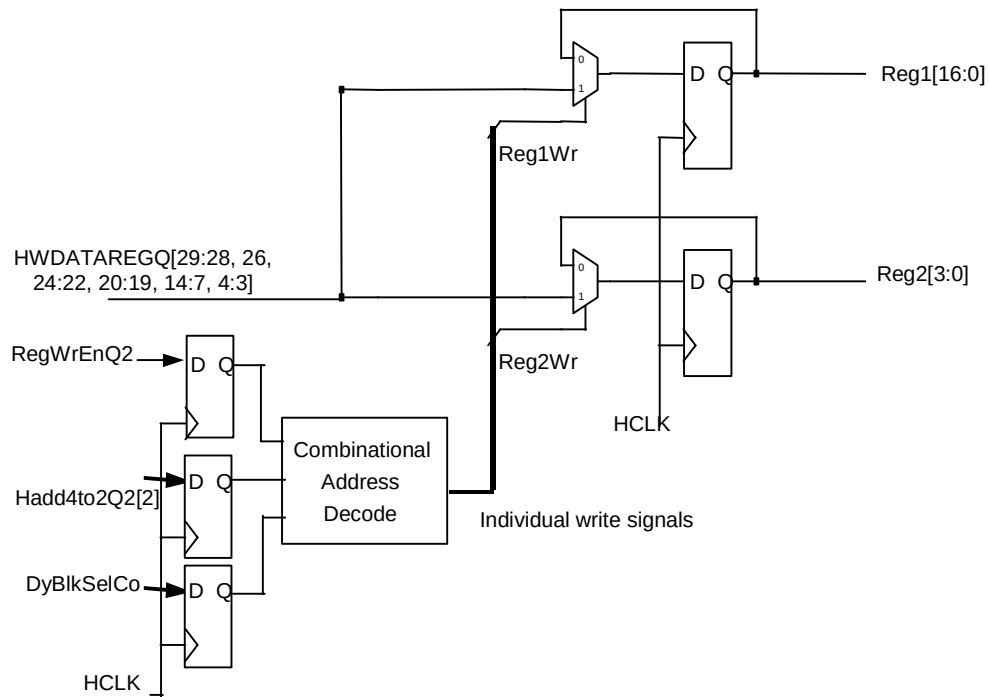


Figure 5.8-1 Dynamic Memory Register Write Logic

Register read logic

The read logic for the dynamic memory registers in the MPMC is as given in **Figure 5.8-2**.

For reads, the registers in the dynamic memory register block are muxed (based on HAddr4to2Q2[2]) to generate the read data bus from the dynamic memory register block, DyBlkRdData[12:0].

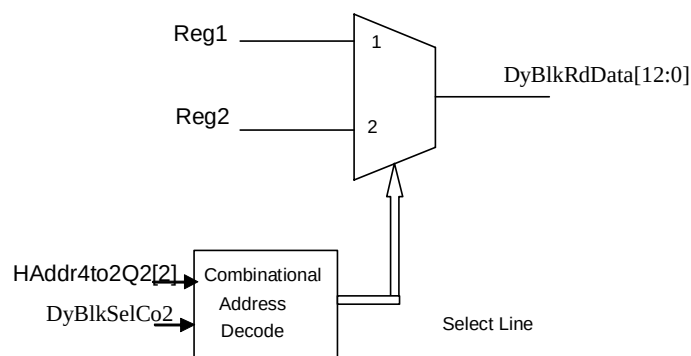


Figure 5.8-2 Dynamic Memory Register Read Logic

Row width and Column width extraction from the address map bits

Based on the address map tables in the Technical Reference Manual [Ref 2.], the row width and column width of the memory device are deduced from the value written into the address map and memory width fields of the MPMCDyConfig[3..0] registers. This generation of row width and column width information is done by processing the

D-input of the MPMCDyConfig register and then registering the processed row and column widths. This avoids an additional propagation delay on the dynamic memory state machine inputs DyRowWidth and DyColWidth.

5.8.2 Dynamic Memory Read Request Generation Block (MpmcDyIntRdGen)

Following functionality is implemented in this block

- Read data request generation to the dynamic memory control state machines
- CAS address generation if the read request is generated internally
- EndofBurst generation, which is used by the buffer control logic
- DyBufFull generation, which is used by the buffer control logic

Need for the multiple read requests can be explained as follows. Dynamic memory control state machines returns 4 words of data for each read request. So in cases where AHB burst size is more than 4 words, this block has to generate more than one read request to complete the AHB burst. RdReqValue generated in this block is used for the internal generation of the dynamic read data request DyPostRdReq.

Following assumptions are used for the generation of the RdReqValue.

- AHB burst size can be calculated by using HSIZE and HBURST information. Registered version of the HBURST and HSIZE values will be available to this block along with the DyPostRReq.
- INCR is internally considered as INCR4.
- Data access state machine provides 4-word of data for each read request.
- Maximum size of the buffer is restricted to 16 words deep. So in case of an INCR/INCR4/INCR8/INCR16 access AHB can cross 16-word address boundary. In this case buffer arbitration is required at the 16-word address boundary and an indication to the buffer block has to be asserted. So in case of incrementing bursts RdReqValue generation is also depending on starting address of the burst.

An internal 3-bit down counter RdReqCnt is used for internal DyPostRdReq generation. RdReqValue is used as the initial load value to this counter. An internal version of the DyBufFull is available to this block. Low on DyBufFull indicates that data access state machine is free to take new read request. Whenever DyBufFull is sampled low and RdReqCnt is greater than zero a new DyPostRdReq is generated from this block. RdReqCnt decrements whenever an internal post read request is generated from this block.

DyBufFullOut that is going to buffer is generated from this block. DyBufFullOut is an OR-ed version of DyBufFull coming into this block and RdReqCnt status. So if there is a pending internal read request DyBufFullOut will be asserted by this block and it prevents the posting of new read request from the buffer control.

EndOfBurst signal - which indicates to the buffer block that all the data that are requested by the AHB has been supplied - is generated from this block. EndOfBurst signal that is originally generated by the data access state machine is processed in this block and is given to the buffer. Each read request to the DASMs provides 4-word of data. EndOfBurst given by the DASMs is a pulse kind of signal that sets when it is providing the last data of the quad word. This EndOfBurst is processed by this block to generate the final EndOfBurst to the buffer control block. EndOfBurst to the buffer is generated under the following conditions

- All the data requested by the AHB has been delivered
- AHB address crosses the 16-word boundary. This restriction comes from the buffer depth.

The Buffer block contains 4 individual buffers. So buffer can post maximum four read request at the same time. So four internal counters are used for the tracking of end of burst signal. This block stores BufAccCode provided by the buffer block at the time of read posting internally. DASM returns a buffer identification code - MemAHBId - along with data. Four comparators are provided in this block that compare the returned MemAHBId with the BufAccCode. This comparators output is used by the end of burst tracking counter and if a match is found in the comparison then the corresponding counter is updated.

5.8.3 Dynamic Memory Controller Data Access State Machine (MpmcDyAxsSM1 and MpmcDyAxsSM2)

There are two Dynamic Memory Data Access State Machines in the MPMC (PL172). These two State Machines are very similar with regards to input/output signals and in functionality but for one difference. The difference comes as a priority coding between the two State Machines. The Data Access State Machine 1 gets default priority for accessing the memory, when both the State Machines are ready to access the memory. However, to prevent MpmcDyAxsSM2 from starving, MpmcDyAxsSM1 relinquishes the external bus for one clock if it finds MpmcDyAxsSM2 waiting after the end of its burst.

This signalling of 'whether DASM1 wishes to proceed or not' is done using the signal SM2Gnt. Whenever an access is to be done by State Machine 1, it proceeds with the access, de-asserting the SM2Gnt signal. The second State Machine starts the access only if SM2Gnt is high, which means that the first State Machine is not performing an access to the memory. The Dynamic Controller Access State Machine transits through different states depending on the type of the request, and the State of the Memory Banks for that particular request.

The Data Access State Machine (DASM) has the following states:

- ST_DY_IDLE: Dynamic memory data access state machine idle state
- ST_DY_CLKON: Dynamic memory data access state machine clock-on state
- ST_DY_SEL: Dynamic memory data access state machine select state
- ST_DY_PRE: Dynamic memory data access state machine precharge state
- ST_DY_ACT: Dynamic memory data access state machine active state
- ST_DY_RDISSUE: Dynamic memory data access state machine read issue state
- ST_DY_DATAFTCH: Dynamic memory data access state machine data fetch state
- ST_DY_WR: Dynamic memory data access state machine write state
- ST_DY_DQMLow: Dynamic memory data access state machine DQM low state.

ST_DY_IDLE:

In this State, DASM waits for a DyPostRReq (DyPostRReq is actually the DyPostRReqOut from the DyIntRdGen block) or DyPostWReq. On getting the request, it clocks the available information like address, chip-select, bank-select etc and moves on to the next state.

The conditions required to trigger a change from ST_DY_IDLE to ST_DY_SEL are:

- MPMC is enabled
- MPMC is not set in low power mode.
- Mode register programming is complete.
- A read or write transfer request is raised.
- The device is not a flash or it is a normal read access to a flash. This information is decoded from the access being a buffered-access.

While transitioning from IDLE State to SEL State, request is raised to the Memory Bus Grant Module. Chip-select, bank-select, byte valid bits, row and column address information for the current access is clocked

The conditions required to trigger a change from ST_DY_IDLE to ST_DY_CLKON are:

- All the conditions required to transition to ST_DY_SEL and
- When the clocks to the SDRAMs are stopped.

ST_DY_CLKON:

If the clock to the memory-devices were stopped, then DASM needs to transition through this State to switch-on the clocks. The State is automatically changed to ST_DY_SEL in this State, after setting the signal to switch on all the clocks.

ST_DY_SEL:

On getting a DyPostRReq or DyPostWReq, the DASM clocks the relevant information and moves to ST_DY_SEL, where it waits for the grant from external-bus-grant logic.

One common condition for moving from SEL state for DASM1 is that Wt1CycForDSM2 is low. That is DASM1 is not in the window where it allows DASM2 to proceed in spite of the default higher priority of DASM1.

The conditions required to trigger a change from ST_DY_SEL to ST_DY_PRE are:

- The two parallel SDRAMC DASMs cannot run on the same bank of the same chip-select. (i.e. CtBankAxsdsM1 != '1')
- The dynamic-memory-controller should have the DyBusGnt and the DASM should have the shared 'lock' to proceed.
- There should be some other row open in the bank (indicated by current bank being 'open' and a different 'last row accessed')
- The latency to precharge should be satisfied.

While transitioning from SEL to PRE state,

- Precharge command is issued.
- 'Bank-accessed' status of the accessed bank is set to high, so that the other data access State Machine does not start operating on the same bank.
- LdPreCmdSM1 is asserted to load all the latency counters, which needs to be triggered off with a precharge command issue.
- Precharge-latency-counter is loaded with tRP value.
- Refresh-latency-counter is loaded with tRP value.
- Active-latency-counter is loaded with tRP value.

The conditions required to trigger a change from ST_DY_SEL to ST_DY_ACT are:

- The two parallel SDRAMC DASMs cannot run on the same bank of the same chip-select. (i.e. CtBankAxsdsM1 != '1')
- The dynamic-memory-controller should have the DyBusGnt and the DASM should have the shared 'lock' to proceed.
- No other row should be open in the bank.
- The latency to the active command should be satisfied.

While transitioning from SEL to ACT State,

- 'Bank-accessed' status of the accessed bank is set to high, so that the other State Machine does not start operating on the same bank.
- "Bank-status" is set as open.
- In the given bank, the row-currently-activated (RowAddrAxsds) is made as the last-row-accessed.
- The LdActCmdSM1 is asserted to load all the latency counters which needs to be triggered off with an active-command issue.
- Precharge-latency-counter is loaded with tRP.
- Active-latency-counter of the same bank is loaded with tRC.
- Active-latency-counter of the different banks is loaded with tRRD.
- RdWrCounter is loaded with ras-to-cas-delay.

The conditions required to trigger a change from ST_DY_SEL to ST_DY_DQMLow are:

- The dynamic-memory-controller should have the DyBusGnt.
- The FifoAvailable signal should be high indicating that the FIFO can receive the next burst of data. Otherwise consecutive read-requests can make the DASMs run one after another and lead to data-loss due to FIFO overflow.
- Data-occupancy-counter should have counted down to a value, where issuing of a CAS by the DASM will not cause data clash OR if the other machine is not in DTFTCH state, then CAS should be issued only if the other DASM is not accessing the same bank and has not asserted the lock.
- The row currently being accessed should already be open in the bank.
- The posted request should be of read type.
- The cas-latency should be equal to unity. For unity-cas-latency devices, the State-machine needs to transition to one extra intermediate state (ST_DY_DQMLow) to honour the two-clock-DQM-latency. If the DQM value is already zero, there is no necessity to transition to DQMLow State. The latency to READ issue is implicitly satisfied. Thus no separate check is done for that.

While transitioning from SEL to DQMLow,

- LockSDRAMC is asserted so that till the end of last read data arrival, the same State Machine continues on memory bus. Interleaving with second-command in the command-buffer is not allowed from READ-issue to last-read-data-arrival.

- 'Bank-accessed' status of the accessed bank is set to high, so that the other DASM does not start operating on the same bank.

The conditions required to trigger a change from ST_DY_SEL to ST_DY_RDISC are:

- The dynamic-memory-controller should have the DyBusGnt.
- The FifoAvailable signal should be high indicating that the FIFO can receive the next burst of data. Otherwise consecutive read-requests can make the DASMs run one after another and lead to data-loss due to FIFO overflow.
- The other DASM's data-occupancy-counter should have counted down to a value, where issuing of a CAS will not cause data clash OR if the other machine is not in DTFTCH state, then CAS should be issued only if the other DASM is not accessing the same bank and has not asserted the lock.
- The row currently being accessed should already be open in the bank.
- The posted request should be of read type.
- The cas-latency should be greater than unity. Otherwise, the State-machine has to transition to one extra intermediate State (ST_DY_DQMLow) to honour the two-clock-DQM-latency for unity-cas-latency devices. However, this condition is not explicitly written as the priority in the State-machine process takes care of this omission. The latency to READ issue is implicitly satisfied. Thus no separate check is done for that.

While transitioning from SEL to RDISC State,

- 'Bank-accessed' status of the accessed bank to high, so that the other State Machine does not start operating on the same bank.
- LockSDRAMC is asserted so that till the end of last read data arrival, the same State Machine continues on memory bus. Interleaving with second-command in the command-buffer is not allowed from READ-issue to last-read-data-arrival.
- If the access is auto-precharge enabled, then the bank will be closed at the end of the access. Thus the bank-status is updated accordingly. Also the latency-counter for the next active- command issue is activated.
- Active-latency-counter is loaded with tAPR + cas-latency + number of read data[8 in case of 16-bit and 4 in-case of 32 bit data]. The same value is also to be loaded in refresh-latency-counter.
- CAS-delay-counter is loaded with CAS-latency.
- LdDtOcpncy is asserted which results in data-occupancy-counter getting loaded. The data-occupancy-counter is used for 'spacing' the command asserted by other DASM, so that no data clash happens on external memory data bus.

The conditions required to trigger a change from ST_DY_SEL to ST_DY_WR are:

- The two parallel SDRAMC DASMs can not run on the same bank of the same chip-select. (i.e. CtBankAxsdSM1 /= '1').
- The dynamic-memory-controller should have the DyBusGnt and the DASM should have the shared 'lock' to proceed.
- The row currently being accessed should already be open in the bank.
- The posted request should be of write type.
- The latency to WRITE issue is implicitly satisfied. Thus no separate check is done for that.

While transitioning from SEL to WR State,

- Write Command is issued
- 'Bank-accessed' status of the accessed bank is set to high, so that the other access State Machine does not start operating on the same bank.
- LockSDRAMC is asserted so that till the end of last read data arrival, the same State Machine continues on memory bus. Interleaving with second-command in the command-buffer is not allowed from WRITE-issue to last-write-data. If the access is auto-precharge enabled, then the bank will be closed at the end of the access. Thus the bank-status is updated accordingly. Also the latency-counter for the next active-command issue is activated.
- When the chip-select is 16 bit wide the value to be loaded in active-latency-counter is tDAL + no. of write data[8 in case of 16-bit data]. The same value is also to be loaded in refresh-latency-counter.
- Value 6 is loaded in the number-of-write-data count which will result in eight data transfers to give the data-burst of required length.
- When the chip-select is 32 bit wide the value to be loaded in active-latency-counter is tDAL + no. of write data[4 in case of 32-bit data]. The same value is also to be loaded in refresh-latency-counter. Value 2 is loaded in the number-of-write-data count (this will result in 4 data transfers) to give the data-burst of required length.

- The active-low-tristate-enable for the tristate-buffer is to be pulled low for write transactions.
- LdDtOcpncy is asserted which results in data-occupancy-counter getting loaded. The data-occupancy-counter is used for 'spacing' the command asserted by other DASM, so that no data clash happens on external memory data bus.

ST_DY_PRE:

If DASM has to access a row in a bank, in which some other row is already open, then it enters ST_DY_PRE and issues a precharge in the process.

The conditions required to trigger a change from ST_DY_PRE to ST_DY_ACT are:

- The latency for active-command issue should be satisfied.
- DASM should have the shared 'lock' to proceed.

While transitioning from PRE to ACT State,

- Active command is issued.
- 'Bank-accessed' status of the accessed bank to high, so that other State Machine does not start operating on the same bank.
- Since an active command is given, "bank-status" is updated as open.
- In the given bank, the row-currently-activated (RowAddrAxsD) becomes the last-row-accessed.
- The LdActCmdSM1 is asserted to load all the latency counters which needs to be triggered off with an active-command issue.
- Precharge-latency-counter is loaded with tRP.
- Active-latency-counter of the same bank is loaded with tRC.
- Active-latency-counter of the different banks is loaded with tRRD.
- RdWrCounter is loaded with ras-to-cas-delay.

ST_DY_ACT:

If the row to be accessed is closed, then DASM transitions to ST_DY_ACT, issues an ACTIVE command in the process and opens the row.

The conditions required to trigger a change from ST_DY_ACT to ST_DY_DQMLow are:

- The other DASM's data-occupancy-counter should have counted down to a value, where issuing of a CAS will not cause data clash OR If the other machine is not in DTFTCH state, then CAS should be issued only if the other DASM has not asserted the lock.
- The posted request should be a read-request.
- The FifoAvailable signal should be high indicating that the FIFO can receive the next burst of data. Otherwise consecutive read-requests can make the DASMs run one after another and lead to data-loss due to FIFO overflow.
- The RAS to CAS latency should be satisfied.
- The device should be a unity cas latency device.
- The mask-value should not already be zero. If it is zero, then the read to the unity-cas-latency devices can be safely issued without transitioning to the DQM_LOW State.

While transitioning from ACT to DQMLow,

- LockSDRAMC is asserted so that till the end of last read data arrival, the same State Machine continues on memory bus. Interleaving with second-command in the command-buffer is not allowed from READ-issue to last-read-data-arrival.

The conditions required to trigger a change from ST_DY_ACT to ST_DY_RDISSUE are:

- DASM should have the shared 'lock' to proceed.
- The posted request should be a read-request.
- The RAS to CAS latency should be satisfied.
- The data occupancy counter of the other state machine should have expired.

While transitioning from ACT to RDISSUE,

- Read command is issued.

- LockSDRAMC is asserted so that till the end of last read data arrival, the same State Machine continues on memory bus. Interleaving with second-command in the command-buffer is not allowed from READ-issue to last-read-data-arrival.
- If the access is auto-precharge enabled, then the bank will be closed at the end of the access. Thus the bank-status is updated accordingly. Also the latency-counter for the next active-command issue is activated.
- The value to be loaded in active-latency-counter is $t_{APR} + \text{cas-latency} + \text{no. of read data}[8 \text{ in case of 16-bit and 4 in-case of 32 bit data}]$. The same value is also to be loaded in refresh-latency-counter.
- CAS-delay-counter is loaded with CAS-latency.
- LdDtOcpncy is asserted which results in data-occupancy-counter getting loaded. The data-occupancy-counter is used for 'spacing' the command asserted by other DASM, so that no data clash happens on external memory data bus.

The conditions required to trigger a change from ST_DY_ACT to ST_DY_WR are:

- DASM should have the shared 'lock' to proceed.
- The posted request should be a write-request.
- The RAS to CAS latency should be satisfied.

While transitioning from ACT to WRITE state,

- Write Command is issued
- 'Bank-accessed' status of the accessed bank is set to high, so that the other access State Machine does not start operating on the same bank.
- LockSDRAMC is asserted so that till the end of last read data arrival, the same state machine continues on memory bus. Interleaving with second-command in the command-buffer is not allowed from WRITE-issue to last-write-data.
- If the access is auto-precharge enabled, then the bank will be closed at the end of the access. Thus the bank-status is updated accordingly. Also the latency-counter for the next active-command issue is activated.
- When the chip-select is 16 bit wide the value to be loaded in active-latency-counter is $t_{DAL} + \text{no. of write data}[8 \text{ in case of 16-bit data}]$. The same value is also to be loaded in refresh-latency-counter.
- Value 6 is loaded in the number-of-write-data count which will result in eight data transfers to give the data-burst of required length.
- When the chip-select is 32 bit wide the value to be loaded in active-latency-counter is $t_{DAL} + \text{no. of write data}[4 \text{ in case of 32-bit data}]$. The same value is also to be loaded in refresh-latency-counter.
- Value 2 is loaded in the number-of-write-data count (this will result in 4 data transfers) to give the data-burst of required length.
- LdDtOcpncy is asserted which results in data-occupancy-counter getting loaded. The data-occupancy-counter is used for 'spacing' the command asserted by other DASM, so that no data clash happens on external memory data bus.

ST_DY_RDISUEE:

After issuing active command, DASM waits for ras-to-cas delay and moves to ST_DY_RDISUEE and issues the read-command. After issue of read, it waits for cas-latency in the ST_DY_RDISUEE state.

The conditions required to trigger a change from ST_DY_RDISUEE to ST_DY_DATAFTCH are:

- The CAS-delay, measuring the cas-latency has expired.

While transitioning from RDISUEE to DATAFTCH state

- DyFIFO block is indicated to start clocking in data.
- When the chip-select is 16 bit wide the value to be loaded in RdNumCnt is 7. [This will result in 8 read data transfers]
- When the chip-select is 32 bit wide the value to be loaded in RdNumCnt is 3. [This will result in 4 read data transfers]

ST_DY_DATAFTCH:

In DATAFTCH state, the memory-device has started giving out the data and these data are clocked into the pad-interface FIFO.

The condition required to trigger a change from ST_DY_DATAFTCH to ST_DY_IDLE is expiry of data-occupancy counter of the other DASM

While transitioning from DATAFTCH to IDLE State,

- The shared 'lock' is unset to allow the other State Machine to proceed if it is pending.
- It is indicated that current State Machine does not require the external-memory-bus as it has finished the burst.
- The bank is no longer accessed by this State Machine, thus the BankAxsD is reset.
- When the dynamic memory's clock-enable (CKE) is configured to be pulled low (whenever it is idle), if chip-select accessed by other DASM is not the same and no refresh request is pending, then the clock enable is pulled low.
- The read-data burst from the dynamic-memory device has stopped. Thus the DyBstStrtS1 is de-asserted.
- Indicate DyFIFO block to start clocking in data.

ST_DY_WR:

In this State, DASM gives out data from its buffer to the external memory.

The conditions required to trigger a change from ST_DY_WR to ST_DY_IDLE are:

- The burst of data-transfers, 4 in case of 32 bit databus and 8 in case of 16-bit data bus should be complete.

While transitioning from WR to IDLE State,

- The shared 'lock' is unset to allow other State Machine to proceed if it is pending.
- It is indicated that this State Machine does not require the external-memory-bus as it has finished the burst.
- The bank is no longer accessed by DASM, thus the BankAxsD flag is reset.
- When the dynamic memory's clock-enable (CKE) is configured to be pulled low (whenever it is idle), if chip-select accessed by other DASM is not the same and no refresh request is pending, then the clock enable is pulled low.
- From the last-data registered, there should be a latency of tWR to the next precharge command. The latency counter for this is activated. The active-low-tristate-enable for the tristate-buffer is to be pulled low for write transactions.
- The write-data is to be driven out with respect to WrNumCnt value irrespective of whether there is a transition from ST_DY_WR to ST_DY_IDLE or the State Machine is staying in ST_DY_WR itself.

ST_DY_DQMLow:

In this State, an automatic transition is made to ST_DY_RDISUE State.

While transitioning to ST_DY_RDISUE,

- Read command is issued. If the access is auto-precharge enabled, then the bank will be closed at the end of the access. Thus the bank-status is updated accordingly.
- Also the latency-counter for the next active-command issue is activated.
- The value to be loaded in active-latency-counter is tAPR + cas-latency + number of read data[8 in case of 16-bit and 4 in-case of 32 bit data]. The same value is also to be loaded in refresh-latency-counter.
- CAS-delay-counter is loaded with CAS-latency.
- LdDtOcpncy is asserted which results in data-occupancy-counter getting loaded. The data-occupancy-counter is used for 'spacing' the command asserted by other DASM, so that no data clash happens on external memory data bus.

Figure 5.8-3 shows the State diagram of the Dynamic Controller Access State Machine

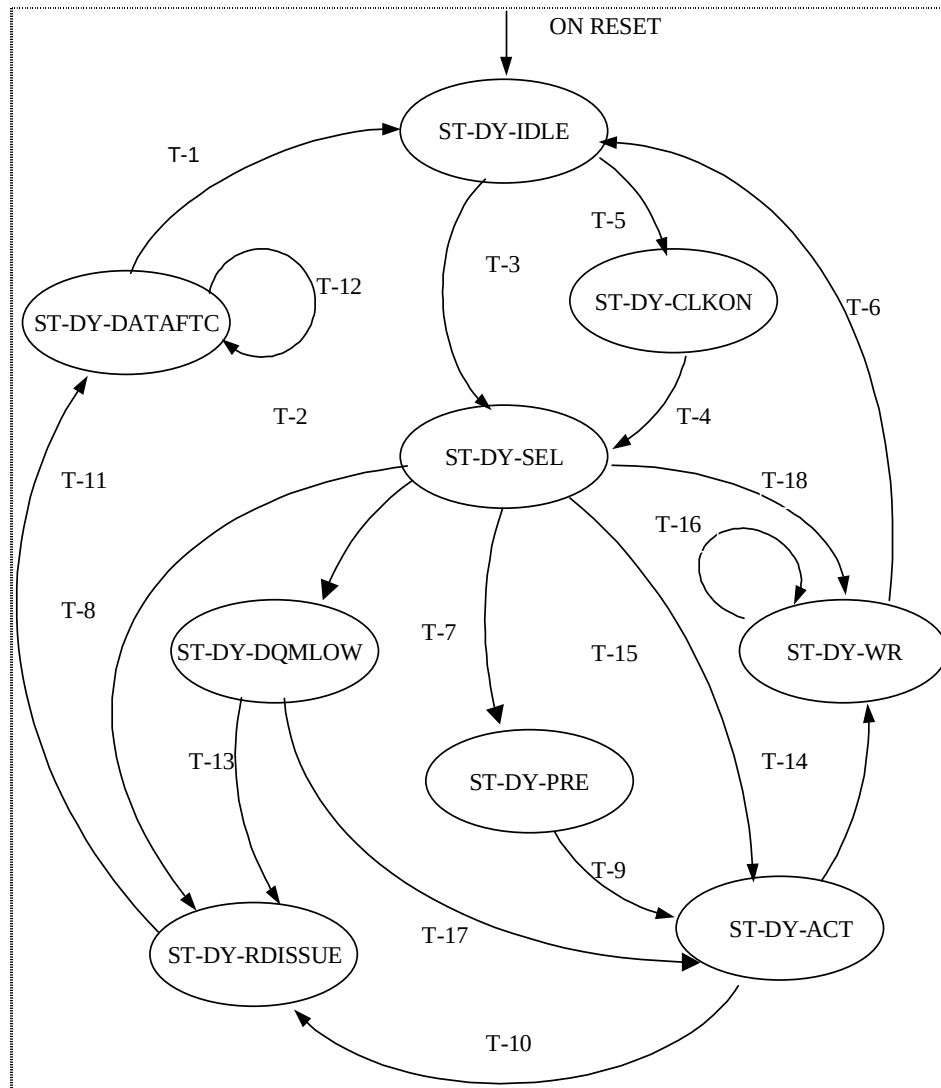


Figure 5.8-3 Dynamic Memory Controller Access State Machine

Table 5.8-1 lists the transitions for the data access state machine.

TRANSITION NO.	DESCRIPTION	QUALIFYING CONDITION
T-1	DATAFTCH to IDLE	((RdNumCnt = "000") and (FifoHE = '1'))
T-2	SEL to DQMLow	((CtBankAxsdsM1 != '1') and (DyBusGnt = '1') and (LockSDRAMC != '1') and (CtBankClsdSM1 = '0') and (IfEqual(LstRwlnBnk4SM1, RwAxsdsM1, NumRowSM1)) and (AxsTypeRd = '1') and (DyCasForSM1 = "01") and (DyDQMHighQ = '1'))
T-3	IDLE to SEL	((MPMCEnQ = '1') and (LowPwrModQ = '0') and (ModeDone = '1'))
T-4	CLKON to SEL	NxtDyAxsSM1St <= ST_DY_SEL; SetClkEnCSSM1 <= '1';

TRANSITION NO.	DESCRIPTION	QUALIFYING CONDITION
T-5	IDLE to CLKON	No qualifying condition. On next clock edge transition happens.
T-6	WR to IDLE	(WrNumCnt = "000")
T-7	SEL to PRE	((CtBankAxsdsM1 != '1') and (DyBusGnt = '1') and (LockSDRAMC != '1') and (CtBankClsdSM1 = '0') and not(IfEqual(LstRwlnBnk4SM1, RwAxsdsM1, NumRowSM1))) and (PreCnt4SM1 = "0000"))
T-8	SEL to RDISSUE	((CtBankAxsdsM1 != '1') and (DyBusGnt = '1') and (LockSDRAMC != '1') and (CtBankClsdSM1 = '0') and (IfEqual(LstRwlnBnk4SM1, RwAxsdsM1, NumRowSM1)) and (AxsTypeRd = '1'))
T-9	PRE to ACT	((LockSDRAMC != '1') and (ActCnt4SM1 = "00000"))
T-10	ACT to RDISSUE	((LockSDRAMC != '1') and (AxsTypeRd = '1') and (RasCntr = "01"))
T-11	RDISSUE to DATAFTCH	(CasCntr = "01")
T-12	Self-Transition	The number of transfers is not equal to zero.
T-13	DQMLOW to RDISSUE	NxtDyAxsSM1St <= ST_DY_RDISSUE; DyCmdSM1 <= CMD_READ; DyAddrSM1 <= ColAxsdsM1(14 downto 11) & AutoPreSM1 & ColAxsdsM1(9 downto 0);
T-14	ACT to WR	((LockSDRAMC != '1') and (AxsTypeRd = '0') and (RasCntr = "01"))
T-15	SEL to ACT	((CtBankAxsdsM1 != '1') and (DyBusGnt = '1') and (LockSDRAMC != '1') and (CtBankClsdSM1 = '1') and (ActCnt4SM1 = "00000"))
T-16	Self-Transition	The number of transfers is not equal to zero.
T-17	ACT to DQMLOW	((LockSDRAMC != '1') and (AxsTypeRd = '1') and (RasCntr = "01") and (DyCasForSM1 = "01") and (DyDQMHighQ = '1'))
T-18	SEL to WR	((CtBankAxsdsM1 != '1') and (DyBusGnt = '1') and (LockSDRAMC != '1') and (CtBankClsdSM1 = '0') and (IfEqual(LstRwlnBnk4SM1, RwAxsdsM1, NumRowSM1)) and (AxsTypeRd = '0'))

Table 5.8-1 Transition table for the access state machine

5.8.4 Dynamic Memory Mode State Machine (MpmcDyModSM)

This block describes the Mode State Machine, which performs Mode Register Programming on the SDRAM chip-selects. This State Machine executes the mode initialisation sequence for SDRAM devices and SyncFlash devices. If in the system, there is a mix of SDRAMs and SyncFlashes on the four dynamic-memory-chip-selects then the mode initialisation sequence of SDRAMs should be followed. The normal sdram initialisation sequence does not affect the

initialisation of SyncFlashes. However, if all the chip selects are populated with syncflashes, then the SyncFlash initialisation sequence can be followed for faster programming.

Mode State Machine transitions through the following states:

- ST_MOD_IDLE: Dynamic memory Mode state machine idle state
- ST_MOD_NOP: Dynamic memory Mode state machine NOP state
- ST_MOD_PALL: Dynamic memory Mode state machine PALL state
- ST_MOD_WPALL: Dynamic memory Mode state machine wait for PALL state
- ST_MOD_MODE: Dynamic memory Mode state machine Mode state
- ST_MOD_MRSREADY: Dynamic memory Mode state machine Mode Register Set ready state
- ST_MOD_MRS: Dynamic memory Mode state machine Mode Register Set state
- ST_MOD_MRSWAIT: Dynamic memory Mode state machine Mode Register Set wait state
- ST_MOD_CKECKON: Dynamic memory Mode state machine CKE and clock on state.

ST_MOD_IDLE:

On power on reset (nPOR), the Mode State Machine enters Idle State. When the sdram initialisation register bits reflect the NOP or mode command the State Machine enters the CKE and clock on State. The bus request is raised, and the CKE and clocks are enabled during transitioning from the idle State.

ST_MOD_CKECKON:

In the CKE and clock on State, the State Machine transitions to the Mode State if a mode command is issued. If a NOP command is issued, it transitions to the NOP State after disabling the bus request.

ST_MOD_NOP:

In the NOP State, if the PALL command is issued, the State Machine transitions to the PALL State. Otherwise it stays in the NOP State.

ST_MOD_PALL:

In PALL State, if bus grant is present, the precharge all command is issued and the State is changed to wait for precharge command, and the bus request is taken low.

ST_MOD_WPALL:

In the wait for precharge State, if a mode command is issued using sdram initialisation bits, then State Machine transitions to the Mode State.

ST_MOD_MODE:

In the Mode State, if a normal command is issued, the State Machine transitions to the idle State. Otherwise, a posted read request will make the State Machine transition from the Mode State to MRS ready State.

ST_MOD_MRSREADY:

In the MRS ready State, the memory address coming from buffer is clocked. The State Machine will transition from MRS ready to MRS State by default.

ST_MOD_MRS:

In the MRS State the mode command is issued when bus grant is present. The mode counter is loaded and the State Machine transitions to the MRS wait State.

ST_MOD_MRSWAIT:

In the MRS wait State, the mode counter is decremented. When the counter expires, the State Machine transitions to the Mode State.

Figure 5.8-4 gives the State transitions for Mode State Machine.

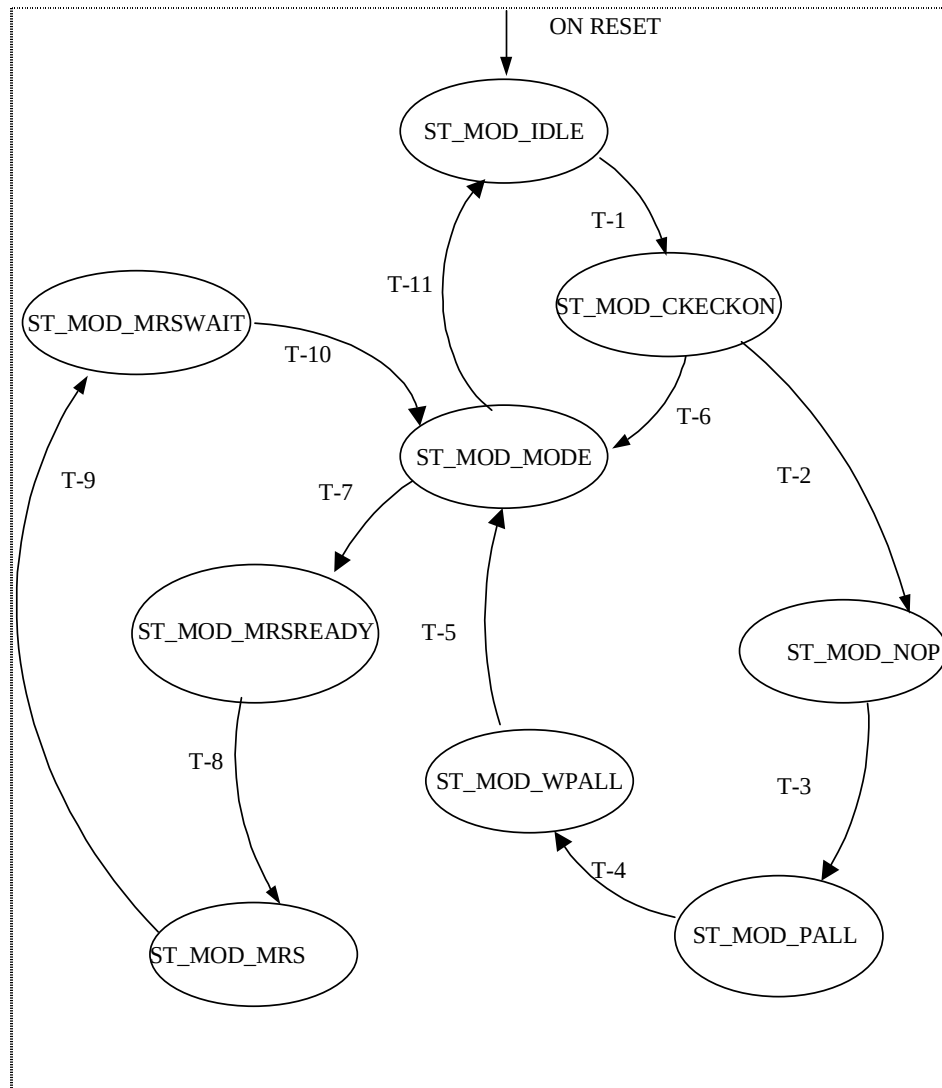


Figure 5.8-4 Dynamic Memory Controller Mode State Machine

Table 5.8-2 lists the transitions for Mode State Machine.

Transition	Description	Condition
T1	IDLE to CKECKON	(SlowClkM = '1')and(MPMCEnQ = '1')and ((SdramInitQ = NOISSUE) or (SdramInitQ = MODISSUE))
T2	CKECKON to NOP	(SdramInitQ = NOISSUE) and (DyBusGnt = '1')
T3	NOP to PALL	(SdramInitQ = PALISSUE) and (SlowClkM = '1')
T4	PALL to WPALL	DyBusGnt = '1'
T5	WPALL to MODE	(SdramInitQ = MODISSUE) and (SlowClkM = '1')
T6	CKECKON to MODE	SdramInitQ = MODISSUE
T7	MODE to MRSREADY	(DyPostRReq = '1') and (SlowClkM = '1') and (SdramInitQ /= NORMALISSUE)

T8	MRSREADY to MRS	Automatic Transition
T9	MRS to MRSWAIT	DyBusGnt = '1'
T10	MRSWAIT to MODE	ModCntr = "0000"
T11	MODE to IDLE	(SdramInitQ = NORMALISSUE) and (SlowClkM = '1')

Table 5.8-2 Transition Table for Dynamic Controller Mode State Machine

5.8.5 Dynamic Memory Refresh State Machine (MpmcDyRefSM)

This block describes the refresh State Machine, which performs self and autorefresh on the SDRAM chip-selects. This block performs Auto Refresh when AutoRefReq signal is asserted, and Self Refresh when SRefReq is asserted.

Auto Refresh Sequence:

Precharge is issued to all the chip selects, in a single clock. After precharge latency has expired, Auto Refresh is issued to all four chip selects in four different cycles. After Refresh Latency is expired the State Machine returns to idle State.

Self Refresh Sequence:

Precharge is issued to all the chip selects, in a single clock. After precharge latency has expired, Self Refresh is issued to all four chip-selects in four different cycles. After Refresh Latency is expired the State Machine waits for Self Refresh Request to go low, before returning to Idle State.

The Refresh State Machine transitions through following states:

- ST_REF_IDLE: Dynamic memory refresh state machine idle state
- ST_REF_REQ: Dynamic memory refresh state machine request state
- ST_REF_CKEON: Dynamic memory refresh state machine CKE on state
- ST_REF_PALL: Dynamic memory refresh state machine PALL state
- ST_REF_WPALL: Dynamic memory refresh state machine wait for PALL state
- ST_REF_REFRESH: Dynamic memory refresh state machine refresh state
- ST_REF_WREFRESH: Dynamic memory refresh state machine wait for refresh state
- ST_REF_WSREFLOW: Dynamic memory refresh state machine wait for SREFstate
- ST_REF_WSREFEXIT: Dynamic memory refresh state machine wait for Self –refresh exit state

ST_REF_IDLE:

When autorefresh signal or selfrefresh signal is asserted, the State Machine will transition from Idle State to bus request State, if both the data access State Machines are idle. Bus request is asserted while transitioning from Idle State.

ST_REF_REQ:

In the Bus Request State the clock status is checked. If the clock has been stopped, then start clock command is issued. If CKE is also low when clock is stopped, then the state machine transitions to CKE switch on State. If the clock is not stopped but if CKE is low then CKE is switched on, and Next State is changed to PALL.

ST_REF_CKEON:

In this State the clock is enabled. An automatic transition is made the PALL State.

ST_REF_PALL:

In PALL State, if Bus grant is present, PALL command is issued to the memory interface. And the State is changed to Wait for PALL, after loading the PALL counter.

ST_REF_WPALL:

In the Wait for PALL State, after the PALL counter has expired, the State is changed to Refresh State.

ST_REF_REFRESH:

In the Refresh State, four refresh commands to four different chip selects are issued. Here CKE is held low if it is autorefresh. After four refresh commands are issued, REFCntr is loaded, and the State is changed to Wait for Refresh.

ST_REF_WREFRESH:

After the REFCntr has expired in the Wait for Refresh State, State is changed to Idle if it is Autorefresh; otherwise it is changed for Wait for Self refresh signal to go low. Bus request is de-asserted while transitioning from this State.

ST_REF_WSREFLOW:

On power-on-reset (nPOR), the State Machine is in self refresh State. In the WSREFLOW State, the selfrefresh request signal is sampled, and when it is low, then a SREF counter is loaded, and the State is changed to SREFEXIT.

ST_REF_WSREFEXIT:

From SREFEXIT, the State transitions to Idle when the counter has expired.

Figure 5.8-5 shows the State diagram for Refresh State Machine

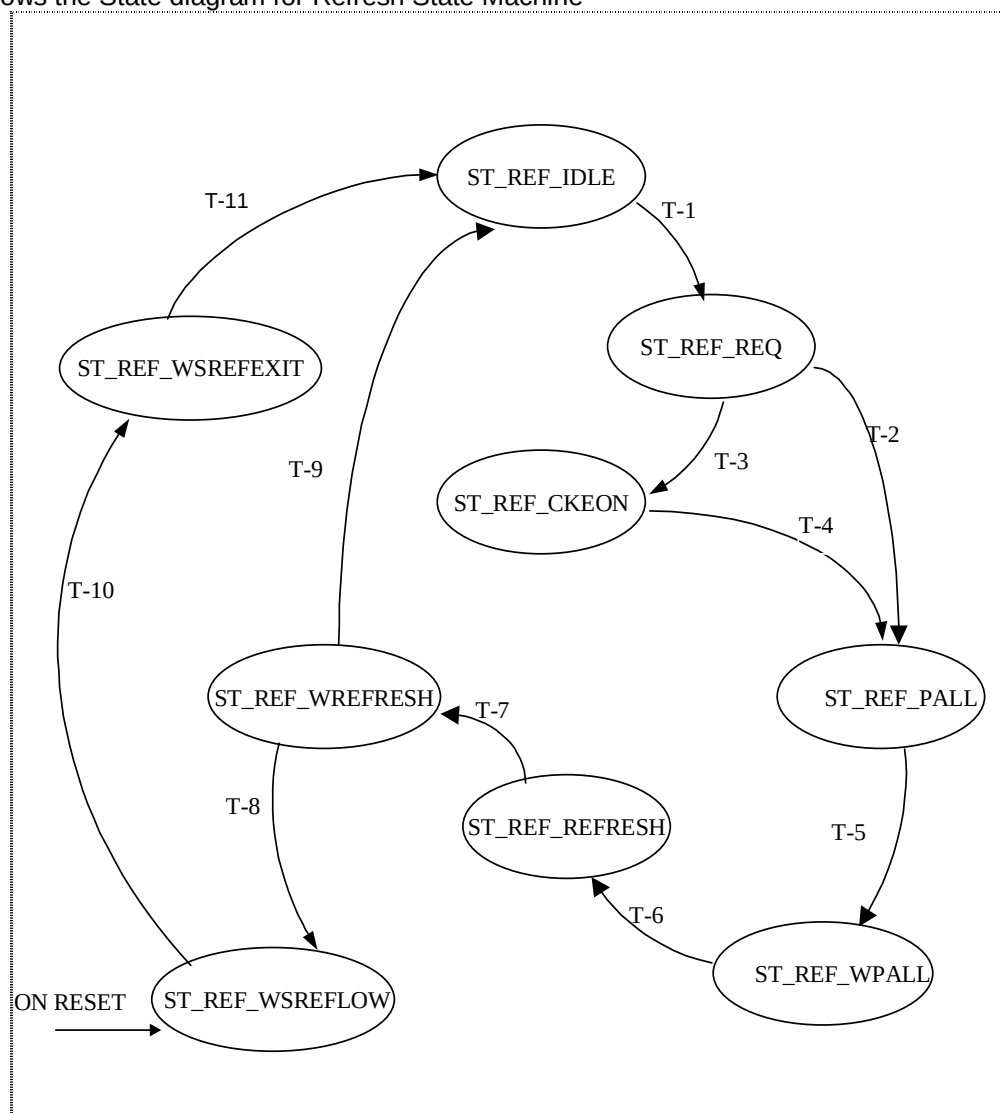


Figure 5.8-5 Refresh State Machine

Table 5.8-3 lists the transitions for Refresh State Machine

Transition	Description	Condition
T1	IDLE to REFREQ	((ARefReqLatQ = '1') and (AutoRefReq = '1') and (SRefReq = '1')) and (MPMCEnQ = '1'))
T2	REFREQ to CKEON	((DyAxsSM1Idle and DyAxsSM2Idle and DySyFlashIdle) = '1') and (ClkStpdStat = '1') and (DyClkEnQ = '0')
T3	REFREQ to PALL	((DyAxsSM1Idle and DyAxsSM2Idle and DySyFlashIdle) = '1') and ((ClkStpdStat = '1') and (DyClkEnQ = '1')) or (ClkStpdStat = '1')
T4	CKEON to PALL	Automatic Transition
T5	PALL to WPALL	(DyBusGnt = '1')
T6	WPALL to REFRESH	(PALLCntrQ = "0000") and (RefLatCnt = "00000")
T7	REFRESH to WREFRESH	(CS2RefreshQ = "0111")
T8	WREFRESH to WSREFLOW	(REFCntrQ = "00000") and (ARefreshQ = '0')
T9	WREFRESH to IDLE	(REFCntrQ = "00000") and (ARefreshQ = '1')
T10	WSREFLOW to WSREFEXIT	(SRefReq = '0')
T11	WSREFEXIT to IDLE	(SREFCntrQ = "000000")

Table 5.8-3 Transition Table for Refresh State Machine

5.8.6 Dynamic Memory Controller SyncFlash State Machine (MpmcDySyFlash)

This block describes the State-machine for executing the flash command set on Micron SyncFlash memories. Address bits 27 and 26, transfer type (read or write), and HWDATA value if the transfer is a write) are used to indicate the type of flash command/transfer to execute. For flash memory reads and writes, the State Machine ensures the correct sequence of LCR-Active-Read/Write.

Flash State Machine transitions through the following states:

- ST_DYF_IDLE: Dynamic memory SyncFlash state machine idle state
- ST_DYF_CLKON: Dynamic memory SyncFlash state machine clock on state
- ST_DYF_SEL: Dynamic memory SyncFlash state machine select state
- ST_DYF_LCR: Dynamic memory SyncFlash state machine load command register state
- ST_DYF_ACT: Dynamic memory SyncFlash state machine activate state
- ST_DYF_WR: Dynamic memory SyncFlash state machine write state
- ST_DYF_RD: Dynamic memory SyncFlash state machine read state
- ST_DYF_WAIT: Dynamic memory SyncFlash state machine wait state

ST_DYF_IDLE:

On power on, the State Machine moves to the idle State. In idle State, if a memory access request to a syncflash is received, the State Machine transitions to the "clock on" State after enabling the clocks and raising the memory bus request.

ST_DYF_CLKON:

In the "clock on" State (ST_DYF_CLKON), the State Machine sets the CKE of the syncflash device and moves to the "selected" State.

ST_DYF_SEL:

In the "selected" State (ST_DYF_SEL) where it waits for the bus grant to be asserted. When the external bus grant is received, the command code extracted from the write-data and the AHB address bits 27 and 26 is driven on the address lines and the State Machine transitions into the appropriate command issue State. The command issue states are the load-command-register (LCR) issue State (ST_DYF_LCR), active command issue State (ST_DYF_ACT), write State (ST_DYF_WR) and read State (ST_DYF_RD).

ST_DYF_LCR:

In case of 'Clear-Status-Register' command, only one LCR is given on the command lines. After that the operation is complete and the FCSM returns to idle State. Thus the State is transitioned from LCR to IDLE when the write-data

indicates com-code for Clear-Status-Register. If it is not a 'Clear-Status-Register' command then the State transitioning is done from LCR to ACT.

ST_DYF_ACT:

When the posted request is a write-request and the RAS to CAS latencies are satisfied, then the State Machine transitions to the Write State. When the posted request is read, and when RAS to CAS latencies are satisfied, then State Machine transitions to Read State.

ST_DYF_WR:

Writes are always single word writes. The FCSM stays in ST_DYF_WR for a clock and then moves to T_DYF_WAIT. If the CE bit in MpmcDynamicControl register indicates the CKE to be pulled low, then CKE is pulled low while transitioning to WAIT State.

ST_DYF_RD:

The FCSM stays in this State for 8 data, which the device always outputs as its burst size is programmed to 8. While staying in this State the FCSM asserts the DyBrstStrtFI (indicating FIFO to take in data) for two clocks. DyBrstStrtFI can be asserted for any two data out of the eight, as the same data is returned by the device for eight consecutive clocks. Currently DyBrstStrtFI is asserted for 4th and 5th data.

ST_DYF_WAIT:

This State is maintained for two clocks to maintain a separation between UseMem-assertion and DyUseMem de-assertion. Then the State Machine transitions to Idle State.

Figure 5.8-6 gives the State diagram for Flash State Machine

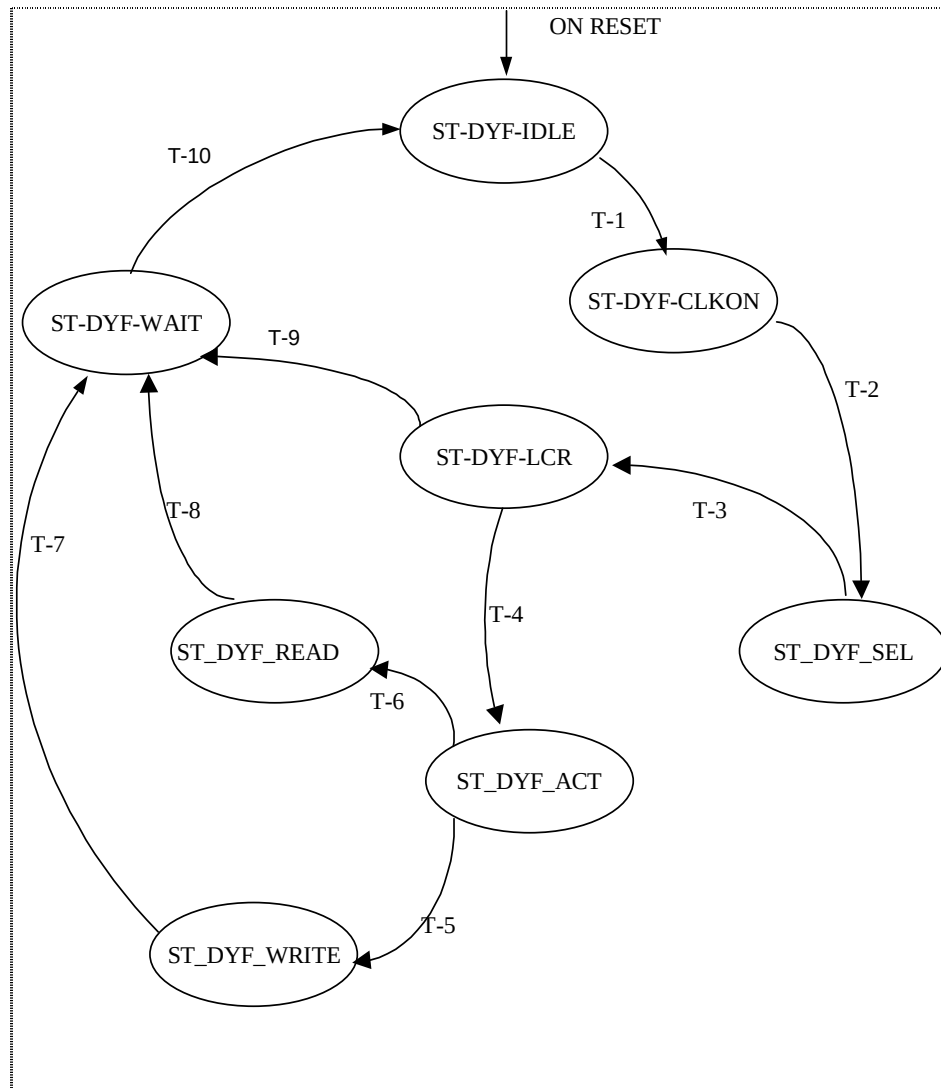


Figure 5.8-6 Dynamic Memory Controller SyncFlash State Machine

Table 5.8-4 lists the transitions for Flash State Machine

Transition	Description	Condition
T1	IDLE to CLKON	(MPMCEnQ = '1') and (LowPwrModQ = '0') and (ModeDone = '1')
T2	CLKON to SEL	Automatic Transition
T3	SEL to LCR	DyBusGnt = '1'
T4	LCR to ACT	(WrDtFISM(15 downto 8) /= COMCODE_CSR)
T5	ACT to WRITE	(RasCntr = "00") and (AxsTypRdFISM = '0')
T6	ACT to READ	(RasCntr = "00") and (AxsTypRdFISM = '1')
T7	WRITE to WAIT	Automatic Transition
T8	READ to WAIT	(CasDataCntr = "0000")

T9	LCR to WAIT	(WrDtFISM(15 downto 8) = COMCODE_CSR)
T10	WAIT to IDLE	(OneCyclnWait = '1')

Table 5.8-4 Transition table for Dynamic Memory Controller SyncFlash State Machine

5.8.7 Dynamic Memory Controller Receive FIFO (MpmcDyFIFO)

The Dynamic memory controller receive FIFO is used to store the data streaming in from the dynamic memory during the quadword reads. The FIFO provides the storage for the read data if the MPMCCLK is faster than HCLK and the AHB is not able to read data at the rate at which the dynamic memory is providing the data.

The MpmcDyFIFO block is a structural block contains the following functionality in the 3 different sub-blocks:

- FIFO registers (MpmcDyRegFile)
- FIFO Control logic (MpmcDyFCntl)
- FBCLK domain to HCLK domain conversion (MpmcDyFBtoH)

FIFO registers

The Dynamic Controller Receive FIFO consists of 8 deep 36-bit wide registers. Each 36-bit register can store a 32-bit memory read data and the A5, A4, A3, A2 bits of the memory address. The state machines supply the address bits A5, A4, A3 and A2 along with the memory access request. These 4 bits need to be stored and incremented for each read data location that is stored. These bits are used to write the memory read data into the correct word of the QQword buffer. Hence there is one A5, A4, A3, A2 storage location each for one location in the FIFO. Read data from the memory is written into the location in the FIFO pointed to by the current value of the WrPtr[1:0] (write pointer) signal on the rising edge of MPMCFBCLKIN on which the DyFWr signal is sampled high. Data in the FIFO location pointed to by the RdPtr[1:0] signal is always driven on the DyMemRData[31:0] output.

FIFO Control logic

This block controls the read pointer and the write pointer. The read and write pointer values are compared to deduce the FIFO fill level. Writes to FIFO occur through the Memory Interface of the Controller. The Data Access State Machine's and the Sync Flash State Machine's assert the Burst Start signals, indicating that data needs to be clocked in.

Data is clocked into the FIFO as long as the burst start signals are kept asserted. The memory data width can be 16-bits or 32-bits. In case of 16-bits, two memory data are combined to form one 32-bit word. The data returned from FIFO is always 32-bits width.

Along with the burst start signals, certain tags are received from the buffer unit to be stored and returned with each data at the end of a read access. Since the incoming data for a given dynamic memory request always consists of four 32-bits of data, and the tag is same for all four words of data, there are only two storage locations for each tag – one for each data access state machine. The tags that are stored in this way are the BufAccCode[8:0] indicating the quadword buffer or the AHB port which is the requestor for the current access.

The FIFO asserts the DyUseMem signal to indicate the data is ready for consumption. Normally, for one SDRAM access request, DyUseMemMC is asserted for four clock cycles to indicate the transfer of 4 words of data. For SyncFlash operation, DyUseMemMC is asserted only for the first data transfer cycle. The remaining data returned by the SyncFlash are the same as the first one and thus should not be returned to the AHB. So in case of SyncFlashes, the DyUseMemMC is asserted for one clock cycle and deasserted for three clock cycles (according to the RdPtr value). If the read pointer indicates the lower four words in FIFO (RdPtr(2) = '0'), then BufAccCode1 is assigned to DyMemAHBIdQ[8:0]. If it points to upper four words (RdPtr(2) = '1'), then BufAccCode2 is assigned. This signal indicates the access requestor code for the current dynamic memory read. The MPMCDyFCntl block also generates the FIFO status signals FifoHE.

FBCLK domain to HCLK domain conversion

MpmcDyFBtoH block converts the dynamic memory receive FIFO data from MPMCFBCLKIN domain to the HCLK domain. This block also generates some of the signals that are to be returned to the quadword buffers along with the read data. The dynamic memory controller always returns all the valid data. So whenever the DyUseMem is high, all the DyMemByte lines are driven high.

5.8.8 Dynamic Memory Controller Internal Register Block (MpmcDyIntRegBlk)

This block stores the internal registers and flags to be used by State Machines. This block stores different shared flags and memory-bank-status of the 16 different banks in the 4 dynamic memory chip selects. The flags and internal registers common to all the state machines are stored in this block.

For many of the common flags and signals, “Set flag” and “Unset flag” signals are received from the corresponding blocks, and corresponding action is taken. The DyBusReq signal, which is used to request the control of the external memory bus is asserted and deasserted in this manner. The ClkStpd signals are also generated in this manner. The ClkStpd signal is used to indicate that the clock to the memory, the MPMCCCLKOUT[3:0] should be disabled. The other flags that are generated in this way are the BankAxs and BankClsd flags for each of the 16 banks.

For the registers used by the state machines, which are common for all the banks, this block gets the chip select and bank select information from the state machine, and outputs the relevant bank information to the state machines. The signals used to indicate the status of the banks that are being accessed by the DASMs and the syncflash state machine are - CtBankAxsSM1/2, CtBankClsdSM1/2, LstRwInBnk4SM1/2, DyRasForSM1/2, DyCasForSM1/2, DyRasForFISM, DyCasForFISM, NumColSM1/2, NumRowSM1/2, MWSM1Co and MWSM2Co.

LockSDRAMC is a signal that indicates that a dynamic memory state machine is currently accessing the memory. DyBufFull is a signal which is generated to indicate to the buffer unit that the dynamic memory controller is servicing two memory access requests and cannot handle any more requests. DyBufFull is asserted, when the dynamic-memory-controller-portion is not able to accept any more DyPostRReq or DyPostWReq. The conditions for assertion are

- Whenever any of the DASM is in idle state, it means that one more DyPost(R/W)Req can be accepted. DyBufFull can be asserted only when both the DASMs are in non-idle state.
- When refresh (auto/self) is continuing, no Post(R/W)Req should be asserted. Thus the DyBufFull is asserted when the refresh-state-machine is active.
- The FlashAccess state-machine (which performs the flash command set operations), can not buffer two requests. Thus the DyBufFull is pulled high, whenever the flash-state machine is in non idle state.

If any one of the state-machines is not idle, then the DyMemBusy is asserted. Only when all the machines are idle, DyMemBusy is de-asserted.

DtOcpncyVal (data occupancy value to be loaded into the counter) and DyLZDstnce (number of clocks after which the data lines are driven to low-impedance after issuing CAS) are determined in DyIntRegBlk block. DtOcpncyVal and DyLZDstnce are used for maintaining 'proper' spacing between the commands issued by the two data-access-state-machines (MpmcDyAxsSM1 and MpmcDyAxsSM2) so that there are no data clashes on the external memory data bus. DtOcpncyVal and DyLZDstnce are function of CAS latency and clock-scheme (command delayed or clock delayed).

5.8.9 Dynamic Memory Latency Counter Block (MpmcDyLatCntr)

This block has a set of counters which track the latency between two command issues to a bank. One bank can be accessed by two DASMs, but the second DASM should honour the latencies, which arise due to the issue of commands by first DASM. Thus these counters are bank-specific. When a command is issued, the related counters are loaded with required-latency values. The DASM checks the expiry-status of the counters to issue subsequent commands.

The counters maintained by this block are Precharge, Active and Refresh Latency Counters. There are 16 Precharge counters, one for each bank, 16 Active counters one for each bank and one refresh counter to cater for the refresh latencies.

The block gets Chip and Bank information from both the Access State Machines for it to identify which bank of which chip is being accessed by the State Machines. This information is derived through signals ChipSM1 and BankSM1 for

State Machine 1, and ChipSM2 and BankSM2 for State Machine 2. Along with the chip and bank information, the State Machines also give the information of the command for which the counters are being loaded. This information is in the signals, LdPreCmdSM1, LdActCmdSM1, LdRdCmdSM1, LdRdACmdSM1, LdWrCmdSM1 and LdWrACmdSM1 from State Machine 1, and LdPreCmdSM2, LdActCmdSM2, LdRdCmdSM2, LdRdACmdSM2, LdWrCmdSM2 and LdWrACmdSM2 from State Machine 2. **Table 5.8-5** describes the signal name, and the command issued along with it.

Signal Name	Command
LdPreCmdSM1, LdPreCmdSM2	Precharge Command
LdActCmdSM1, LdActCmdSM2	Active Command
LdRdCmdSM1, LdRdCmdSM2	Read Command
LdWrCmdSM1, LdWrCmdSM2	Write Command
LdRdACmdSM1, LdRdACmdSM2	Read Command with Autoprecharge
LdWrACmdSM1, LdWrACmdSM2	Write Command with AutoPrecharge

Table 5.8-5 Commands issued to Latency Counter

With the command information, the latency to be loaded is also supplied by the State Machines. This information is in the signals, PreCntValSM1, RefCntValSM1, ActCntValSM1, DifActCntValSM1 from State Machine 1 and PreCntValSM2, RefCntValSM2, ActCntValSM2, DifActCntValSM2 from State Machine 2. **Table 5.8-6** describes the latency counter name and the counter for which it is intended to.

Signal Name	Command
PreCntValSM1	To load Precharge Latency Counter pointed to by ChipSM1 and BankSM1.
PreCntValSM2	To load Precharge Latency Counter pointed to by ChipSM2 and BankSM2.
RefCntValSM1, RefCntValSM2	To load the Refresh Latency Counter
ActCntValSM1	To load Active Latency Counter pointed to by ChipSM1 and BankSM1.
ActCntValSM2	To load Active Latency Counter pointed to by ChipSM2 and BankSM2.
DifActCntValSM1	To load all Active Latency Counters for ChipSM1 other than which is pointed to by BankSM1.
DifActCntValSM2	To load all Active Latency Counters for ChipSM2 other than which is pointed to by BankSM2

Table 5.8-6 Latency Counters

All the latency counters are loaded only when the value being loaded is higher than the current value of the counter. These counters are decremented every clock until they reach zero. The Data State Machines check that the value has reached zero before issuing a corresponding command.

5.9 Static Memory Controller (MpmcStMemCntl)

In this section, the sub-blocks within the Static Memory Controller are described in detail. The static memory controller block controls all the static memory accesses.

Before description, three keywords namely transfer size; memory width, burst size and extra read are described. These three keywords have been repeatedly used in the document.

- **Transfer size:** For unbuffered memory accesses from the AHB interface, it indicates the size of the data transfer as indicates by HSIZE. For buffered accesses, it indicates the size of the data that is to be transferred in the current memory access.
- **Memory width:** It is the memory bus width of the static memory being accessed.
- **Burst size:** It is the size of the AHB burst indicated by HBURST and used for unbuffered prefetch read bursts.
- **Extra read:** A static memory read access that has already been initiated to the memory is termed an extra read or a extra prefetch read when an AHB burst read is terminated prematurely due to re-arbitration or due to busy transfers in the burst.

5.9.1 Static memory register block (MpmcStRegBlk)

Details of the MPMC static memory register block functionality are described in this sub-section. The static memory register block is instantiated four times in the static memory controller to realise the register blocks associated with each static memory chip select. The static memory register block select (StBlkSelCo2[3:0]) signal is used to select one static memory register block among the four.

Each static memory controller register block implements the following functionality:

- Static memory registers
- Register write logic
- Register read logic

Registers implemented in the static memory register block

- The static memory configuration register, MPMCStConfig. This register is used to program the configuration of the static memory connected to the chip select associated with the static memory register block. This register contains the write protect bit (StWrProtQ), the buffer enable bit (StBufEnQ), the extended write enable bit (StExdWrEnQ), the byte lane State bit (StByteLaneStateQ), the chip select polarity bit (StCSPolQ), the page mode enable bit (StPageModeRdQ) and the memory width bits (ChipSelMWQ[1:0]).
- The static memory timing control registers. These include the static memory write access time register (MPMCStWtWrQ), normal read access time register (MPMCStWtRdQ), page mode read access time register (MPMCStWtPgQ), the turnaround time register (MPMCStWtTurnQ), the write enable delay time register (MPMCStWtWenQ) and the output enable delay time register (MPMCStWtOenQ). The static memory controller issues static memory accesses with the timings programmed in these registers.

Register write logic

The register write logic for the static memory register block is as in **Figure 5.9-1**. A static memory register is written with the data on HWDATAREG when the write enable for that register is sampled high on the rising edge of HCLK. The register to be written is selected by decoding the sampled registered address (HAddrReg4to2Q2[4:0]) and the static memory register block select (StBlkSelCo2) from the AHB register interface block. The write enable signals are generated by combining the address decode with the sampled register write enable signal (RegWrEnQ2) from the AHB register interface block. Registered versions of the data on HWDATAREG is written into the registers decoded by using registered HAddrReg4to2Q2[4:2], StBlkSelCo2 and RegWrEnQ2 to meet synthesis timings. For all writes to the static memory registers, one wait State is added.

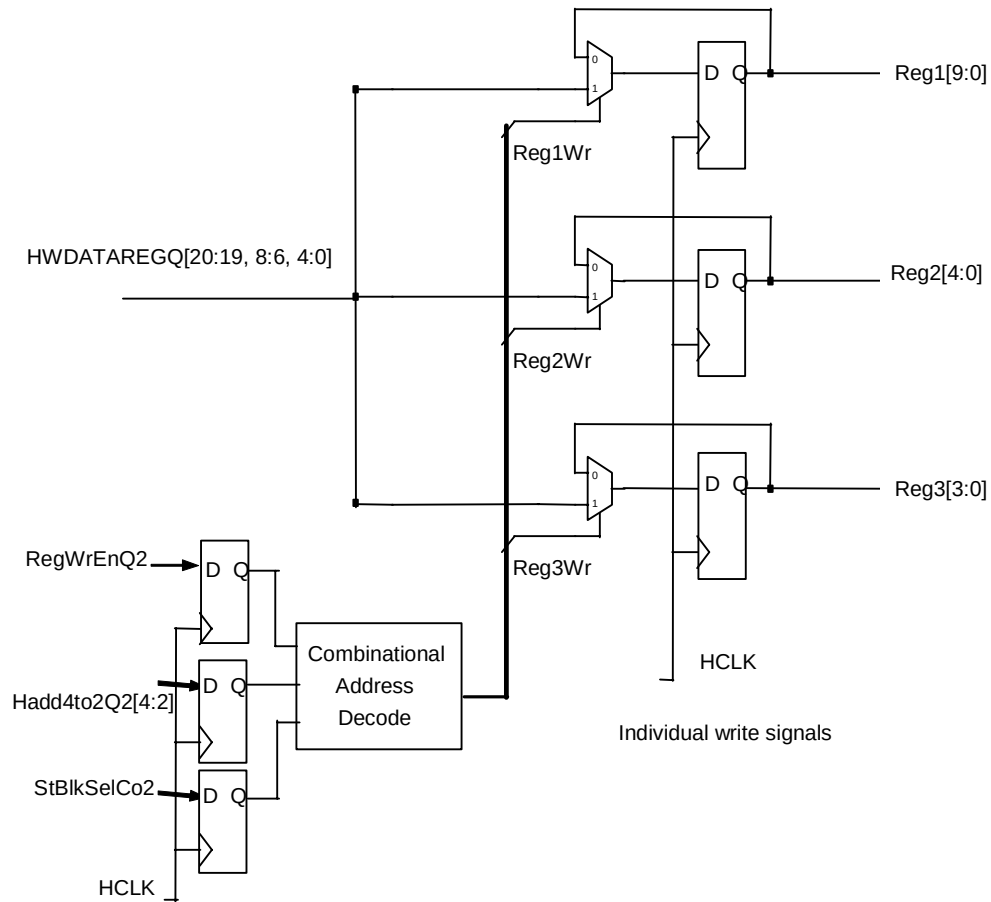


Figure 5.9-1 Static Memory Register Write Logic

Register read logic

The read logic for the static memory registers in the MPMC is as given in **Figure 5.9-2**.

For reads, the registers in the static memory register block are muxed (based on HAddr4to2Q2[4:2]) to generate the read data bus from the static memory register block, StBlkRdData[9:0].

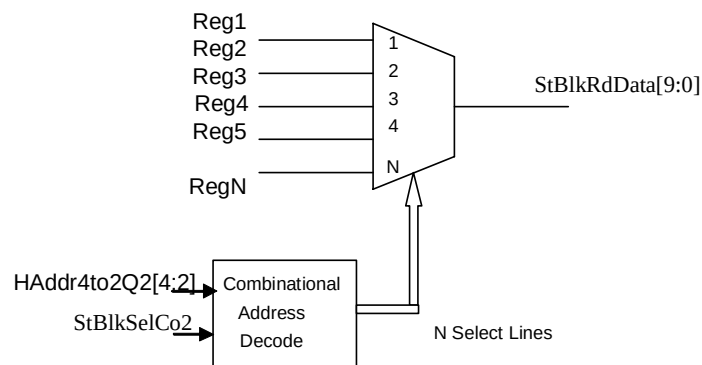


Figure 5.9-2 Static Memory Register Read Logic

5.9.2 Static memory controller internal interface (MpmcStIntIf)

The function of the static memory controller internal interface is to sample and store the valid static memory read and write requests from the buffer control block. It then provides the processed memory access requests to the static memory transfer State Machine and the static memory controller external interface to initiate the memory access on the external memory bus. It also processes the address, data and control signals of the static memory access. The handshake signals, which are required for the interaction of the static memory controller with the buffer control block and the AHB memory interfaces, are provided by this block.

The following functionality is implemented in this block:

- Processing the memory access request based on the type of burst and whether an access to the static memories is already in progress.
- Processing the quad-quad-word address, chip select and burst type information for static memory access requests, storing the information for use during the memory read or write.
- Processing of the write data and byte valid information available from the buffer unit along with the static memory read or write request.
- LSBAddr generation to be used as the lowest 4 bits of the address to be driven on the MPMCADDR[3:0] lines during the static memory access.
- Target static memory configuration and timing control bits selection based on the chip select information from the buffer unit
- TransferSize generation to indicate the data size of the current access request. This indicates the size of the AHB transfer (HSIZE) or the size of data requested by the buffer.
- StBufFullQ generation to indicate that the static memory controller is busy and cannot accept any more requests.
- Burst start and burst break detection to start and stop read data prefetching
- Generation of BufferedWrQ and BufByPass generation to indicate whether the static memory controller should accumulate multiple writes from the AHB to do a single write to the memory.
- Generation of AHB RdOverQ to indicate that the AHB has finished reading the data read from the static memory during the previous read.
- Generation of handshake signals to the buffer control block
- Logic for selection between pin values and register values

Processing the memory access requests

StReadReq and StWriteReq indicate valid static memory read and write requests from the buffer control block.

StReadReq is asserted only when the read request (StPostRReq) from the buffer control block is asserted under the following conditions

- It is the start of an AHB burst read or the first SEQ after a BUSY transfer on the AHB.
- It is a SINGLE type of burst on the AHB
- The static memory read request was not asserted for a BUSY transfer from the AHB.

StWriteReq is asserted when the write request (StPostWReq) from the buffer control block is asserted and it is not asserted for a BUSY transfer from the AHB. However, if the BUSY transfer comes during a sequence of HSIZE < memory width type of writes where the data from multiple writes from the AHB is stored and then written into the memory through a single write, StWriteReq is kept asserted. At the end of servicing the current read or write request, StReadReq and StWriteReq are cleared. StRReq and StWReq indicate the first clock of StReadReq and StWriteReq respectively. ValidReq indicates that either StPostWReq or StPostRReq is asserted. ValidReqCo is asserted if either StRReq or StWReq is asserted.

Processing the quad word address, chip select and burst type information for static memory access requests

The quad word address (StMemAddr[27:4]) driven by the buffer control block along with the assertion of StPostRReq or StPostWReq is stored in the static memory internal interface block as StMemAddrQ[27:4] when ValidReq is asserted. The chip select (StChipSel[3:0]) and unbuffered access (StUnBufMemAcc) information are also stored similarly as ChipSelQ[3:0] and UnBufMemAccQ.

The selection of the static memory configuration bits and access timings for the current access requires to be done whenever a valid request to the static memory is detected. If ChipSelQ[3:0] was used for this purpose, there would be a delay of one clock between the start of a memory access and the detection of the memory configuration. To avoid this, ChipSelCo[3:0] is generated by assigning StChipSel[3:0] from the buffer control block to it when ValidReq is asserted. When it is not asserted, the registered chip select value, ChipSelQ[3:0] is used. Thus the chip select information is available and the corresponding memory configuration and access timings are selected before the memory access can be initiated on the external memory bus.

BufferedPageRd is asserted when a buffered static memory access is initiated to a page mode enabled device. When a valid request is detected, the new value of the SelHBurstQ[2:0] is stored in CurAxsHburstQ[2:0] if it is a unbuffered access. If it is a buffered page mode access, CurAxsHburstQ[2:0] indicates a INCR4 burst since one full quad word of data has to be returned to the buffer for buffered page mode reads. If it is any other type of buffered access, CurAxsHburstQ[2:0] indicates a SINGLE burst. ChipSelCmpQ is asserted if the previous access and the current access are to the same chip select. This is used in the TSM to determine the number of turnaround cycles to be inserted between writes to different chip selects.

Processing of data and byte valid information

The data for the static memory writes (BufDataToSt[127:0]) from the buffer control block is sampled and stored as StBuffDataQ[128:0] on the clock after the write request is asserted by the buffer control block. For unbuffered writes from the AHB with transfer size less than the memory width, the valid data received for multiple sequential writes from the AHB are accumulated together and written into the memory with a single write access. For example, in a case where the transfer size is 8-bits and memory width is 32 bits, the data for upto 4 sequential writes from the AHB can be accumulated together and written into the memory with a single 32-bit write. The accumulation of data is done by selecting the valid data bytes indicated by SelByteCo2[3:0] from the AHB interface and overwriting only those bytes of StBuffDataQ[127:0]. The write data bits are cleared after servicing the write request.

StBufValByteQ[15:0] is loaded with the byte valid bits (StBufDriveByte[15:0]) from the buffer control block when there is a valid request. On writes, for the case where transfer size is less than the memory width, the byte valid information for successive writes needs to be overlapped. OR logic is used to combine the previous valid byte information with the current information. Data for multiple AHB writes can be written as a single write access to memory due to this logic. StBufValByteQ[15:0] is cleared after the current request has been serviced.

StBufValByteCo[15:0] is generated combinationally from StBufDriveByte[15:0], ValidReq and StBufValByteQ[15:0] to be used for LSBAddr[3:0] and TransferSize[1:0] generation. When ValidReq is asserted and it is not a BUSY transfer, StBufDriveByte[15:0] is driven on StBufValByteCo[15:0] and when ValidReq is not asserted, or if it is a BUSY transfer, StBufValByteQ[15:0] is driven on StBufValByteCo[15:0].

LSBAddr generation

LSBAddr[3:0] is generated to keep track of the address corresponding to the selected byte valid bit from the StBufValByteCo[15:0] bits. For unbuffered accesses only one memory address is required to be generated since only one memory access is required. For buffered writes, if multiple bits of StBufValByteCo[15:0] are set, more than one memory accesses may be required for a single buffer-flush write request. In such a case, the address corresponding to each memory access has to be generated, one after the other. LSBAddr[3:0] points to the bit at which the first '1' is encountered in StBufValByteCo[15:0] for the first access to the memory. After the first memory access for the current request being serviced, the next valid byte address is detected from the TagByteValid[15:0] bits. TagByteValid[15:0] is generated on the next clock after LSBAddr[3:0] generation. TagByteValid[15:0] is loaded with StBufValByteCo[15:0] when a valid static memory access request is detected. Once a memory access has been initiated corresponding to a byte valid bit in TagByteValid[15:0], that byte valid bit in the TagByteValid[15:0] is made '0' so that the next '1' can be detected to indicate the next valid byte after the current access has been completed.

StMemAddrStr[27:0] is generated by concatenating StMemAddrQ[27:4] and LSBAddr[3:0]. StMemAddrStr[27:0] is the address that is driven to the static memory controller external interface block to drive on MPMCADDR[27:0] with the correct timings. For buffered page mode reads, StMemAddrStr[3:0] are driven with '0' since the full quad word of data has to be fetched from the static memory.

Depending on the word address indicated by LSBAddr[3:2], one word of data is selected from StBuffDataQ[127:0] and driven on StMemWData[31:0].

Target static memory configuration and timing control bits selection

The chip select specific parameters for the static memory chip select which is currently being accessed are selected depending on the value of ChipSelCo[3:0] for signals that are used on the very next clock after a valid request is sampled. The configuration and timing control bits selected by using ChipSelCo[3:0] are the memory width (StMemWidthQ[1:0]), page mode read enable (StPgModeRdQ), extended wait enable (StExtendWtEnQ), write enable delay (StWtWenQ[3:0]), output enable delay (StWtOenQ[3:0]), normal read access time (StWtRdQ[4:0]), page mode read access time (StWtPgQ[4:0]) and write access time (StWtWrQ[4:0]). Turnaround time (StWtTurn[3:0]) and byte lane State (StBLState) are selected depending on the value of ChipSelStr[3:0] since these are not used in the very next clock after a valid request is sampled.

TransferSize generation

TransferSize[1:0] indicates the size of the current data transfer to memory. The possible values of TransferSize[1:0] are "00" (byte), "01" (half-word) and "10" (word). For buffered mode reads, the transfer size is fixed at the maximum transfer size of word since a quad word of data has to be read from the static memory. For other types of accesses, the transfer size is generated from the valid bytes in the current word being accessed. If all the bytes are valid, TransferSize[1:0] is "10". If a half-word is valid, TransferSize[1:0] is "01" and if only a byte is valid, TransferSize[1:0] is "00".

StBufFullQ generation

StBufFullQ indicates that the static memory command buffer is full, that is the static memory controller is servicing a static memory access request from a buffer or AHB interface. StBufFullQ is asserted along with StReadReq or StWriteReq for unbuffered reads and buffered reads and writes. If it is an unbuffered write, StBufFullQ is asserted along with StWriteReq if transfer size is larger than or equal to the memory width. If transfer size is less than the memory width, it is asserted when the size of the valid data stored in the buffer becomes equal to the memory width. StBufFullQ is cleared at the end of the read or write accesses to memory if it is an unbuffered access. StBufFullQ is kept asserted during prefetch read bursts as long as the static memory controller is accessing the static memory device. If it is a buffered access, it is cleared when the last write from the buffer or the last read to the buffer has been completed on the memory side.

Burst start and burst break detection

BurstStartQ is asserted if there is a valid static memory read request for unbuffered and buffered accesses. It is cleared after it has been used by the TSM to transition into a non-idle State.

BurstBreak is asserted if ValidArbQ is de-asserted or SelBusy is asserted for unbuffered read accesses. It is de-asserted when BurstStart is asserted indicating the start of the next burst. Both BurstBreak and BurstStartQ are used to start or stop prefetching data for an AHB burst of reads.

Generation of BufferedWrQ and BufByPass

BufferedWrQ is set high when there is a write request for buffered accesses. For unbuffered accesses, it is set high when there is a write request and the transfer size is greater than or equal to the memory width for the current access. When there is an unbuffered write with transfer size lesser than memory width, it is asserted when enough valid data has been accumulated to do a write to the memory. BufByPass is used to indicate that the transfer size is greater than or equal to the memory width. These signals are used in the TSM to determine whether to initiate a write request or wait for multiple writes to accumulate data to be written to the memory.

Generation of AhbRdOverQ

AhbRdOverQ is asserted when all the data read from the memory during the previous static memory reads has been read by the AHB. If transfer size is greater than or equal to memory width, AhbRdOverQ is asserted along with MemRdOverQ. If transfer size is lesser than memory width, AhbRdOverQ is asserted after all the data read from the memory during the previous memory read access has been read by the AHB.

Generation of handshake signals to the buffer control block

StUseMem is asserted at the end of an unbuffered or buffered read or unbuffered write access to the static memory controller block if it is not during an extra prefetch read. It indicates the completion of a data transfer to or from the buffer or AHB to the static memory controller. For unbuffered reads with transfer size less than memory width, one read to the memory fetches data for multiple reads. In such a case, StUseMem is kept asserted until all the data retrieved from the memory has been read by the AHB. StMemRd is asserted along with StUseMem for reads. This signal is used as a select for the memory read data buses from the static memory controller and the dynamic memory controller in the memory controller interface block. StEndOfBurstQ is asserted at the end of a read burst transfer to the static memory controller block if it is not during an extra prefetch read. Static memory read data is driven on StMemRdDataQ[31:0] only when StUseMem is asserted for reads. Else zeroes are driven on the read data bus. StMemAHBIdBuf[3:0] indicates the access requestor buffer for the current read access. BuffAccCode[3:0] from the buffer control block is stored at the start of the read and it is returned on StMemAHBIdBuf[3:0] with StUseMem at the end of a read. The valid byte information and address bits 5, 4, 3 and 2 of the current access are returned as StMemByteQ[3:0] and StA5A2Q[5:2] to the buffer when StUseMem is generated for reads.

Logic for selection between pin values and register values

Multiplexers are used to select between the pin values or integration test input register values of the memory width (for chip select 1), the chip select polarity (for all static memory chip selects) and byte lane state (for chip select 1) or the value programmed in the static memory configuration register, (MPMCStConfigx, x = 0, 1, 2 or 3 indicating the chip select). The select for the multiplexers are derived from the write enable of the MPMCStConfigx register (MpmcStConfigxWr). The pin value or integration test input register value is used until the first write to the MPMCStConfigx register after reset. On reset, the signal StConfigxUpd is cleared and it is set when MPMCStConfigx is updated through a write. After the first write StConfigxUpd remains set. When StConfigxUpd is cleared, the pin value is used and when it is set, the register programmed value is used.

5.9.3 Static memory transfer State Machine (MpmcStTSM)

Details of the static memory transfer State Machine block are described in this sub-section.

The transfer State Machine is used to control the read and write transactions from the static memory controller block to the external memory device. In addition to the TSM, this block also contains the logic to generate some the control signals for the State Machine. The following functionality is described in detail below:

- Generation of BurstTransQ and FastRdOpQ to distinguish between prefetch reads and normal reads
- Generation of BeatCount and LastBeat to determine the last beat in a prefetch burst.
- Static Memory transfer state machine to control the data transfer to and from any static memory device connected to the MPMC.

Generation of BurstTransQ and FastRdOpQ

BurstTransQ is used to indicate that an AHB burst read transfer is in progress and when this signal is high, the next read data transfer is started in advance before receiving the StPostRReq is sampled high by the static memory controller. This improves the performance of the static memory controller for reads. The burst of prefetch reads is broken when BurstBreak is sampled high or if it is the last beat in the burst or if a quad-quad-word boundary cross has been detected or if it is a SINGLE type of AHB burst or if a write request is detected.

If transfer size is less than memory width, then more data than requested is read from the memory with a single read access to the memory. FastRdOpQ is used to refrain the static memory controller from speculatively starting the next read in advance as the data for the next sequential address can be returned from the 32-bit read buffer (RdBuf[31:0]) of the static memory controller external interface.

Generation of BeatCount and LastBeat

BeatCount[4:0] is cleared when an AHB burst read is completed. It is incremented with each read access to the memory. The number of reads to the static memory that are required for a AHB burst read is calculated by taking into account the transfer size, the memory width, the burst start address, the burst size and the endian mode. The burst size is generated from CurAxsHburstQ[2:1]. INCR bursts are treated as INCR4 type of bursts for calculating the number of read data to be prefetched in the burst. In the case where transfer size is smaller than the memory width, if the burst start address is not aligned to the memory address (depending on memory width), one more read than aligned reads will be required to complete the burst. When BeatCount[4:0] becomes equal to the number of reads in the AHB burst read, LastBeat is asserted. This signal is used to stop the prefetch reads to the memory.

Static memory transfer State Machine

In this sub-section the State-machine handling the transfers to the static memory is explained.

The static memory transfer state machine has the following states:

- ST_TSM_IDLE: Static memory transfer state machine idle state
- ST_TSM_BUFWR: Static memory transfer state machine internal buffer write state
- ST_TSM_WENCNT: Static memory transfer state machine write enable count state
- ST_TSM_MEMWR: Static memory transfer state machine memory write state
- ST_TSM_SEQWR: Static memory transfer state machine sequential write state
- ST_TSM_TURNARND: Static memory transfer state machine turn around state
- ST_TSM_OENCNT: Static memory transfer state machine output enable count state
- ST_TSM_MEMRD: Static memory transfer state machine memory read state
- ST_TSM_AHBRD: Static memory transfer state machine AHB read state

ST_TSM_IDLE:

After reset and when there are no outstanding static memory access requests, the state machine resides in this state. It moves to any other state only if the MPMCEn bit is set and LowPwrMod bit is not set. The different triggering points for the memory transfer are:

- When a write transfer is detected, depending on the BufBypass status the state machine moves to either the ST_TSM_BUFWR or the ST_TSM_MEMWR state. If the TransferSize = StMemWidthQ then the buffer is bypassed and external write transfer is started immediately, by routing the write data directly to the external data bus. The transfer is initiated only if the external bus is granted. If the sizes are different then the data is first collected into the buffer and the state machine positions itself in the ST_TSM_BUFWR. The bus-request is asserted before moving out.
- When a fresh memory read request is initiated, a request for the bus is issued and when the bus is granted, the nMPMCSTCSOUT is asserted with the start of the read access count. The nMPMCOEOUT is asserted depending on the amount of delay required after the CS assertion. If delay is greater than 0, then the TSM goes to the ST_TSM_OENCNT state before proceeding to the ST_TSM_MEMRD state.
- During read from the memory when the Static Memory Controller is de-granted in between and few more packets of data are remaining to be fetched, then the TSM will come in from the ST_TSM_MEMRD state and remain in this state till the bus is granted to the static memory controller. The AHB master will be waited till bus is granted and all the data packets are read in.

ST_TSM_BUFWR:

This is the state where the data from AHB is collected into an internal 32-bit buffer due to mismatch in AHB transfer size and memory width. For example, in the case of TransferSize=8-bit and StMemWidthQ=32-bit, upto a maximum of 4 bytes can be collected into the buffer before writing into memory. Once the buffer is full, the Static Memory

Controller can start flushing data into the device. The subsequent state transition depends on when the write enable can be asserted, either immediately or after some delay. Due to dynamic priority arbitration of the external data bus, checks are performed to ensure that the Static Memory Controller has got the Bus Grant, otherwise the static memory controller will remain in the same state till the bus is granted again.

ST_TSM_WENCNT:

In this state the TSM waits for the delay completion indication to assert the nMPMCWEOUT

ST_TSM_MEMWR:

The Write to the memory is in progress in this state. When the TransferSize is greater than StMemWidthQ, then multiple accesses to the memory devices are required which is indicated MemWrOverQ being de-asserted. The TSM then moves to the ST_TSM_SEQWR so as to de-assert the output. The CntEnd signal in this state indicates the end of the write access time.

ST_TSM_SEQWR:

The MemWrOverQ=0 indicates that the TransferSize is greater than StMemWidthQ and multiple transfers are required to complete the write transfer. The MPMCADDROUT incremented and depending on the value of WEnCnt the next state is determined. If the write transfer is successful as indicated by MemWrOverQ=1, then depending on whether any new write or read transition to appropriate state is made. If the next transfer is a write to different bank, then the CS is disabled. If on the end of the current write transfer, no valid transfers are initiated, then the TSM moves to IDLE state. Due to dynamic priority arbitration of the external data bus, checks are performed to ensure that the static memory controller has got the memory bus grant; otherwise the Static Memory Controller will remain in the same state till the bus is granted again. The memory bus could be de-granted before all the write data packets have been flushed out to the memory from the internal Buffer in the TransferSize greater than StMemWidthQ case. In the event of this, the StBusReq is de-asserted for one HCLK cycle and again asserted. This is required to satisfy the memory bus grant logic protocol.

ST_TSM_TURNARND:

Turn-Around cycles are required to ensure that there is no conflict of data on the data Bus. The bus turnaround cycles are carried out when the static memory controller is in this state. The number of turn around cycles are determined by the StWtTurn value programmed corresponding to the particular memory device. Bus turnarounds are inserted for the following cases:

- After completion of a read transaction and the next transaction is a write or a read to a different bank.
- On successful completion of a read transfer if there are no more transfers on the AHB.
- One HCLK cycle turn-around is inserted when a read follows a write transfer.

When the Turn-Around cycles are in progress, if there is any new read or a write transfer request to the memory, then these requests are stored. These requests will be serviced after the completion of the turn-around transition to the subsequent state of the TSM from this state depends on the next transfer to be serviced. If there are no other transfers from the Buffer Control block, then the TSM goes to the IDLE state. Before going into any other state, the static memory controller waits for the completion of the turn around cycles indicated by CntEnd.

ST_TSM_OENCNT:

The TSM waits in this state for the completion of the delay after which the nMPMCOEOUT can be asserted.

ST_TSM_MEMRD:

During the read transfer the Static Memory Controller is in this state and waits for the access time completion. On CntEnd assertion, if all the data packets have been read (as indicated by MemRdOverQ=1), then the next state is enabling the read from the buffer/AHB. If there are still some data packets left to be read from the memory device because of the TransferSize to StMemWidthQ differences, then access counters are enabled again by asserting the address incremter signal, AddrIncCo, and the remaining data packets are read in and stored till the AHB/Buffer read is complete. The AHB burst information is used to perform fast speculative burst reads with the memory address

value, MPMCADDROUT, the memory device generated in advance. During the fast burst read mode the static memory controller's TSM continues to remain in this state.

The various triggers for the state transitions are:

- During burst read operation from the memory, the static memory controller does speculative reads, which are ahead of real AHB request. When performing speculative fast burst reads, the static memory controller in parallel also checks for termination of the burst read transfer. The burst could have been aborted due to next transfer, which could either be a write or a read to a different bank or a new read transfer. The next state transition will depend on all these conditions. If the burst read request continues from the AHB, the TSM will continue in this state. The nMPMCSTCSOUT & the nMPMCOEOUT will be kept continuously asserted when doing burst reads. If it is a page mode read capable device (StPgModeRdQ = '1'), the initial longer access time is indicated by the assertion of the RdCntLd signals. Whereas the shorter page mode access time is indicated by the assertion of the RdBMCntLd signal. During burst reads to page mode read capable devices, even if the AHB is continuously doing burst reads, the static memory controller aborts the burst on reaching the Quad-Boundary address of the device. So if the 1st address is aligned, then the static memory controller does 1 initial longer access and subsequent 3 short accesses. The hitting of the Quad-Boundary address is indicated by the PgLendQ signal from the MpmcStExtIf module.
- There is one exception to the speculative burst read from the page mode memory device when the static memory controller does not perform advance speculative reads. This happens for the case when the TransferSize is less than StMemWidthQ and the RdBuf[31:0] already has cached in more data than required by the current AHB read address. If the subsequent reads from the AHB are sequential in the same address range, then the data is already available in the RdBuf and the data is returned with zero wait cycles. This is indicated by the FastRdOp signal. In this scenario the next transition is to the ST_TSM_AHBRD state so as to route the data to the AHB.
- In the non-burst read mode (HBURST is SINGLE or a buffered read), once the data is read in from the memory device & the RdBuf is full as indicated by MemRdOverQ=1 the TSM will move to the ST_TSM_AHBRD state so as to route the data to the AHB.

During reads if the External Bus is de-granted with some more reads yet to be performed from the device, then after the current read is complete, the RdRemain is asserted, StBusReq is de-asserted and the subsequent state is ST_TSM_IDLE. The address increment signal for the Burst address, BrstAddInc, is generated during continuous Burst reads as the read data from the memory fills the RdBuf[31:0] and the data is routed to the AHB.

ST_TSM_AHBRD:

This state controls the routing of read data to the MemRData bus in the to enable AHB / Buffer Reads. The various state transition triggers are:

- At the completion of the read operation, indicated by the MemRdOverQ, if the Bus is de-granted, then the TSM transitions to the TURNARND state after de-asserting all the control signals.
- When the AHB / Buffer completes its read operation from the RdBuf[31:0], indicated by the AhbRdOver, then depending on whether the next transfer is continuation of SEQ reads to the same device or read to a different Bank or a write operation, the state change decisions are taken. If during Burst Read operation for the exception case when the TransferSize is less than StMemWidthQ, the RdBuf will have cached in more data than required. The subsequent sequential read data are returned from the internal RdBuf and the state machine remains in the ST_TSM_AHBRD state until the data in RdBuf has been read by the AHB. If the subsequent Reads requests from AHB / Buffer are still sequential and if the Quad-Boundary Address location (for page mode reads) is not yet reached, then faster read access is initiated by moving to the ST_TSM_MEMRD state. nMPMCSTCSOUT & nMPMCOEOUT are kept asserted during this state.
- If the next transfer after the completion of the AHB Read is a write or a read to a different bank, then the static memory controller will perform bus turn-around cycles before starting the memory access. These requests are stored appropriately.
- In the event of not further transfer requests from the AHB, then Turn-Around cycles are performed and the state machine returns to the idle state after the turnaround states have been introduced.

In **Figure 5.9-3**, the State diagram is shown. The details of the State-transition-triggers are described in **Table 5.9-1**. The tasks that are done on each transition are described after the table.

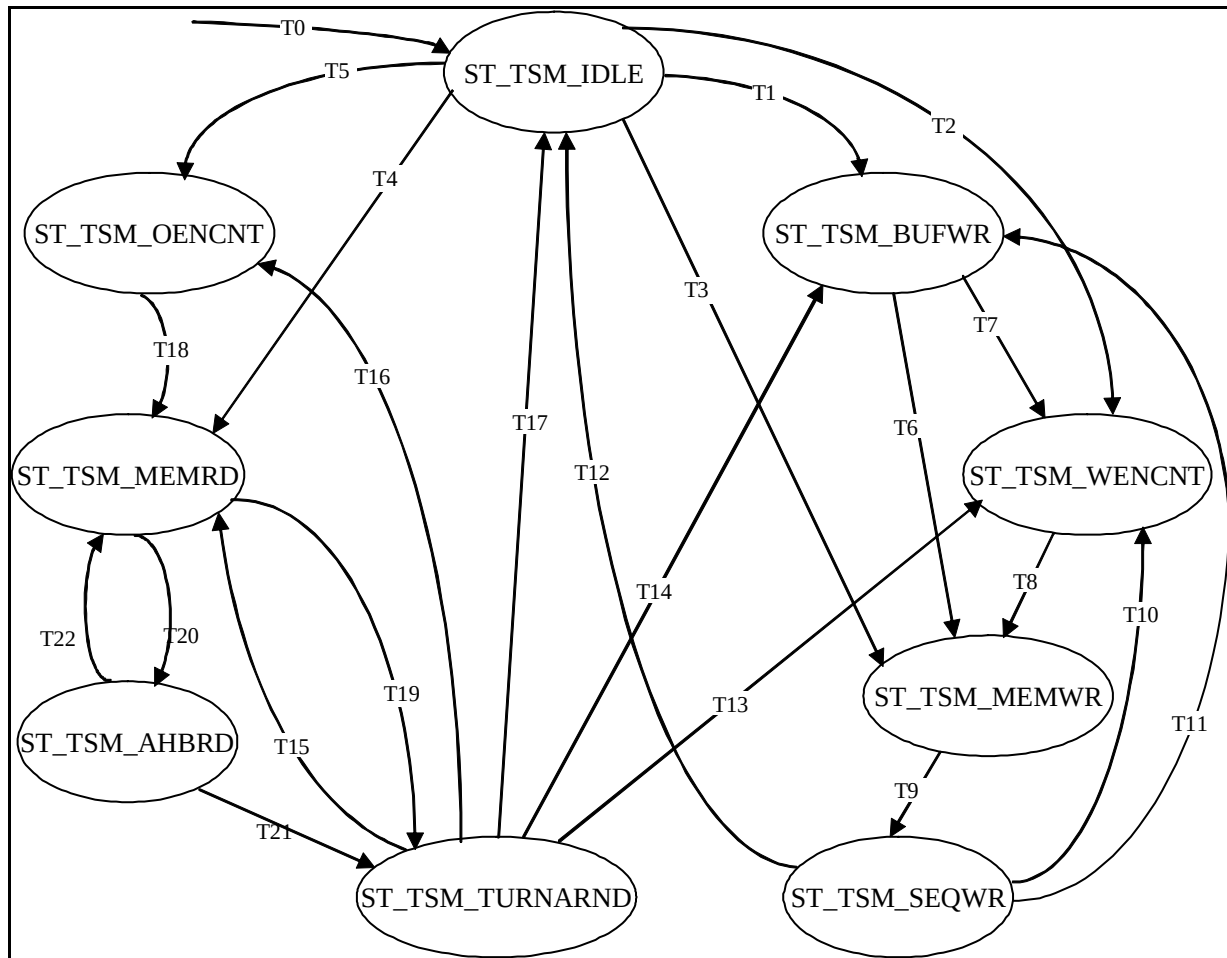


Figure 5.9-3 Static Memory Transfer State Machine

Transition Number	Description	Condition
T0	Reset to IDLE	When nPOR is asserted, transition T0 takes place.
T1	IDLE to BUFWR	When a write request is detected (StWriteReq = '1') and the transfer size is not greater than or equal to memory width (BufByPass = '0') for unbuffered accesses (UnBufMemAccQ = '1'), transition T1 takes place.
T2	IDLE to WENCNT	When a write request is detected and the transfer size is greater than or equal to memory width (BufByPass = '1') or if it is a buffered access (UnBufMemAccQ = '0'), transition T2 takes place if the write enable delay value is not zero (WenCntEZ = '0') and the static memory controller has been granted the external memory bus (StBusGnt = '1').
T3	IDLE to MEMWR	When a write request is detected and the transfer size is greater than or equal to memory width or if it is a buffered access (UnBufMemAccQ = '0'), transition T3 takes place if the write enable delay value is zero (WenCntEZ = '1') and the static memory controller has been granted the external memory bus.
T4	IDLE to MEMRD	When a read request is detected (StReadReq = '1'), transition T4 takes place if

Transition Number	Description	Condition
		the output enable delay value is zero (OenCntEZ = '1') and the static memory controller has been granted the external memory bus.
T5	IDLE to OENCNT	When a read request is detected, transition T5 takes place if the output enable delay value is not zero (OenCntEZ = '0') and the static memory controller has been granted the external memory bus.
T6	BUFWR to MEMWR	When the data for a write to the static memory has been accumulated (BufferedWrQ = '1' or BufWrOverStT = '1') and the static memory controller has been granted control of the external memory bus, transition T6 takes place if the write enable delay is zero.
T7	BUFWR to WENCNT	When the data for a write to the static memory has been accumulated and the static memory controller has been granted control of the external memory bus, transition T7 takes place if the write enable delay is non-zero.
T8	WENCNT to MEMWR	When the delay counter expires (DelayEnd = '1') or the write enable delay is zero, transition T8 takes place.
T9	MEMWR to SEQWR	When the timer counter expires (CntEnd = '1' or CntEZEnd = '1'), transition T9 takes place.
T10	SEQWR to WENCNT	When transfer size is greater than the memory width MemWrOverQ is not asserted on the clock after the timer counter expires. In this condition, if StBusGnt is asserted transition T10 takes place. OR When MemWrOverQ is asserted on the clock after the timer counter expires, and a write request is sampled high on the same clock, and transfer size is greater than or equal to memory width, transition T10 takes place if StBusGnt is high.
T11	SEQWR to BUFWR	When MemWrOverQ is asserted on the clock after the timer counter expires, and a write request is sampled high on the same clock, and transfer size is less than memory width, transition T11 takes place if StBusGnt is high.
T12	SEQWR to IDLE	When BusDeGnt is sampled low and there is no pending write access to the memory, transition T12 takes place.
T13	TURNARND to WENCNT	If a valid write request is detected when the timer counter expires and if transfer size is greater than or equal to memory width, transition T13 takes place if StBusGnt is high.
T14	TURNARND to BUFWR	If a valid write request is detected when the timer counter expires and if transfer size is less than memory width, transition T14 takes place.
T15	TURNARND to MEMRD	If a valid read request is detected and the AHB burst has not been broken when the timer counter expires, transition T15 takes place if output enable delay is non-zero and StBusGnt is high.
T16	TURNARND to OENCNT	If a valid read request is detected and the AHB burst has not been broken when the timer counter expires, transition T16 takes place if the output enable delay is zero and StBusGnt is high.
T17	TURNARND to IDLE	If there is no valid request when the timer counter expires, transition T17 takes place.

Transition Number	Description	Condition
T18	OENCNT to MEMRD	When the delay counter expires, transition T18 takes place.
T19	MEMRD to TURNARND	Transition T19 takes place when there is a break in the AHB burst of reads, or if it is the last read in the burst, or if a write request is sampled high when the timer counter expires OR If StBusGnt is de-asserted during an AHB read and transfer size is not less than memory width.
T20	MEMRD to AHBRD	Transition T20 takes place if BurstTransQ is sampled high and the timer counter expires with transfer size less than memory width OR If it is a SINGLE type of burst with transfer size less than memory width.
T21	AHBRD to TURNARND	Transition T21 takes place when AhbRdOverQ is asserted for reads with transfer size less than memory width indicating that the data that was read from the memory in the previous read has been read by the AHB if StBusGnt is low OR When MemRdOverQ is asserted for reads with transfer size greater than or equal to memory width if StBusGnt is low OR When a break in the burst or a quad-quad-word boundary or the last beat in the read prefetch burst is detected for AHB bursts with burst size greater than 1.
T22	AHBRD to MEMRD	During an AHB burst read with burst size greater than 1 if the data read from the memory during the previous read access has been read by the AHB, transition T22 takes place

Table 5.9-1 Transition table for Static Memory Transfer State Machine

Tasks done on the transition edges

Transition T0:

- The State-machine is initialised to ST_TSM_IDLE.

Transition T1:

- StBusReq is asserted requesting for the external memory bus.

Transition T2:

- This transition indicates that a static memory write access has to be initiated with a non-zero write enable delay. So the static memory bus request (StBusReq), the chip select enable signal (StCSEnCo), the enable signal for the external memory data enable (WrXdatEn), the write access time count load signal (WrCntLd) and the write enable delay count load signal (WenCntLd) are asserted. The stored buffered-write-completion-indicator (BufWrOvrStrT) is cleared.

Transition T3:

- This transition indicates that a static memory write access has to be initiated with zero write enable delay. So StBusReq, StCSEnCo, WrXdatEn, WrCntLd and the enable signal for the external memory write enable (ExtWrEn) are asserted. BufWrOvrStrT is cleared.

Transition T4:

- This transition indicates that a static memory read access has to be initiated with a zero output enable delay. So StBusReq, StCSEnCo, RdXdatEn, XoutEn and RdCntLd are asserted. The stored read requests (RdReqStr and RdRemain) are cleared.

Transition T5:

- This transition indicates that a static memory read access has to be initiated with non-zero output enable delay. So StBusReq, StCSEnCo, RdXdatEn, RdCntLd and OenCntLd are asserted. RdReqStr and RdRemain are cleared.

Transition T6:

- BufWrOvrStrT and BusDeGnt are cleared. StCSEnCo, WrXdatEn, WrCntLd and WenCntLd are asserted to initiate a write access with a zero programmed write enable delay.

Transition T7:

- BufWrOvrStrT and BusDeGnt are cleared. StCSEnCo, WrXdatEn, WrCntLd and ExtWrEn are asserted to initiate a write access with non-zero programmed write enable delay.

Transition T8:

- ExtWrEn is asserted to assert the external memory write enable.

Transition T9:

- ExtWrDis is asserted to de-assert the external memory write enable.

Transition T10:

- BusDeGnt is reset. AddrIncCo, StCSEnCo and WrXdatEn are asserted to increment the external memory address by memory width in order to initiate a new write access if transfer size is less than memory width. StCSEnCo and WrXdatEn are set to initiate a new write access if it is a new write request with transfer size greater than or equal to memory width.

Transition T11:

- If ChipSelCmpQ is not sampled high, StCSEnCo is cleared.

Transition T12:

- StBusReq, StCSEnCo and ExtWrDis are reset. XdatDis is set.

Transition T13:

- RdReqStr and BufWrOvrStrT are reset. StCSEnCo and WrXdatEn are set to initiate a new write access.

Transition T14:

- RdReqStr are reset.

Transition T15:

- RdReqStr is reset. StCSEnCo, RdXdatEn, RdCntLd and XoutEn are set to initiate a new read access with zero output enable delay.

Transition T16:

- RdReqStr is reset. StCSEnCo, RdXdatEn, RdCntLd and OEnCntLd are set to initiate a new read access with non-zero output enable delay.

Transition T17:

- StCSEnCo and StBusReq are reset. XdatDis and XoutDis are set.

Transition T18:

- XoutEn is asserted to assert the output enable.

Transition T19:

- StCSEnCo is cleared. TrnCntLd, XoutDis and XdatDis are asserted. If StBusGnt was sampled low at the end of a read access, which is a part of an AHB, read burst, RdReqStr and BrstAddInc are asserted. If it was sampled low when the data for a single AHB read has not been completely fetched from the memory, RdRemain is set.

Transition T20:

- No actions on this state transition.

Transition T21:

- StCSEnCo is cleared. TrnCntLd, XoutDis and XdatDis are asserted. If StBusGnt was sampled low at the end of a read access, which is a part of a AHB, read burst, RdReqStr is asserted. If MemRdOverQ is asserted when AhbRdOverQ is asserted for a burst, BrstAddInc are asserted. If a new read or write request is detected during this transition, it is stored by asserting RdReqStr or WrReqStr.

Transition T22:

- StCSEnCo is asserted. XoutEn is set. For AHB burst reads, if it is a page mode enabled device (StPgModeRdQ = '1') and the end of a page is not reached (PgLenEndQ = '0'), RdBMcntLd is set to load the timer counter with the page mode read access time. If the end of a page is reached for a page mode enabled device or if it is an INCR or SINGLE type of burst, RdCntLd is asserted. If MemRdOverQ is asserted when AhbRdOverQ is asserted for a burst, BrstAddInc are asserted.

In ST_MEM_RD State, if transfer size is greater than memory width and the timer counter expires (CntEnd = '1' or CntEZEnd = '1') and MemRdOverCo is not asserted, it indicates that some more reads from the memory are required to return data for the AHB read request. In such a case, AddrIncCo is asserted to increment the static memory address by the memory width and a new access to the memory is initiated by maintaining the asserted condition of the chip select and the output enable. If MemRdOverCo is asserted indicating that there is enough data to be returned for the AHB read request, BrstAddIncCo is asserted to increment the static memory address by burst size for an AHB burst of reads. RdBMcntLd is asserted if it is a page mode enabled device and a page boundary is not reached. If a page boundary is reached or if it is a not a page mode enabled device, RdCntLd is asserted.

If StBusGnt is de-asserted when the static memory transfer State Machine is about to issue a read or write to the memory, the State Machine waits in the same state for StBusGnt to be asserted. The pending read requests that are asserted during StBusGnt low or during turnaround cycles are stored as RdReqStr. BufferedWrQ is stored as BufWrOvrStrT if it is asserted during the deasserted State of the static memory controller. If StBusGnt is de-asserted during a read or write access, the access is completed and then StBusReq is cleared so that the dynamic memory controller can be given the grant. If StBusGnt is sampled low in ST_TSM_BUFWR or ST_TSM_SEQWR states, StBusReq is de-asserted for one clock and asserted again. So the dynamic memory controller or the other master of the external memory bus will be granted the external memory bus and the static memory controller will regain the bus immediately after the external memory bus has been freed up by the other master or the dynamic memory controller.

5.9.4 Static memory timing control block (MpmcStTimCntl)

The details of the static memory timing control block functionality are described in detail in this sub-section.

This block receives the load enable signals for various delay values from the transfer State Machine and loads the corresponding access times and delay values into the timer counter and delay counter respectively and checks till the counters expire. The access time or turnaround time completion is indicated by the assertion of CntEnd. The delay count expiry is indicated by the assertion of DelayEnd.

The following functionality is implemented in this block:

- Generation of the write enable and output enable delay values
- Generation of signals to indicate a counter load value of zero
- Generation of CntEZEnd
- Timer State Machine

Generation of the write enable and output enable delay values

The chip select assertion to write enable assertion delay and the chip select assertion to output enable assertion delay is selected from the value in the selected chip selects output or write enable delay counts and the access time counts depending on which one is smaller. If StWtOenQ is greater than StWtRdQ, OEnCount follows the StWtRdQ value. During page mode reads, if StWtOenQ is less than StWtPgQ, OenCount follows the StWtPgQ value. Else OEnCount follows StWtOenQ value. If StWtWenQ is greater than StWtWrQ, WrEnCount follows the StWtWrQ value. Else WrEnCount follow StWtWenQ value. OEnCount and WrEnCount are used to load the delay count register. This logic prevents the transfer State Machine from hanging if the user erroneously programs the write enable time greater than the write access time or the output enable time greater than read access time.

Generation of signals to indicate a counter load value of zero

When the normal mode read access time, page mode read access time, write access time and turnaround count load values are zero, the signals StWtRdEZ, StWtPgEZ, StWtWrEZ and StWtTurnEZ are set. When OEnCount is zero, OEnCntEZ is asserted, and when WrEnCount is zero, WEnCntEZ is asserted.

Generation of CntEZEnd

When the count values for the read and write access times and the turnaround time are zero, CntEZEnd is asserted without waiting for the timer State Machine transitions when their respective counter load values are asserted.

Timer State Machine

The timer State Machine is a two-bit one hot encoded State Machine with two states :

- ST_TW_IDLE: Static memory timer state machine idle state
- ST_TW_CNT: Static memory timer state machine count state.

This State Machine controls the timer counter for the read and write access times and the turnaround time.

ST_TW_IDLE:

The timer State Machine transitions to the ST_TW_IDLE State on reset. In the ST_TW_IDLE State, if a timer count load signal (RdCntLd, RdBMcntLd, WrCntLd or TrnCntLd) is sampled high, it transitions to State ST_TW_CNT if the count value is non-zero. The corresponding count value is loaded into the timer counter during this transition.

ST_TW_CNT:

If a timer count load signal is sampled high during this State, the new count value is loaded in the timer counter. CntEnd is asserted when the timer counter reaches a value of one. The State Machine transitions to the ST_TW_IDLE State and the timer counter is cleared when the counter reaches a value of one. Otherwise, the timer counter decrements by one at every rising edge of HCLK in this State.

Delay State Machine

The delay State Machine is a two-bit one hot encoded State Machine with two states :

- ST_OEWE_IDLE: Static memory output enable write enable state machine idle state
- ST_OEWE_CNT: Static memory output enable write enable state machine count state.

This State Machine controls the delay counter for the chip select to output enable and chip select to write enable assertion delays.

ST_OEWE_IDLE:

The delay State Machine transitions to the ST_OEWE_IDLE State on reset. In the ST_OEWE_IDLE State, if a delay count load signal (WrEnCntLd or OEnCntLd) is sampled high, it transitions to State ST_OEWE_CNT if the count value is non-zero. The corresponding count value is loaded into the timer counter during this transition.

ST_OEWE_CNT:

If a delay count load signal is sampled high during this State, the new count value is loaded in the delay counter. The State Machine transitions to the ST_TW_IDLE State when the delay counter reaches a value of one. Otherwise, the delay counter decrements by one at every rising edge of HCLK in this State.

Generation of DelayEnd

DelayEnd is asserted when the delay counter reaches a value of one in the ST_OEWE_CNT State. However, if the load value of the delay counter is '1', DelayEnd is asserted on the clock after the delay count load signal is asserted.

5.9.5 Static memory controller external interface (MpmcStExtIf)

The details of the static memory controller external interface block are described in this sub-section.

This module provides the interface with the external static memory devices for facilitating the read and write accesses. The memory device control signals along with the address and data paths are generated based on the signals from the TSM. When the external memory bus is not granted to the static memory controller, the external memory bus signals that are shared by the static memory controller and dynamic memory controller are driven to zero.

The following functionality is implemented in this block:

- Chip select generation according to the chip select polarity
- Advance generation of burst address for use during prefetch reads
- External address, StAddrQ[27:0] generation logic
- Write enable generation for the 32-bit read buffer to store read data from memory
- Read data path from the external memory to the 32-bit read buffer
- Write data path to the external memory from the static memory controller
- Data enable generation
- External memory output enable generation
- External memory write enable generation
- Write enable generation for nMPMCBLSOUT[3:0] for byte wide devices
- External memory byte lane select generation
- MemWrOverQ and MemRdOverQ generation logic
- Page length end detection for page mode enabled devices

Chip select generation

When the chip select enable (StCSEnCo) is asserted by the TSM, the chip select for the current access is asserted while the other chip selects are de-asserted. When the chip select enable (StCSEnCo) is de-asserted, all the chip selects are deactivated. The chip select polarities are taken into account for this activation and deactivation.

Advance generation of burst address

The next address in the burst (BurstAddr[9:0]) is generated in advance during AHB burst reads when BrstAddIncCo is sampled high. This is the advance evaluation of the next address depending on the transfer size, memory width, endian mode and burst type. This address is used for starting the prefetching of read data before the read request has been received by the static memory controller.

External address, StAddrOutQ[27:0] generation logic

- StMemAddrStr[27:0] from the static memory controller internal interface is driven on StAddrQ[27:0] at the start of a read or write access if the external memory bus is granted to the static memory controller.
- If the TSM asserts the AddrIncCo signal, the address is incremented by 1, 2 or 4 depending on whether the external memory width is 8-bits, 16-bits or 32-bits respectively.
- If the TSM indicates that it is a AHB burst read by asserting the BurstTransQ signal, and the previous read is over, BurstAddr[9:0] is concatenated with the previous StAddrQ[27:10] and driven on StAddrQ[27:0] if the external memory bus is granted to the static memory controller.
- If the external memory bus is not granted to the static memory controller, and a new read or write request is seen, the request is stored as ValidReqStr and used after the bus is granted to drive the valid values on StAddrQ[27:0].
- On the clock after the bus is granted to the static memory controller, the address is driven from StAddrStore[27:0] in order to continue the previous burst, which was terminated by degranting of the external memory bus. StAddrStore[27:0] is the internal version of StAddrQ[27:0]. However, StAddrStore[27:0] is not cleared when the static memory controller is degranting the external memory bus.
- StAddrOutQ[27:0] is generated from StAddrQ[27:0]. When Rel1ConfigInCo is asserted, StAddrQ[27:0] is driven out directly on StAddrOutQ[27:0]. When Rel1ConfigInCo is not asserted, StAddrQ[27:0] is right shifted by one bit and driven on StAddrOutQ[27:0] if the external memory width is 16-bits; if the external memory width is 32-bits, StAddrQ[27:0] is right shifted by two bits and if the external memory width is 8-bits, no right shifting is done to generate StAddrOutQ[27:0].

Write enable generation for the 32-bit read buffer to store read data from memory

At the end of a read, as indicated by assertion of CntEnd or CntEZEnd in the ST_TSM_MEMRD State, the relevant read buffer enables (RdBufWrEn[3:0]) are asserted depending on the value of StMemWidthQ, the memory width.

- If StMemWidthQ[1:0] indicates a 32-bit memory, all the bits of RdBufWrEn are asserted (RdBufWrEn = "1111")
- If StMemWidthQ[1:0] indicates a 16-bit memory lower or upper 2 bits of RdBufWrEn are asserted depending on whether StAddr[1] indicates an access to halfword 0 or halfword 1 respectively.
- If StMemWidthQ[1:0] indicates a 8-bit memory, bit 0, 1, 2, or 3 of RdBufWrEn is asserted depending on whether StAddr[1:0] indicates an access to byte 0, byte 1, byte 2 or byte 3 respectively.

Read data path from the external memory to the 32-bit read buffer

Data is read in from the memory device at the end of the read access time. Data is driven on the byte lanes of TmpRdDat[31:0] at the end of read access time depending on the external memory width and the address driven to the memory. This data is then routed to the 32-bit read buffer, RdBuf based on the value of the RdBufWrEn[3:0] bits.

- If the external memory width is 32-bits, MPMCDATAIN[31:0] is driven on TmpRdDat[31:0]
- If the external memory width is 16-bits, MPMCDATAIN[15:0] is driven on TmpRdDat[31:16] or TmpRdDat[15:0] depending on whether StAddr[1] is 1 or 0 respectively.
- If the external memory width is 8-bits, MPMCDATAIN[8:0] is driven on TmpRdDat[31:24], TmpRdDat[23:16], TmpRdDat[15:8] or TmpRdDat[7:0] depending on whether StAddr[1:0] indicates byte 3, 2, 1 or 0

Each byte of data from TmpRdDat[31:0] is stored in the corresponding byte of RdBuf[31:0] depending on whether the RdBufWrEn[3:0] bit corresponding to that byte is set.

Write data path to the external memory from the static memory controller

StDataOutQ[31:0] is the write data output to be driven out on MPMCDATAOUT[31:0].

During writes, depending on the external memory width and StAddrQ[1:0], the corresponding byte lanes are driven.

- If the external memory width is 32-bits, StMemWData[31:0] from the static memory controller internal interface block is driven on StDataOutQ[31:0]
- If the external memory width is 16-bits, StMemWData[31:16] or StMemWData[15:0] is driven on StDataOutQ[15:0] depending on whether StAddrQ[1] is 1 or 0 respectively.
- If the external memory width is 8-bits, StMemWData[31:24], StMemWData[23:16], StMemWData[15:8] or StMemWData[7:0] are driven on StDataOutQ[7:0] depending on whether StAddrQ[1:0] indicates byte 3, 2, 1 or 0 respectively.

During reads, at the end of the read access time the relevant MPMCDATAIN[31:0] byte lanes are routed to the byte lanes of StDataOutQ[31:0] for recirculation of the data. If StBusReq is not set, zeros are driven on StDataOutQ[31:0].

Data enable generation

nStDataEn[3:0] is the data enable signal driven by the static memory controller. This is used to generate the output pins nMPMCDataEn[3:0] in the pad interface block. The nStDataEn[3:0] bits are asserted or de-asserted depending on the type of the transfer in progress. For writes, all the bits of nStDataEn are asserted. For reads, only the nStDataEn[3:0] bits corresponding to the valid bytes in the read are de-asserted. The other bits are kept asserted for the recirculation logic. When StBusReq is de-asserted, nStDataEn bits are de-asserted.

External memory output enable generation

The nMPMCOEOUT is enabled or disabled depending on the control signals from the TSM. When XoutEn is asserted, nMPMCOEOUT is asserted low. When XoutDis is asserted, nMPMCOEOUT is de-asserted high.

External memory write enable generation

nStWEQ is the external memory write enable generated by the static memory controller. This is used to generate the nMPMCWEOUT signal in the pad interface block of the MPMC. nStWEAssert is an intermediate signal for the generation of StWEQ. nStWEAssert is de-asserted high when ExtWrDis is asserted by the TSM. When ExtWrEn is asserted, nStWEAssert is asserted low if it is not a byte wide static memory. nStWEAssert is registered to generate nStWEQ.

Write enable generation for nMPMCBLSOUT[3:0] for byte wide devices

Write enables, WrBufWrEn[3:0] will be used to generate the nMPMCBLSOUT[3:0] outputs with write enable timings for byte wide devices (StBLState = '0'). WrBufWrEn[3:0] bits are generated as active low since nMPMCBLSOUT is an active low signal. During a write,

- If TransferSize[1:0] indicates a 32-bit access from an AHB or a buffer, all the bits of WrBufWrEn[3:0] are asserted.
- If TransferSize[1:0] indicates a 16-bit access from an AHB or buffer, WrBufWrEn[1:0] is asserted if ByteAddr[1] indicates that it is an access to halfword 0 and WrBufWrEn[3:2] is asserted if ByteAddr[1] indicates that it is an access to halfword 1. In addition to the above 2 bits, if a third bit is also valid as in the case of three 8-bit writes to a 32-bit memory, the relevant bit of WrBufWrEn (bit 2 or bit 1) is asserted for such a case.
- If TransferSize[1:0] indicates a 8-bit access from an AHB or a buffer, WrBufWrEn[3], WrBufWrEn[2], WrBufWrEn[1] or WrBufWrEn[0] are asserted based on which byte is indicated by ByteAddr[1:0]. In addition to the above one bit, if another bit is also valid as in the case of two 8-bit writes to a 32-bit memory, the relevant bit of WrBufWrEn (bit 3 or 2) for such a case.

At the end of a write, all the write enable bits are de-asserted (WrBufWrEn = "1111").

External memory byte lane select generation

StBLSAssert[3:0] is an intermediate signal used to generate the byte lane signal, nMPMCBLSOUT[3:0]. All bits of nStBLSAssert[3:0] are de-asserted if it is a byte wide static memory as indicated by StBLState when ExtWrDis is asserted by the TSM. When ExtWrEn is asserted, some bits of nStBLSAssert[3:0] are asserted if it is a byte wide static memory. The selection of bits to be asserted is as follows.

If transfer size is greater than or equal to memory width as indicated by BufByPass = '1',

- All the bits of nStBLSAssert are asserted if it is an access to a 32-bit device
- nStBLSAssert[1:0] are asserted if it is an access to a 16-bit device
- nStBLSAssert[0] is asserted if it is an access to a 8-bit device

If transfer size is less than memory width as indicated by BufByPass = '0',

- The D-input of WrBufWrEn[3:0] is driven on nStBLSAssert if it is an access to a 32-bit device
- The D-input of WrBufWrEn[3:2] is or WrBufWrEn[1:0] is driven on nStBLSAssert[1:0] and the upper 2 bits of nStBLSAssert[3:0] are de-asserted if it is a 16-bit device and ByteAddr[1] is '1' or '0' respectively
- A '0' is driven on nStBLSAssert[0] and the upper 3 bits of nStBLSAssert[3:0] are de-asserted if it is a 8-bit device

When StBLState is asserted indicating that it is not a byte wide static memory device and XoutEn is asserted by the TSM all the bits of nStBLSAssert[3:0] are asserted. All the bits of nStBLSAssert are cleared if the chip select enable is not asserted. nStWEAssert.

BlsSel is the select to determine whether nMPMCBLSOUT[3:0] should follow the timings for the external memory write enable or the byte lane select timings. When StBLState is asserted indicating that it is not a byte wide static memory device and ExtWrEn or XoutEn are asserted, BlsSel is set so that nMPMCBLSOUT[3:0] will follow the byte lane select timings. BlsSel is cleared at the write enable assertion if it is a byte-wide static memory and nMPMCBLSOUT[3:0] has to follow the write enable timings. A multiplexer is used to select between the two versions of nMPMCBLSOUT[3:0]. If BlsSel is high, nStBLSAssert[3:0] is driven on nMPMCBLSOUT[3:0]. If BlsSel is cleared, registered nStBLSAssert[3:0] is driven on nMPMCBLSOUT[3:0].

MemWrOverQ and MemRdOverQ generation logic

MemWrOverQ assertion indicates the completion of writes to the memory device for the current write request. MemWrOverQ generation depends on the memory width and the number of memory writes required for the single write from an AHB or a buffer. The end of a write to memory is indicated by the assertion of CntEnd or CntEZEnd in the ST_TSM_MEMWR State.

- If StMemWidthQ[1:0] indicates a 32-bits memory width or TransferSize[1:0] indicates a 8-bit write from an AHB or buffer or if both StMemWidthQ and TransferSize indicate 16-bits, MemWrOverQ is asserted at the end of each memory write access irrespective of the address driven to memory.
- If the transfer size is 32-bits and memory width is 16-bits, MemWrOverQ is asserted when StAddrQ[1] is set indicating the second memory write for the current write request from the AHB or buffer
- If the transfer size is 32-bits and memory width is 8-bits, MemWrOverQ is asserted when StAddrQ[1:0] are set indicating the fourth memory write for the current write request from the AHB or buffer
- If the transfer size is 16-bits and memory width is 8-bits, MemWrOverQ is asserted when StAddrQ[0] is set indicating the second memory write for the current write request from the AHB or buffer

MemRdOverQ is asserted to indicate completion of reads to the memory device for the current read request. MemRdOverQ generation depends on the StMemWidthQ[1:0] value and the number of memory reads required for the single write from the AHB or buffer. The end of a read to memory is indicated by the assertion of CntEnd or CntEZEnd in the ST_TSM_MEMRD State.

- If StMemWidthQ[1:0] indicates a 32-bits memory or TransferSize[1:0] indicates a 8-bit read from the AHB or buffer or if both StMemWidthQ[1:0] and TransferSize[1:0] indicate 16-bits, MemRdOverQ is asserted at the end of each memory read access irrespective of the address driven to memory.

- If the transfer size is 32-bits and the memory width is 16-bits, MemRdOverQ is asserted when StAddrQ[1] is set indicating the second memory read for the current read request from the AHB or buffer
- If the transfer size is 32-bits and the memory width is 8-bits, MemRdOverQ is asserted when StAddrQ[1:0] are set indicating the fourth memory read for the current read request from the AHB or buffer.
- If the transfer size is 16-bits and memory width is 8-bits, MemRdOverQ is asserted when StAddrQ[0] is set indicating the second memory read for the current read request from the AHB or the buffer

Page length end detection for page mode enabled devices

PgLenEnd generation is required to determine the end of the page length and the crossover of the address quad-boundary for page mode enabled devices. This logic is used for the purpose of introducing the burst length restriction. Only burst length of four is supported by the MPMC for page mode enabled devices

5.10 Memory Controller Interface (MpmcMemCntlIf)

The details of the memory controller interface block are described in this section.

This block contains the logic for combining the response signals from the dynamic and static memory controllers to provide a single set of signals to the AHB interface and buffer blocks.

The following functionality is implemented in this block:

- Combining the dynamic and static memory register read data buses
- Combining the status and handshake signals from the static and dynamic memory controllers
- Combining the dynamic and static memory read data buses

Combining the register read data buses

The register read data buses from the static memory register banks and the dynamic memory register banks are logically ored and the registered out as a common memory register read data bus (MemBlkRdDataQ[14:0]) to the AHB slave register interface block.

Combining the status and handshake signals

MemBusy is a memory controller busy flag. This signal generated by combining the busy flags from the static memory controller and the dynamic memory controller blocks. The arbiter block uses MemBusy to generate a global MPMC busy status indicator.

MemAHBIdBuf[8:0] is generated by combining the requestor codes returned from the static memory controller and the dynamic memory controller. MemAHBIdBus[8:0] is used by the buffer control block to determine the buffer to which the read data should be written into.

MemA5A2[5:2] and MemByte[3:0] are generated by combining StMemA5A2Q[5:2] and DyMemA5A2[5:2] together StMemByteQ[3:0] and DyMemByte[3:0] together by using simple OR gates.

Combining the read data from static and dynamic memory

For memory read data to be returned to a buffer, StMemRDataQ[31:0] and DyMemRData[3:0] are combined using a multiplexer whose select is the StMemRd signal from the static memory controller. For memory read data to be returned to an AHB interface, StMemRDataQ[31:0] and DyMemRDataQ[3:0] are multiplexed. The difference in the two versions of read data is that the registered dynamic memory read data is used for the AHB interface in order to avoid synthesis timing issues on the AHB interface paths.

5.11 Memory Bus Granting Logic (MpmcMemBGnt)

The memory bus granting logic block arbitrates between external memory bus control requests from the static and dynamic memory controller blocks. This block receives the bus requests from the static and dynamic memory controllers and the TIC and generates the bus grants depending on whether the memory bus is being used by the other controller. The MPMC can enter the test mode only when MPMCTESTIN is asserted during reset. So on reset, the MPMC enters either the functional mode or the test mode. The requesting controller is granted the bus only if MPMCEBIGNT is asserted. The grant to the static memory is deasserted if MPMCEBIBACKOFF is asserted when the static memory controller is granted the bus. The dynamic memory controller uses MPMCEBIBACKOFF internally to block further memory access requests from being posted.

The following logic is implemented in this block:

- Memory bus arbiter state machine
- DyBusGnt generation logic
- StBusGnt generation logic
- TICBUSGNT generation logic
- MPMCEBIREQ generation

Memory bus arbiter State Machine

The memory bus arbiter State Machine is described in this sub-section. This state machine has the following states:

- ST_GNT_DMC: Memory bus arbiter state machine dynamic memory grant state
- ST_GNT_SMC: Memory bus arbiter state machine static memory grant state

ST_GNT_DMC:

This state is entered after reset. This is also the default state when none of the masters are active. The dynamic memory controller is granted the bus by default if no other controller is requesting for the external memory bus. This state indicates that the dynamic memory controller has control of the bus. If the lower priority master static memory controller requests for the bus it is granted only after the dynamic memory controller completes all its operation indicated by de-assertion on the DyBusReq request.

ST_GNT_SMC:

This state indicates that the static memory controller is active and has control of the bus. Whenever the dynamic memory controller requests for the bus, the StBusGnt is de-asserted and waits for the static memory controller to complete its current operation and de-assert StBusReq.

In **Figure 5.11-1**, the state diagram for the memory arbiter state machine is shown. The details of the state-transition-triggers are described in **Table 5.11-1**.

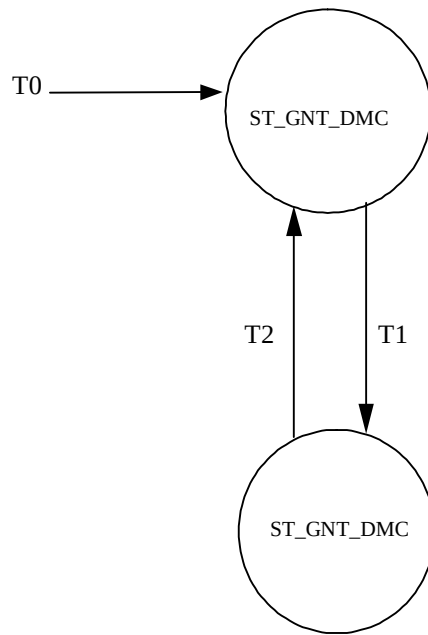


Figure 5.11-1 State Machine for Memory Bus Granting Logic

Transition Number	Description	Condition
T0	Reset to DMC	When nPOR is asserted, transition-0 takes place.
T1	DMC to SMC	When StBusReq is asserted indicating that the static memory controller is requesting control of the memory bus when DyBusReq is not asserted indicating that the dynamic memory controller is not requesting control of the memory bus, transition-1 is made.
T2	SMC to DMC	When StBusReq is not asserted indicating that the static memory controller is not requesting the control of the memory controller bus, transition-2 is made.

Table 5.11-1 Transition table for Memory Bus Granting Logic

StBusGnt generation

StBusGnt is asserted only if StBusReq is asserted, the memory bus arbiter state machine is in ST_GNT_SMC state, MPMCEBIGNT is asserted and DyBusReq is not asserted and MPMCEBIBACKOFF is not asserted. It is also generated in the idle State if DyBusReq is not asserted so that the static memory controller is granted by default on reset.

DyBusGnt generation

DyBusGnt is asserted only if the current State of the memory bus arbiter State Machine is ST_GNT_DMC and MPMCEBIREQ and MPMCEBIGNT are asserted.

TICBUSGNT generation

TICBUSGNT is asserted only if TICBUSREQ is asserted and MPMCEBIGNT is asserted. It is assumed that the MPMC will not be used to access memories in the TIC test mode. Hence there is no need to qualify the generation of TICBUSGNT with the bus requests and grants of the static and dynamic memory controllers.

MPMCEBIREQ generation

MPMCEBIREQ is asserted when StBusReq or DyBusReq or TICBUSREQ is asserted.

5.12 Pad Interface (MpmcPadIf)

The details of the pad interface block are described in this section.

This block takes signals from the static memory controller, the dynamic memory controller and TIC and drives them to the output pads of the MPMC. The signals from the dynamic memory controller are registered on MPMCCLK, and driven out. The signals from the static memory controller are registered on HCLK in the static memory controller block. This block uses the SDRClkScheme information (bit 1:0 in MpmcDyReadConfig register) to implement either 'clock-delayed-mode' or the 'command-delayed-mode' of operation to access dynamic memories. The descriptions of the two modes of operation are given below.

Clock Delayed

In this mode of operation, the clock is given an additional delay from the controller end (outside the MPMC boundary). This prevents the race between capture edge and the memory command at the memory pins. Moreover by making the 'same' edge capture the data at memory, one clock is saved.

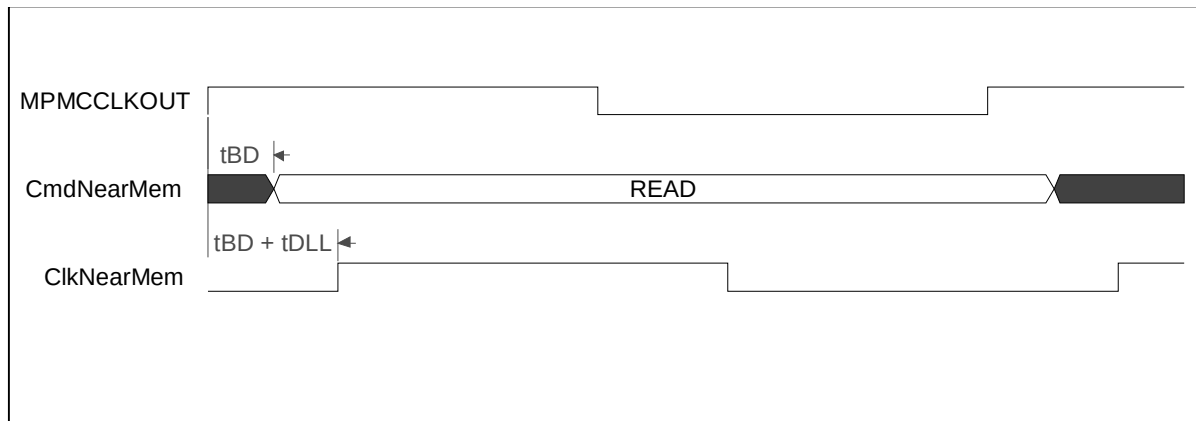


Figure 5.12-1 Clock delayed mode of operation

A board delay of t_{BD} is introduced on both clock trace and command-lines. However there is an additional delay of t_{DLL} put on the clock trace, which makes the launching edge of the command at controller, the capture edge at memory pins.

Command delayed

In this mode of operation, the race between capture edge and memory command at the memory pins is solved by delaying the commands at the controller. The command lines are delayed by clocking it out with respect to MPMCCLKDELAY.

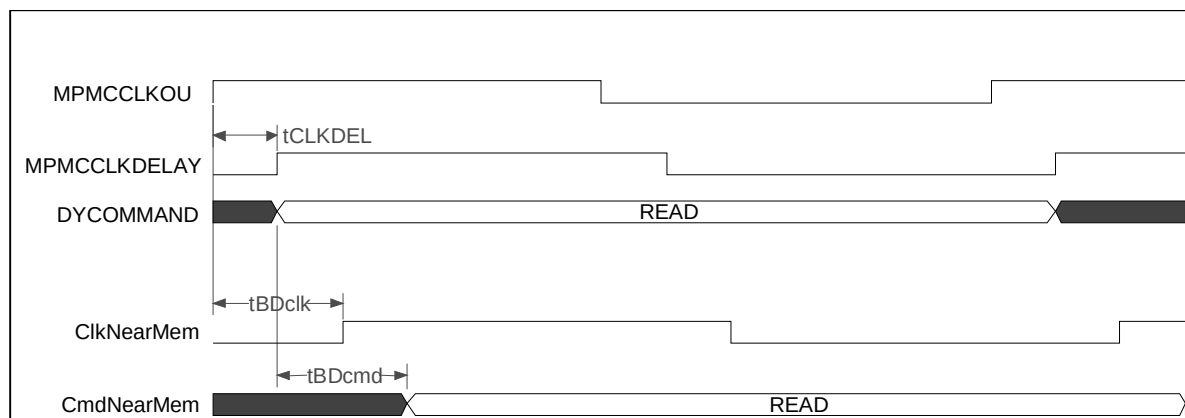


Figure 5.12-2 Command delayed mode of operation

tBDclk is the board delay introduced on clock trace. tBDcmd is the board delay introduced on the command lines. tBDclk and tBDcmd are similar but not exactly same. The race between the capture edge and the command is prevented by the additional delay of tCLKDEL (with respect to which, the commands are clocked).

Apart from the clock/command delayed mode of operation, the following functionalities are implemented in this block:

- Generation of SlowClkM to indicate the high phase of the slower clock
- Generation of MPMCDQMOUT[3:0]
- Generation of MPMCADDRROUT[27:0]
- Generation of MPMCDATAOUT[31:0]
- Generation of nMPMCDATAEN[3:0]
- Generation of nMPMCWEOUT

Generation of SlowClkM

SlowClkM is used to indicate the slower of MPMCCLK and HCLK to the dynamic memory controller. This signal is generated by generating a signal called MPMCCLKDiv2, which is at half the frequency of MPMCCLK. MPMCCLKDiv2 is driven on SlowClkM if the ClkRatioQ bit from the AHB register interface indicates that the HCLK:MPMCCLK frequency ratio is 1:2. Else, a '1' is driven on SlowClkM. SlowClkM is used in the dynamic memory controller to drive the data and command outputs to the AHB interface or buffer control block on the low phase of HCLK.

Generation of MPMCDQMOUT[3:0]

The D-input for MPMCDQMOUT[3:0] is generated by

- combining the DQM information from the different State Machines - DASM1, DASM2 and FCSM by using OR logic.
- It is registered on the rising edge of MPMCCLK.
- The output of the MPMCCLK register is further clocked with respect to MPMCCLKDELAY.
- Depending on the clocking scheme (clock delayed or command delayed), MPMCCLKed flip flop or MPMCCLKDELAYed flip flop is routed out.

Generation of MPMCADDRROUT[27:0]

DyAddrOutQ[14:0] is generated by

- combining the address information from DASM1, DASM2, FCSM, Mode State Machine and Refresh State Machine by using OR logic
- registering it on rising edge of MPMCCLK.
- The output of the MPMCCLK register is further clocked with respect to MPMCCLKDELAY.
- Depending on the clocking scheme (clock delayed or command delayed), MPMCCLKed flip flop or MPMCCLKDELAYed flip flop is routed as DyAddrOutQ.

Zeros are driven on DyAddrQ[27:15]. DyAddrOutQ[27:0] is then logically “OR”ed with StAddrQ[27:0] to generate MPMCADDRROUT[27:0]

Generation of MPMCDATAOUT[31:0]

DyWDataQ[31:0] is generated by

- combining the write data from DASM1, DASM2 and FCSM using OR logic
- registering it on rising edge of MPMCCLK.
- The output of the MPMCCLK register is further clocked with respect to MPMCCLKDELAY.
- Depending on the clocking scheme (clock delayed or command delayed), MPMCCLKed flip flop or MPMCCLKDELAYed flip flop is routed as DyWDataQ.

DyWDataQ is combined with the data outputs of the static memory controller {StDataOutQ[31:0] and the TIC (TBUSOUT[31:0]) by using OR logic to generate the final MPMCDATAOUT.

Generation of nMPMCDATAEN[3:0]

DyDataEnOutQ[3:0] is generated by

- duplicating DyDataEn on all four bits
- registering on the rising edge of MPMCCLK.
- The output of the MPMCCLK register is further clocked with respect to MPMCCLKDELAY.
- Depending on the clocking scheme (clock delayed or command delayed), MPMCCLKed flip flop or MPMCCLKDELAYed flip flop is routed as DyDataEnOutQ.

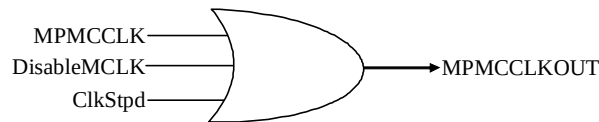
DataEnOutQ[3:0] is combined with the data enables of the static memory controller (nStDataEnQ[3:0]) and TIC (nTicDataEn[3:0]) by using AND logic to generate nMPMCDATAEN.

Generation of nMPMCWEOUT

nMPMCWEOUT is generated by combining the write enables from the static memory controller (nStWEQ), the dynamic memory controller (nDyWEOutQ) and the test acknowledge (TESTACK) from the TIC by using AND logic. As with the other dynamic signals, nDyWEOutQ is generated based on the clocking scheme (command delayed or clock delayed).

5.13 Clock gating logic (MpmcClkOr)

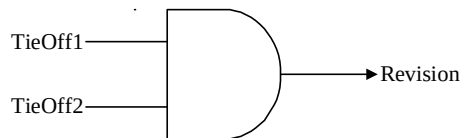
This module contains the clock gating logic for the synchronous memory clocks that are driven on the output ports, MPMCCLKOUT[3:0] of the MPMC. This is implemented by a single OR gate. The signal ClkStpd, which is driven by the dynamic memory controller is used for controlling the MPMCCLKOUT[3:0] during the idle time of the dynamic memories. If ClkStpd is LOW, MPMCCLKOUT follows MPMCCLK. If ClkStpd is high, MPMCCLKOUT goes to HIGH and the clock to the memories is thus disabled. Similarly, DisableMCLK signal, which is the input from the register interface to the Clock gating logic, is used to control MPMCCLKOUT irrespective of the state of the dynamic memory controller state machines.

**Figure 5.13-1** Clock gating component

This OR gate is instantiated four times at the top level, since 4 clocks are to be driven out on MPMCCLKOUT[3:0].

5.14 Revision Designator block (MpmcRevAnd)

This module contains a single AND gate to be used as a place-holder cell to mark the Revision of the controller. The 2 input pins are tied-off at the top level of the hierarchy. These "TieOffs" can be identified during layout and re-wired to "VDD" or "VSS" if needed.

**Figure 5.14-1** RevisionAnd component

This AND gate is instantiated four times at the top level, since the revision number is a four bit quantity. In the present version of the MPMC, all the inputs are tied to zero at the top level. Outputs of all AND gates are concatenated and connected to the output read multiplexer of the AHB slave interface logic. This helps software to identify the revision number through an AHB read.

5.15 MPMC Test Interface Controller (MpmcTIC)

The MPMC TIC bus master module helps to allow externally applied test vectors to be converted into internal AHB transfers.

The TIC is a State Machine that provides an AMBA bus master for system test. It controls the External Test Bus and AHB data buses HRDATATIC and HWDATATIC during the application of Test Interface Format (TIF) test vectors.

The TIC consists of three main blocks:

- Test Vector State Machine - Uses the TESTREQA and TESTREQB signals to decide which type of vector is being applied on the external TESTBUS.
- Address and Control Registers - Holds values for the address and control signals and includes an address incrementer for burst accesses.
- AMBA Bus Master Interface - Includes the AMBA bus master State Machine and the control of the AMBA bus signals.

The details of the TIC can be referred to, in the ARM PrimeCell Multiport Memory Controller (PL172) Technical Reference Manual [Ref. 1] and chapter 6, of the AMBA Specification (Rev 2.0) [Ref. 3].

The MpmcTIC uses the MPMCTESTIN input pin value to enable entry into test mode. The TESTACK output of the MpmcTIC shares the output pin with the active low write enables of the static and dynamic memories. So the MpmcTIC drives a HIGH on the TESTACK output when not in test mode to allow a simple AND logic in the Pad interface for driving the nMPMCWEOUT pad. This is the only difference between Micropack's TIC and the MpmcTIC.

6 DESIGN ASSUMPTIONS

Some assumptions have been made in the design of the MPMC (PL172). The user is required to keep the system assumptions in mind while connecting the MPMC (PL172) in an AHB system. The software assumptions are to be kept in mind while programming the MPMC and while accessing the memory through the MPMC.

6.1 Software Assumptions

The following software assumptions have been made regarding programming of the MPMC:

6.1.1 Mode programming/ Initialisation

- The autorefresh value programmed in the MPMCDyRef register should be greater than the precharge period and the time required for applying the auto refresh requests to all the dynamic memory chip selects.

$\text{AutorefreshValue} > ((\text{MpmcDytRPQ} + 1) + (\text{MpmcDytRFCQ} + 1) + 4)$ where MpmcDytRPQ is the precharge period, MpmcDytRFCQ is the auto refresh latency and the 4 extra clock cycles are required to apply the auto refresh cycles.

A value programmed equal to or lesser than this will always keep the Refresh State Machine running and the MPMC will end up giving only refresh cycles.

- The SDRAM memory initialisation can take two possible variations

- NOP -> PALL -> MODE -> NORMAL (normal sdram sequence)

(1) (2) (3) (4)

- MODE -> NORMAL (syncflashes sequence)

(1) (2)

In both the cases, there should be at-least one idle cycle between step (1) and step (2). For normal SDRAMs, a 200uS delay is to be maintained between (1) and (2) [as per the MPMC PL172 TRM [Ref. 2]]. Therefore, the one-clock-idle is automatically satisfied. Nothing specific has been mentioned in SyncFlash initialisation sequence. This restriction comes because of the necessity of the Mode-State-machine to be able to restart the clock and CKE, after exit from deep-sleep-State. It is to be noted that the normal sequence for SDRAMs can also be applied on syncflashes (especially in the situation where the slots are populated with both syncflashes and sdrams). However, if all the slots are populated with only syncflashes, the syncflash-sequence can be used.

- Initialisation sequence:

The steps mentioned below are to be taken-care-of in addition to the sequence mentioned in the MPMC PL172 TRM [Ref. 2].

- First program the DyConfig register for disabling the buffers.
- Keep the CE and CS bit in MpmcDyControl high.
- Perform the mode initialisation. During the mode-programming process one has to write into the MpmcDyControl register repeatedly (For altering the SdramInit field). It is to be taken care that the CE and CS bits (residing in the same register) are always kept at "11" value.
- After mode-programming is finished, reprogram to enable/disable the buffers (as per requirement)
- Also program the CS and CE bits to the desired configuration, once the mode-programming is done.

If the CE and CS bits are not kept high, then the refresh-SM will switch off the clock/clock-enable during mode-programming-sequence. To prevent the refresh-machine from switching off the clock/clock-enable, additional logic will be required. By having the restriction in place, we can avoid additional logic. Since mode-programming

does not happen frequently, thus keeping CE/CS high is not going to have any significant impact on power consumption.

- Mode-register programming is done only at the very beginning (after reset). Multiple mode programming is required only if the controller is programmed for deep-sleep-mode entry. If there is no deep-sleep-entry, then no multiple-mode-programming is allowed (in fact it is not necessary).
- After the controller exits deep-sleep mode, all the chip-selects should be mode-programmed (in case a few chips are deep-sleep-sdrams and the rest normal sdrams).
- In the startup sequence, first all the MPMC control registers should be programmed. During programming of these control registers, SRefReq bit in MpmcDyControl register (bit[2]) should not be reset. It should be maintained high. After all the registers are programmed, the SRefReq bit should be written with '0' and then the mode-initialisation routine should be started. This restriction has come because of the requirement of maintaining data in the memory device over controller reset (nPOR).
- The reset value of SRefReq bit in MpmcDyControl register has been made '1'. This is for the requirement of maintaining data in the memory device over controller reset. In case the MPMC is intended to be used only for static memories, even then it is mandatory to program the SRefReq bit to '0' before proceeding with normal operations.

6.1.2 Self Refresh Entry and Exit

- MPMCSREFREQ pin or the MPMCSREFREQ bit of the MPMCDyControl register should not go low until the MPMCSREFACK pin and the MPMCSREFACK bit of the MPMCDyControl register are asserted by the MPMC

6.1.3 SyncFlash

- During SyncFlash initialisation, the RP line is to be handled entirely by software. The nMPMCRPOUT and MPMCRPVHOUT outputs are directly controlled by the respective bits in the MPMCDyControl register.
- The Flash-command-set operations (as against the normal array read mode, similar to SDRAMs) are to be performed in 'unbuffered' mode.
- For programming the syncflashes, the syncflashes should be connected only in 16-bit external width mode.
- Every time a complete halfword access should be attempted, that is HSIZE should only indicate HalfWord. When HalfWord accesses are attempted, the halfword should be duplicated on both the lanes.

6.1.4 Low Power Deep Sleep

- Before setting the deep-sleep bit in the MPMC, it should be ensured that all the data in the quadword buffers are flushed to the memory. This can be ensured by checking that the Write-buffer-State (in MPMCStatus register) shows buffer empty.
- The low power deep sleep exit is not automatic. That is, the low-power-deep-sleep bit has to be explicitly reset by software and then only normal accesses should be started.
- Deep sleep entry is to be done 'cleanly'. To be precise,
 - When DeepSleep bit is being set, no state-machines should be in motion (reflected by the busy-bit in MpmcStatus). For e.g. when mode-programming is in process, deep-sleep entry should not be attempted.
 - As mentioned above, the buffers should be flushed to memory.

6.1.5 Static memory extended wait transfers

- The static memory transfer time programmed in the MpmcStExdWt register should be small enough to allow the dynamic memory to be refreshed at regular intervals. Using extremely long transfer times might mean that SDRAM devices are not refreshed correctly.

6.1.6 Byte Lane State

- For static memory banks constructed from 8-bit or non-byte-partitioned memory devices, the Byte Lane State (PB) bit should be cleared to 0 within the respective MPMCStConfig register.
- For static memory banks constructed from 16 or 32-bit memory devices, the Byte Lane Select (PB) bit should be set to 1 within the respective MPMCStConfig register.

6.1.7 General

- The 16-bit chip-selects HAVE to be configured for sdram-burst of eight. The 32-bit-chip-selects have to be configured for sdram-burst of four. The mode registers of the respective dynamic memory devices should be programmed accordingly.
- The CE = 1 and CS = 0 configuration is not allowed. (CE and CS are bits in MpmcDyControl register).
- MpmcEnable bit of MpmcControl register can be made low only when
- Busy bit of MPMCStatus register is low.
- Buffer status(S) bit of MPMCStatus register is low.
- Memory control and configuration registers (both dynamic and static) can be initialised only during system initialisation or with MPMcEnable bit is disabled.

6.2 System Assumptions

6.2.1 Power-on reset memory map

- On power-on reset, memory chip select 1 is mirrored onto memory chip select 0 and chip select 4. Therefore, any transactions to memory chip select 0 or chip select 4 (or chip select 1) access memory chip select 1. Hence the boot code should be located in the memory device connected to chip select 1. Clearing the address mirror bit (M) in the MPMCCControl register disables address mirroring. Memory chip select 0, chip select 4, and memory chip select 1 can then be accessed as normal.

6.2.2 Power-on reset values of Input pins

- On power on reset, the static memory chip select polarity pins (MPMCSTCS[3..0]POL) should indicate the polarity of the respective chip selects (0 for active LOW and 1 for active HIGH). These values will be used until the MPMCStConfig[3..0] register is written into. Once the corresponding chip select's configuration register is programmed, the register value the chip select polarity bit in the register will be used to determine the chip select polarity.
- On power on reset, the address mirror bit in the MPMCCControl register is asserted. Hence the memory databus width of chip select 1 should be driven on the MPMCSTCS1MW[1:0] ("00" for 8-bit, "01" for 16-bit and "10" for 32-bit databus width) and the byte lane state of chip select 1 should be driven on MPMCSTCS1PB on power on reset. This value will be used until the MPMCStConfig1 register is written into. Once the register is programmed, the memory width bits of the register will be used to determine the memory width.

- On power on reset, the endian mode of the MPMC should be driven on the MPMCBIGENDIAN pin. This value will be used until the MPMCConfig register is written into. Once this register is programmed, the register value of the Endian mode bit will be used to determine the endian mode.

6.2.3 Self-refresh mode

- The MPMCSREFREQ pin should be connected to the power management unit (PMU) in the system. The PMU should assert the MPMCSREFREQ signal when self-refresh mode has to be entered. Once asserted, MPMCSREFREQ pin should not go low until the MPMCSREFACK pin is asserted by the MPMC. In case the PMU does not support this mechanism, the self-refresh mode can be entered manually by setting the MPMCSREFREQ bit of the MPMCDyControl register and polling the MPMCSREFACK bit of the MPMCStatus register.

6.2.4 Write enable connection of static memory devices

- For static memory banks constructed from 8-bit or non byte-partitioned memory devices, the nMPMCBLSOUT[3:0] signals should be connected to write enable (nWE) inputs of each 8-bit memory. The nMPMCWEOUT signal from the MPMC should not be used.
- For static memory banks constructed from 16-bit or 32-bit memory devices, the nMPMCBLSOUT[3:0] signals should be connected to the byte select inputs of each memory device. The nMPMCWEOUT signal from the MPMC should be connected to the write enable inputs of each memory device.

6.2.5 Address connection of static memory device

- On power on reset, the MPMCREL1CONFIG input pin of the MPMC should reflect the mode of connecting the address pins of the static memory devices to the MPMCADDRROUT[27:0] lines. When HIGH, it indicates the static memory addressing mode of PL172 release 1v0 in which the static memory address should be connected as follows:
 - When external memory width is 8-bits, MPMCADDRROUT[27:0] should be connected to the A[27:0] pins of the static memory device
 - When external memory width is 16-bits, MPMCADDRROUT[27:1] should be connected to the A[26:0] pins of the static memory device and MPMCADDRROUT[0] is not used
 - When external memory width is 32-bits, MPMCADDRROUT[27:2] should be connected to the A[25:0] pins of the static memory device and MPMCADDRROUT[1:0] is not used.

When LOW, it indicates that the static memory addresses should be connected as follows:

- When external memory width is 8-bits or 16-bits or 32-bits, MPMCADDRROUT[27:0] should be connected to the A[27:0] pins of the static memory device.

6.2.6 Entry into Test mode

- On the AHB, it is assumed that the processor is always requesting access to the bus, but the TIC has the highest priority to bus access.
- Once the MPMC has entered test mode, it is assumed that there will be no normal mode accesses. The MPMC will enter test mode only if the sequence of events as indicated below is followed. The following sequence of events has to be used to use the MPMC's TIC to apply test patterns:
 - Apply reset (HRESETn) asynchronously.
 - Assert the MPMCTESTIN test input to enter test mode.
 - When the reset is removed asynchronously, the MPMC enters TIC test mode.

- The TIC block in the MPMC requests for the access to the external bus through the assertion of the MPMCEBIREQ signal. Once MPMCEBIGNT is sampled asserted, the TIC block takes control of the external bus and relinquishes the bus only after it has completed all the transfers on the AHB master interface.
- The TIC block asserts its HBUSREQTIC signal to the arbiter, requesting access to the AHB bus. Because the TIC is the highest priority, HGRANTTIC is asserted and the granted TIC takes ownership of the AHB bus.
- The MPMCTESTIN signal is usually fed to the clock generation logic so that HCLK can be switched to test speed on entry into test mode.

7 MPMC VERIFICATION TESTBENCH

The MPMC Functional Verification testbench is an AHB BusTalk testbench. The VHDL and Verilog BusTalk testbench files reside in the **verification/vhdl** and **verification/verilog** directories respectively.

7.1 MPMC VHDL Verification Testbench

The **Figure 7.1-1** shows the test environment of MPMC. The MPMC and Trickbox are connected to 4 AHB buses. The slave Testbench is responsible for programming the slave registers in Trickbox, TrickMem and UUT (MPMC). The Testbench is interfaced to the UUT and Trickbox via all AHB Buses. The Testbench is interfaced with TrickMem via AHB Bus3 to check the static memory controller functionality. The TrickMem acts as the static memory model. It can be accessed by both AHB3 and by the MPMC. The Synchronous memory model is connected to the MPMC to check the synchronous memory controller functionality. The Trickbox also does all the protocol checking and drives the non-AMBA signals to the MPMC.

Figure 7.1-1 illustrates the BusTalk VHDL testbench for running test vectors.

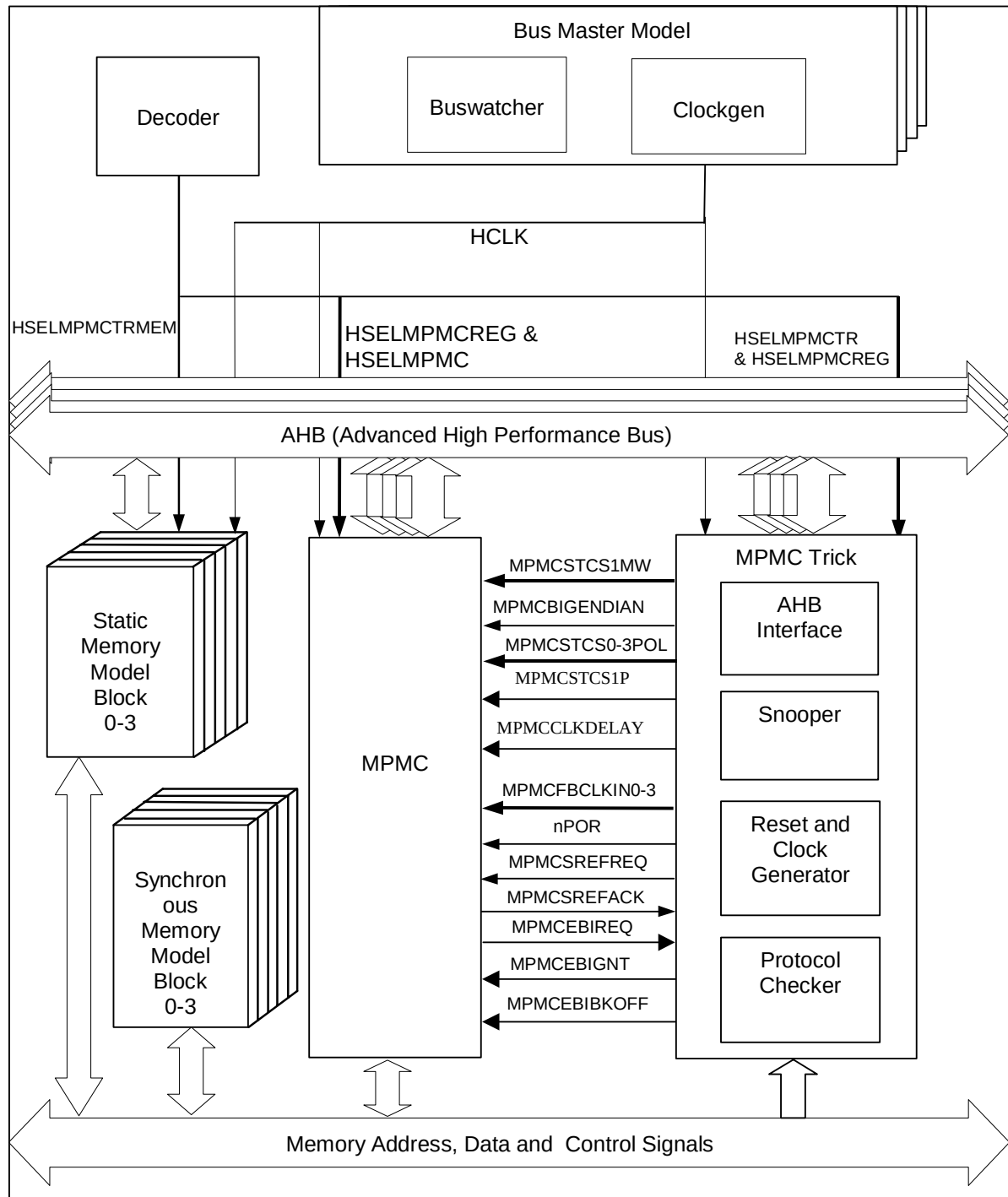
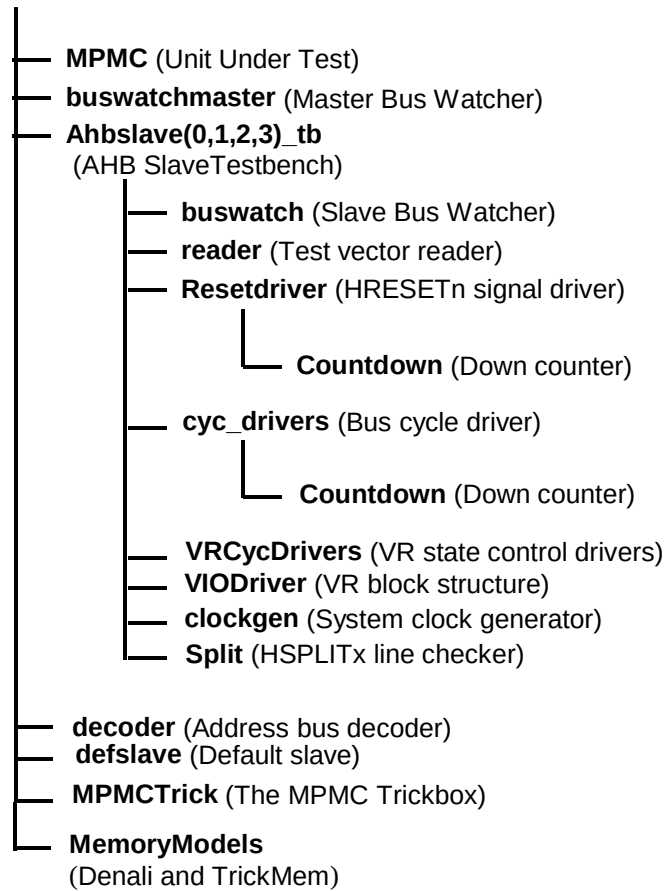


Figure 7.1-1 VHDL Functional Verification Testbench

Figure 7.1-2 illustrates the hierarchy for the VHDL testbench.

tbench**Figure 7.1-2 VHDL Functional Verification Testbench hierarchy****7.1.1 Testbenches**

The testbenches, **tbench.vhd** and **tbench1.vhd** to **tbench13.vhd** are used for functional testing of the Multiport Memory Controller (MPMC). Every testbench block instantiates the MPMC (Unit Under Test), the trickbox and the memory models. The behavioural models of the decoder, the default slave and the master bus watcher are replicated on all four AHB buses in each testbench. Different memory models are used in different test benches. The memory models used are of two types – trick memory models that are the behavioural model of a static memory device and the Denali memory models.

All the Scan input pins are tied low to keep them from interfering with the test process.

The HCLK frequency and AHB signal timing budgets are defined in the timing.vhd file.

7.1.2 Unit Under Test

The Unit Under Test may be one of the following

- MPMC VHDL RTL
- MPMC scan-inserted VHDL netlist from VHDL Source

One out of these two versions may be referenced by creating a link [named **uut**] in the **verification/vhdl** directory to the corresponding source directory. [Note that this link will be created automatically by the simulation makefiles].

7.1.3 Bus Master Protocol Checker

The testbench instantiates the master bus watcher, `buswatchmaster.vhd` that checks the protocol of master accesses initiated via the test code. This module implements the protocol checks for the AHB master's output signals and will flag warning and error messages if any timing or protocol violations are detected during simulations.

7.1.4 AHB Master/Arbiter Model

The UUT is stimulated primarily by AHB Master/Arbiter behavioural model. The AHB Master/Arbiter block issues commands on the AHB bus under program control.

There are four AHB masters hence four AHB Master/Arbiter model, `ahbslave0_tb.vhd`, `ahbslave1_tb.vhd`, `ahbslave2_tb.vhd`, `ahbslave3_tb.vhd` instantiates the buswatcher, the reader, the reset driver, the cycle drivers, the VR cycle driver, the split checker and the clock generator. The reader block reads the test vectors from the .bif file (from the **verification/bustest/invec** directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the HRESETn signal with the correct timings. The `cyc_drivers` drive the address, control and data signals for each AHB cycle type, i.e. HSR and HSW etc. The `VRcycdrivers` selects the appropriate VR registers for a read or write operation. The buswatch module implements the protocol checks for the AHB slave's output signals and flags warning and error messages on detection of mismatches. The Split module verifies that the HMASTER number is asserted on the HSPLITx lines on the expected clock. The clockgen module generates the system clock, HCLK.

The reader reads test vectors from the `infile.bif` residing in the **verification/bustest/invec** directory.

7.1.5 Decoder

This module decodes the addresses and generates the HSELx signals for the slaves. It can generate the select signals for a maximum of 16 slaves. There are 4 AHBs in the tbench. Each has their own decoders (Except AHB3, all decoding is implemented in the `tbench.vhd`, `tbench1.vhd` to `tbench13.vhd` itself).

AHB0

Any access to the address range

0x00000000 to 0x0FFFFFFF

will access the UUT memory interface via the HSEL0[0] select line.

Any access to the address range

0x10000000 to 0x1FFFFFFF

will access the UUT memory interface via the HSEL0[1] select line.

Any access to the address range

0x20000000 to 0x2FFFFFFF

will access the UUT memory interface via the HSEL0[2] select line.

Any access to the address range

0x30000000 to 0x3FFFFFFF

will access the UUT memory interface via the HSEL0[3] select line.

Any access to the address range

0x40000000 to 0x4FFFFFFF

will access the UUT memory interface via the HSEL0[4] select line.

Any access to the address range

0x50000000 to 0x5FFFFFFF

will access the UUT memory interface via the HSEL0[5] select line.

Any access to the address range

0x60000000 to 0x6FFFFFFF

will access the UUT memory interface via the HSEL0[6] select line.

Any access to the address range

0x70000000 to 0x7FFFFFFF

will access the UUT memory interface via the HSEL0[7] select line.

Any access to the address range

0xC0000000 to 0xCFFFFFFF

will access the Trickbox via the HSEL0[8] select line.

Accesses to any address outside the above ranges will select the Default Slave.

AHB1

Any access to the address range

0x00000000 to 0x0FFFFFFF

will access the UUT memory interface via the HSEL1[0] select line.

Any access to the address range

0x10000000 to 0x1FFFFFFF

will access the UUT memory interface via the HSEL1[1] select line.

Any access to the address range

0x20000000 to 0x2FFFFFFF

will access the UUT memory interface via the HSEL1[2] select line.

Any access to the address range

0x30000000 to 0x3FFFFFFF

will access the UUT memory interface via the HSEL1[3] select line.

Any access to the address range

0x40000000 to 0x4FFFFFFF

will access the UUT memory interface via the HSEL1[4] select line.

Any access to the address range

0x50000000 to 0x5FFFFFFF

will access the UUT memory interface via the HSEL1[5] select line.

Any access to the address range

0x60000000 to 0x6FFFFFFF

will access the UUT memory interface via the HSEL1[6] select line.

Any access to the address range

0x70000000 to 0x7FFFFFFF

will access the UUT memory interface via the HSEL1[7] select line.

Any access to the address range

0xC0000000 to 0xCFFFFFFF

will access the Trickbox via the HSEL1[8] select line.

Accesses to any address outside the above ranges will select the Default Slave.

AHB2

Any access to the address range

0x00000000 to 0x0FFFFFFF

will access the UUT memory interface via the HSEL2[0] select line.

Any access to the address range

0x10000000 to 0x1FFFFFFF

will access the UUT memory interface via the HSEL2[1] select line.

Any access to the address range

0x20000000 to 0x2FFFFFFF

will access the UUT memory interface via the HSEL2[2] select line.

Any access to the address range

0x30000000 to 0x3FFFFFFF

will access the UUT memory interface via the HSEL2[3] select line.

Any access to the address range

0x40000000 to 0x4FFFFFFF

will access the UUT memory interface via the HSEL2[4] select line.

Any access to the address range

0x50000000 to 0x5FFFFFFF

will access the UUT memory interface via the HSEL2[5] select line.

Any access to the address range

0x60000000 to 0x6FFFFFFF

will access the UUT memory interface via the HSEL2[6] select line.

Any access to the address range

0x70000000 to 0x7FFFFFFF

will access the UUT memory interface via the HSEL2[7] select line.

Any access to the address range

0xC0000000 to 0xCFFFFFFF

will access the Trickbox via the HSEL2[8] select line.

Accesses to any address outside the above ranges will select the Default Slave.

AHB3

Any access to the address range

0x00000000 to 0x0FFFFFFF

will access the UUT memory interface via the HSEL3[0] select line.

Any access to the address range

0x10000000 to 0x1FFFFFFF

will access the UUT memory interface via the HSEL3[1] select line.

Any access to the address range

0x20000000 to 0x2FFFFFFF

will access the UUT memory interface via the HSEL3[2] select line.

Any access to the address range

0x30000000 to 0x3FFFFFFF

will access the UUT memory interface via the HSEL3[3] select line.

Any access to the address range

0x40000000 to 0x4FFFFFFF

will access the UUT memory interface via the HSEL3[4] select line.

Any access to the address range

0x50000000 to 0x5FFFFFFF

will access the UUT memory interface via the HSEL3[5] select line.

Any access to the address range

0x60000000 to 0x6FFFFFFF

will access the UUT memory interface via the HSEL3[6] select line.

Any access to the address range

0x70000000 to 0x7FFFFFFF

will access the UUT memory interface via the HSEL3[7] select line.

Any access to the address range

0x80000000 to 0xBFFFFFFF

will access the UUT register interface via the HSEL3[8] select line.

Any access to the address range

0xC0000000 to 0xCFFFFFFF

will access the Trickbox via the HSEL3[9] select line.

Any access to the address range

0xD0000000 to 0xDFFFFFFF

will access the TrickMem0 (u0MpmcTrickMem) via the HSEL3[10] select line.

Any access to the address range

0xE0000000 to 0xEFFFFFFF

will access the TrickMem1 (u1MpmcTrickMem) via the HSEL3[11] select line.

Any access to the address range

0xF0000000 to 0xF7FFFFFF

will access the TrickMem2 (u2MpmcTrickMem) via the HSEL3[12] select line.

Any access to the address range

0xF8000000 to 0xFFFFFFFF

will access the TrickMem3 (u3MpmcTrickMem) via the HSEL3[13] select line.

Accesses to any address outside the above ranges will select the Default Slave.

7.1.6 Default Slave

This module drives the response when an access is detected to an unmapped region. The Default slave drives its HREADY output high, when other slaves are not selected. It drives a zero wait State OK response for IDLE and BUSY transfers and a two cycle ERROR response for an NSEQ or SEQ transfer access to an unmapped address.

7.1.7 Protocol Checker

The testbench includes a protocol checker, which reports any protocol or timing violations during accesses to any AHB slave. Timing parameters can be set using the timing.vhd file in the **verification/vhdl/tbench** directory. The buswatch module residing in the **verification/vhdl/buswatcher** directory checks the Read/Write databus timings and reports any timing violations.

7.1.8 Trickbox

The functionality of the MPMC is verified via the Trickbox. The Trickbox is a programmable AHB slave designed to drive stimulus on the non-AHB input terminals of the MPMC and sample its non-AHB outputs. It samples the output lines from the MPMC and checks for the protocols. It flags error messages for any unexpected behaviour detected on the MPMC memory signals.

7.1.9 Trickbox Memory Model

The TrickMem is a programmable static memory model, which also is an AHB slave. The TrickMem module is used for testing memory related functionality of the MPMC. The MPMC writes into the TrickMem memory model and the data can be verified via both through the MPMC and the AHB. The TrickMem model checks for the timings of the memory control signals. When timing or protocol violation is detected on the MPMC memory control signals, the TrickMem flags error messages.

7.1.10 Denali Memory Models

The Denali Memory simulation models are used for verification of the MPMC. Memory transfers are done to the Denali models through the MPMC and transfers are verified by reading out the data from the memory through the MPMC. The SOMA and initialising files for the Denali models are located in the **verification/denali** directory. The SOMA files are *.soma files and the memory initialising files are *.dat files.

7.1.11 Simulation environment

Generics

The MPMC testbench provides some configurability options using generics. All these generics except SuppressOnReset are configurable through the tbench.vhd file. SuppressOnReset is configurable through the ahbslave_tb.vhd file in the **verification/vhdl/tbench** directory.

- XonSig

XonSig is used in the cyc_drivers.vhd file that resides in **verification/vhdl/bid**.

This generic is used to eliminate the 'X's that appear on the address and control signals when these signals are not being actively driven. If XonSig is set to 0 in the top level of the testbench, these signals go to '0' when the corresponding line driver is inactive. If set to 1, the signals go to 'X' when the line driver is idle.

- Verbosity

Verbosity is used in the reset_driver.vhd and the cyc_drivers.vhd which reside in **verification/vhdl/bid**. It is also used in the buswatch.vhd file that resides in **verification/vhdl/buswatcher** and the VIODrivers.vhd which reside in **verification/vhdl/vr**.

This generic is used to control the verbosity of the transcript generated during simulation. If Verbosity is set to 1, every command issued will have a corresponding message in the transcript file. If Verbosity is set to 0, a message is issued only if an error occurs on a read.

- HaltOnMismatch

This is used in the cyc_drivers.vhd, which resides in **verification/vhdl/bid**, the buswatch.vhd which resides in **verification/vhdl/buswatcher** and the VIODriver.vhd which resides in **verification/vhdl/vr**.

This generic is used to stop the simulation if an error occurs. If set to 1, the simulation halts after an error is reported. If set to 0, the error is flagged, but the simulation continues.

- Databuswidth

Databuswidth is used in the cyc_drivers.vhd, which resides in **verification/vhdl/bid**.

This generic controls the width of the AHB databus. For the MPMC testbench this is set to 32.

- SuppressOnReset

SuppressOnReset is used in the buswatch.vhd, which resides in **verification/vhdl/buswatcher**.

This generic is used to suppress all protocol checks on the slave's output signals when the HRESETn signal is asserted. If SuppressOnReset is TRUE, the set-up and hold timing violations on these signals are not flagged when HRESETn is asserted. If SuppressOnReset is set to FALSE, the set-up and hold timing violations are flagged irrespective of HRESETn. When SuppressOnReset is FALSE the slave's output signals are not checked for 'X'es when HRESETn is asserted.

Test Vectors

The test vectors for the verification of the MPMC are coded into separate C files, in the **verification/bustest** directory. There are separate test files created for each category tests described in the verification plan.

Running Simulations

The **README** file in the **verification/vhdl/tbench** directory gives step by step instructions for running simulations using the MPMC test vectors on the VHDL source code.

Run time logs for all the combinations are available in the **verification/vhdl/MPMC_rtl** and **verification/vhdl/MPMC_scaninsert_vhdl_net_max** directories.

7.2 MPMC Verilog Verification Testbench

The **Figure 7.2-1** shows the test environment of MPMC. The MPMC and Trickbox are connected to 4 AHB buses. The slave Testbench is responsible for programming the slave registers in Trickbox, TrickMem and UUT (MPMC). The Testbench is interfaced to the UUT and Trickbox via all AHB Buses. The Testbench is interfaced with TrickMem via AHB Bus3 to check the static memory controller functionality. The TrickMem acts as the static memory model. It can be accessed by both AHB3 and by the MPMC. The Synchronous memory model is connected to the MPMC to check the synchronous memory controller functionality. The Trickbox also does all the protocol checking and drives the non-AMBA signals to the MPMC.

Figure 7.2-1 illustrates the MPMC Verilog testbench for simulating test vectors.

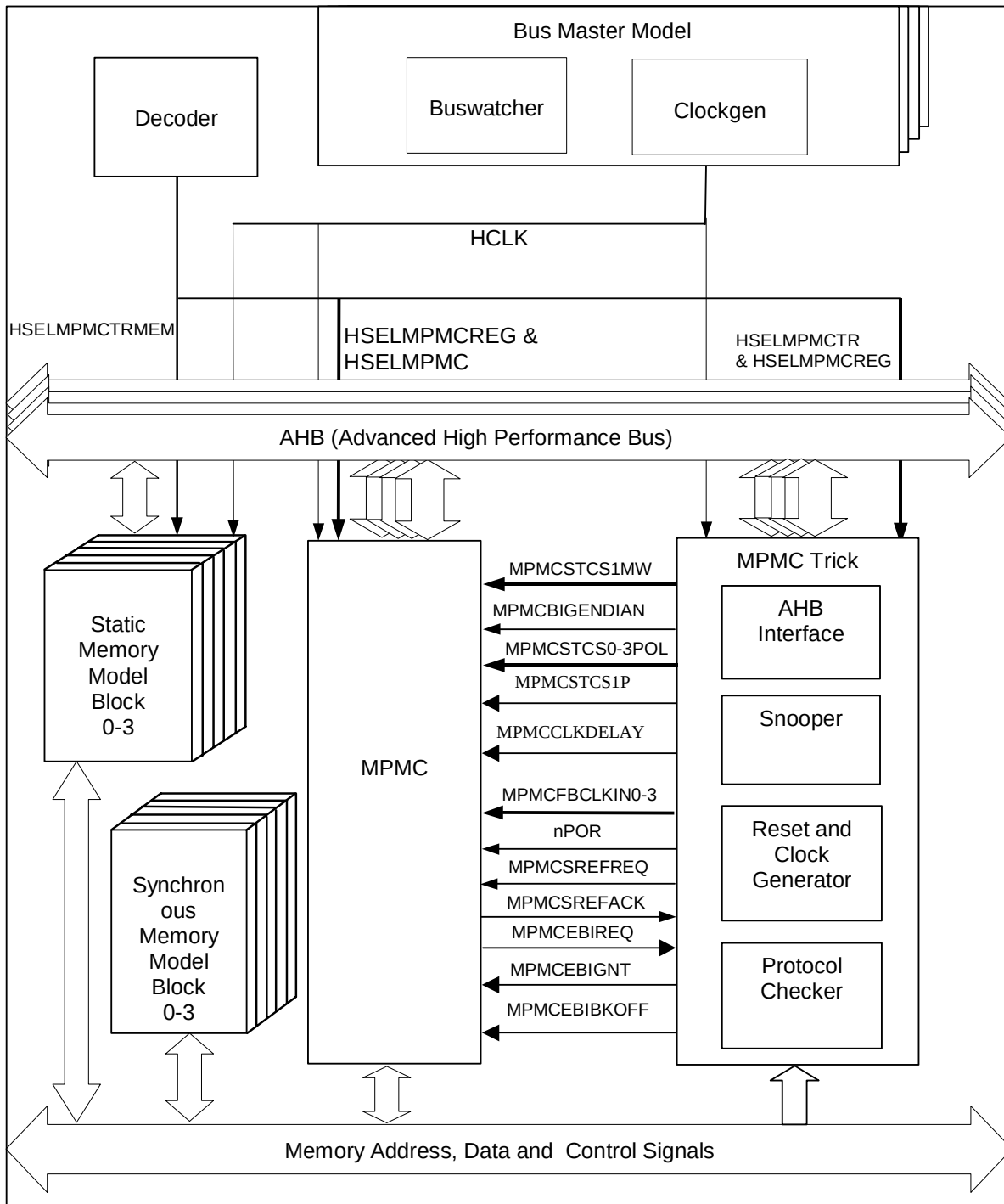
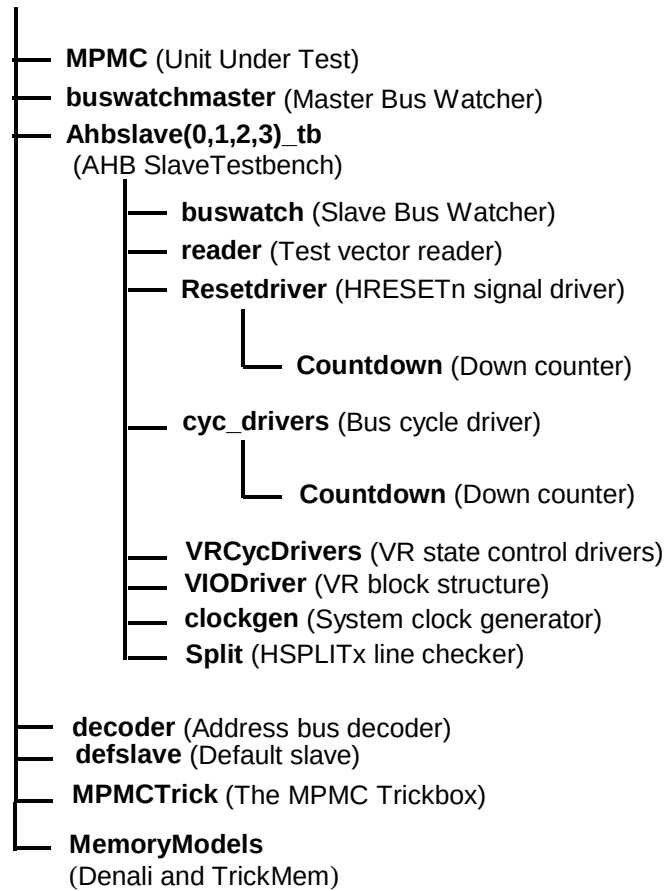


Figure 7.2-1 Verilog Functional Verification testbench

Figure 7.2-2 illustrates the hierarchy for the Verilog testbench.

tbench**Figure 7.2-2 Verilog Functional Verification Testbench hierarchy****7.2.1 Testbenches**

The testbenches, **tbench.v** and **tbench1.v** to **tbench13.v** are used for functional testing of the Multiport Memory Controller (MPMC). Every testbench block instantiates the MPMC (Unit Under Test), the trickbox and the memory models. The behavioural models of the decoder, the default slave and the master bus watcher are replicated on all four AHB buses in each testbench. Different memory models are used in different test benches. The memory models used are of two types – trick memory models that are the behavioural model of a static memory device; and the Denali memory models.

All the Scan input pins are tied low to keep them from interfering with the test process.

The HCLK frequency and AHB signal timing budgets are defined in the timing.v file.

7.2.2 Unit Under Test

The Unit Under Test may be one of the following,

MPMC Verilog RTL

MPMC scan-ready Verilog netlist from Verilog Source

MPMC non-scan Verilog netlist from VHDL Source

One out of these three versions may be referenced by creating a link [named **uut**] in the **verification/verilog** directory to the corresponding source directory. [Note that this link will be created automatically by the simulation makefiles].

7.2.3 Bus Master Protocol Checker

The testbench instantiates the master bus watcher, `buswatchmaster.v`, which checks the protocol of master accesses initiated via the test code. This module implements the protocol checks for the AHB master's output signals and flags warning and error messages if any timing or protocol violations are detected during simulations.

7.2.4 AHB Master/Arbiter Model

The UUT is stimulated primarily by AHB Master/Arbiter behavioural model. The AHB Master/Arbiter block issues commands on the AHB bus under program control.

There are four AHB masters and hence four AHB Master/Arbiter model, `ahbslave0_tb.v`, `ahbslave1_tb.v`, `ahbslave2_tb.v`, `ahbslave3_tb.v` instantiates the buswatcher, the reader, the reset driver, the cycle drivers, the VR cycle driver, the split checker and the clock generator. The reader block reads the test vectors from the .sim file (from the **verification/bustest/invec** directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the `HRESETn` signal with the correct timings. The `cyc_drivers` drive the address, control and data signals for each AHB cycle type, i.e. HSR and HSW etc. The `VRcycdrivers` selects the appropriate VR registers for a read or write operation. The `buswatch` module implements the protocol checks for the AHB slave's output signals and flags warning and error messages on detection of mismatches. The `split` module verifies that the `HMASTER` number is asserted on the `HSPLITx` lines on the expected clock. The `Clockgen` module generates the system clock, `HCLK`.

The reader reads test vectors from the `bif.sim` residing in the **verification/bustest/invec** directory.

7.2.5 Decoder

This module decodes the addresses and generates the `HSELx` signals for the slaves. It can generate the select signals for a maximum of 16 slaves. There are 4 AHBs in the `tbench`. Each has their own decoders (Except AHB3, all decoding is implemented in the `tbench.vhd` itself).

AHB0

Any access to the address range

`0x00000000 to 0x0FFFFFFF`

will access the UUT memory interface via the `HSEL0[0]` select line.

Any access to the address range

`0x10000000 to 0x1FFFFFFF`

will access the UUT memory interface via the `HSEL0[1]` select line.

Any access to the address range

`0x20000000 to 0x2FFFFFFF`

will access the UUT memory interface via the `HSEL0[2]` select line.

Any access to the address range

`0x30000000 to 0x3FFFFFFF`

will access the UUT memory interface via the `HSEL0[3]` select line.

Any access to the address range

0x40000000 to 0x4FFFFFFF

will access the UUT memory interface via the HSEL0[4] select line.

Any access to the address range

0x50000000 to 0x5FFFFFFF

will access the UUT memory interface via the HSEL0[5] select line.

Any access to the address range

0x60000000 to 0x6FFFFFFF

will access the UUT memory interface via the HSEL0[6] select line.

Any access to the address range

0x70000000 to 0x7FFFFFFF

will access the UUT memory interface via the HSEL0[7] select line.

Any access to the address range

0xC0000000 to 0xCFFFFFFF

will access the Trickbox via the HSEL0[8] select line.

Accesses to any address outside the above ranges will select the Default Slave.

AHB1

Any access to the address range

0x00000000 to 0x0FFFFFFF

will access the UUT memory interface via the HSEL1[0] select line.

Any access to the address range

0x10000000 to 0x1FFFFFFF

will access the UUT memory interface via the HSEL1[1] select line.

Any access to the address range

0x20000000 to 0x2FFFFFFF

will access the UUT memory interface via the HSEL1[2] select line.

Any access to the address range

0x30000000 to 0x3FFFFFFF

will access the UUT memory interface via the HSEL1[3] select line.

Any access to the address range

0x40000000 to 0x4FFFFFFF

will access the UUT memory interface via the HSEL1[4] select line.

Any access to the address range

0x50000000 to 0x5FFFFFFF

will access the UUT memory interface via the HSEL1[5] select line.

Any access to the address range

0x60000000 to 0x6FFFFFFF

will access the UUT memory interface via the HSEL1[6] select line.

Any access to the address range

0x70000000 to 0x7FFFFFFF

will access the UUT memory interface via the HSEL1[7] select line.

Any access to the address range

0xC0000000 to 0xCFFFFFFF

will access the Trickbox via the HSEL1[8] select line.

Accesses to any address outside the above ranges will select the Default Slave.

AHB2

Any access to the address range

0x00000000 to 0x0FFFFFFF

will access the UUT memory interface via the HSEL2[0] select line.

Any access to the address range

0x10000000 to 0x1FFFFFFF

will access the UUT memory interface via the HSEL2[1] select line.

Any access to the address range

0x20000000 to 0x2FFFFFFF

will access the UUT memory interface via the HSEL2[2] select line.

Any access to the address range

0x30000000 to 0x3FFFFFFF

will access the UUT memory interface via the HSEL2[3] select line.

Any access to the address range

0x40000000 to 0x4FFFFFFF

will access the UUT memory interface via the HSEL2[4] select line.

Any access to the address range

0x50000000 to 0x5FFFFFFF

will access the UUT memory interface via the HSEL2[5] select line.

Any access to the address range

0x60000000 to 0x6FFFFFFF

will access the UUT memory interface via the HSEL2[6] select line.

Any access to the address range

0x70000000 to 0x7FFFFFFF

will access the UUT memory interface via the HSEL2[7] select line.

Any access to the address range

0xC0000000 to 0xCFFFFFFF

will access the Trickbox via the HSEL2[8] select line.

Accesses to any address outside the above ranges will select the Default Slave.

AHB3

Any access to the address range

0x00000000 to 0x0FFFFFFF

will access the UUT memory interface via the HSEL3[0] select line.

Any access to the address range

0x10000000 to 0x1FFFFFFF

will access the UUT memory interface via the HSEL3[1] select line.

Any access to the address range

0x20000000 to 0x2FFFFFFF

will access the UUT memory interface via the HSEL3[2] select line.

Any access to the address range

0x30000000 to 0x3FFFFFFF

will access the UUT memory interface via the HSEL3[3] select line.

Any access to the address range

0x40000000 to 0x4FFFFFFF

will access the UUT memory interface via the HSEL3[4] select line.

Any access to the address range

0x50000000 to 0x5FFFFFFF

will access the UUT memory interface via the HSEL3[5] select line.

Any access to the address range

0x60000000 to 0x6FFFFFFF

will access the UUT memory interface via the HSEL3[6] select line.

Any access to the address range

0x70000000 to 0x7FFFFFFF

will access the UUT memory interface via the HSEL3[7] select line.

Any access to the address range

0x80000000 to 0xBFFFFFFF

will access the UUT register interface via the HSEL3[8] select line.

Any access to the address range

0xC0000000 to 0xCFFFFFFF

will access the Trickbox via the HSEL3[9] select line.

Any access to the address range

0xD0000000 to 0xDFFFFFFF

will access the TrickMem0 (u0MpmcTrickMem) via the HSEL3[10] select line.

Any access to the address range

0xE0000000 to 0xEFFFFFFF

will access the TrickMem1 (u1MpmcTrickMem) via the HSEL3[11] select line.

Any access to the address range

0xF0000000 to 0xF7FFFFFF

will access the TrickMem2 (u2MpmcTrickMem) via the HSEL3[12] select line.

Any access to the address range

0xF8000000 to 0xFFFFFFFF

will access the TrickMem3 (u3MpmcTrickMem) via the HSEL3[13] select line.

Accesses to any address outside the above ranges will select the Default Slave.

7.2.6 Default Slave

This module drives the response when an access is detected to an unmapped region. The Default slave drives its HREADY output high, when other slaves are not selected. It drives a zero wait State OK response for IDLE and BUSY transfers and a two cycle ERROR response for an NSEQ or SEQ transfer access to an unmapped address.

7.2.7 Protocol Checker

The testbench includes a protocol checker, which reports any protocol or timing violations during accesses to any AHB slave. Timing parameters can be set using the timing.v file in the **verification/verilog/tbench** directory. The buswatch module residing in the **verification/verilog/buswatcher** directory checks the Read/Write databus timings and reports any timing violations.

7.2.8 Trickbox

The functionality of the MPMC is verified via the Trickbox. The Trickbox is a programmable AHB slave designed to drive stimulus on the non-AHB input terminals of the MPMC and sample its non-AHB outputs. It samples the output lines from the MPMC and checks for the protocols. It flags error messages for any unexpected behaviour detected on the MPMC memory signals.

7.2.9 Trickbox Memory Model

The TrickMem is a programmable memory model, which also is an AHB slave. The TrickMem module is used for testing memory related functionality of the MPMC. The MPMC writes into the TrickMem memory model and the data can be verified via both through the MPMC and the AHB. The TrickMem model checks for the timings of the memory control signals. When timing or protocol violation is detected on the MPMC memory control signals, the TrickMem flags error messages.

7.2.10 Denali Memory Models

The Denali Memory simulation models are used for verification of the MPMC. Memory transfers are done to the Denali models through the MPMC and transfers are verified by reading out the data from the memory through the MPMC. The SOMA and initialising files for the Denali models are located in the **verification/denali** directory. The SOMA files are *.soma files and the memory initialising files are *.dat files.

7.2.11 Simulation Environment

Simulation configuration (Parameters)

The MPMC testbench provides some configurability options via parameters. All the parameters are configurable through the tbench.v file.

- XonSig

XonSig is used in the cyc_drivers.v file that resides in **verification/verilog/bid**.

This parameter is used to eliminate the 'X's that appear on the address and control signals when these signals are not being actively driven. If XonSig is set to 0 in the top level of the testbench, these signals go to '0' when the corresponding line driver is inactive. If set to 1, the signals go to 'X' when the line driver is idle.

- Verbosity

Verbosity is used in the reset_driver.v and the cyc_drivers.v which reside in **verification/verilog/bid**. It is also used in the buswatch.v that resides in **verification/verilog/buswatcher** and the VIODrivers.v which reside in **verification/verilog/vr**.

This parameter is used to control the verbosity of the transcript generated during simulation. If Verbosity is set to 1, every command issued will have a corresponding message in the transcript file. If Verbosity is set to 0, a message is issued only if an error occurs on a read.

- HaltOnMismatch

This is used in the cyc_drivers.v, which resides in **verification/verilog/bid**, the buswatch.v which resides in **verification/verilog/buswatcher** and the VIODriver.v which resides in **verification/verilog/vr**.

This parameter is used to stop the simulation if an error occurs. If set to 1, the simulation halts after an error is reported. If set to 0, the error is flagged but the simulation continues.

- SuppressOnReset

SuppressOnReset is used in the buswatch.v, which resides in **verification/verilog/buswatcher**.

This parameter is used to suppress all protocol checks on the slave's output signals when the HRESETn signal is asserted. If SuppressOnReset is TRUE, the set-up and hold timing violations on these signals are not flagged when HRESETn is asserted. If SuppressOnReset is set to FALSE, the set-up and hold timing violations are flagged irrespective of HRESETn. When SuppressOnReset is FALSE the slave's output signals are not checked for 'X'es when HRESETn is asserted.

- TestMode

TestMode is used in the cyc_drivers.v, which resides in **verification/verilog/bid**.

If the TestMode parameter is set, the testbench is configured to run in Big Endian mode. By default this parameter is cleared.

- Databuswidth

Databuswidth is used in the cyc_drivers.v, which resides in **verification/verilog/bid**.

This parameter controls the width of the AHB databus. For the MPMC testbench this is set to 32.

Test Vectors

The test vectors for the verification of the MPMC are coded into separate C files, in the **verification/bustest** directory. There are separate test files created for each category tests described in the verification plan. Make options have been provided to use them individually.

Running Simulations

The **README** file in the **verification/verilog/tbench** directory gives step by step instructions for running simulations using tests on the Verilog source code.

Run time logs for all the combinations are available in the **verification/verilog/MPMC_rtl**, **verification/verilog/MPMC_noscan_vhdl_net_min** and **verification/verilog/MPMC_scanready_verilog_net_typ** directories.

7.3 Trickbox Description

The Trickbox is a programmable AHB slave, designed to drive stimulus on the non-AHB input terminals of the Multiport Memory Controller and to sample the non-AHB outputs of the same. The Trickbox also provides a way for Data Transfer Protocol Validation.

The Trickbox provides an AHB interface for accessing the internal registers and controlling the data transfer.

The basic blocks of the trickbox are illustrated in **Figure 7.3-1 Block diagram of the trickbox**.

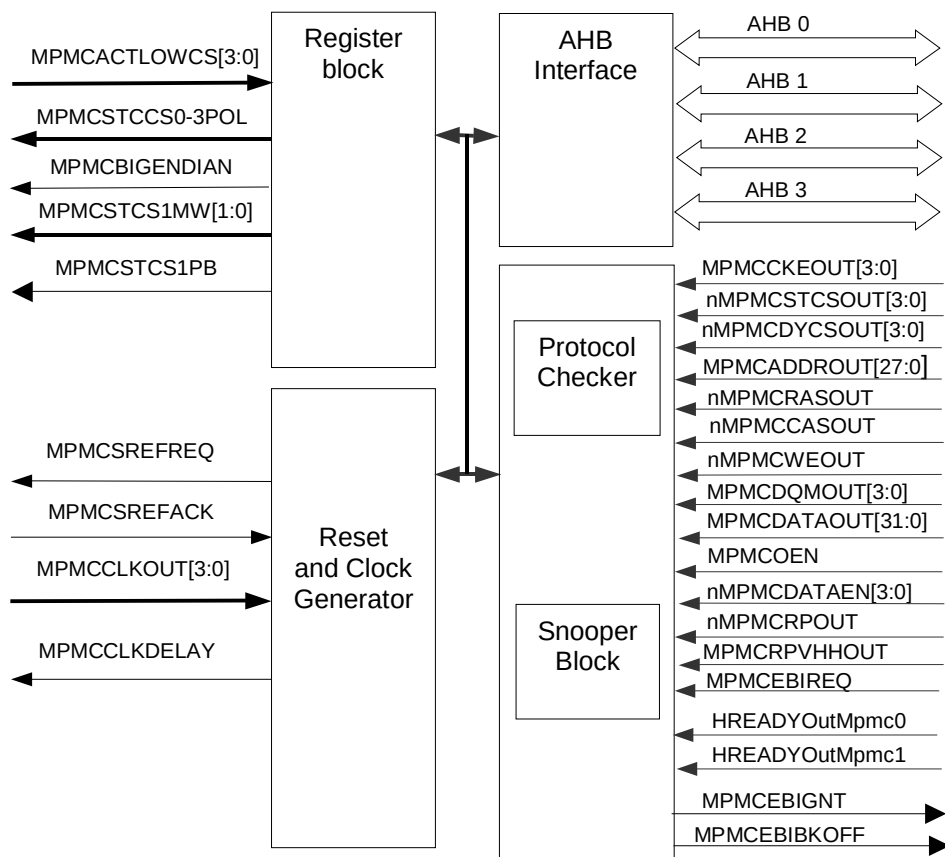


Figure 7.3-1 Block diagram of the trickbox

7.3.1 Trickbox Signal Description

Multiport Memory Controller Behavioural TrickBox Signals are listed in **Table 7.3-1** and **Table 7.3-2**

Name	Type	Source / Destination	Description
HCLK	IN	From AHB master model 3	AHB clock
HRESETn	IN	From AHB master model 3	Bus reset signal, Active Low
HADDRx[11:2]	IN	From AHB master model x (x = 0 to 3)	Subset of AHB address bus
HTRANSx[1:0]	IN	From AHB master model x (x = 0 to 3)	Indicates the type of the current transfer.
HSELMPMCTRx	IN	From AHB master model x (x = 0 to 3)	Trickbox select signal from decoder. When set to 1 this signal indicates the Trickbox is selected and that a data transfer is required
HSELMPMCREG	IN	From AHB master model 3	MPMC register select signal from decoder.
HSIZEx[2:0]	IN	From AHB master model x (x = 0 to 3)	AHB Data Transfer Size Indicator
HWRITEx	IN	From AHB master model x (x = 0 to 3)	AHB transfer direction signal, indicates a write access when HIGH, read access when LOW
HBURSTx[2:0]	IN	From AHB master model x (x = 0 to 3)	AHB Burst type indicator
HREADYINx	IN	From AHB master model x (x = 0 to 3)	AHB Bus Free Input
HWDATAx[31:0]	IN	From AHB master model x (x = 0 to 3)	AHB Write databus
HRESPx[1:0]	OUT	From AHB master model x (x = 0 to 3)	AHB Slave Response
HREADYOUTx	OUT	From AHB master model x (x = 0 to 3)	AHB Slave Ready Response
HRDATAx[31:0]	OUT	From AHB master model x (x = 0 to 3)	Output Databus to AHB

Table 7.3-1 AHB signal descriptions

Name	Type	Source/Destination	Description
MPMCCLKOUT[3:0]	IN	From Controller	Synchronous memory Clocks
MPMCCLKDELAY	OUT	To Controller	Delayed MPMCCLK
MPMCCKEOUT[3:0]	IN	From Controller	Synchronous memory clock enables.
nPOR	OUT	To Controller	Power on reset

Name	Type	Source/Destination	Description
nMPMCSTCSOUT[3:0]	IN	From Controller	Static memory chip selects.
nMPMCDYCSOUT[3:0]	IN	From Controller	Synchronous memory chip selects.
MPMCADDRROUT[27:0]	IN	From Controller	Address output. Used for both static and Synchronous memory devices. Synchronous memories use bits [14:0]. Static memories use bits [25:0].
nMPMCRASOUT	IN	From Controller	Row address strobe.
nMPMCCASOUT	IN	From Controller	Column address strobe.
nMPMCWEOUT	IN	From Controller	Write enable.
MPMCDQMOUT[3:0]	IN	From Controller	Data mask output
MPMCDATAOUT[31:0]	IN	From Controller	Data output to memory.
MPMCACTLOWCS[3:0]	IN	From MpmcTrickMem	Active low Chip Select signals from MpmcTrickMem. These active LOW signals (i.e. independent of the Chip select polarity programmed) are used in the multiple chip select assertion protocol checking.
nMPMCOENOUT	IN	From Controller	Output enable for static memories.
MPMCDATAEN[3:0]	IN	From Controller	Tri-State I/O pad enable for the byte lanes of the external memory data bus MPMCDATA[31:0], active LOW. Enables the byte lanes [31:24], [23:16], [15:8], [7:0] of the data bus independently.
MPMCSREFREQ	OUT	To Controller	Self-refresh request.
nMPMCRP	IN	From Controller	Reset power down to SyncFlash memory.
MPMCRPVHHOUT	IN	Form Controller	Reset power down to SyncFlash memory, which has to be driven to VHH.
MPMCSREQACK	IN	From Controller	Self-refresh acknowledgement.
HREADYOutMpmc0	IN	From Controller	HREADY of AHB0
HREADYOutMpmc1	IN	From Controller	HREADY of AHB1
MPMCBIGENDIAN	OUT	To Controller	Endianness of the system.
MPMCSTCS0POL	OUT	To Controller	Chip select polarity of CS0
MPMCSTCS1POL	OUT	To Controller	Chip select polarity of CS1
MPMCSTCS2POL	OUT	To Controller	Chip select polarity of CS2

Name	Type	Source/Destination	Description
MPMCSTCS3POL	OUT	To Controller	Chip select polarity of CS3
MPMCSTCS1PB	OUT	To Controller	ByteLaneSelect pin of CS1
MPMCSTCS1MW[1:0]	OUT	To Controller	Memory width of memory chip 1

Table 7.3-2 MPMC Interface signal description

7.3.2 Trickbox register summary

The Base address of the trickbox is not fixed and depends on the system implementation. However, the offset of any particular register from the Base address is fixed.

MPMCTrCR: Control Register

This is a 5-bit read/write register for controlling the MPMCSREFREQ and nPOR pins.

Bits	Name	Read/Write	Function
31:5	Reserved	-	-
4	ProtChkMask	R/W	Masks protocol checks when it is set.
3	BigEndian	R/W	System endianness.
2	LatencyChkEn	R/W	Enables the RAS and CAS latency check
1	POR	R/W	Controls nPOR line.
0	SREFREQ	R/W	Controls MPMCSREFREQ line.

Table 7.3-3 MPMCTrCR bit description

The MPMCSREFREQ line is driven continuously if the SREFREQ Bit is set. If SREFREQ Bit is reset, MPMCSREFREQ line is pulled low after sampling a high on MPMCSREFACK pin.

If POR bit is set, the nPOR line will be asserted at positive edge of HCLK.

If BigEndian bit is set, MPMCBIGENDIAN line will be asserted which indicates that the system is operating in BIGENDIAN mode.

MPMCTrSR: MPMC Trickbox Status Register

This is a 9-bit read/write register for setting and reading the control signals.

Bits	Name	Read/Write	Function
31:9	Reserved	-	-
8	RefErrSt	R/W	Refresh frequency error Status.
7	RefCheckEn	R/W	Refresh frequency Check Enable.
6:3	CKEOUT	R	CKE output from the Controller
2	SREF	R	This will be set when a self-refresh command appears at the memory command signals.
1	InitSeqErrSt	R/W	Initial Sequence Error Status. Write will clear it.
0	InitCheckEn	R/W	Enables the Initial Sequence Check

Table 7.3-4 MPMC Trickbox Status Register

MPMCTrSNPFIFO1: MPMC Trickbox Snooper Fifo1

This is a 32 bit Read/Write register for reading the Snooper FIFO1 contents. This FIFO is used to snoop the sequence of nMPMCCASOUT, nMPMCRASOUT, nMPMCWEOUT, nMPMCDYCSOUT, nMPMCSTCSOUT and MPMCADDRROUT.

Bits	Name	Read/Write	Function
31:0	FIFO1	R	Snooper FIFO1 Output.

Table 7.3-5 MPMC Trickbox Snooper Fifo1

Encoding of the FIFO1 is given in the table below:

31:29 – Command		28:26 – Chip select		25:0 – Address out
Value	Code	Value	Chip select	MPMCADDRROUT[25:0]
000	LMR	000	nMPMCSTCSOUT[0]	
001	BURST TERMINATE	001	nMPMCSTCSOUT[1]	
010	NOP	010	nMPMCSTCSOUT[2]	
011	ACTIVE	011	nMPMCSTCSOUT[3]	
100	WRITE	100	nMPMCDYCSOUT[0]	
101	READ	101	nMPMCDYCSOUT[1]	
110	PRECHARGE	110	nMPMCDYCSOUT[2]	
111	REFRESH/LCR	111	nMPMCDYCSOUT[3]	

Table 7.3-6 MPMC Trickbox Snooper Fifo1 Encoding

MPMCTrSNPFIFO2: MPMC Trickbox Snooper Fifo2

This is a 32 bit Read/Write register for reading the Snooper FIFO2 contents. This FIFO is used to snoop the MPMCDATAOUT from MPMC.

Bits	Name	Read/Write	Function
31:0	FIFO2	R	Snooper FIFO2 Output.

Table 7.3-7 MPMC Trickbox Snooper Fifo2

MPMCTrStCS: MPMC Trickbox static chip select polarity and memory width register.

This is a 7 bit Read/Write register which drives the appropriate pins of the MPMC according to the value programmed to these bits. These bits are basically used to drive the PowerOnReset values of the chipselect polarity of the four chip select. Memory width and the Byte Lane State of the chip select 1.

Bits	Name	Read/Write	Function
31:7	Reserved	-	-
6	MPMCSTCS1PB	R/W	ByteLaneState of CS1
5:4	CS1MW	R/W	Memory width of CS1
3	CS3POL	R/W	Chip select polarity of CS3
2	CS2POL	R/W	Chip select polarity of CS2
1	CS1POL	R/W	Chip select polarity of CS1
0	CS0POL	R/W	Chip select polarity of CS0

Table 7.3-8 MPMC Trickbox static chip select polarity and memory width register

MPMCTrTES: MPMC Trickbox Test End Register

This 4-bit register is used for achieving the synchronisation of 4-AHBs. Each bit of this register is writable through only the corresponding AHB while it is readable from all AHBs. Bit 0 of this register corresponds to AHB0 and so are other bits.

Bits	Name	Read/Write	Function
3:0	TES	R/W	Test end status.

Table 7.3-9 MPMC Trickbox Test end status register

MPMCTrSNPCR: MPMC Trickbox Snooper control Register

This is a 4-bit register to control the functionality of the FIFOs in the snooper block.

Bits	Name	Read/Write	Function
3	FIFO2 Clear	R/W	Snooper FIFO2 clear. When set, the contents of FIFO2 is cleared.
2	FIFO2 Enable	R/W	Snooper FIFO2 Enable.
1	FIFO1 Clear	R/W	Snooper FIFO1 clear. When set, the contents of FIFO1 is cleared.
0	FIFO1 Enable	R/W	Snooper FIFO1 Enable.

Table 7.3-10 MPMC Trickbox Snooper control Register

MPMCTrExpRef: MPMC Trickbox Expected refresh cycles Register

This 4-bit register specifies the number of refresh cycles required during the initialisation sequence of SDRAMs. The protocol checker flags error message when this number is violated during the initialisation process.

Bits	Name	Read/Write	Function
3:0	ExpRefresh	R/W	Expected number of refresh cycles during SDRAM initialisation.

Table 7.3-11 MPMC Trickbox expected refresh cycles Register

MPMCTrExBkOff: MPMC Trickbox Backoff assertion Counter value

This 6-bit register helps to assert the BackOff signal to the granted controller. When the bus grant is given to the controller, this count value is loaded to the counter. When this counter value reaches zero, BackOff is asserted.

Bits	Name	Read/Write	Function
6:0	ExBkOff	R/W	Counter value to assert the BackOff signal

Table 7.3-12 MPMC Trickbox expected refresh cycles Register

HREADY0CNT: MPMC Trickbox AHB0 latency limitation Counter Value

This 10-bit register contains the maximum value of AHB0 latency. If HREADY0 is low for more than the value specified in this register, the protocol checker displays the error message.

Bits	Name	Read/Write	Function
9:0	HREADY0CNTR	R/W	Counter value to check the AHB0 latency

Table 7.3-13 MPMC Trickbox AHB0 latency register

HREADY1CNT: MPMC Trickbox AHB1 latency limitation Counter Value

This 10-bit register contains the maximum value of AHB1 latency. If HREADY1 is low for more than the value specified in this register, the protocol checker displays the error message.

Bits	Name	Read/Write	Function
9:0	HREADY1CNTR	R/W	Counter value to check the AHB1 latency

Table 7.3-14 MPMC Trickbox AHB1latency register

MPMCTrControl: MPMC Trickbox Control Register

The MPMCTrControl register is a Read / Write register that is used to check the following protocol of the MPMC.

Checking Mpmc disable protocol

Checking Low power mode protocol

This register is a mirrored version of the MPMCControl register of the MPMC

Bit	Name	R/W	Reset value	Function
31-3	Reserved	-	0x0	-
2	Low power mode (L)	R/W	0x0	0 = Normal mode 1 = Low power mode
1	Address Mirror(M)	R/W	0x1	0 = Normal memory map 1 = CS1 is mirrored onto CS0 and CS4
0	Enable(E)	R/W	0x1	0 = MPMC Disabled 1 = MPMC Enabled

Table 7.3-15 MPMC Trickbox Control Register

MPMCTrConfig: MPMC Trickbox Configuration Register

The MPMCTrConfig register is a Read / Write register that is used to check HCLK to MPMCCLK ratio of MPMC.

This register is a mirrored version of the MPMCCConfig register of the MPMC

Bit	Name	R/W	Reset value	Function
31-9	Reserved	-	0x0	-
8	HCLK : MPMCCLK ratio (CLK)	R/W	0x0	0 = 1:1 1 = 1:2
7:1	Reserved	-	0x0	-
0	Endian mode (N)	R/W	0x0	0 = Little endian mode. 1 = Big endian mode.

Table 7.3-16 MPMC Trickbox Configuration Register

MPMCTrDynCntl: MPMC Trickbox Dynamic memory Control Register

The MPMCTrDynCntl register is a Read / Write register that is used to check the following protocol of the MPMC.

Monitoring the MPMCCKEOUT with respect to CE bit field

Monitoring the MPMCCLKOUT with respect to CS bit field

Checking the software control over Self-refresh.

Monitoring the signal nMPMCRPOUT and MPMCRPVHHOUT with respect to RP bit field

This register is a mirrored version of the MPMCDynCntrl register of the MPMC

Bit	Name	R/W	Reset value	Function
31-16	Reserved	-	0x0	-
15	SyncFlash Reset/Power down signal to be driven to VHH	R/W	0x0	0 = MPMCRPVHHOUT signal low 1 = MPMCRPVHHOUT signal high
14	SyncFlash Reset/Power down signal (nRP)	R/W	0x0	0 = nMPMCRPOUT signal low 1 = nMPMCRPOUT signal high
13	Low-power SDRAM deep sleep mode(DP)	R/W	0x0	0 = normal operation 1 = enter deep power down mode
12-9	Reserved	-	0x0	-
8-7	SDRAM Initialisation (I)	R/W	0x0	00 = Issue SDRAM NORMAL operation command. 01 = Issue SDRAM MODE command. 10 = Issue SDRAM PALL (pre-charge all) command. 11 = Issue SDRAM NOP (no-operation) command.
6	Reserved	-	0x0	-
5	DMC	R/W	0x0	0 = Enables MPMCCLKOUT 1 = Disable MPMCCLKOUT
4-3	Reserved	-	0x0	-
2	Self refresh (SR)	R/W	0x0	0 = Normal mode. 1 = Self-refresh mode.
1	Synchronous memory Clock control (CS)	R/W	0x1	0 = MPMCCLK stops when all SDRAMs are idle 1 = MPMCCLK runs continuously.
0	Synchronous memory Clock enable control (CE)	R/W	0x0	0 = Clock enable of idle devices are de-asserted to save power. 1 = All clock enables are driven HIGH continuously.

Table 7.3-17 MPMC Trickbox Dynamic memory Control Register

MPMCTrDynRfrsh: MPMC Trickbox Dynamic memory Refresh Register

The MPMCTrDynRfrsh register is a Read / Write register that is used to check the protocol of refresh cycle of the MPMC.

This register is a mirrored version of the MPMCSyncRefresh register of the MPMC.

Bit	Name	R/W	Reset value	Function
31-11	Reserved	-	0x0	-
10-0	REFRESH	R/W	0x000	0x0 = Refresh disabled 0x1 = 1(x 16) = 16 HCLK ticks between SDRAM refresh cycles. 0x8 = 8(x 16) = 128 HCLK ticks between SDRAM refresh cycles. 0x1 to 0x3ff = n(x 16) = 16n HCLK ticks between SDRAM refresh cycles.

Table 7.3-18 MPMC Trickbox Dynamic memory Refresh Register**MPMCTrDynRC[0,1,2,3]: MPMC Trickbox Dynamic memory RAS to CAS delay count Register**

The MPMCTrDynRC register is a Read / Write register that is used to check the protocol of RAS to CAS latencies of the MPMC.

This register is a mirrored version of the MPMCDynRasCas register of the MPMC.

Bit	Name	R/W	Reset value	Function
31-10	Reserved	-	0x0	-
9-8	CAS latency(CAS)	R/W	0x3	00 = Reserved 01 = 1 10 = 2 11 = 3
7-2	Reserved	-	0x0	-
1-0	RAS to CAS latency (RAS)	R/W	0x3	00 = Reserved 01 = 1 10 = 2 11 = 3

Table 7.3-19 MPMC Trickbox Dynamic memory RAS to CAS Register.**MPMCTrExtWt: MPMC Trickbox static memory extended wait register**

The MPMCTrExtWt register is a Read/Write register that is used to program the extended wait delay to static memory access.

This register is a mirrored version of the MPMCStaticExtended Wait register of the MPMC.

Bit	Name	R/W	Reset value	Function
31:10	Reserved	-	-	-
9-0	EXTENDEDWAIT	R/W	0x00	Purpose of this register is to time long static memory read and write transfers, when the EW bit of the MPMCStaticConfig register is enabled.

Table 7.3-20 MPMCTrickMem Extended Wait Register

MPMCTrDynCnfg[0,1,2,3]: MPMC Trickbox Dynamic memory configuration Register

The MPMCTrDynCnfg register is a Read / Write register that is used to configure the dynamic memory.

This register is a mirrored version of the MPMCDynConfig register of the MPMC.

Bit	Name	R/W	Reset value	Function
31:21	Reserved	-	-	-
20	Write Protect[P]	R/W	0x0	0 = writes not protected 1 = write protected
19	Buffer enable[B]	R/W	0x0	0 = buffers disabled 1 = buffers enabled
18-15	Reserved	-	-	-
14-7	Address Mapping[AM]	R/W	0x0	Bit 14 : 16/32 bit external bus Field [13:12] : RBC/BRC Field [11:9] : Part Size Field [8:7] : MemPartWidth
6-5	Reserved	-	-	-
4-3	Memory device[MD]	R/W	0x0	00 = SDRAM 01 = Low-Power SDRAM 10 = Micron SyncFlash 11 = Reserved
2-0	Reserved	-	-	-

Table 7.3-21 MPMCTrickbox Dynamic memory configuration Register

7.3.3 MPMC Static Memory Model (TrickMem)

The TrickMem serves the purpose of a static memory model, required for the verification of the MPMC.

The following features of the TrickMem are programmable.

Memory type – can be programmed as SRAM, ROM and BURST ROM

Memory width – can be programmed as 8-bit, 16-bit and 32-bit

Read access time for both page and non page mode

Write access time

Memory data bus turn around time

Independent chip select polarity for all the 4 memory banks

Chip select to nMPMCWEOUT assertion delay during writes to Memory devices.

Chip select to nMPMCOENOUT assertion delay during reads from Memory devices.

The following sections detail the functionality of the TrickMem.

TrickMem Signal Description

The following tables summarise the MPMC TrickMem signal list. **Table 7.3-22** lists the AHB interface signals while **Table 7.3-23** lists the block specific non-AMBA signals.

Signal Name	Type	Source / Destination	Description
-------------	------	----------------------	-------------

Signal Name	Type	Source / Destination	Description
HCLK	IN	Clock Generator	AHB Bus Clock. This clock times all bus transfers. All signal timings are referenced to the rising edge of HCLK.
HRESETn	IN	Reset Controller	Bus reset (active low).
HADDR[15:0]	IN	AHB Master	Address bus. Subset of 32-bit system address bus.
HTRANS[1:0]	IN	AHB Master	Indicates the type of the current transfer.
HSIZE[2:0]	IN	AHB Master	Indicates the size of the current transfer.
HBURST[2:0]	IN	AHB Master	Indicates the transfer type
HWRITE	IN	AHB Master	AHB transfer direction signal, indicates a write access when HIGH, read access when LOW.
HRDATA[31:0]	OUT	To Test Bench	AHB read data bus. It is used to transfer data from bus slaves to the master during read operations.
HWDATA[31:0]	IN	AHB Master	AHB write data bus. During write operations, HWDATA is used to transfer data from the bus master to the slaves.
HREADYIN	IN	External Slave(s)	The slave will start the transfer only if HREADYIN is HIGH.
HREADYOUT	OUT	AHB Slave	HREADYOUT along with an OK response indicates the successful completion of a data transfer.
HRESP[1:0]	OUT	AHB Slave	The bus transfer response.
HSELMPCMCTRMEM	IN	AHB Decoder	MPMC TrickMem select

Table 7.3-22 AMBA AHB Signals description

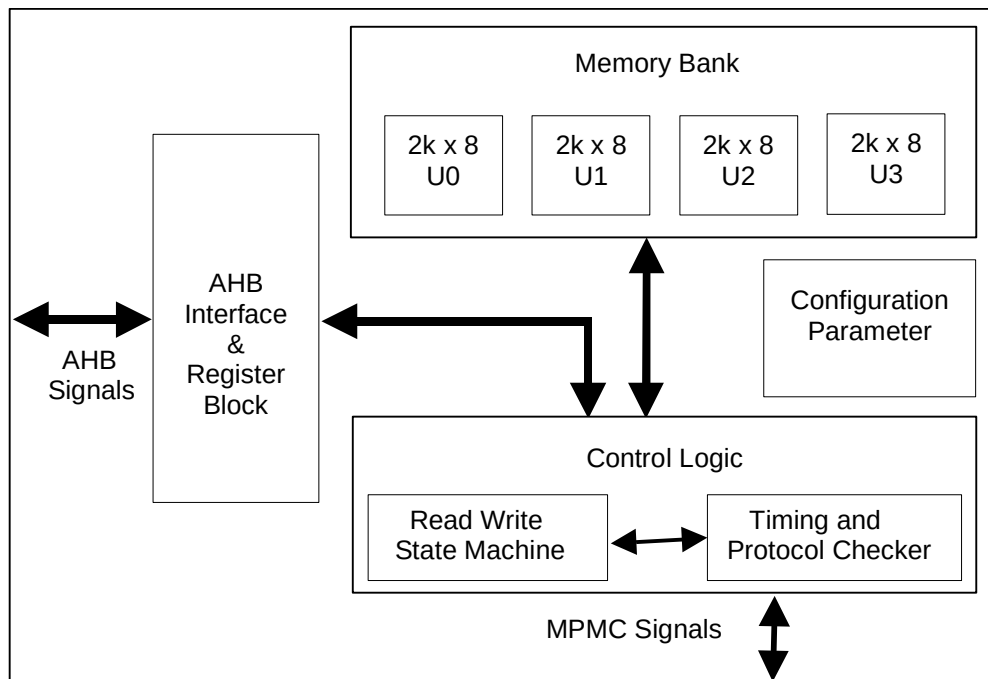
Signal Name	Type	Source / Destination	Description
MPMCDATA[31:0]	INOUT	MPMC	Memory data bus. It is a 32-bit bi-directional bus
MPMCADDR[27:0]	IN	MPMC	Address bus to the memory bank.
nMPMCWEOUT	IN	MPMC	Write enable for the external memory bank (active low).
nMPMCOENOUT	IN	MPMC	Output enable for external memory bank
MPMCDQMOUT[3:0]	IN	MPMC	Byte enables (byte-wide) for the external memory bank
MPMCTrAlICS[3:0]	IN	MPMC	The CS status of all the eight banks connected with the MPMC.
MPMCTrCS	IN	MPMC	Individual Select line for a TrickMem (bank).

Signal Name	Type	Source / Destination	Description
MPMCACTLOWCS[3:0]	OUT	Trickbox	Used for checking the multiple assertion of Chip Select.

Table 7.3-23 Block specific non-AMBA Memory signals**TrickMem Functional Description**

The TrickMem contains the following major modules.

The block diagram of the TrickMem is shown in the **Figure 7.3-2**

**Figure 7.3-2 TrickMem block diagram**

- **AHB Interface and the Register Block**

This block interfaces the TrickMem with the AHB. The TrickMem is an AHB slave and it supports read/write operations of transfer size 32-bit and burst type as INCR. The AHB interface block performs the following operations.

Generation of all AHB slave related response.

Generation of read and write decodes for all registers.

- **TrickMem Register Summary**

The MPMC TrickMem registers are shown in **Table 7.3-24**.

Address	Read Location	Write Location
MPMC TrickMem Base + 0x0000 – 1FFF	MPMCTrMEMR	MPMCTrMEMR
MPMC TrickMem Base + 0x2000	MPMCTrIDCY	MPMCTrIDCY
MPMC TrickMem Base + 0x4000	MPMCTrMEMT	MPMCTrMEMT

Address	Read Location	Write Location
MPMC TrickMem Base + 0x5000	MPMCTrMEMB	MPMCTrMEMB
MPMC TrickMem Base + 0x6000	MPMCTrCS2OEN	MPMCTrCS2OEN
MPMC TrickMem Base + 0x7000	MPMCTrWaitRd	MPMCTrWaitRd
MPMC TrickMem Base + 0x8000	MPMCTrCS2WEN	MPMCTrCS2WEN
MPMC TrickMem Base + 0x9000	MPMCTrWaitWr	MPMCTrWaitWr
MPMC TrickMem Base + 0xA000	MPMCTrWaitPg	MPMCTrWaitPg
MPMC TrickMem Base + 0xF000	MPMCTrCSPOL	MPMCTrCSPOL

Table 7.3-24 TrickMem Register map

The Memory model (TrickMem) is instantiated four times for four static memory banks of MPMC. Different AHB base addresses are used for different Memory models.

- Memory Block

This block contains a basic memory array of 8-bit width and 2K depth. This memory array is instantiated four times in parallel to create 8Kb of memory of 32-bit width and 2K depth. Depending on the value programmed in the MPMCTrMEMT register, it is possible to configure this block as an 8-bit, 16-bit or a 32-bit wide memory. Parameter value MemDeep, in the MpmcTrPackage/MpmcTrParams file determines the depth of the memory block. It is possible to access this memory both from the AHB and from the MPMC.

- Control Logic

Control logic contains two parts:

State machine:

The State Machine is shown in **Figure 7.3-3**. It contains the following states

ST_IDLE: This is the default State after reset. When a memory write request is issued, the State Machine moves to the ST_WRITE State. Similarly, when a memory read request is issued, the State Machine moves to the ST_READ State.

ST_WRITE: The State Machine can move only to the ST_IDLE State from this State. The rising edge of the write signal is used for this purpose (ST_WRITE to ST_IDLE State change).

ST_READ: Depending on the input parameters, it is possible to move to the ST_IDLE State and to the ST_BURSTREAD State from this State.

The State Machine moves to

the ST_IDLE State on the rising edge of read enable signal, nMPMCOENOUT.

the ST_BURSTREAD State if the TrickMem is programmed as a Burst ROM and the read access initiated by the controller is a burst read.

ST_BURSTREAD: The State Machine can move only to the ST_IDLE State. The rising edge of read enable signal, nMPMCOENOUT is used for this purpose (ST_BURSTREAD to ST_IDLE State change).

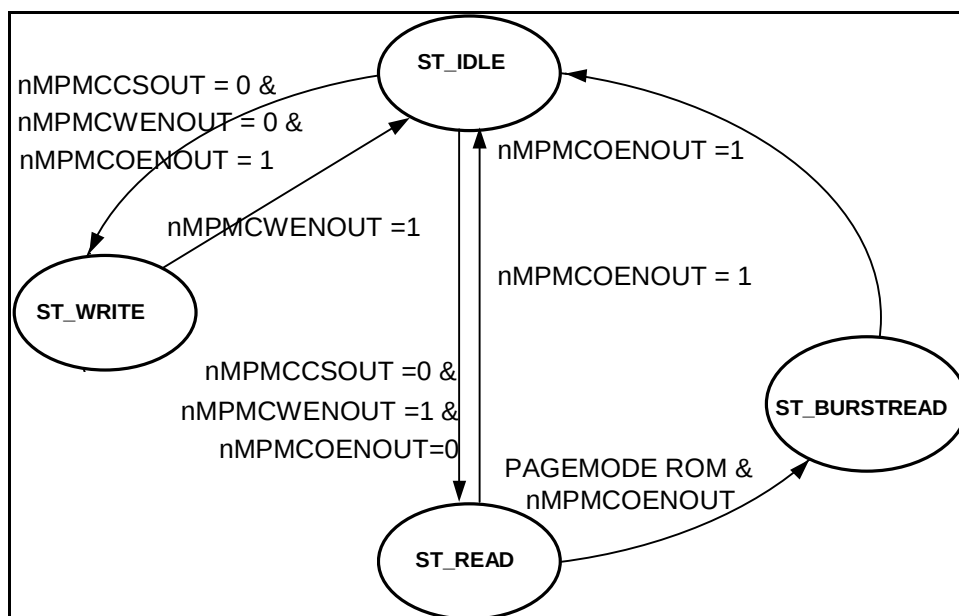


Figure 7.3-3 TrickMem State Machine

Timing and protocol check block:

This block displays error messages when the following unexpected operations happen:

Access time violation for non-burst read. The access time starts with assertion of chip select and ends with de-assertion of output enable or chip select or address change.

Access time violation for burst read. The access time starts with change in Address and ends with change in address or de-assertion of chip select.

In the above two protocol checks, the actual access time measured is compared with MPMCTrWaitRd and MPMCTrWaitPg registers taking MPMCTrCS2OEN register value into account.

Maximum time limit violation for non-burst read access. This maximum access time is little more than normal access time as mentioned in Protocol 1.

Maximum time limit violation for burst read access. This maximum access time is little more than normal access time as mentioned in Protocol 2.

Access time violation for writes. The access time starts with assertion of chip select and ends with de-assertion of write enable.

In the above protocol check, the actual access time measured is compared with MPMCTrWaitWr register taking MPMCTrCS2WEN register value into account.

Maximum time limit violation for write-accesses. This maximum access time is little more than normal access time as mentioned in Protocol 5.

Read and write enable asserted at the same time.

Address changes when write enable is active.

Chip select controlled write.

Chip select changes when read is active.

Address hold-time violation for write accesses with respect to write enable de-assertion.

Data hold-time violation for write accesses with respect to write enable de-assertion.

Data set-up time violation for write accesses with respect to write enable assertion.

Write to read idle time violation for same and different bank with idle time as one clock period.

Read to write idle time violation for same and different bank with idle time as $(1 + IDCY) \times \text{clock period}$.

Read to read idle time violation for different bank with idle time as $(1 + IDCY) \times \text{clock period}$

Write-enable byte-select violation.

Write to ROM or PAGE MODE ROM or Write Protected SRAM.

Chip select assertion to Output Enable assertion time violation. This check is not performed for Wait enabled mode.

Chip select assertion to Write Enable assertion time violation. This check is not performed for Wait enabled mode.

Chip select changes when Write is active.

MPMCDQMOUT (nMPMCBLSOUT) protocol checks with respect to PB (Byte Lane State) bit in the MPMCStaticConfig register.

- Configuration Parameters

The following parameter values are set via the configuration file

Address Set-up / Hold time

Data Set-up / Hold time

Depth of Memory

TrickMem Register Description

The base address of the TrickMem is not fixed and may be different based on system implementations. However, the offset of any particular register from the base address is fixed.

MPMCTrMEMR: MPMC TrickMem Memory Array Register

Bit	Name	R/W	Reset value	Function
31- 0	MEMR	R/W	0x00	Purpose of this register is to provide read write accessibility to the memory. This represents 2k-memory space of 32 bit wide memory. Figure 7.3-4 shows this memory space.

Table 7.3-25 MPMCTrickMem Memory array register

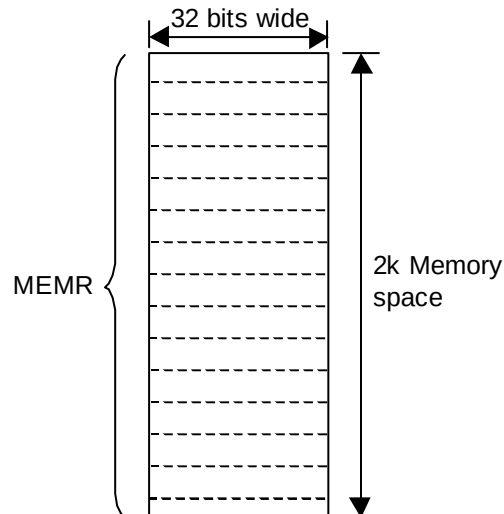


Figure 7.3-4 Memory space 32-bit wide

MPMCTrIDCY : MPMC TrickMem Idle Time Register

Bit	Name	R/W	Reset value	Function
31-4	Reserved	-	-	-
3 – 0	IDCY	R/W	0x00	Memory data bus turn around time. The turn around time is $(IDCY + 1) \times T_{HCLK}$

Table 7.3-26 MpmcTrickMem Idle Time register

MPMCTrMEMT : MPMC TrickMem Memory Type Register

Bits	Name	R/W	Reset value	Function
31-9	Reserved	-	-	-
8	Extended Wait	R/W	0x0	0 = Extended wait access disabled 1 = Extended wait access enabled
7	CSPolarity	R/W	0x00	0 = Memory banks respond when CS is active low 1 = Memory banks respond when CS is active high.
6	PB (Byte Lane State)	R/W	0x00	0 = Memory banks configured as Write Enable controlled. 1 = Memory banks configured as Byte Enable controlled.
5	AccErMask	R/W	0x00	1 = Error message disable 0 = Display error message if any violation happened in the read access time.

Bits	Name	R/W	Reset value	Function
4	MaxAccErMask	R/W	0x00	1 = Error message disable 0 = Display error message if the read access time exceeded then the expected value and whether there is a bank to bank idle time violation or not.
3 – 2	MEMTYPE	R/W	0x00	MEMTYPE bits determine the type of the memory model simulated by the TrickMem 0x00 for SRAM 0x01 for ROM 0x02 for PAGE MODE ROM
1 – 0	MEMSIZE	R/W	0x00	MEMSIZE bits determine the width of the memory model simulated by the TrickMem. 0x00 for 8-bit wide memory 0x01 for 16-bit wide memory 0x02 for 32-bit wide memory

Table 7.3-27 MpmcTrickMem Memory Type register**MPMCTrMEMB: MPMC TrickMem Memory addresses base register**

Bit	Name	R/W	Reset value	Function
31-17	Reserved	-	-	-
16 – 0	BASEVALUE	R/W	0x00	The TrickMem contains only 8K of memory. Using MPMCTrMEMB register value as a base value and the 8K TrickMem internal memory as an offset it is possible to simulate 64-MB memory requirement for the test.

Table 7.3-28 MpmcTrickMem Memory address base Register**MPMCTrCS2OEN: MPMC TrickMem CS assertion to OEN assertion delay register**

Bit	Name	R/W	Reset value	Function
31-4	Reserved	-	-	-
3 – 0	CS2OEN	R/W	0x00	In the case of SRAMs and ROMs this register determines the time duration between Chip select assertion and the assertion of nMPMCOENOUT signal. In the case of Page mode ROMs initial access this register determines the time duration between Chip select assertion and the assertion of nMPMCOENOUT signal and is insignificant in successive Page mode ROM accesses.

Table 7.3-29 MpmcTrickMem CS assertion to nMPMCOENOUT assertion delay register**MPMCTrWaitRd: MPMC TrickMem Read accesses delay register**

Bit	Name	R/W	Reset value	Function
31-5	Reserved	-	-	-
4 – 0	WaitRead	R/W	0x00	In the case of SRAMs and ROMs this register determines the read access time. The time is marked as the interval between the CS assertion to CS/OEN de-assertion, CS assertion to address change or address change to address change. In the case of Page mode ROMs initial access, this register determines the time duration between Chip select assertion and the address change and is insignificant in successive Page mode ROM accesses.

Table 7.3-30 MpmcTrickMem Read access delay register**MPMCTrCS2WEN: MPMC TrickMem CS assertion to write-enable assertion wait register**

Bit	Name	R/W	Reset value	Function
31-4	Reserved	-	-	-
3 – 0	CS2WEN	R/W	0x00	In the case of SRAMs this register determines the time duration between Chip select assertion and the assertion of nMPMCWENOUT signal. In the case of Page mode ROMs and ROMs this field is insignificant.

Table 7.3-31 MpmcTrickMem CS assertion to nMPMCWENOUT assertion register**MPMCTrWaitWr: MPMC TrickMem Write accesses delay register**

Bit	Name	R/W	Reset value	Function
31-5	Reserved	-	-	-
4 – 0	WaitWrite	R/W	0x00	In the case of SRAMs this register determines the time duration between Chip select assertion and the Write access.

Table 7.3-32 MpmcTrickMem Write access delay register**MPMCTrWaitPg: MPMC TrickMem Wait Page delay register**

Bit	Name	R/W	Reset value	Function
31-5	Reserved	-	-	-
4 – 0	WaitPage	R/W	0x00	Asynchronous page mode read delay after the first read in page-mode.

Table 7.3-33 MpmcTrickMem Wait Page delay register**MPMCTrCSPOL: MPMC TrickMem Chip Select Polarity Register**

Bit	Name	R/W	Reset value	Function
31-8	Reserved	-	-	-
3 – 0	CSPOL	R/W	0x00	This register contains the Chip Select polarity of all the other banks. In the MPMCTrMEMT register there is already a bit which determines the Chip Select Polarity of the memory connected to that particular bank. But for protocol checking, it is also required to know about the Chip Select Polarity of other banks. Hence this additional register.

Table 7.3-34 MpmcTrickMem Chip Select Polarity register

7.4 Testbench Description

For the functional verification of the MPMC, Denali memory models are used. These memory models are connected to the MPMC in various testbenches (tbench, tbench1 through tbench13). Each testbench has a different set of memory models connected to each memory chip select. Some of the testbenches use only Denali memory models while some use both Denali models as well as the trick memory model.

Tbench12 and tbench13 are tbenches for command delayed testing where memory is provided with undelayed clock. MPMCFBCLKs are adjusted according to the memory connected in the tbench.

Tbench, tbench1 to tbench11 are test benches for the clock delayed mode where clocks are delayed. MPMCFBCLKs are adjusted according to the memory connected in the tbench.

Static memory connections for the tbench12 has been done as per the MPMC-PL172Rel1 memory connections to test the dual type of memory connection facility.

7.4.1 Denali Memory Models parts used in test benches

A limited period license can be downloaded from the Denali web-site (<http://www.denalisoftware.com>). The required SOMA (Specification Of Memory Architecture) files can be downloaded from the following web-site: <http://www.ememory.com/>

Downloaded SOMA files are kept in the **verification/denali** directory. The .soma files need to be saved as .soma and the VHDL/Verilog source needs to be created using the Denali memmaker utility.

A List of part-names of all required models are given below:

Synchronous Memory

The Denali models used for synchronous memories in the MPMC (PL172) testbenches are shown in the **Tables 7.4-1, Table 7.4-2** and **Table 7.4-3** below.

Memory Size	Organisation	Vendor	Part Name
16Mb	1M X 16	Micron	48LC1M16A1S
	2M X 8	Samsung	K4S160822D-G/F7
64Mb	2M X 32	Micron	MT48LC2M32B2-6
	2M X 32	Samsung	K4S643232C-60

Memory Size	Organisation	Vendor	Part Name
128Mb	4M X16	Micron	MT28S4M16LC-10
	8M X 8	Hitachi	HM5264805F-75
	4M X 32	Micron	MT48LC4M32B2
	8M X 16	Micron	MT48LC8M16A2
	16M X 16	Samsung	K4S641632D-60
	16M X 8	Micron	MT48LC16M8A2
	8M X 32	Hitachi	HM5225325F-B60
	16M X 16	Micron	MT48LC16M16A2-8E
256Mb	32M X 8	Hitachi	HM5225805B-75
	64M X 8	Elpida	HM5257805B-A6

Table 7.4-1 Denali models for SDRAM devices

Memory Size	Organisation	Vendor	Part Name
64Mb	4M X 16	Micron	MT28S4M16LC-10
	4M X 16	Micron	MT28S4M16LC-12

Table 7.4-2 Denali memory models for SyncFlash devices

Memory Size	Organisation	Vendor	Part Name
64Mb	2M X 32	Micron	MT48LC2M32LFFC_8
128 Mb	16M X 8	Infineon	hyb25l128800ac-75

Table 7.4-3 Denali memory models for Low Power SDRAM devices**Static Memory**

The Denali memory models used for static memories in the MPMC (PL172) testbenches are shown in the **Tables 7.4-4, Table 7.4-5** and **Table 7.4-6** below.

Memory Size	Organisation	Vendor	Part Name	Page Mode support
16Mb	512K X 32	Samsung	K6T8016C3M	No
4Mb	256K X 16	Samsung	K6R4016C1C-12	No
4Mb	256K X 16	Samsung	K6R4016C1C-10	No
16Mb	512K X 32	Samsung	K6T8016C3M-70	No
1Mb	64K X 16	Samsung	K6R1016V1C-12	No

Memory Size	Organisation	Vendor	Part Name	Page Mode support
256Kb	32K X 8	Micron	MT5C2568-20	No
1Mb	32K X 32	Micron	MT5C2568-12	No
8Mb	512K X 16	Samsung	K6T8016C3M-55	No
256Kb	32K X 8	Micron	MT28F004B5-6T2	No
1Mb	32K X 32	Micron	MT5C2568-12	No

Table 7.4-4 Denali memory models for SRAM Devices

Memory Size	Organisation	Vendor	Part Name	Page Mode support
128Mb	8M X 16	Samsung	K3P9U100M	Yes
8Mb	256K X 32	SST	SST37VF020	No
2Mb	256K X 8	ST	M27W201-80	No
4Mb	512K X 8	Atmel	AT27LV040A-90	No
4Mb	512K X 8	Atmel	AT27LV040A-15	No
256Kb	32K X 8	Xicor	X28C256-15	Yes

Table 7.4-5 Denali memory models for Page Mode ROM devices

Memory Size	Organisation	Vendor	Part Name
32Mb	2M X 16	Intel	I28f320w18b_70
4Mb	256K X 16	Micron	MT28F400B5SG-8T
2Mb	256K X 8	Micron	MT28F002B3VG-9B
4Mb	512K X 8	Micron	MT28F004B5-8B-ET
32Mb	1M X 32	AMD	AM29PL320DT-60R
32Mb	1M X 32	AMD	AM29PL320DB-90
4Mb	512Kb X 8	Micron	MT28F004B5-6T1
2Mb	256K X 8	Micron	MT28F002B3VG-10TET

Table 7.4-6 Denali memory models for Flash devices

Memory Size	Organisation	Vendor	Part Name
4Mb	512K X 8	Atmel	at27lv040a-12
128Mb	8M X 16	Samsung	k3n9vu1000m

Table 7.4-7 Denali memory models for ROM devices

7.4.2 Testbenches

Details of the memory model connections in the testbenches - tbench, tbench[1..13] are given in the following tables:

Testbench: tbench

Chip select	Part name	Vendor	Size	SOMA
nDYCS[0]	MT48LC16M16A2_75	Micron	16Mx16	mt48lc16m16a2_75.soma
nDYCS[1]	MT48LC4M32B2	Micron	4Mx32	mt48lc4m32b2_6.soma
nDYCS[2]	MT48LC1M16A1_6S	Micron	1Mx16	mt48lc1m16a1_6s.soma
nDYCS[3]	HM5264165F_75	Elpida	4Mx16	hm5264165f_75.soma
nSTCS[0]	MpmcTrickMem			
nSTCS[1]	MpmcTrickMem			
nSTCS[2]	MpmcTrickMem			
nSTCS[3]	MpmcTrickMem			

Table 7.4-8 Memory connections done in tbench

Testbench: tbench1

Chip select	Part name	Vendor	Size	SOMA
nDYCS[0]	MT48LC2M32B2-6	Micron	2Mx32	mt48lc2m32b2_6.soma
nDYCS[1]	MT48LC8M16A2-7E	Micron	8Mx16	mt48lc8m16a2_7e.soma
nDYCS[2]	UPD4516821G5	Elpida	2M X 16	upd4516821g5.soma x 2
nDYCS[3]	MT48LC4M32B2_6	Micron	4Mx32	mt48lc4m32b2_6.soma
nSTCS[0]	MT5C2568-20	Micron	32Kx8	mt5c2568-20.soma
nSTCS[1]	MT28F004B5_6T1	Micron	512Kx8	mt28f004b5_6t1.soma
nSTCS[2]	K6R4016C1C-10	Samsung	64Kx16	k6r4016c1c_10.soma
nSTCS[3]	K6T8016C3M-55	Samsung	512Kx16	k6t8016c3m_55.soma

Table 7.4-9 Memory connections done in tbench1

Testbench: tbench2

Chip select	Part name	Vendor	Size	SOMA
nDYCS[0]	MT48LC16M16A2_75	Micron	16Mx32	(mt48lc16m16a2_75.soma) X 2
nDYCS[1]	MT48LC4M32B2	Micron	4Mx32	mt48lc4m32b2_6.soma
nDYCS[2]	MT48LC1M16A1_6S	Micron	1Mx16	mt48lc1m16a1_6s.soma
nDYCS[3]	HM5264165F_75	Elpida	4Mx16	hm5264165f_75.soma
nSTCS[0]	MpmcTrickMem			

nSTCS[1]	MpmcTrickMem
nSTCS[2]	MpmcTrickMem
nSTCS[3]	MpmcTrickMem

Table 7.4-10 Memory connections done in tbench2**Testbench: tbench3**

Chip select	Part name	Vendor	Size	SOMA
nDYCS[0]	HM5257805B-A6	ELPIDA	32Mx32	hm5257805b_a6.soma x 4
nDYCS[1]	HM5257805B-A6	ELPIDA	32Mx16	hm5257805b_a6.soma x 2
nDYCS[2]	HM5257805B-A6	ELPIDA	32Mx32	hm5257805b_a6.soma x 4
nDYCS[3]	MT48LC4M16A2_75	Micron	4Mx32	mt48lc4m16a2_75.soma x 2
nSTCS[0]	MT28F400B5SG-8T	Micron	256Kx16	mt28f400b5sg_8t.soma
nSTCS[1]	K3N9VU1000M	Samsung	8Mx16	k3n9vu1000m.soma
nSTCS[2]	MT28F002B3VG-9B	Micron	256Kx8	mt28f002b3vg_9b.soma
nSTCS[3]	AT27LV040A-12	Atmel	512Kx8	at27lv040a_12.soma

Table 7.4-11 Memory connections done in tbench3**Testbench: tbench4**

Chip select	Part name	Vendor	Size	SOMA
nDYCS[0]	HYB25L128800AC	Infineon	16Mx16	hyb25l128800ac.soma x 2
nDYCS[1]	MT48LC32M16A2-75	Micron	32Mx16	mt48lc32m16a2_75.soma
nDYCS[2]	MT48LC2M32LFFC_8	Micron	2Mx32	mt48lc2m32lffc_8.soma
nDYCS[3]	MT28S4M16LC-10	Micron	4Mx16	mt28s4m16lc_10.soma
nSTCS[0]	K3N9VU1000M	Samsung	8Mx16	k3n9vu1000m.soma
nSTCS[1]	SST37VF020-90	SST	256Kx32	sst37vf020_90.soma x 4
nSTCS[2]	MT5C2568-20	Micron	32Kx8	mt5c2568_20.soma
nSTCS[3]	K6R1016V1C-12	Samsung	64Kx16	k6r1016v1c_12.soma

Table 7.4-12 Memory connections done in tbench4**Testbench: tbench5**

Chip select	Part name	Vendor	Size	SOMA
nDYCS[0]	K4S561632A-75	Samsung	16Mx32	k4s561632a_75.soma x 2
nDYCS[1]	MT48LC32M16A2-75	Micron	32Mx32	mt48lc32m16a2_75.soma x 2

Chip select	Part name	Vendor	Size	SOMA
nDYCS[2]	MT48LC1M16A1_6s	Micron	1Mx16	mt48lc1m16a1_6s.soma
nDYCS[3]	MT28S4M16LC-12	Micron	4Mx16	mt28s4m16lc_12.soma
nSTCS[0]	K6R1016V1C-15	Samsung	64Kx16	k6r1016v1c_15.soma
nSTCS[1]	Int28f800f3b95	Intel	128Kx16	Int28f800f3b95.soma
nSTCS[2]	K3P9VU1000M	Samsung	8Mx16	k3n9vu1000m.soma
nSTCS[3]	Int28f800f3b115	Intel	128Mx16	Int28f800f3b115.soma

Table 7.4-13 Memory connections done in tbench5**Testbench: tbench6**

Chip select	Part name	Vendor	Size	SOMA
nDYCS[0]	MT28S4M16LC-10	Micron	4Mx16	mt28s4m16lc_10.soma
nDYCS[1]	MT28S4M16LC-10	Micron	4Mx32	mt28s4m16lc_10.soma x 2
nDYCS[2]	MT28S4M16LC-12	Micron	4Mx32	mt28s4m16lc_12.soma x 2
nDYCS[3]	MT28S4M16LC-12	Micron	4Mx16	mt28s4m16lc_12.soma
nSTCS[0]	K3P9VU1000M_3.0V	Samsung	8Mx16	k3p9vu1000m_3_0v.soma
nSTCS[1]	K3P9VU1000M_3.0V	Samsung	8MX32	k3p9vu1000m_3_0v.soma x 2
nSTCS[2]	X28C256-15	Xicor	32Kx8	x28c256_15.soma
nSTCS[3]	K3P9VU1000M_3.3V	Samsung	8Mx16	k3p9vu1000m_3_3v.soma

Table 7.4-14 Memory connections done in tbench6**Testbench: tbench7**

Chip select	Part name	Vendor	Size	SOMA
nDYCS[0]	MT48LC1M16A1_6S	Micron	1Mx32	mt48lc1m16a1_6s.soma x 2
nDYCS[1]	UPD4516821G5	Elpida	2Mx16	upd4516821g5.soma x 2
nDYCS[2]	HM5264805F-75	Hitachi	8Mx16	hm5264805f_75.soma x 2
nDYCS[3]	MT48LC16M8A2-75	Micron	16Mx32	mt48lc16m8a2_75.soma x 4
nSTCS[0]	K6T8016C3M-70	Samsung	512Kx32	k6t8016c3m_70.soma x 2
nSTCS[1]	K6R4016C1C-12	Samsung	64Kx16	k6r4016c1c_12.soma
nSTCS[2]	MT5C2568_20	Micron	32Kx8	mt5c2568_20.soma
nSTCS[3]	MT5C2568-12	Micron	32Kx32	mt5c2568_12.soma x 4

Table 7.4-15 Memory connections done in tbench7**Testbench: tbench8**

Chip select	Part name	Vendor	Size	SOMA
nDYCS[0]	HM5212165F-75a	Hitachi	8Mx32	hm5212165f_75a.soma x 2
nDYCS[1]	MT48LC1M16A1_6s	Micron	1Mx16	mt48lc1m16a1_6s.soma
nDYCS[2]	HM5225805B-75	Hitachi	32Mx16	hm5225805b_75.soma x 2
nDYCS[3]	MT48LC2M32B2-6	Micron	2Mx32	mt48lc2m32b2_6.soma
nSTCS[0]	28F320W18B_70	Intel	2Mx16	l28f320w18b_70.soma
nSTCS[1]	28F800F3B95-EXT	Intel	512Kx32	Int28f800f3b95.soma X 2
nSTCS[2]	28F800F3B115-AUTO	Intel	512Kx16	Int28f800f3b115.soma
nSTCS[3]	28F800F3B115-AUTO	Intel	512Kx32	Int28f800f3b115.soma X 2

Table 7.4-16 Memory connections done in tbench8

Testbench: tbench9

Chip select	Part name	Vendor	Size	SOMA
nDYCS[0]	UPD4516821G5	Elpida	2Mx32	upd4516821g5.soma x 4
nDYCS[1]	K4S641632D_60	Samsung	16Mx16	k4s641632d_60.soma
nDYCS[2]	UPD4516821G5	Elpida	2Mx8	upd4516821g5.soma x 2
nDYCS[3]	K4S643232C_60	Samsung	2Mx32	k4s643232c_60.soma
nSTCS[0]	MT28F004B5-6T1	Micron	512Kx8	mt28f004b5_6t1.soma
nSTCS[1]	MT28F004B5-6T1	Micron	512Kx16	mt28f004b5_6t1.soma x 2
nSTCS[2]	MT28F004B5-6T1	Micron	512Kx32	mt28f004b5_6t1.soma x 4
nSTCS[3]	MT28F004B5-8B-ET	Micron	512Kx32	mt28f004b5_8b_et.soma x 4

Table 7.4-17 Memory connections done in tbench9

Testbench: tbench10

Chip select	Part name	Vendor	Size	SOMA
nDYCS[0]	MT28S4M16LC-10	Micron	4Mx16	mt28s4m16lc_10.soma
nDYCS[1]	MT48LC2M32B2-6	Micron	2Mx32	mt48lc2m32b2_6.soma
nDYCS[2]	MT48LC16M8A2-75	Micron	16Mx16	mt48lc16m8a2_75.soma x 2
nDYCS[3]	MT48LC2M32LFFC_8	Micron	2Mx32	mt48lc2m32lffc_8.soma
nSTCS[0]	K6R1016V1C-15	Samsung	64Kx16	k6r1016v1c_15.soma
nSTCS[1]	K6R4016C1C-12	Samsung	64Kx16	k6r4016c1c_12.soma
nSTCS[2]	K3P9VU1000M	Samsung	8Mx16	k3n9vu1000m.soma

Chip select	Part name	Vendor	Size	SOMA
nSTCS[3]	MT28F002B3VG	Micron	256Kx8	mt28f002b3vg.soma

Table 7.4-18 Memory connections done in tbench10**Testbench: tbench11**

Chip select	Part name	Vendor	Size	SOMA
nDYCS[0]	MT48LC4M32B2_6	Micron	4Mx32	mt48lc4m32b2_6.soma
nDYCS[1]	MT48LC2M32LFFC_8	Micron	2Mx32	mt48lc2m32lffc_8.soma
nDYCS[2]	MT28S4M16LC-10	Micron	4Mx32	mt28s4m16lc_10.soma x 2
nDYCS[3]	MT48LC32M16A2-75	Micron	32Mx16	mt48lc32m16a2_75.soma
nSTCS[0]	K3N9VU1000M	Samsung	8Mx16	k3n9vu1000m.soma
nSTCS[1]	K3N9VU1000M	Samsung	8Mx32	k3n9vu1000m.soma x 2
nSTCS[2]	AT27LV040A-12	Atmel	512Kx32	at27lv040a-12.soma x 4
nSTCS[3]	AT27LV040A-15	Atmel	512Kx8	at27lv040a-15.soma

Table 7.4-19 Memory connections done in tbench11**Testbench: tbench12**

Chip select	Part name	Vendor	Size	SOMA
nDYCS[0]	MT28S4M16LC_10	Micron	4Mx16	mt28s4m16lc_10.soma
nDYCS[1]	MT28S4M16LC_10	Elpida	4MX32	mt28s4m16lc_10.soma x 2
nDYCS[2]	MT28S4M16LC_12	Hitachi	4Mx32	mt28s4m16lc_12.soma x 2
nDYCS[3]	MT28S4M16LC_12	Micron	4MX16	mt28s4m16lc_12.soma
nSTCS[0]	K6T8016C3M-70	Samsung	512Kx32	k6t8016c3m_70.soma x 2
nSTCS[1]	K6R4016C1C-12	Samsung	64Kx16	k6r4016c1c_12.soma
nSTCS[2]	MT5C2568_20	Micron	32Kx8	mt5c2568_20.soma
nSTCS[3]	MT5C2568-12	Micron	32Kx32	mt5c2568_12.soma x 4

Table 7.4-20 Memory connections done in tbench12**Testbench: tbench13**

Chip select	Part name	Vendor	Size	SOMA
nDYCS[0]	MT48LC16M16A2_75	Micron	16Mx16	mt48lc16m16a2_75.soma
nDYCS[1]	MT48LC4M32B2_6	Micron	4Mx32	mt48lc4m32b2_6.soma
nDYCS[2]	MT48LC1M16A1_6S	Micron	1Mx16	mt48lc1m16a1_6s.soma
nDYCS[3]	HM5264165F_75	Elpida	4Mx16	hm5264165f_75.soma

Chip select	Part name	Vendor	Size	SOMA
nSTCS[0]	MpmcTrickMem			
nSTCS[1]	MpmcTrickMem			
nSTCS[2]	MpmcTrickMem			
nSTCS[3]	MpmcTrickMem			

Table 7.4-21 Memory connections done in tbench13

8 VERIFICATION TESTS

The test vectors that are applied to the verification (BusTalk) testbenches in the MPMC (PL172) are in the **verification/bustest** directory.

The following table gives the mapping for the testbenches in which each of the tests is run:

Test Case	Testbench Name
AddrMirrTest	Tbench
AddrToggleTest	Tbench7
Arb2DyStWrRd	Tbench
Arb2DyWrRd	Tbench7
Arb2StWrRd	Tbench
Arb3DyStWrRd	Tbench
Arb3DyWrRd	Tbench7
Arb3StWrRd	Tbench
Arb4DyStWrRd	Tbench7
Arb4DyWrRdClkRat	Tbench7
Arb4DyWrRd	Tbench7
Arb4IdleBusy	Tbench
Arb4StWrRd	Tbench
BLSDelayValueTests	Tbench
BigEndHburstTest	Tbench
BigEndianPinTest	Tbench
BusDegrantTest	Tbench
BusyInsertionTest	Tbench
CSPolarityTest	Tbench
ClkCtrlTest	Tbench
ClkEnableTest	Tbench
ControllerBusyTest	Tbench
CornerCases1	Tbench
CornerCases2	Tbench3
CornerCases3	Tbench
Arb2CornerCase4	Tbench10
DelayValueTests	Tbench

Test Case	Testbench Name
MemModelHburstTest	Tbench7
DyHburstClkRatTB7Test	Tbench7
DyHburstClkRatTB8Test	Tbench8
DyHburstClkRatTB9Test	Tbench9
DyHburstTest1	Tbench
DyHburstTest2	Tbench
DyHburstTestTB10	Tbench10
DyHburstTestTB5	Tbench5
DyHburstTestTB7	Tbench7
DyHburstTestTB8	Tbench8
DyHburstTestTB9	Tbench9
EndiannessTests	Tbench
ExdWaitDelTest	Tbench
HResetTest	Tbench
HburstTest	Tbench
LowPwrModeTest	Tbench
LowPwrSdramFunTest	Tbench4
MPMCEnableTest	Tbench
MpmcRegisterTests	Tbench
PORResetTest	Tbench
ROMTest	Tbench
RandHburstTest	Tbench
RandHburstTest1	Tbench
RandHburstTest2	Tbench
RandomTest1	Tbench
RdRefAlnTest	Tbench
RefFreqChk	Tbench1
SdramInitRtnChk	Tbench
SelRefPriorityTest	Tbench
SelfRefChk	Tbench1
StWtPgBufEnTest	Tbench

Test Case	Testbench Name
StWtPgTest	Tbench
SyncFlashTest	Tbench6
TransferTest	Tbench
MemModelPgROMTest	Tbench6
MemModelROMTest	Tbench11
MemModelStPgModeTest	Tbench8
DyHburstCmdDelTest1	Tbench13
Rel1HburstTest	Tbench12
IntegrationTest	Tbench7
SyncFlashCmdDelTest	Tbench12
Arb2HMastLockTest	Tbench
WrProtTest	Tbench7
MultiPortPgAccessTest	Tbench2
MultiPortPgAccessTest1	Tbench2
MultiPortPgAccessTest2	Tbench2
MultiPortPgAccessTest3	Tbench2
MultiPortPgAccessTest4	Tbench2
MultiPortPgAccessTest5	Tbench2
MultiPortWrProtTest	Tbench
RefreshMissTest	Tbench
DirectDyDtFtchTest	Tbench
LatencyTest1	Tbench2
LatencyTest2	Tbench2
LatencyTest3_1	Tbench2
LatencyTest3_2	Tbench2
LatencyTest4_1	Tbench2
LatencyTest4_2	Tbench2
LatencyTest5_1	Tbench2
LatencyTest5_2	Tbench2
LatencyTest6_1	Tbench2
LatencyTest6_2	Tbench2

Test Case	Testbench Name
LatencyTest7	Tbench2
LatencyTest8	Tbench2
LatencyTest9	Tbench2
LatencyTest10	Tbench2
MemBGntTest	Tbench
MultiPortDiffComb1	Tbench2
MultiPortDiffComb2	Tbench2
MultiPortDiffComb3	Tbench2
MultiPortDiffComb4	Tbench2
MultiPortDiffComb5	Tbench2
MultiPortDiffComb6	Tbench2
MultiPortDiffComb7	Tbench2
MultiPortDiffComb8	Tbench2
AHBLockIdleInsertTest1	Tbench2
AHBLockIdleInsertTest2	Tbench2
CornerCases4	Tbench7
CornerCases5	Tbench10
CornerCases6	Tbench
CornerCases7	Tbench7
CornerCases8	Tbench7

Table 8-1 Test Case versus Testbench mapping

8.1 Functional Tests

8.1.1 HReset value Test

Objective of this test is to check the reset values of the registers when HRESETn is applied.

Test Identification No: MPMC_REGRES_1

Apply HRESETn.

Read the register value with HSIZE=010.

Verify whether response is OKAY.

8.1.2 POReset Test

Objective of the test is to verify the values of the registers when POReset is applied.

Apply nPOR

Read the register value with HSIZE=010.

Verify whether response is OKAY.

Register HRESETn and POReset values are listed below in **Table 8.1-1**.

Register	Address	RESET VALUE(nPOR)	Reset value (HRESETn)
MPMCControl	0x000	0x003	0x001
MPMCStatus	0x004	0x004	N/A
MPMCConfig	0x008	0x000	N/A
MPMCDyCntl	0x020	0x006	N/A
MPMCDyRef	0x024	0x000	N/A
MPMCDyRdCfg	0x028	0x000	N/A
MPMCDyRP	0x030	0x00F	N/A
MPMCDyRAS	0x034	0x00F	N/A
MPMCDySREX	0x038	0x00F	N/A
MPMCDyAPR	0x03C	0x00F	N/A
MPMCDyDAL	0x040	0x00F	N/A
MPMCDyWR	0x044	0x00F	N/A
MPMCDyRC	0x048	0x01F	N/A
MPMCDyRFC	0x04C	0x01F	N/A
MPMCDyXSR	0x050	0x01F	N/A
MPMCDyRRD	0x054	0x01F	N/A
MPMCDyMRD	0x058	0x01F	N/A
MPMCStExdWt	0x080	0x000	N/A
MPMCDyConfig0	0x100	0x000	N/A
MPMCDyRasCas0	0x104	0x303	N/A
MPMCDyConfig1	0x120	0x000	N/A
MPMCDyRasCas1	0x124	0x303	N/A
MPMCDyConfig2	0x140	0x000	N/A
MPMCDyRasCas2	0x144	0x303	N/A
MPMCDyConfig3	0x160	0x000	N/A
MPMCDyRasCas3	0x164	0x303	N/A
MPMCStConfig0	0x200	0x000	N/A
MPMCStWtWen0	0x204	0x000	N/A
MPMCStWtOen0	0x208	0x000	N/A
MPMCStWtRd0	0x20C	0x01F	N/A
MPMCStWtPg0	0x210	0x01F	N/A
MPMCStWtWr0	0x214	0x01F	N/A
MPMCStWtTurn0	0x218	0x00F	N/A
MPMCStConfig1	0x220	0x000	N/A

Register	Address	RESET VALUE(nPOR)	Reset value (HRESETn)
MPMCStWtWen1	0x224	0x000	N/A
MPMCStWtOen1	0x228	0x000	N/A
MPMCStWtRd1	0x22C	0x01F	N/A
MPMCStWtPg1	0x230	0x01F	N/A
MPMCStWtWr1	0x234	0x01F	N/A
MPMCStWtTurn1	0x238	0x00F	N/A
MPMCStConfig2	0x240	0x000	N/A
MPMCStWtWen2	0x244	0x000	N/A
MPMCStWtOen2	0x248	0x000	N/A
MPMCStWtRd2	0x24C	0x01F	N/A
MPMCStWtPg2	0x250	0x01F	N/A
MPMCStWtWr2	0x254	0x01F	N/A
MPMCStWtTurn2	0x258	0x00F	N/A
MPMCStConfig3	0x260	0x000	N/A
MPMCStWtWen3	0x264	0x000	N/A
MPMCStWtOen3	0x268	0x000	N/A
MPMCStWtRd3	0x26C	0x01F	N/A
MPMCStWtPg3	0x270	0x01F	N/A
MPMCStWtWr3	0x274	0x01F	N/A
MPMCStWtTurn3	0x278	0x00F	N/A
MPMCITCR	0xF00	0x000	0x000
MPMCITIP	0xF20	N/A	0x000
MPMCITOP	0xF40	N/A	0x001
MPMCPeriphId4	0xFD0	0x033	0x033
MPMCPeriphId5	0xFD4	0x000	0x000
MPMCPeriphId6	0xFD8	0x000	0x000
MPMCPeriphId7	0xFDC	0x000	0x000
MPMCPeriphId0	0xFE0	0x072	0x072
MPMCPeriphId1	0xFE4	0x011	0x011
MPMCPeriphId2	0xFE8	0x014	0x014
MPMCPeriphId3	0xFEC	0x007	0x007
MPMCPCellId0	0xFF0	0x00D	0x00D
MPMCPCellId1	0xFF4	0x0F0	0x0F0
MPMCPCellId2	0xFF8	0x005	0x005
MPMCPCellId3	0xFFC	0x0B1	0x0B1

Table 8.1-1 Register reset values description

8.1.3 MpmcRegisterTest

This test performs the write operation on the MPMC registers and reads back the register values and verifies that correct data has been written.

- Different data patterns are written to registers.
- Each read only bit should return Read Only values only.
- Other bits should return the values, which are written to the particular bits.

There are two types of registers in MPMC. Read only registers and read-write registers.

Read only registers

Write operation cannot be performed on read only register. These registers return only Read-Only values. Write data to read only registers and verify that register values are not affected.

Address and read only bits of Read-Only registers are listed below in **Table 8.1-2**.

Register	Bits	Address
MPMCStatus	2 – 0	0x004
MPMCPeriphId4	7 – 0	0xFD0
MPMCPeriphId5	7 – 0	0xFD4
MPMCPeriphId6	7 – 0	0xFD8
MPMCPeriphId7	7 – 0	0xFDC
MPMCPeriphId0	7 – 0	0xFE0
MPMCPeriphId1	7 – 0	0xFE4
MPMCPeriphId2	7 – 0	0xFE8
MPMCPeriphId3	7 – 0	0xFEC
MPMCPCellId0	7 – 0	0xFF0
MPMCPCellId1	7 – 0	0xFF4
MPMCPCellId2	7 – 0	0xFF8
MPMCPCellId3	7 – 0	0xFFC

Table 8.1-2 Address and read only bits of Read Only registers

Read/write registers

Writing known values to the write/read register bits and reading it back can test read-write registers. Bits that are not masked are reserved bits.

Address, read/write bits and mask values of Read/write registers are listed below in **Table 8.1-3**.

Register	Bit	Address Value	Mask Value
MPMCControl	2 – 0	0x000	0x00000007
MPMCConfig	8, 0	0x008	0x00000101
MPMCDyCntl	15 – 13, 8, 7, 2 – 0	0x020	0x0000E187
MPMCDyRef	10 – 0	0x024	0x000007FF
MPMCDyRdCfg	1 – 0	0x028	0x00000003
MPMCDytRP	3 – 0	0x030	0x0000000F

Register	Bit	Address Value	Mask Value
MPMCDytRAS	3 – 0	0x034	0x0000000F
MPMCDytSREX	3 – 0	0x038	0x0000000F
MPMCDytAPR	3 – 0	0x03C	0x0000000F
MPMCDytDAL	3 – 0	0x040	0x0000000F
MPMCDytWR	3 – 0	0x044	0x0000000F
MPMCDytRC	4 – 0	0x048	0x0000001F
MPMCDytRFC	4 – 0	0x04C	0x0000001F
MPMCDytXSR	4 – 0	0x050	0x0000001F
MPMCDytRRD	4 – 0	0x054	0x0000000F
MPMCDytMRD	4 – 0	0x058	0x0000000F
MPMCDyConfig0	20 – 19, 14, 12 – 7, 4 – 3	0x100	0x00185F98
MPMCDyRasCas0	9 – 8, 1 – 0	0x104	0x00000303
MPMCDyConfig1	20 – 19, 14, 12 – 7, 4 – 3	0x120	0x00185F98
MPMCDyRasCas1	9 – 8, 1 – 0	0x124	0x00000303
MPMCDyConfig2	20 – 19, 14, 12 – 7, 4 – 3	0x140	0x00185F98
MPMCDyRasCas2	9 – 8, 1 – 0	0x144	0x00000303
MPMCDyConfig3	20 – 19, 14, 12 – 7, 4 – 3	0x160	0x00185F98
MPMCDyRasCas3	9 – 8, 1 – 0	0x164	0x00000303
MPMCStConfig0	20 – 19, 8 – 6, 3, 1 – 0	0x200	0x001801CB
MPMCStWtWen0	3 – 0	0x204	0x0000000F
MPMCStWtOen0	3 – 0	0x208	0x0000000F
MPMCStWtRd0	4 – 0	0x20C	0x0000001F
MPMCStWtPg0	4 – 0	0x210	0x0000001F
MPMCStWtWr0	4 – 0	0x214	0x0000001F
MPMCStWtTurn0	3 – 0	0x218	0x0000000F
MPMCStConfig1	20 – 19, 8 – 6, 3, 1 – 0	0x220	0x001801CB
MPMCStWtWen1	3 – 0	0x224	0x0000000F
MPMCStWtOen1	3 – 0	0x228	0x0000000F
MPMCStWtRd1	4 – 0	0x22C	0x0000001F
MPMCStWtPg1	4 – 0	0x230	0x0000001F
MPMCStWtWr1	4 – 0	0x234	0x0000001F
MPMCStWtTurn1	3 – 0	0x238	0x0000000F
MPMCStConfig2	20 – 19, 8 – 6, 3, 1 – 0	0x240	0x001801CB
MPMCStWtWen2	3 – 0	0x244	0x0000000F
MPMCStWtOen2	3 – 0	0x248	0x0000000F

Register	Bit	Address Value	Mask Value
MPMCStWtRd2	4 – 0	0x24C	0x0000001F
MPMCStWtPg2	4 – 0	0x250	0x0000001F
MPMCStWtWr2	4 – 0	0x254	0x0000001F
MPMCStWtTurn2	3 – 0	0x258	0x0000000F
MPMCStConfig3	20 – 19, 8 – 6, 3, 1 – 0	0x260	0x001801CB
MPMCStWtWen3	3 – 0	0x264	0x0000000F
MPMCStWtOen3	3 – 0	0x268	0x0000000F
MPMCStWtRd3	4 – 0	0x26C	0x0000001F
MPMCStWtPg3	4 – 0	0x270	0x0000001F
MPMCStWtWr3	4 – 0	0x274	0x0000001F
MPMCStWtTurn3	3 – 0	0x278	0x0000000F
MPMCITCR	0	0xF00	0x00000001

Table 8.1-3 Address, read/write bits and mask values of Read/write registers**Test Identification No: MPMC_REG_1**

All register accesses should be a word access (HSIZE = 10).

Write 0x00000000 to all registers.

Read back the expected values with the insertion of one or zero WAIT State wherever required.

Check that there are only OKAY responses returned by the MPMC.

Repeat the above steps with the following write patterns: 0xFFFFFFFF, 0x55555555, 0xAAAAAAAA, 0x33333333, 0xCCCCCCCC, 0x0F0F0F0F, 0xF0F0F0F0, 0x00FF00FF, 0xFF00FF00, 0x0000FFFF, and 0xFFFF0000.

Test Identification No: MPMC_REG_2

The following test ensures that every register is independent of other registers in MPMC.

All the register accesses should be a word access (HSIZE = 10).

Write 0x00000000 to all even registers (Reg0, Reg2, and Reg4...) and 0xFFFFFFFF to all odd registers (Reg1, Reg3, and Reg5...)

Read them back with the expected value (Write pattern AND Mask pattern)

Check that there are only OKAY responses returned by the MPMC.

Invert every bit of the write pattern and repeat the above steps.

Group two successive registers (Reg0 and Reg1 forms the first group, Reg2 and Reg3 form the second group...)

Write 0x00000000 to all even groups of registers (Reg0, Reg1, Reg4, and Reg5...) and 0xFFFFFFFF to all odd group of registers (Reg2, Reg3, Reg6, and Reg7...)

Read them back with the expected value (Write pattern AND Mask pattern)

Check that there are only OKAY responses returned by the MPMC.

Invert every bit of the write pattern and repeat the above steps.

Repeat the above test with each group consisting of 4 / 8 / 16 / 32 registers.

Test Identification No: MPMC_REG_3

Write and read access to every register with a random data.

Write access to every register with HSIZE as word and HBURST as INCR4, INCR8, and INCR16.

Read them back with the expected value (Write pattern AND Mask pattern)

Check that there are only OKAY responses returned by the MPMC.

Byte / Half word access

Test Identification No: MPMC_REG_BHW_1

Write and read access to every register with a random data.

Write access to every register with HSIZE as Byte / Half word / Double Word and HTRANS as NSEQ / SEQ.

Check that there are only ERROR responses returned by the MPMC.

Read access to every register with HSIZE as Word and check that the data is not modified.

Read access to every register with HSIZE as Byte / Half word / Double Word and HTRANS as NSEQ / SEQ.

Check that there are only ERROR responses returned by the MPMC

Test Identification No: MPMC_REG_BHW_2

Read / Write access with HSIZE as Byte / Half word / Double Word and HTRANS as IDLE / BUSY to MPMC.

Check that there are only OKAY responses returned by the MPMC.

Test Identification No: MPMC_REG_nPOR

It checks whether the value of MPMCEndian, CSPOL, MPMCSTCS1PB pin values are reflected in the registers or not.

Drive 1 on the MPMCEndian pin from the trickbox. Apply nPOR.

Read the MPMCConfig register to check whether the pin value reflected in the register bit.

Write 0 to the MPMCConfig register and read back to check the data written properly or not.

Drive 0 on the MPMCEndian pin from trickbox. Apply nPOR.

Read the MPMCConfig register to check whether the pin value reflected in the register bit.

Write 1 to the MPMCConfig register and read back to check the data written properly or not.

Drive different values on MPMCSTCS0POL, MPMCSTCS1POL, MPMCSTCS2POL, MPMCSTCS3POL and MPMCSTCS1PB pins.

Check the value of MPMCSTConfig registers whether the pin values are reflected or not during power on reset.

8.1.4 Initialisation Routine Check Test

This test verifies whether the proper initialisation sequence for SDRAM is followed or not.

Test identification No: MPMC_INITCHK_1

The Trickbox is programmed for checking the initialisation sequence for SDRAM by setting the InitCheckEn bit of the MPMCTrSR register of trickbox. Disable address mirror bit.

Program MPMCDyConfig[0, 1, 2, 3] registers with buffers disabled.

Program MPMCDyRasCas[0, 1, 2, 3] registers.

Wait for 100us after the power is applied and clocks have stabilised.

Set the I field of MPMCDyCntl register to NOP.

Wait for 200us.

Perform static memory read operation, which confirms the bus degranting, is happening properly during initialisation of sdram.

Issue PALL by writing 10 to I field of MPMCDyCntl register.

A small value is programmed to REFRESH field of register MPMCDyRef.

Wait for a time period equivalent to 2 or 8 refresh-cycles depending on the memory connected.

Program the operational value into the refresh timer.

Set the SDRAM initialisation value I field to MODE in the MPMCDyCntl register.

Perform a read operation to the device. The address of the read is used to configure the mode register for its burst length, burst type, CAS latency, operating mode and Write burst mode. And mode data has to be given over Row address lines.

Program I of MPMCDyCntl to 00 so that SDRAM is in NORMAL mode.

Program the MPMCDyConfig [0,1,2,3] and the MPMCDyRasCas [0,1,2,3] registers with the buffers enabled.

Check InitSeqSt bit of MPMCTrSR of trickbox is verified to check whether the initialisation sequence is properly followed or not.

8.1.5 Address Toggle Test

This test toggles all memory-related address bits.

Test Identification No: MPMC_AddTog_1

Disable address mirror bit.

Initialise static memory controller registers, dynamic memory controller registers and Sdram as well.

Write 0x00000000 on address bus and write valid data to memory.

Perform memory write operation with different combinations of address bits.

Read data from the locations where different data have been written.

Above steps are repeated for different addresses like 0xFFFFFFFF, 0x55555555, 0xAAAAAAAA, 0x55AA55AA, 0xAA55AA55, 0x0000FFFF, 0xFFFF0000, 0xAAAA5555, 0x5555AAAA.

8.1.6 MPMC enable Test

Objective of the test is to check the functionality of E bit of MPMCControl register.

Test Identification No: MPMC_En_1

Disable address mirror bit.

Initialise static memory controller registers, dynamic memory controller registers and Sdrams as well.

Poll for busy bit. When the controller is in idle State, reset E bit of MPMCControl register and perform a memory access and verify that response is ERROR.

Set E bit of MPMCControl register, again perform a memory access and verify that response is OKAY.

8.1.7 Clock control Test

This test tests the Clock Stop(CS) and DMC bits of MPMCDyCntl register.

Test Identification No: MPMC_ClkCtl_1

Disable address mirror bit. Initialise Sdram and dynamic memory controller registers.

Write 0 to Clock stop(CS), 1 to DMC and 0 to Clock enable (CE) of MPMCDyCntl register. Trickbox checks the MPMCCLKOUT according to the value programmed in the mirror register.

A single write is performed from port 3. Flush write buffer. Check the status of Write buffer status (S) bit of MPMCStatus register. It must be 0 indicating that write buffer is empty.

Read the data back. Compare the value with written one.

Above steps are repeated with different types of bursts.

Write 1 to CS bit and 0 to DMC bit. Disabling the MPMCCLKOUT stops the clock to be applied to the memory. Trickbox checks whether MPMCCLKOUT stopped or not.

Write 1 to CE bit, 1 to CS and 0 to DMC. And repeat the above step.

Set Clock control (CS) and DMC to 1.

8.1.8 Clock Enable Test

This test, tests Clock Enable(CE) bit of MPMCDyCntl register.

Test Identification No: MPMC_ClkEn_1

Initialise Sdram and registers related to dynamic memory controller.

Write CS = 0 and DMC = 1 of MPMCDyCntl register.

Write CE = 0 and DMC = 1 of MPMCDyCntl register. Write valid data. Flush data to memory. The Trickbox checks MPMCCKEOUT according to the value programmed to its register.

Test Identification No: MPMC_ClkEn_2

Write CS = 1 and DMC = 1 of MPMCDyCntl register.

Write CE = 0 of MPMCDyCntl register. Write valid data to buffer. Flush data to memory.

Check whether S bit of MPMCStatus register is 0.

Read data back and compare it with the written one.

Write CE = 1. Write data to memory. Flush the data

Read data back and compare it with written one.

The Trickbox checks MPMCCKEOUT according to the value programmed to its register.

Programming the synchronous memory clock enable high (CE = 1) and synchronous memory clock control low (CS = 0) will result in UNPREDICTABLE behaviour. This condition must be avoided.

8.1.9 Read and refresh alignment Test

This test verifies that refresh for sync memory has highest priority over memory access when they are applied simultaneously.

Test Identification No: MPMC_RdRefAln_1

Initialise Sdram and registers of dynamic memory controller.

Program REFRESH field of MPMCDyRef register with 30.

Write a data to buffer and insert Wait State so that data is flushed to memory.

When REFRESH counter becomes zero, issue single read from port 3 to the memory so that read is aligned with refresh.

Verify that refresh gets higher priority than read operation.

Repeat the above test with REFRESH counter values 10, 70, 300, 700.

8.1.10 Refresh Frequency Check Test

It verifies that the refreshes for sync memory are separated by approximately the same number of clocks as the value that is written into the Refresh register of the controller.

Test Identification No: MPMC_RefFreq_1

Initialise Sdram and registers related to dynamic memory access.

Write 40 into the REFRESH field of MPMCDyRef register.

Set RefCheckEn bit of MPMCTrSR register.

The RefErrSt bit of the MPMCTrSR register is read after enough number of idle cycles to verify that it is cleared.

The RefErrSt bit being low indicates that the expected number of clocks separates the refreshes.

Reset RefCheckEn bit, indicating refresh frequency check is over.

Test is repeated for 10, 60, 250, 500, 700 values of refresh cycles.

8.1.11 SelfRefreshChk

This test verifies SelfRefreshReq(SR) bit of MPMCDyCntl register and SelfRefAck(SA) bit of MPMCStatus register.

Test Identification No: MPMC_SelfRefAck_1

Initialise Sdram and registers related to dynamic memory access.

Self refresh for sync memory is requested by writing 1 to SREFREQ (SR) of MPMCDyCntl register.

Poll SREFAK (SA) bit of MPMCStatus register. It acknowledges the self refresh.

Perform memory access when self-refresh is still asserted and verify that response is ERROR.

After getting ACK for the self-refresh, de-assert SR. Read SA bit. It will be 0 indicating controller has entered normal mode.

Perform dummy read operation to keep the controller in IDLE condition after the read operation.

Assert SR bit to indicate self refresh and poll SA bit.

De-assert SR and poll SA bit.

Write 1 to SREFREQ bit of MPMCTrCR register of the Trickbox that drives MPMCSREFREQ line continuously. Trickbox samples MPMCSREFAK line and verifies the MPMCSREFREQ line and MPMCSREFAK lines are behaving properly.

When self-refresh is requested externally, perform memory access and find that response is ERROR.

Post a request for self refresh by asserting SR bit of MPMCDyCntl register. Verify whether memory has entered the self-refresh by observing the SA bit of MPMCStatus register.

When memory has entered self-refresh mode, assert MPMCSREFREQ line by writing 1 to SREFREQ bit of MPMCTrCR register. Perform memory access to memory and verify that ERROR response has returned.

8.1.12 SelfRefPriority Test

This test verifies the functionality of REFRESH and SREFREQ when both appear simultaneously.

Test Identification No : MPMC_SelfRef_1

Disable address mirror bit. Initialise Sdram and dynamic memory controller registers.

Keep MPMC in IDLE State.

Program REFRESH field of MPMCDyRef register with a value.

As soon as REFRESH becomes zero, write 1 to SREFREQ bit of MPMCDyCntl register so that both self refresh and normal refreshes are applied at a time.

Verify that auto refresh is served first and then self-refresh.

Apply self refresh little earlier (few clocks) before normal refresh.

Verify that self refresh finishes without getting delayed. But refresh gets delayed by a few clocks.

8.1.13 Chip select polarity Test

Objective of this test is to check whether polarity of chip selects is behaving properly or not for SRAM.

Test Identification No: MPMC_CSPol_1

Disable address mirror bit.

Initialise TrickMem and static memory controller registers.

Disable buffers.

Write 0 to Chip Select Polarity (PC) bit of MPMCStConfig[0,1,2,3] register which issues active low chip select. Trickbox verifies the polarity of chip select based on the value of mirror register.

Write specific words to different memory locations varying HADDR[30:28] and keeping HADDR[27:0] constant so that data has been written to a particular memory location of all chips.

Read data back and verify with the written data. If it matches then unique CS assertion for each address is proved.

Repeat the test by writing 0 to PC bit of MPMCStConfig[0,1] and 1 to PC bit of MPMCStConfig[2,3] and verify through trickbox.

Try different combinations of chip select polarity for different chips.

Repeat the test with PC = 1 for all chip-selects.

During Power On Reset, chip select polarity follows the MPMCSTCS0POL,

MPMCSTCS1POL, MPMCSTCS2POL and MPMCSTCS3POL pin values till MPMCStConfig[0,1,2,3] registers are written with actual values. Set different values to these pins and suitable value to MPMCSTCS1PB pin and apply power on reset. Verify that chip selects follow the pin values.

Program MPMCStCS1MW field of MPMCTrStCS register of the Trickbox with value 01. Apply nPOR. Verify that memory width of CS1 follows the external pins before MW field of MPMCStConfig[1] is programmed.

8.1.14 Busy Insertion Test

This function tests the functionality of the controller when BUSY states are inserted randomly during burst transfer.

Test Identification No: MPMC_BusyInsert_1

Disable address mirror bit and initialise Sdram, controller's registers.

Perform burst write operation with different size and to different memory.

Insert BUSY transfers randomly.

After the BUSY cycles, burst is rebuilt with same burst type.

Read the data back and check the data integrity using the same or different burst type with the insertion of BUSY-State randomly.

8.1.15 Bus Degrant Test

This function tests the functionality of the controller when IDLE states are inserted randomly during burst transfer.

Test Identification No: MPMC_BusDegrant_1

Disable address mirror bit and initialise Sdram, controller's registers.

Perform burst word write operation with different size to different memory.

In between the burst operation insert BUSY states randomly.

In between the burst operation insert Idle cycles randomly which is nothing but master gets degranted.

After the IDLE cycles, the burst is rebuilt with the same type of burst.

Read the data back using the same burst type or of different burst type and checked.

Perform write operation and insert IDLE cycle to degrant the bus.

Read data back with the insertion of IDLE-State to degrant the bus and check for data integrity.

8.1.16 Endianness Test

This test verifies the controller for different types of endian modes.

Disable address mirror bit, initialise Sdram, TrickMem and controller's registers.

Set the endian mode to little endian.

Perform write operation to chip select 0 with HSIZE as BYTE and HBURST as SINGLE.

Set the endian mode to big endian.

Read same data and verify that data read back is in big endian.

Repeat the same procedure for different HSIZE, HBURST and for different chip selects i.e. for different memory width.

Set the endian mode to big endian.

Perform write operation to chip select 0 with HSIZE as BYTE and HBURST as SINGLE.

Set the endian mode to little endian.

Read same data and verify that data read back is in little endian.

Repeat the same procedure for different HSIZE, HBURST and for different chip selects i.e. for different memory width.

8.1.17 BigEndian Pin Test

When power on reset is applied, endianness mode has to follow the status of the BigEndian pin till the endian register bit is written. This test verifies this functionality.

Disable address-mirror bit and initialise Sdram, TrickMem and controller's registers.

Write 0 to BIGENDIAN bit of MPMCTrCR register of the Trickbox so that MPMCBIGENDIANIN pin programs the controller to little endian mode.

Apply power on reset by asserting nPOR bit of MPMCTrCR register of the Trickbox.

Initialise static memory controller registers and the TrickMem registers with relevant values.

Perform memory write and read operations and verify that operation is being performed in little endian mode.

Write 1 to BIGENDIAN bit of MPMCTrCR register of the Trickbox so that MPMCBIGENDIANIN pin programs the controller to big endian mode.

Apply power on reset by asserting nPOR bit of MPMCTrCR register of the Trickbox.

Initialise static memory controller registers and the TrickMem registers with relevant values.

Perform memory write and read operations and verify that operation is being performed in big endian mode.

8.1.18 Write protect Test

This test verifies the write protection of memory by setting the write protect(WP) bit of MPMCStConfig[0,1,2,3] and MPMCDyConfig[0,1,2,3] registers.

Test Identification No: MPMC_WrProt_1

Disable address mirror bit. Initialise Sdram, SRAM memory models and controller registers.

Set write protect bit of MPMCDyConfig[0,1,2,3] and MPMCStConfig[0,1,2,3] registers.

Perform write operation and verify that response is ERROR.

Set the write protect bit of configuration registers selectively and verify that there is an ERROR response for write protected chip and for other chip selects there is OK response.

8.1.19 Address Mirror Test

This test verifies the Address Mirror functionality of the controller. When Address mirror(M) bit of MPMCControl register is set (reset value), chip select 1 is mirrored on to chip select 0 and chip select 4.

Test Identification No: MPMC_AddrMirr_1

Initialise Sdram, TrickMem and controller's registers.

Write 0 to M bit of MPMCControl register.

Write to a location with HADDR[30:28] = 000 and by reading the data back verify that data has been written to chip select 0.

Write to a memory location with HADDR[30:28]= 001 and verify that data has been written to chip1 by reading the data back.

Write R = 1. Now CS1 data is mirrored onto CS0 and CS4.

Write to a memory location with HADDR[30:28]= 000. Read the same data back from same address but from CS1.

Write a word to memory location with HADDR[30:28] = 100. Read the same location from CS1 and verify the data integrity.

8.1.20 Controller Busy Test

This test verifies the busy bit of MPMCStatus register.

Test Identification No: MPMC_ContrBusy_1

Disable address mirror bit, initialise Sdram, TrickMem and controller's registers.

Write valid data to static memory and read data back.

Check the busy bit B of MPMCStatus register. It will be set indicating that controller is in BUSY State.

Apply refresh to memory and check whether B bit is set or not.

Perform write operation to dynamic memory. Check the busy bit. It will be set indicating that controller is in busy State.

After this operation insert sufficient idle cycles. Check the busy bit. It will be 0 indicating that controller is in Idle State.

8.1.21 Memory Test

Objective of these set of tests is to perform different types of memory transfers over different benches.

It performs all types of memory access to different types of memory that have different timing parameters. Starting from a single data transfer of various bus sizes, it initiates various combinations of read and write transactions to test the functionality of the controller. The memory is first filled with valid data before initiating the combination of read and write accesses.

The tests include

- Single data transfers to test the generation of data mask for every byte of a word individually.

- Multiple burst at various times with respect to the data transfer from the memory.

- Single and multiple burst transfers targeting the specific write buffer.

- Single and multiple burst transfers targeting different write buffer.

- Tests are repeated with different values for HADDR[30 : 28] which selects different chip select.

- Tests are repeated for different types of HBURST like INCR4, INCR8, INCR16, WRAP4, WRAP8, WRAP16

SRAM memory transactions test

HburstTest

This test targets SRAM memory with different HBURST, HSIZE and memory width.

Test Identification No: MPMC_HBURST_1

- Disable address mirror bit. Initialise the TrickMem and static memory controller registers.

- Perform write to all chip selects with insertion of BUSY and IDLE transfers randomly. Read the data back with Different HBURST and check for the data integrity.

- Perform write to the particular chip select with HSIZE as BYTE and HBURST as SINGLE. Read the data back and check the data integrity.

- Repeat the above step to different chip select, HSIZE and HBURST.

MemoryModelHburstTest

This test uses SRAM Denali memory models for memory transactions.

Test Identification No: MPMC_MemModelHburst_1

- Disable address mirror bit. Initialise Denali memory model with data.

- Delay values are programmed according to the characteristics of memory connected.

- The test performs different types of transactions as described in the Test Identification No : MPMC_HBURST_1.

RandomHburstTest

This test uses TrickMem as the memory model.

Test Identification No: MPMC_RandHburst_1

- Disable address mirror bit. Initialise the TrickMem as well as static memory controller registers.

- This test selects the chip select, HSIZE and HBURST randomly and does the memory accesses to the randomly selected address.

- Reads data back and verify for the data integrity.

BigEndHburstTest

It performs the memory access to static memory with endian set to BIG endian.

Test Identification No: MPMC_BigEndHburst_1

Disable address mirror bit. Initialise TrickMem as well as static memory controller registers.

Set the endian as BIG endian.

Perform different types of memory transfers as described in the Test Identification No.: MPMC_HBURST_1.

Perform memory access with different types of burst with the insertion of BUSY and continue the burst. Read data back and check for the data integrity.

Perform memory access with different types of burst and degrant the bus by inserting the IDLE. Restart either the same type of burst or start with different type of burst. Read data back and check for the data integrity.

MemModelROMTest

It performs the read operation to ROM memory models connected to the Chip select0 to Chip select3.

Test Identification No: MPMC_MemModelROMTest_1

Initialise the memory with predefined data.

Initialise the static memory controller registers according to the memory connected.

Perform read operation to the chip select 0 with different combinations of HSIZE, HBURST.

Insert BUSY trans randomly during the read operation.

Repeat the above read process to the different chip select.

MemModelStPgModeTest

It performs the write operation to the Intel page mode flash and performs the page mode read operation.

Test Identification No.: MPMC_MemModelStPgModeTest_1

Initialise the static memory controller registers according to the memory connected.

Disable the write buffers.

Unprotect the memory block by writing 0x60 in the first cycle and then 0xD0 in the second cycle of write.

In the first cycle of write operation, write 0x40 to the memory to put the memory in write mode with address as the memory address where the actual data is supposed to be written.

In the second cycle write the actual data to be written with NSEQ as the HTRANS and SINGLE as the HBURST.

Read the status register bit (SR7) to know whether the memory is in BUSY state or it is ready to take next command. After reading the status register clear the status register by writing 0x50 to the same address.

Write 100 data to the memory by repeating the above steps.

Write 0xFF to the same memory block to put the memory in read mode.

Read the data back from the memory and check for the data integrity.

Enable the buffers. And read the data back once again and check the data integrity.

Disable the buffers and unlock and erase the memory.

Perform the write operation as described earlier.

Read the data back and check the data integrity as described earlier.

Enable the buffers and read the data back and check for the data integrity.

MemModelPgROMTest

It performs the read operation to the page mode ROM memory.

Test Identification No.: MPMC_MemModelStPgModeTest_1

Initialise the memory models with predefined data.

Initialise the static memory controller registers according to the memory connected.

Enable the page mode bit.

Perform read operation to the chip select 0 with different combinations of HSIZE, HBURST.

Insert BUSY trans randomly during the read operation.

Repeat the above read process to the different chip select.

Dynamic memory transaction tests

These test cases run on different test benches. These test benches are connected with different types of memory models. Controller can be tested effectively with different timing values, different latency values, clock ratio and different burst length.

Test case	Bank	Memory size	Memory width	RAS latency	CAS latency	ClkRatio	Test Identification No
DyHburstTest1	Bank4	16MX16	16	2	3	1:1	MPMC_DyHburstTest1
	Bank5	4MX32	32	2	2		
	Bank6	1MX16	16	3	2		
	Bank7	4MX16	16	2	3		
DyHburstTest2	Bank4	16MX16	16	1	1	1:1	MPMC_DyHburstTest2
	Bank5	4MX32	32	>=2	>=2		
	Bank6	1MX16	16	>=2	>=2		
	Bank7	4MX16	16	>=2	>=2		
DyHburstTestTB10	Bank4	4MX16 (sflash)	16	>=2	>=2	1:1	MPMC_DyHburstTestTB10_1
	Bank5	2M X 32	32	>=2	>=2		
	Bank6	16MX16	16	>=2	>=2		
	Bank7	2MX32	32	3	3		
DyHburstTestTB5	Bank4	16X32	32	3	3	1:1	MPMC_DyHburstTestTB5_1
	Bank5	32MX32	32	3	3		
	Bank6	1MX16	16	3	3		
	Bank7	4MX16	16	3	3		
DyHburstTestTB7	Bank4	1MX32	32	>=2	>=2	1:1	MPMC_DyHburstTestTB7_1
	Bank5	2MX16	16	>=2	>=2		
	Bank6	8MX16	16	>=2	>=2		

Test case	Bank	Memory size	Memory width	RAS latency	CAS latency	ClkRatio	Test Identification No
	Bank7	16X32	32	3	3		
DyHburstTestTB8	Bank4	8MX32	32	>=2	>=2	1:1	MPMC_DyHburstTestTB8_1
	Bank5	1MX16	16	1	2		
	Bank6	32MX16	16	>=2	>=2		
	Bank7	2MX32	32	3	3		
DyHburstTestTB9	Bank4	2MX32	32	1	1	1:1	MPMC_DyHburstTestTB9_1
	Bank5	8MX16	16	3	3		
	Bank6	2MX16	16	>=2	>=2		
	Bank7	4MX32	32	1	1		
DyHburstClkRatTB7Test	Bank4	1MX32	32	>=2	>=2	1:2	MPMC_DyHburstClkRatTB7_1
	Bank5	2MX16	16	>=2	>=2		
	Bank6	8MX16	16	>=2	>=2		
	Bank7	16X32	32	3	3		
DyHburstClkRatTB8Test	Bank4	8MX32	32	>=2	>=2	1:2	MPMC_DyHburstClkRatTB8_1
	Bank5	1MX16	16	1	2		
	Bank6	32MX16	16	>=2	>=2		
	Bank7	2MX32	32	3	3		
DyHburstClkRatTB9Test	Bank4	2MX32	32	1	1	1:2	MPMC_DyHburstClkRatTB9_1
	Bank5	8MX16	16	3	3		
	Bank6	2MX16	16	>=2	>=2		
	Bank7	4MX32	32	1	1		

Table 8.1-4 Different test cases with different combinations of latency values

DyHburstTest1, DyHburstTest2 and DyHburstTest10 tests are signed off at the end of test by reading all data from memory and data integrity is checked.

RandomHburstTest1

Test Identification No: MPMC_DyRandHburst1

This test performs the memory transactions for Sdram.

Disable address mirror bit. Initialise Sdram and controller registers as well.

Keep the HCLK : MPMCCLK clock ratio is 1 : 1.

Perform different types of memory transfers in random fashion.

RandomHburstTest2**Test Identification No: MPMC_DyRandHburst2**

This test performs memory transactions for Sdram.

Disable address mirror bit. Initialise Sdram and controller registers as well.

Keep HCLK : MPMCCLK clock ratio as 1 : 2.

Perform different types of memory transfers in random fashion.

Multi port memory access

These types of tests perform memory transfers from different ports simultaneously. Initialisation of SDRAM/SRAM models and controller registers initialisations are performed from port3. Tests are performed either by two, three or four ports. Memory targeted may be static memory alone, dynamic alone or it may be the combinations of static memory and dynamic memory.

Multi port tests do the following operations,

Initialise SDRAM/SRAM memory models as well as controller registers from Port3. Till then other ports are kept in IDLE State.

When the initialisation of memory is completed, start memory transfers from different ports.

Check the priority of ports when there are simultaneous accesses from different ports.

Check the data integrity by reading the data back.

The following table gives the test name, memory used and number of ports used.

Test case	Number of ports used	Memory types used	Clock Ratio	Test Identification No.
Arb2StWrRd	2	Static memory	N/A	MPMC_Arb2St_1
Arb3StWrRd	3	Static memory	N/A	MPMC_Arb3St_1
Arb4StWrRd	4	Static memory	N/A	MPMC_Arb4St_1
Arb2DyWrRd	2	Dynamic memory	1:1	MPMC_Arb2DyWrRd_1
Arb3DyWrRd	3	Dynamic memory	1:1	MPMC_Arb3DyWrRd_1
Arb4DyWrRd	4	Dynamic memory	1:1	MPMC_Arb4DyWrRd_1
Arb2DyStWrRd	2	Dynamic as well as static memory	1:1	MPMC_Arb2DySt_1
Arb3DyStWrRd	3	Dynamic as well as static memory	1:1	MPMC_Arb3DySt_1
Arb4DyStWrRd	4	Dynamic as well as static memory	1:1	MPMC_Arb4DySt_1

Test case	Number of ports used	Memory types used	Clock Ratio	Test Identification No.
Arb4DyWrRdClkRat	4	Dynamic memory	1:2	MPMC_Arb4DyClkRatio_1

Table 8.1-5 Different test cases for multiple port testing with Clock ratio used.

Arb4IdleBusy

It is a multi port test. Objective of the test is to perform the memory access from different ports with insertion of BUSY and IDLE states.

Test Identification No: MPMC_Arb4IdleBusy_1

Initialise SDRAM/SRAM memory models from Port3. Till that time keep other ports in IDLE State.

As soon as the initialisation is completed, start memory access from different ports.

In between the transfers, insert BUSY State and continue the burst once the sufficient number of BUSY states is inserted.

In between the burst, de-grant the burst by inserting the IDLE State. This breaks the burst. After the insertion of IDLE State, restart the burst by different type or of the same type of HBURST.

8.1.22 Transfer Test

It is the memory transfer test for static memory where BUSY and IDLE states are inserted at certain points.

Test Identification No: MPMC_Trans_1

Initialise SRAM memory models.

Perform memory access from Port3.

Insert BUSY states and restart memory access by same burst type.

Insert IDLE states and de-grant the burst. Restart the burst either by different burst or of the same burst.

Different combinations of NSEQ, SEQ, BUSY and IDLE are shown below,

HBURST	HTRANS	HSIZE	Starting Address
INCR	NS – B – S	32	0x1E1
INCR	NS – I – NS	32	0x12C
INCR4	NS – S – S – I – NS	32	0x004
INCR4	NS – S – S – B – NS	32	0x034
INCR	NS – B – I – NS	32	0x038
WRAP4	NS – B – I – NS	32	0x7D4
INCR	NS – S – B – S	32	0x7DC
INCR	NS – B – S – B – S	32	0x1C4
WRAP16	NS – S – S ... B – S – ... B – I – NS – I	8	0x029
WRAP16	NS – S – S ... B – S – ... B – I – NS – I	16	0x060
INCR16	NS – S – B – S – S ...	32	0x7A0

Table 8.1-6 Different combinations of HTRANS

8.1.23 Low power mode Test

Objective of the test is to verify the functionality of L bit of MPMCControl register.

Test Identification No: MPMC_LowPwrMode_1

Disable address mirror bit, initialise Sdram, TrickMem and controller registers.

Set Low Power mode bit when controller is in idle State and buffer is empty which enables the low power mode.

Write different data patterns from different AHB interfaces and observe that response is ERROR.

Observe refresh activity. Refresh operation should remain unaffected.

Disable the low power mode.

8.1.24 Delay Value Test

Objective of this test is to verify the functionality of the MPMCStWtWen, MPMCStWtOen and MPMCStWtRd, MPMCStWtWr and MPMCStWtTurn registers.

Test Identification No: MPMC_DeIVal_1

Disable address mirror bit. Initialise TrickMem, configure different chip selects with different memory width.

Configure MPMCStWtWen, MPMCStWtOen and MPMCStWtRd, MPMCStWtWr, MPMCStWtTurn registers with different values. Value of MPMCStWtRd must be greater than MPMCStWtOen value. Similarly value of MPMCStWtWr must be greater than MPMCStWtWen value.

Configure MPMCTrMEMT, MPMCTrCS2OEN, MPMCTrCS2WEN, MPMCTrCS2Rd, MPMCTrWtPg, MPMCTrCS2Wr, MPMCTrIDCY, MPMCTrCSPOL and MPMCTrMEMB as per the requirement.

Perform different memory access operation.

All combinations mentioned below of read, write and burst read are tried.

Read followed by burst reads to same and different banks.

Read followed by writes to same and different banks.

Burst read followed by reads to same and different banks.

Burst read followed by writes to same and different banks.

Write followed by reads to same and different banks.

Write followed by burst reads to same and different banks.

Repeat the test for all four memory-banks.

Repeat the test with different values to different chip select.

8.1.25 StWtPg Test

This test verifies WAITPAGE field of MPMCStWtPg[0,1,2,3] register and PageMode(Pm) bit of MPMCStConfig[0,1,2,3] register.

Test Identification No: MPMC_StWtPg_1

Disable address mirror bit. Initialise the TrickMem and controller's registers.

Disable the buffers.

Write Memory device (MD) field of MPMCStConfig[0,1,2,3] register with "00" and MEMTYPE field of MPMCTrMEMT register with "01" indicating that memory is ROM.

Write 1 to Page mode Read (Pm) of MPMCStConfig[0,1,2,3] so that page mode is enabled.

Write 0 to EW of MPMCStConfig[0,1,2,3] register.

Program different values like 00000,00001...11111 to WAITPAGE field of MPMCStWtPg register, WaitPage field of MPMCTrWaitPage[0,1,2,3] register of TrickMem and WAITRD of MPMCStWtRd[0,1,2,3] register, CS2Rd of MPMCTrCS2Rd[0,1,2,3] register of the TrickMem.

Number of wait-states added after the first read are verified by the TrickMem.

8.1.26 StWtPgBufEn Test

This test verifies WAITPAGE field of MPMCStWtPg [0,1,2,3] register and PageMode(Pm) bit of MPMCStConfig[0,1,2,3] register with buffers enabled.

Test Identification No: MPMC_StWtBufEn_1

Disable address mirror bit. Initialise the TrickMem and controller's registers.

Enable buffers.

Write Memory device (MD) field of MPMCStConfig[0,1,2,3] register with "00" and MEMTYPE field of MPMCTrMEMT register with "01" indicating that memory is ROM.

Write 1 to Page mode Read (Pm) of MPMCStConfig[0,1,2,3] so that page mode is enabled.

Write 0 to EW of MPMCStConfig[0,1,2,3] register.

Program different values to WAITPAGE field of MPMCStWtPg register, WaitPage field of MPMCTrWaitPage[0,1,2,3] register of TrickMem and WAITRD of MPMCStWtRd[0,1,2,3] register, CS2Rd of MPMCTrCS2Rd[0,1,2,3] register of the TrickMem.

Initialise the trickmemory with valid data. Perform memory read operations from the memory with different HBURST, HTRANS and HSIZE and check for the data integrity.

8.1.27 ExdWaitDelTest

This test verifies the functionality of Extended Wait(EW) bit of MPMCStConfig[0,1,2,3] and MPMCStExdWt register.

Test Identification No: MPMC_WAIT_1

Initialise the memory banks and TrickMem with required memory width.

Set EW field of MPMCStConfig[0,1,2,3] register to 1 thus enabling extended wait operation.

Program MPMCStExdWt registers with different values.

Perform different types of accesses and combinations of burst operations.

Trick Memory verifies whether delay programmed in ExdWtDel register is used as the actual delay.

Repeat the above test with different values of delays.

8.1.28 BLS Delay Value Test

This test verifies Byte Lane State(PB) bit of MPMCStConfig[0,1,2, 3] register.

Test Identification No: MPMC_ByteLane_1

Configure TrickMem and static registers for different values.

Configure different chip selects with 16 or 32 bit memory width.

Write 1 to Byte Lane State(PB) field of MPMCStConfig[0,1,2,3] register as well as RBLE field of MPMCTrMEMT register of TrickMem .

TrickMem will check the nMPMCDQMOUT[3:0] line for the required function.

Repeat the above test with different values of delay values.

Make MPMCSTCS1PB '1' and MPMCSTCS1MW as 16 bit memory by programming MPMCTrStaticCS register of trickbox.

Apply nPOR.

Perform memory write and read operation and check for data integrity.

Reprogram the MPMCTrStaticCS to drive 0 on MPMCSTCS1PB and 8 bit memory width on MPMCSTCS1MW.

Apply nPOR.

Perform memory write, perform read operation and check the data integrity.

8.1.29 ROM Test

This test verifies the functionality of controller when the memory connected is ROM.

Test Identification No: MPMC_ROM_1

Configure TrickMem of different chip selects with different memory width.

Configure TrickMem as read only memory.

Program MPMCStConfig[0,1,2,3] registers as read only memory.

Write 00 to Memory Device (MD) of MPMCStConfig[0,1,2,3] and "01" to MEMTYPE field of MPMCTrMEMT register, which sets memory to ROM.

Initialise data to ROM from AHB interface side.

Program WAITRD of MPMCStWtRd register and CS2Rd of MPMCTrCS2Rd register of TrickMem with valid period.

Read data from controller side.

Try to write to memory from controller side and verify that response is ERROR.

8.1.30 LowPwrSdramFunTest

In this test initialisation of LPSPDRAM, memory access to LPSPDRAM, functionality of deep sleep mode and re initialisation of LPSPDRAM after coming out of deep sleep are tested.

Test Identification No: MPMC_LowPwrSdram_1

Disable address mirror bit. Configure MPMCDyRasCas and MPMCDyConfig registers.

Disable buffers.

Wait for 100us.

Apply PALL

Wait till two refreshes are applied to memory.

Apply mode command to memory.

Program the mode registers according to the requirement. Mode register value has to be given over Row address from AHB.

Program the extended mode register. Programming extended mode register enables the reduction in power consumption.

Bring the memory to normal mode.

Enable buffers and perform memory access to LPSPDRAM.

Wait till refresh is applied to memory. When buffer is empty and controller is in idle State, write 1 to RP bit of MPMCDyCntl register that allows the memory to enter low power mode.

Perform memory access operation and verify that response is error.

Write 0 to MPMCDyCntl register that allows the memory to enter normal mode.

Disable the buffers and perform the initialisation as described earlier.

Enable buffers and perform memory operation and verify the data integrity.

8.1.31 SyncFlash Memory Test

This test performs initialisation of sync flash, reads device configuration, protects the block, memory programming and array reads from sync flash.

Test Identification No: MPMC_SyncFlash_1

Disable address mirror bit. Initialise MPMCDyConfig[0,1,2,3] registers and MPMCDyRasCas[0,1,2,3].

Disable the buffers.

Set RP bit.

Wait for 100us.

Apply mode command to memory.

Program the mode registers according to the required burst length, burst type, CAS latency, op mode and write burst mode.

Apply normal command to memory.

Perform clear status register by writing 1 to HADDR[27] and 0xXXXX50XX on HWDATA where X is don't care.

Wait till SR7 becomes high so that it is ready to take another command.

Read device manufacturer compatible ID by writing 1 to HADDR[27], 0 to HADDR[26], Row address and Column addresses as zero.

Wait till SR7 becomes high so that it is ready to take another command.

Read device ID by writing 1 to HADDR[27] and 0 to HADDR[26], Row address as zero and Column addresses as 2.

Wait till SR7 becomes high so that it is ready to take another command.

Perform program operation by resetting HADDR[27]. Since sync flash is not supporting the burst write, after each write operation status of SR7 bit of status has to be observed.

Enable the buffers before doing array read. Perform array read and check for the data integrity.

Verify that controller is in idle State and buffers are empty. Disable the buffers. Block protect the block1 of bank1 by setting HADDR[27] = 1 and giving appropriate row and column address and protect set up and confirm on data line.

Wait till ISM comes out of busy State by polling the SR7 bit.

Write 0 to RP bit of MPMCDyCntl.

8.1.32 Random Test1

This test verifies the overall functionality of the controller when a number of operations are performed randomly. Following points can be followed while running random tests.

Test Identification No: MPMC_Rndom_1

Dedicate few memory chips as write protected.

Disable address mirror bit. Initialise sdram, TrickMem and controller registers.

Memory accesses are issued from port3 randomly either for read or write operation with variable burst length and to different chip selects.

8.1.33 CornerCases1

Test Identification No: MPMC_CornerCase1_1

This tests the functionality of static memory controller when small delay values are programmed to the delay registers.

Disable address mirror bit. Initialise TrickMem as well as static memory controller registers.

Program small values (0x0 or 0x1) as delay values.

Perform memory write operation with different types of HBURST, HSIZE and HTRANS.

Read data back and check the data integrity.

8.1.34 CornerCases2

Test Identification No: MPMC_CornerCase2_1

It does the memory access to 512M memory and hit the upper row address for memory access.

Disable address mirror bit. Initialise Sdram according to the burst length.

Write burst of data to upper row addresses of different chip select.

Read data back and check the data integrity.

8.1.35 CornerCases3

Test Identification No: MPMC_CornerCase3_1

It does different types of memory transfers with different combinations of HTRANS to both static as well as dynamic memory with buffers enabled.

Disable address mirror bit. Initialise TrickMem, Sdram memory models, static memory controller registers and dynamic memory registers.

Enable buffers.

Perform different types of memory transfers to both static and dynamic memory with different combinations of HTRANS.

Read data back and check the data integrity.

Different combinations of HTRANS are shown below,

HBURST	HTRANS
INCR	NS – B – S
INCR	NS – I – NS
INCR4	NS – S – S – I – NS
INCR4	NS – S – S – B – NS
INCR	NS – B – I – NS
WRAP4	NS – B – I – NS
INCR	NS – S – B – S
INCR	NS – B – S – B – S

WRAP16	NS – S – S ...B – S – ... B – I – NS – I
WRAP16	NS – S – S ...B – S – ... B – I – NS – I
INCR16	NS – S – B – S – S ...

Table 8.1-7 Different combinations of HTRANS

8.1.36 Arb2CornerCase4

This test performs the simultaneous access to the static as well as sync flash memory from different ports.

Test Identification No: MPMC_Arb2CornerCase4_1

Disable address mirror bit, initialise sync flash and controller register from port3.

Perform write operation to sync flash from one port and start memory write operation to static memory from another port.

Read data back and check the data integrity.

8.1.37 Arb2HMastLockTest

This test checks whether the locked transfer is happening successfully or not.

Test Identification No: MPMC_Arb2HMastLockTest

Initialise the sdram controller, static controller and trickmemory registers.

Initialise the trickmemory from the address 0x2 from Port3.

Perform locked transfers from Port3 to the memory location other than the initialised memory location.

Simultaneously, perform memory write to the initialised memory from Port2. Since Port3 is doing locked transfers, Port2 is not able to do the memory write.

Read the memory where the data are initialised before starting the test and verify that they are not altered.

8.1.38 Rel1HburstTest

This test does the memory accesses with the MPMCADDROUT lines connected to the memory as in MPMC-PL172 Release1v0, with MPMCREL1CONFIG tied HIGH.

Test Identification No: MPMC_Rel1HburstTest_1

Connect the MPMCADDROUT to the memory models as done in the MPMC-PL172Rel1.

Assert MPMCREl1Config pin.

Initialise the memory controller registers according to the memory connected in the tbench.

Perform write operation to the memory. Check the data integrity by reading the data back.

Perform different types of memory accesses with different HBURST, HSIZE and HTRANS and check for data integrity.

8.1.39 Command Delayed Tests

These tests are primarily to check the functionality of the command delayed mode of the controller. In command delayed mode, commands for the memory are delayed instead of MPMCCLKOUT as in Clock delayed approach.

DyHburstCmdDelTest1

Test Identification No: MPMC_DyHburstCmdDelTest1

Initialise the controller register. Write 1 to MpmcDyRdCfg register to enable the command delayed mode.

Initialise the sdram memory.

Perform different types of memory accesses with different HBURST, HSIZE and HTRANS and check for data integrity.

SyncFlashCmdDelTest**Test Identification No: MPMC_SyncFlashCmdDelTest_1**

Initialise the controller registers. Write 1 to MpmcDyRdCfg register to enable the command delayed mode.

Initialise the syncflash memory.

Perform different types of memory accesses with different HBURST, HSIZE and HTRANS and check for data integrity.

8.1.40 IntegrationTest

This test is primarily to toggle all the AHB signals and memory signals of the PrimeCell.

Test Identification No: MPMC_IntegrationTest_1

Initialise the controller registers and trickmemory registers.

Initialise the sdram memory.

Perform memory write with address from which it is possible to toggle all the address lines and also write the data, which can toggle all data lines from all ports.

Perform these operations with different HSIZE, HTRANS and HBURST.

Read the data back and check for the data integrity.

8.1.41 Latency Tests

These tests perform accesses through different AHB ports and tests whether the AHB priority scheme is properly followed, as far as returning data to HRDATA bus is concerned. To check the latency of AHB0, program the HREADY0CNT register of trickbox with the maximum allowable value. If the HREADY0 is low for maximum allowable clocks, the trickbox displays the message. If it is required to check the latency of AHB1, then program the HREADY1CNT register to the maximum allowable value. Different types of accesses are tried to find out to find the maximum latency on AHBs.

MultiPortPgAccessTest:**Test Identification No: MPMC_MultiPortPgAccessTest**

This test performs the memory accesses from Port3 and Port0.

Initialise the SDRAMs and MPMC registers from Port3. Program the HREADY0CNT register with the value of 85. Enable the latency checking for AHB0.

Perform write to the 16Mx32 memory with BRC from port3. Increment the address by 8000 so that each time different page is written.

Try to perform read from the Port0 to the same bank.

Verify that the Port0 gets the bus at the earliest. If HREADY0 is low for more than 85 clocks, the trickbox displays the error message.

MultiPortPgAccessTest1:**Test Identification No: MPMC_MultiPortPgAccessTest1**

This test performs the memory accesses from Port2 and Port0.

Initialise the SDRAMs and MPMC registers from Port3. Program the HREADY0CNT register with the value of 85. Enable the latency checking for AHB0.

Perform write to the 16Mx32 memory with BRC from port2. Increment the address by 8000 so that each time different page is written.

Try to perform read from the Port0 to the same bank.

Verify that the Port0 gets the bus at the earliest. If HREADY0 is low for more than 85 clocks, the trickbox displays the error message.

MultiPortPgAccessTest2:**Test Identification No: MPMC_MultiPortPgAccessTest2**

This test performs the memory accesses from Port1 and Port0.

Initialise the SDRAMs and MPMC registers from Port3. Program the HREADY0CNT register with the value of 85. Enable the latency checking for AHB0.

Perform write to the 16Mx32 memory with BRC from port1. Increment the address by 8000 so that each time different row is written in the same bank. The column address is so selected that every write request performs an auto-precharge.

Try to perform read from the Port0 to the same bank.

Verify that the Port0 gets the bus at the earliest. If HREADY0 is low for more than 85 clocks, the trickbox displays the error message.

MultiPortPgAccessTest3:**Test Identification No: MPMC_MultiPortPgAccessTest3**

This test performs the memory accesses from Port3 and Port0.

Initialise the SDRAMs and MPMC registers from Port3. Program the HREADY0CNT register with the value of 85. Enable the latency checking for AHB0.

Perform write to the 16Mx32 memory with BRC from port3. Increment the address by 8000 so that each time different page is written.

Try to perform burst of reads from the Port0 to the same bank.

Verify that the Port0 gets the bus at the earliest. If HREADY0 is low for more than 85 clocks, the trickbox displays the error message.

MultiPortPgAccessTest4:**Test Identification No: MPMC_MultiPortPgAccessTest4**

This test performs the memory accesses from Port3 and Port1.

Initialise the SDRAMs and MPMC registers from Port3. Program the HREADY1CNT register with the value of 85. Enable the latency checking for AHB1.

Perform write to the 16Mx32 memory with BRC from port3. Increment the address by 8000 so that each time different row is written in the same bank. The column address is so selected that every write request performs an auto-precharge.

Try to perform read from the Port1 to the same bank.

Verify that the Port1 gets the bus at the earliest. If HREADY1 is low for more than 85 clocks, the trickbox displays the error message.

MultiPortPgAccessTest5:

Test Identification No: MPMC_MultiPortPgAccessTest5

This test performs the accesses from Port2 and Port1 and checks whether the AHB priority scheme is properly followed as far as returning data to HRDATA bus is concerned.

Initialize the SDRAMs and MPMC registers from Port3. Program the HREADY1CNT register with the value of 85. Enable the latency checking for AHB1. Program the refresh counter with 0x32.

From Port2, wait till Port3 completes the initialization. Once the initialization is over, perform single burst writes from Port2 to the address 0x4042A7F0, so that the writes are happening to the last locations of the page. Increment the address and perform writes in such a way that the accesses are happening to different pages each time.

Simultaneously, perform continuous INCR16 reads from Port1 with the starting address as 0x4031A2FC.

If there is any access which is resulting in Port1 read latency of more than 85 clocks, it can be detected with the help of trickbox error messages.

LatencyTest1:

Test Identification No: MPMC_LatencyTest1

This test performs the memory accesses from Port2, Port1 and Port0.

Initialise the SDRAMs and MPMC registers from Port3. Program the HREADY0CNT register with the value of 85. Enable the latency checking for AHB0.

Perform write to the 16Mx32 memory with BRC from port2. Increment the address by 8000 so that each time different page is written.

Try to perform read from the Port0 and Port1 to the same bank.

Verify that the Port0 gets the bus at the earliest. If HREADY0 is low for more than 85 clocks, the trickbox displays the error message.

LatencyTest2:

Test Identification No: MPMC_LatencyTest2

This test performs the memory accesses from Port2 and Port1.

Initialise the SDRAMs and MPMC registers from Port3. Program the HREADY1CNT register with the value of 85. Enable the latency checking for AHB1.

Try to perform read from the Port1 to the same bank.

Verify that the Port1 gets the bus at the earliest. If HREADY1 is low for more than 85 clocks, the trickbox displays the error message.

LatencyTest3_1:

Test Identification No: MPMC_LatencyTest3_1

This test performs the memory accesses from Port2 and Port0.

Initialise the SDRAMs and MPMC registers from Port3. Program the HREADY0CNT register with the value of 85. Enable the latency checking for AHB0.

Perform read from Port0. The read operation from Port0 is repetitive. I.e. between two reads of 150 clocks of clocks of idle states are added.

From Port2 perform different types of writes are initiated. I.e. continuous SINGLE write, continuous INCR write, continuous INCR4 write, continuous INCR8, continuous INCR16, continuous WRAP4, continuous WRAP8, continuous WRAP16.

In each case, find the worst-case latency on AHB0. If the HREADY of AHB0 is low for more than 85 clocks, the trickbox displays the error message.

LatencyTest3_2:

Test Identification No: MPMC_LatencyTest3_2

This test performs the memory accesses from Port2, Port1 and Port0.

Initialise the SDRAMs and MPMC registers from Port3. Program the HREADY0CNT register with the value of 85. Enable the latency checking for AHB0.

Perform read from Port0. The read operation from Port0 is repetitive. I.e. between the two reads of 150 clocks of idle states are added.

From AHB1 and AHB2 perform different types of accesses. Below table shows the different combinations of AHB1 and AHB2.

AHB1	AHB2
Continuous WRAP16 access	Continuous SINGLE access
Continuous WRAP8 access	Continuous INCR access
Continuous WRAP4 access	Continuous INCR4 access
Continuous INCR16 access	Continuous INCR8 access
Continuous INCR8 access	Continuous INCR16 access
Continuous INCR4 access	Continuous WRAP4 access
Continuous INCR access	Continuous WRAP8 access
Continuous SINGLE access	Continuous WRAP16 access

Verify that the Port0 gets the bus at the earliest. If HREADY0 is low for more than 85 clocks, the trickbox displays the error message.

LatencyTest4_1:

Test Identification No: MPMC_LatencyTest4_1

This test is similar to LatencyTest3_1 except for AHB0 is performing writes instead of reads.

LatencyTest4_2:

Test Identification No: MPMC_LatencyTest4_2

This test is similar to LatencyTest3_2 except for AHB0 is performing writes instead of reads.

LatencyTest5_1:

Test Identification No: MPMC_LatencyTest5_1

This test is similar to LatencyTest3_1 except for AHB2 is performing reads instead of writes.

LatencyTest5_2:**Test Identification No: MPMC_LatencyTest5_2**

This test is similar to LatencyTest3_2 except for AHB2 is performing reads.

LatencyTest6_1:**Test Identification No: MPMC_LatencyTest6_1**

This test is similar to LatencyTest3_1 except for AHB2 is performing reads instead of writes and AHB0 is performing writes.

LatencyTest6_2:**Test Identification No: MPMC_LatencyTest6_2**

This test is similar to LatencyTest3_2 except for AHB2 is performing reads instead of writes and AHB0 is performing writes.

LatencyTest7:**Test Identification No: MPMC_LatencyTest7**

This test performs the memory accesses from the AHB2, AHB1 and AHB0.

Initialise the SDRAMs and MPMC registers from Port3. Program the HREADY0CNT register with the value of 85. Enable the latency checking for AHB0.

Perform locked transfers write followed by read followed by write followed by read to memory region. Perform access from AHB1 and AHB0.

Confirm that AHB1 and AHB2 do not perform access when AHB0 is doing the locked transaction.

LatencyTest8:**Test Identification No: MPMC_LatencyTest8**

This test performs the memory accesses from the AHB2, AHB1 and AHB0.

Initialise the SDRAMs and MPMC registers from Port3. Program the HREADY1CNT register with the value of 85. Enable the latency checking for AHB1.

Perform transfers write followed by read followed by write followed by read to memory region from Port0.

Perform locked transactions from Port1.

Perform normal transactions from Port2. Confirm that Port1 has granted the bus at the earliest.

LatencyTest9:**Test Identification No: MPMC_LatencyTest9**

This test performs the memory accesses from the AHB2, AHB1 and AHB0.

Initialise the SDRAMs and MPMC registers from Port3. Program the HREADY0CNT register with the value of 85. Enable the latency checking for AHB0.

Perform locked transfers write followed by read followed by write followed by read to memory region from Port0.

Perform locked transactions from Port1.

Perform normal transactions from Port2. Confirm that Port1 is not allowed to do the transactions when AHB0 is doing the transfers.

LatencyTest10:**Test Identification No: MPMC_LatencyTest10**

This test performs the memory accesses from the AHB2, AHB1 and AHB0.

Initialise the SDRAMs and MPMC registers from Port3. Program the HREADY0CNT register with the value of 85. Enable the latency checking for AHB0.

Perform locked transfers write followed by read followed by write followed by read to memory region from Port0.

Perform locked transactions from Port1.

Perform locked transactions from Port2. Confirm that Port0 has the got the bus at the earliest.

MultiPortDiffComb1:**Test Identification No: MPMC_MultiPortDiffComb1**

Initialise the SDRAMs and MPMC registers from Port3. Program the HREADY0CNT register with the value of 85. Enable the latency checking for AHB0.

Perform single writes from Port3.

Perform burst of INCR reads from Port0 and check for the AHB0 latency.

MultiPortDiffComb2:**Test Identification No: MPMC_MultiPortDiffComb2**

Initialise the SDRAMs and MPMC registers from Port3. Program the HREADY0CNT register with the value of 85. Enable the latency checking for AHB0.

Perform continues SINGLE reads from Port3. Make sure that the addresses are incremented in such a way that each time different page is accessed.

Perform SINGLE writes from Port0 and check for the AHB0 latency.

MultiPortDiffComb3:**Test Identification No: MPMC_MultiPortDiffComb3**

Initialise the SDRAMs and MPMC registers from Port3. Program the HREADY0CNT register with the value of 85. Enable the latency checking for AHB0.

Perform continues SINGLE reads from Port3. Make sure that the addresses are incremented in such a way that each time different page is accessed.

Perform burst of INCR reads from Port0 and check for the AHB0 latency.

MultiPortDiffComb4:**Test Identification No: MPMC_MultiPortDiffComb4**

Initialise the SDRAMs and MPMC registers from Port3. Program the HREADY0CNT register with the value of 85. Enable the latency checking for AHB0.

Perform continues SINGLE reads from Port3. Make sure that the addresses are incremented in such a way that each time different page is accessed.

Perform burst of reads from Port0 and check for the AHB0 latency.

MultiPortDiffComb5:**Test Identification No: MPMC_MultiPortDiffComb5**

Initialise the memory and registers from Port3. Perform single write from Port3. The address is incremented in such a way that each write is executed for different page.

Perform incr8 read from the Port0. And check that the Port0 got the bus grant at the earliest.

MultiPortDiffComb6:**Test Identification No: MPMC_MultiPortDiffComb6**

Initialise the memory and registers from Port3. Perform incr8 read from Port3.

Perform single write from the Port0. Increment the address in such a way that each write is to the different page.

Check that the latency for Port0 is minimum.

MultiPortDiffComb7:**Test Identification No: MPMC_MultiPortDiffComb7**

Initialise the memory and registers from Port3.

Perform incr8 read from Port0.

Perform incr8 read from the Port3. Increment the address in such a way that each read is to the different page.

Check that the latency for Port0 is minimum.

MultiPortDiffComb8:**Test Identification No: MPMC_MultiPortDiffComb8**

Initialise the memory and registers from Port3.

Perform incr8 write from Port0.

Perform single write from the Port3. Increment the address in such a way that each write is to the different page.

Check that the latency for Port0 is minimum.

8.1.42 RefreshMissTest**Test Identification No: MPMC_RefreshMissTest**

This test checks whether the refreshes are applied to the dynamic memory when there are continuous access to the static memory.

Initialise the dynamic memory and MPMC registers. Program a small value to the refresh counter. Enable the refresh check bit of trickbox.

Perform continuous accesses to the static memory.

If the refresh is not applied to the memory then the trickbox issues the error message.

8.1.43 DirectDyDtFtchTest

This function tests the data integrity in certain cases, where the MPMC directly passes the dynamic-memory data to the AHB, instead of routing it through buffer.

The normal flow of dynamic-memory-read-data is

Pad interface --> Buffer --> AHB.

For improving the AHB read latency, sometimes the Buffer is bypassed and data directly passed from pad-interface to AHB. The above-mentioned bypass is done only in certain conditions. This test checks for the bypass happening in proper conditions.

Test Identification No: MPMC_DIRECTDYDATA

Initialise the dynamic memory and MPMC registers.

Perform continuous write to the dynamic memory. After completing writes insert wait states.

Perform write to the write-protected static memory and expect the ERROR response.

Perform read operation to dynamic memory, which is already written, and check for the data integrity.

8.1.44 MemBGntTest

This function tests the controller in a particular corner case, where the autorefresh is applied at the point of last static access.

Test Identification No: MPMC_MemBGnt_1

This test performs burst of read to the unbuffered static chip select. At the last beat apply the refresh so that the static is degraded.

Perform single read to the buffered static chip select and observe that the AHB locks up or not.

8.1.45 AHBLockIdleInsertTest1

This test performs the locked accesses from the AHB1 and inserts idle states along with Lock signal high and checks whether the arbitration should not happen in the idle window.

Test Identification No: MPMC_AHBLockIdleInsertTest1_1

Initialise the SDRAMs from Port3.

Perform locked write to the 16Mx16 memory with BRC from port2. When the locked burst is completed keep the lock signal stays active (HMASTLOCK = 1) and perform IDLE transfers.

Perform locked write transactions from the AHB0 while AHB1 is doing idle transfer to the same address.

A locked read transfer occurs on AHB1 (HMASTLOCK = 1). The value read back should be from the AHB1 write transfer.

8.1.46 AHBLockIdleInsertTest2

This test performs the locked accesses from the AHB1 and inserts idle states along with Lock signal high and checks whether the arbitration should not happen in the idle window.

Test Identification No: MPMC_AHBLockIdleInsertTest2_1

Initialise the SDRAMs from Port3.

Perform locked write to the 16Mx16 memory with BRC from port2.

When the locked burst is completed keep the lock signal stays active (HMASTLOCK = 1) and perform IDLE transfers with HSELMPMC Low.

Perform locked write transactions from the AHB0 while AHB1 is doing idle transfer to the same address.

A locked read transfer occurs on AHB1 (HMASTLOCK =1). The value read back should be from the AHB1 write transfer.

8.1.47 CornerCases4

This test performs the access to the unity cas latency dynamic memory.

Test Identification No: MPMC_CornerCases4_1

Initialise the SDRAMs from Port3.

Perform single data write with different types of size and read the data back to check the data integrity to the unity CAS latency memory.

It also does the multiple data write with different size depending on the burst type and reads the data back.

It checks whether precharge is applied or not by accessing the last column of a row.

8.1.48 CornerCases5

This test performs the syncflash accesses with CE bit low.

Test Identification No: MPMC_CornerCases5_1

Initialise the SDRAMs from Port3.

Perform different types of accesses to the sync flash memory I.e. LCR/ACT/READ, syncflash write, syncflash read with CE bit low.

Perform different accesses to the dynamic memory also and check for data integrity.

8.1.49 CornerCases6

This test checks the functionality of all AHB bursts on dynamic memory.

Test Identification No: MPMC_CornerCases6_1

Initialise the SDRAMs from Port3.

Perform read modify write sequence when all buffers are occupied.

8.1.50 CornerCases7

This test checks multiport access.

Test Identification No: MPMC_CornerCases7_1

Initialise the SDRAMs from Port3.

Perform read burst, which is then broken. Start a new read on a different port.

8.1.51 CornerCases8

This test checks multiport access.

Test Identification No: MPMC_CornerCases8_1

Initialise the SDRAMs from Port3.

Perform semaphore type transactions on two AHB ports and check results.

8.1.52 MultiPortWrProtTest

Checks write protect feature with multiport access.

Program Chip select 0 (static memory) as write protected, 32 bit memory width.

Perform write to the CS0 from the Port3 with HSIZE as BYTE and expect ERROR response. After read perform read to the same chip select.

Perform read from the Port2 to the dynamic memory with HSIZE as WORD.

If the controller posts the write request to the memory in spite of write protect enable, the trick memory is gives the error messages.

8.2 Verification Test Vector Generation

A ***make sourcefilename*** (without the .c extension) in the ***verification/bustests*** directory will create ***.bif*** formatted and ***.sim*** formatted vectors in the ***verification/bustest/invec*** directory. The makefile in ***verification/bustest*** directory can be referred to for the make options.

9 INTEGRATION TESTBENCH

The integration testbench is based on an AHB TICTalk testbench. The VHDL and Verilog TICTalk testbench files reside in the *integration/vhdl* and *integration/verilog* directories respectively. This testbench is used for testing the MPMC's TIC module as well as for running the integration tests of the MPMC (PL172). For running the integration tests, the MPMC's TIC is used.

9.1 VHDL Integration testbench

The AMBA TICTalk VHDL testbench used for integration testing and for testing of the MPMC's TIC is located in the *integration/vhdl/tbench* directory. The Integration test vectors are located in the *integration/bustest* directory.

Figure 9.1-1 illustrates the VHDL testbench to apply TICTalk test vectors.

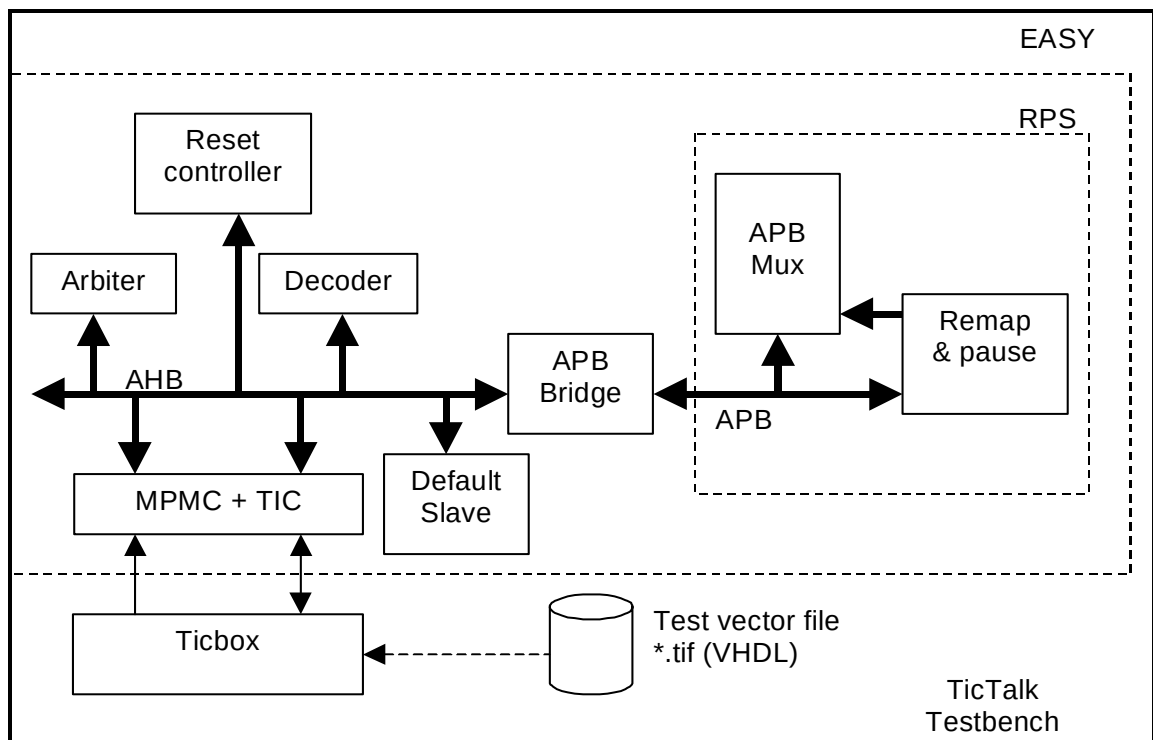


Figure 9.1-1 VHDL Testbench for simulating TICTalk test vectors

The VHDL TicTalk testbench is a stripped down version of the standard EASY system testbench.

It has decoder which is used to provide a select signal HSELx for each slave on the bus, arbiter which ensures that only one master has access to the bus, default slave which provides response signals when non-existence memory address is accessed, reset controller which generates reset.

For further information refer to the Example AMBA System user guide which is part of ARM Micropack documentation.

Figure 9.1-2 illustrates the hierarchy for the TICTalk VHDL testbench.

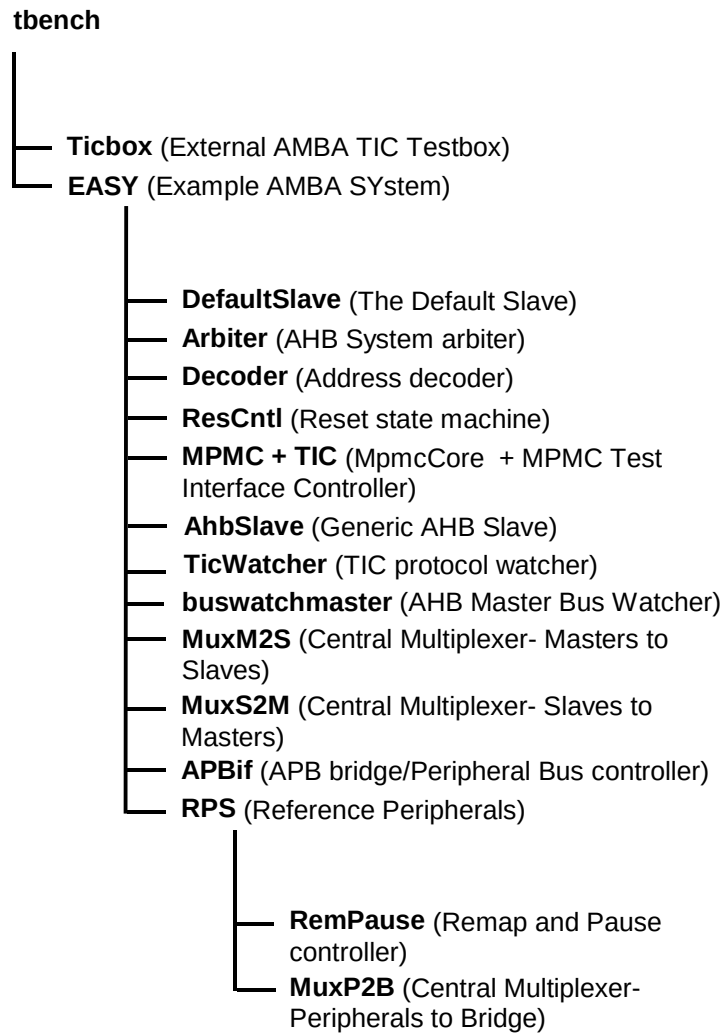


Figure 9.1-2 VHDL Testbench hierarchy for simulating TICTalk test vectors

The testing of the TIC module is done in the TICTALK world. The Ticbox module in the TICTALK world drives the TIC vectors onto the TIC module. The Ticbox reads the infile.tif in the *integration/bustest/invec* directory and generates TIC vectors. A generic AHB slave is used for the testing of the MpmcTIC module functionality. The TIC generates AHB transfers to the generic AHB slave. Different types of transfers are initiated and the behaviour of the TIC is tested. A TICWatcher module checks for the data integrity and protocol checks on the TIC generated AHB signals.

9.1.1 Bus Master Protocol Checker

The EASY instantiates the master bus watcher, buswatchmaster.vhd which checks the protocol of TIC master accesses initiated via the test code. This module implements the protocol checks for the AHB master's output signals and will flag warning and error messages if any timing or protocol violations are detected during simulations.

9.1.2 TIC Protocol Checker

The TIC Watcher module instantiated in the EASY is a TIC data integrity/protocol checker. This module mirrors the logic of the TIC and generates the AHB signals internally. The internally generated AHB signals are compared with that from the TIC module. It flags error messages whenever a protocol violation is detected.

9.1.3 Generic AHB Slave

The Generic AHB Slave is instantiated in the EASY. The TIC is validated by using a generic AHB Slave, which will respond to the AHB transfers generated by the TIC. By tracking the responses from the generic slave on each transfer, protocol and timing checks are done. The Generic AHB Slave is a Split-capable slave device that aids the validation of the TIC. In addition to the bus interface logic, it contains programmable registers and data arrays that can be used to control the way in which each AHB transfer is responded to.

9.1.4 Test Vectors

The test vectors for the verification of the TIC are coded into separate C files, in the *integration/bustest* directory, depending on the logical region of the TIC, which these vectors are testing. Make options have been provided to use all the test vectors together or to use them individually.

9.1.5 Running Simulations

The **README** file in the *integration/vhdl/tbench* directory gives step-by-step instructions for running simulations using tests on the VHDL source code.

Run time logs for all the combinations are available in the *integration/vhdl/Mpmc_rtl* and *integration/vhdl/Mpmc_scaninsert_vhdl_net_max* directories.

9.2 Verilog Integration testbench

The AMBA TICTalk Verilog testbench used for testing of the TIC is located in the *integration/verilog/tbench* directory. The Integration test vectors are located in the *integration/bustest* directory.

Figure 9.2-1 illustrates the Verilog testbench to apply TICTalk test vectors.

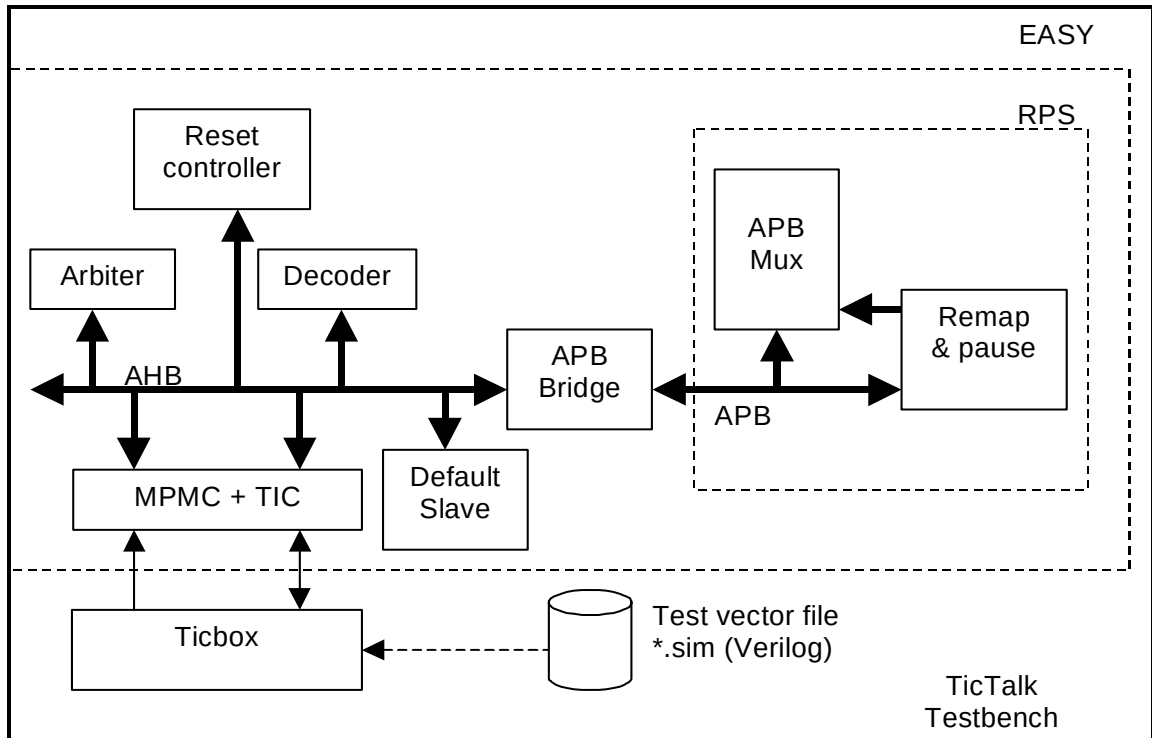


Figure 9.2-1 Verilog Testbench for simulating TICTalk test vectors

The Verilog TicTalk testbench is a stripped down version of the standard EASY system testbench.

It has decoder which is used to provide a select signal HSELx for each slave on the bus, arbiter which ensures that only one master has access to the bus, default slave which provides response signals when non-existence memory address is accessed and reset controller which generates reset for the system.

For further information refer to the Example AMBA System user guide which is part of ARM Micropack documentation.

Figure 9.2-2 illustrates the hierarchy for the TICTalk Verilog testbench.

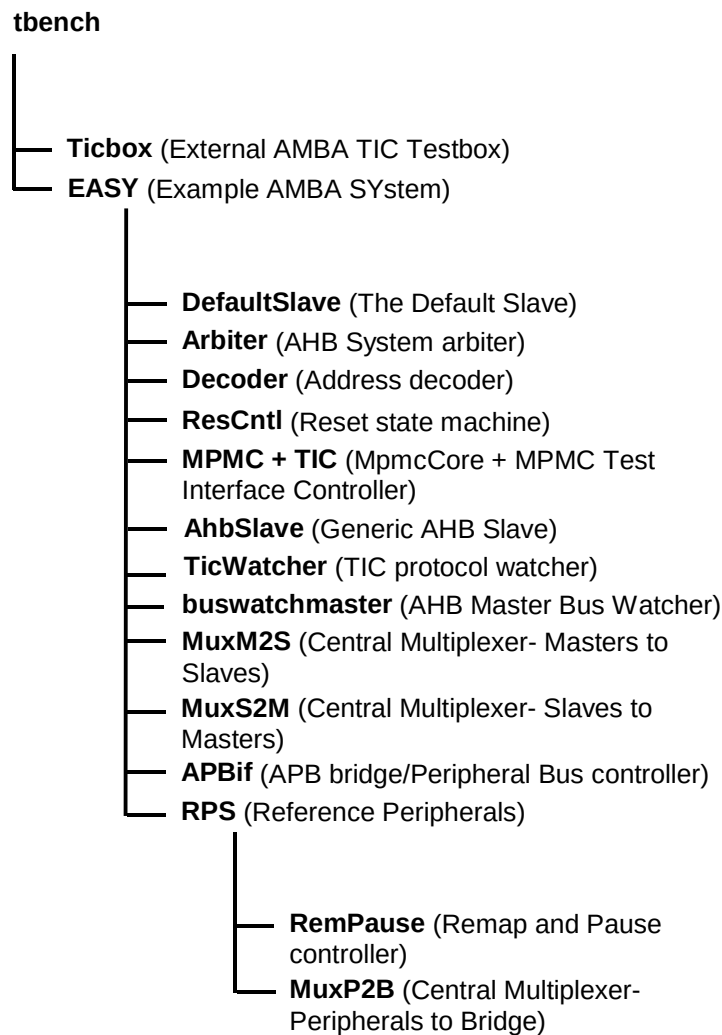


Figure 9.2-2 Verilog Testbench hierarchy for simulating TICTalk test vectors

The testing of the TIC module is done in the TICTALK world. The Ticbox module in the TICTALK world drives the TIC vectors onto the TIC module. The Ticbox reads the tif.sim in the **integration/bustest/invec** directory and generates TIC vectors. A generic AHB slave is used for the testing of the MpmcTIC module of the MPMC. The TIC generates AHB transfers to the generic AHB slave. Different types of transfers are initiated and the behaviour of the TIC is tested. A TICWatcher module checks for the data integrity and protocol checks on the TIC generated AHB signals.

9.2.1 Bus Master Protocol Checker

The TicTalk testbench instantiates the master bus watcher, buswatchmaster.v that checks the protocol of TIC master accesses initiated via the test code. This module implements the protocol checks for the AHB master's output signals and will flag warning and error messages if any timing or protocol violations are detected during simulations.

9.2.2 TIC Protocol Checker

The TIC Watcher module instantiated in the EASY is a TIC data integrity/protocol checker. This module mirrors the logic of the TIC and generates the AHB signals internally. The internally generated AHB signals are compared with that from the TIC module. It flags error messages whenever a protocol violation is detected.

9.2.3 Generic AHB Slave

The Generic AHB Slave is instantiated in the EASY. The TIC is validated by using a generic AHB Slave, which will respond to the AHB transfers generated by the TIC. By tracking the responses from the generic slave on each transfer, protocol and timing checks are done. The Generic AHB Slave is a Split-capable slave device that aids the validation of the TIC. In addition to the bus interface logic, it contains programmable registers and data arrays that can be used to control the way in which each AHB transfer is responded to.

9.2.4 Test Vectors

The test vectors for the verification of the TIC are coded into separate C files, in the *integration/bustest* directory, depending on the logical region of the TIC, which these vectors are testing. Make options have been provided to use all the test vectors together or to use them individually.

9.2.5 Running Simulations

The **README** file in the *integration/verilog/tbench* directory gives step by step instructions for running simulations using tests on the Verilog source code.

Run time logs for all the combinations are available in the *integration/verilog/Mpmc_rtl*, *integration/verilog/Mpmc_noscan_vhdl_net_min* and *integration/verilog/Mpmc_scanready_vhdl_net_typ* directories.

9.3 Trickbox Description

9.3.1 Generic AHB Slave

Using a generic AHB Slave, which will respond to the AHB transfers generated by the TIC, validates the TIC. By tracking the responses from the generic slave on each transfer, protocol and timing checks are done. The Generic AHB Slave is a Split-capable slave device that aids the validation of the TIC. In addition to the bus interface logic, it contains programmable registers and data arrays that can be used to control the way in which each AHB transfer is responded to.

The block diagram of the Generic AHB Slave is shown in **Figure 9.3-1**.

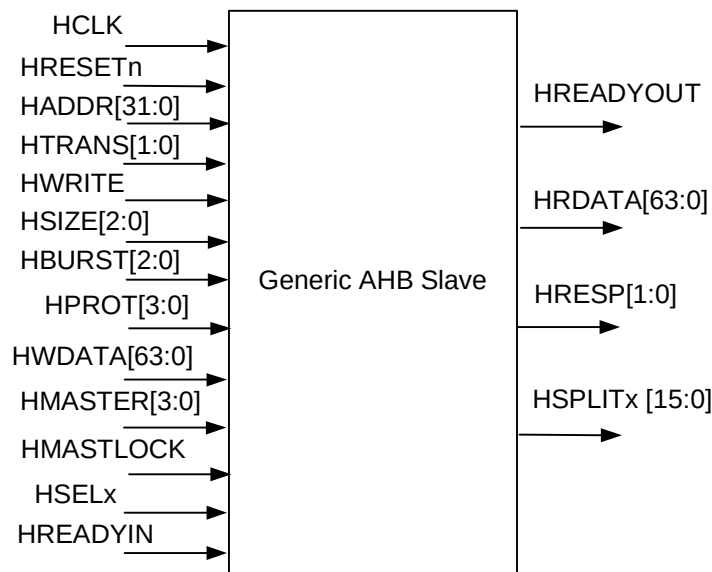


Figure 9.3-1 Generic AHB Slave Block diagram

Generic Slave Specifications

Functional Mode Registers:

Address	Type	Width	Reset Value	Name	Description
Base Address + 0x00	R/W	32	0x0000	WSC1	Wait State Register 1 defines the wait states to be introduced while accessing memory array 1
Base Address + 0x04	R/W	32	0x0000	WSC2	Wait State Register 2 defines the wait states to be introduced while accessing memory array 2
Base Address + 0x08	R/W	32	0x0000	CR	Control Register defines DELAYEN bits for all output signals and PROTECTION bits for HRESP and HRDATA. Also defines the endianness of the system.
Base Address + 0x0C	R/W	32	0x0000	TMR	Timeout register defines the number of clock cycles after which the transfer times out in a SPLIT/RETRY transfer.
Base Address + 0x10	R/W	32	0x0000	XR	Transfer delay register determines when the slave has to respond to an access to a RETRY/SPLIT space with an OK response

Table 9.3-1 Functional Mode Registers

Memory Arrays:

Address	Type	Width	Reset Value	Name	Description
Base Address + (0x100 – 0x4FF)	R/W	64	0x00	ARRAY1	Memory Array 1 to store data to check read and write operations
Base Address + (0x500 – 0x8FF)	R/W	64	0x00	ARRAY2	Memory Array 2 to store data to check read and write operations

Table 9.3-2 Memory Arrays

Generic Slave Description

The AHB slave contains two register arrays and five configuration registers as per the following description. In the following all addresses are word addresses:

Memory Array

This is an array of 128 dwords (64 bits), which is accessible for reads or writes over the AHB. Aligned byte / half-word, word and dword accesses to this array of 1KB are supported with wait states controlled on a per-beat basis as per the Array Wait State Control Register. The array is divided into four parts such that accessing each part will give a different type of HRESP response.

Wait State Control Register

Any R/W access to a particular array location will introduce that many wait states as is programmed in the wait State register. The 32 bit wait State register is split into 8 fields with each field denoting the number of wait states to be introduced during an access to each beat of a burst transfer to that particular array. It is intended to do read/write operation to the register with zero wait states.

For a 4 word burst

- bits 7:0 = wait states in beat 0
- bits 15:8 = wait states in beat 1
- bits 23:16 = wait states in beat 2
- bits 31:24 = wait states in beat 3

For an 8 word burst

- bits 3:0 = wait states in beat 0
- bits 7:4 = wait states in beat 1
- bits 11:8 = wait states in beat 2
- bits 15:12 = wait states in beat 3
- bits 19:16 = wait states in beat 4
- bits 23:20 = wait states in beat 5
- bits 27:24 = wait states in beat 6
- bits 31:27 = wait states in beat 7

Control Register

The Control register contains 32 bits of which 6 are used and the others are reserved. Each of the response signals generated depend upon the two programmable bits in the generic slave control register. These two bits helps to find out whether the TIC is capable enough to detect the error in the responses given out by the generic slave.

Bit [0]: BIGENDIAN – This bit will determine whether the Slave will behave as little endian or big endian.

Bit [1]: ENDIANEN – When this bit is set, Slave will behave as a Big endian system, driving the data only in the required byte lanes.

Bit[2] : ForceErrResp – When this bit is set, will force the slave to drive the ERROR response. If not set then slave is intended to behave normally.

Bit[3] : ForceDataErr – When this bit is set, will force the slave to drive the complement of expected data on HRDATA bus.

Bit [4]: RESPEN – This bit, if not set, will force the slave to drive the OK response. Otherwise, if set, slave is intended to behave normally.

Bit [5]: DELAYEN – If this bit is enabled, all the output signals are delayed for the default value programmed to check the timing protocols of the signal. This will help in checking the timing protocols of the signals.

Timeout Register

The 32 bit register determines when the slave will timeout for a retry/split transfer. After timeout, the slave won't proceed with the ongoing split/retry transfer.

Transfer Delay Register

The first 16 bits will determine on which re-issue of the SPLIT transfer, the generic slave has to respond with an OK response. The higher order 16 bits will determine the number of re-issues required in a retry transfer.

Transfer Modes

The generic slave supports almost all modes in AHB Slave architecture. The modes supported are listed below.

HTRANS[1:0]	All combinations are supported
HSIZE[2:0]	All transfers upto DWORD are supported
HBURST[2:0]	All combinations are supported
HRESP[1:0]	All combinations are supported
HPROT[3:0]	Not supported

9.3.2 TIC Watcher Module

The TIC Watcher module is a TIC data integrity/protocol checker. This module mirrors the logic of the TIC and generates the AHB signals internally. The internally generated AHB signals are compared with that from the TIC module. It flags error messages whenever a protocol violation is detected.

TIC Watcher Signal Description

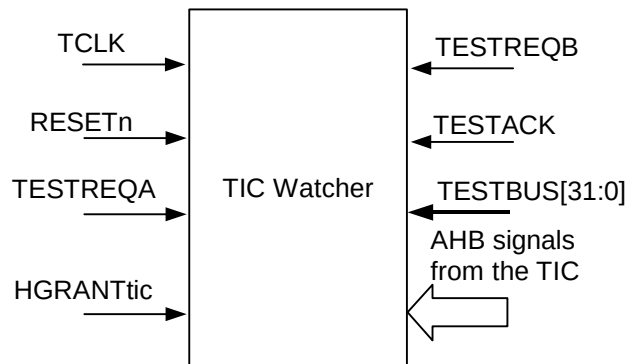
The following table summarises the TIC Watcher signal list.

Signal Name	Type	Source / Destination	Description
TCLK	IN	Clock Generator	Test mode clock input. This clock times all TIC bus transfers. All signal timings are referenced to the rising edge of TCLK.
RESETn	IN	Reset Controller	Bus reset (active low).
TESTREQA	IN	TICbox (External tester)	Test bus request A
TESTREQB	IN	TICbox (External tester)	Test bus request B
TESTACK	IN	TICbox (External tester)	Test Acknowledge
TESTBUS[31:0]	IN	TICbox (External tester)	Test data bus
HADDROUT[31:0]	IN	TIC	The 32-bit system address bus.
HTRANSOUT[1:0]	IN	TIC	Indicates the type of the current transfer, which can be Non-Sequential, Sequential or Idle. The TIC does not use the Busy transfer type.
HWRITEOUT	IN	TIC	When HIGH this signal indicates a write transfer and when LOW a read transfer.

Signal Name	Type	Source / Destination	Description
HSIZEOUT[2:0]	IN	TIC	Indicates the size of the transfer, which is typically byte (8-bit), halfword (16-bit) or word (32-bit). The TIC does not support larger transfer sizes.
HBURSTOUT[2:0]	IN	TIC	Indicates if the transfer forms part of a burst. The TIC always performs incrementing bursts of unspecified length.
HPROTOUT[3:0]	IN	TIC	The Protection control signals indicate if the transfer is an opcode fetch or data access, as well as if the transfer is a supervisor mode access or user mode access. These signals can also indicate whether the current access is cacheable or bufferable.
HWDATAOUT[31:0]	IN	TIC	The write data bus is used to transfer data from the master to bus slaves during write operations. A minimum data bus width of 32 bits is recommended, however this may easily be extended to allow for higher bandwidth operation.

Table 9.3-3: The TIC Watcher Signal Description**TIC Watcher Functional Description**

The TIC Watcher block diagram is shown in **Figure 9.3-4**.

**Figure 9.3-4 The TIC Watcher Block Diagram**

The TIC Watcher receives the same vectors as the TIC does, from the Ticbox and generates AHB signals internally. It then compares the internally generated signals with the corresponding signals from the TIC and flags error messages whenever a protocol violation is detected on the bus.

Test Vectors

The test vectors for the integration testing of the MPMC are coded into separate C files for testing accesses to all the MPMC registers and for Integration testing. Make options are provided to run the tests together or individually.

10 INTEGRATION TESTS

The test vectors that are applied to the TICTalk testbenches are in the *integration/bustest* directory. The testvectors for integration testing are written in BusTalk and then converted into *.tif* format to be applied to the TICTalk integration testbench. The TIC validation test cases and the integration tests are used to test the MPMC's TIC and for integration testing respectively.

10.1 TIC Validation Test Cases

The TIC validation tests are used to test the functionality of the TIC in the MPMC. These tests are performed in the integration test environment rather than the standard BusTalk verification environment because of the need to stimulate the TIC interface through the ticbox, which is not available in the BusTalk verification environment.

10.1.1 Transfer tests

In this test, vectors are applied in different sequences with different HSIZE values. Tests are done in Address Increment enabled/disabled mode of the TIC. Address boundary check is done by applying writes and reads to the boundary address in the address increment enabled mode.

10.1.2 Response tests

In this test, the generic slave is configured to give out different responses (ERROR/SPLIT/RETRY) and the TIC performance is tested.

10.2 Integration Tests

10.2.1 AMBA signal integration test

The AHB signals connected to the AHB Slave ports of the MultiPort Memory Controller signals are tested with AHB transactions to the MPMC.

10.2.2 Non-AMBA intra-chip integration test

Integration Testing of Intra-chip Inputs

When Integration tests are run with the MPMC in a stand-alone test set-up:

Write a '1' to the ITEN bit in the Control register. This selects the test path from the MPMCITIP[0] register bits to the internal signal.

Write different values to the MPMCITIP[0] register address and read the same register address to ensure that the value written is read out.

Repeat the above steps for other valid bits of MPMCITIP register.

When Integration tests are run with the MPMC as part of an integrated system:

Write a '0' to the ITEN bit in the Control register. This selects the normal path from the external Intra-chip Input pin to the internal core logic.

Write different values into output register of source intra-chip device so as to toggle the Intra-chip signals. Read from the MPMCITIP[9:0] register to verify the value.

Integration Testing of intra-chip outputs

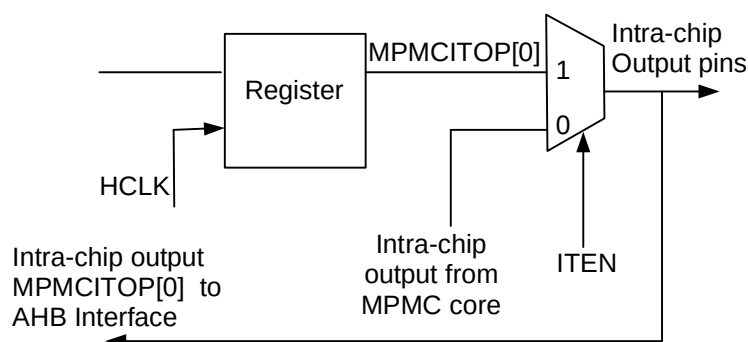


Figure 10.2-1 Output integration test harness Intra-chip

When Integration tests are run with the MPMC in a stand-alone test set-up:

Write a '1' to the ITEN bit in the Control register. This selects the test path from the MPMCITOP[0] register bit to the intra-chip output signal.

Write a '1' and '0' to the MPMCITOP[0] register address and read the same register address to ensure that the value written is read out.

Repeat the above steps with MPMCITOP[1] register bit.

When Integration tests are run with the MPMC as part of an integrated system:

Write a '1' to the ITEN bit in the Control register. This selects the test path from the MPMCITOP[0] register bit to the Intra-chip Outputs.

Write '1' and '0' into the MPMCITOP[0] register and verify it by reading from input register of target intra-chip device.

Repeat the above step with MPMCITOP[1] register bit.

10.2.3 Primary I/O signal integration test

The Primary inputs and outputs (external memory signals) can be tested external to the chip with some memory accesses through the MPMC.

10.3 Test Vector Generation in the Integration Test environment

A **make sourcefilename** (without the .c extension) in the *integration/bustests* directory will create .tif formatted and .sim formatted vectors in the *integration/bustest/invect* directory. The makefile in *integration/bustest* directory can be referred for the make options.

The **make all** option can be used to create a test vector file including all the tests.

11 CUSTOMISATION

The ARM PrimeCell MultiPort Memory Controller can be modified to suit specific user configurations, but the full benefit of a fast integration will be only obtained when the design is used without modification. In the event of modification, it is the user's responsibility to ensure that the design is fully functional by appropriate amendment of the functional test program and the testbenches where required. Synthesis will have to be rerun to ensure that timing is met. Some of the customisations that are possible, and the changes that need to be done for each of them are listed in the following sub-sections.

11.1 Changing the number of AHB memory interface ports

The ARM PrimeCell MPMC (PL172) currently implements 4 AHB memory interface ports. The number of AHB memory interface ports can easily be changed. The design supports upto 8 AHB memory interface ports with minor modifications to the code.

If the number of memory interface ports need to be changed, the following changes will need to be done:

- The AHB interface block is generic and the same block (MpmcAhbif) may be instantiated 'N' times in the top-level of the AHB memory interfaces (MpmcAhbTop), where N stands for the number of AHB memory interface ports.
- The signals related to the unconnected AHB interfaces in the top-level of the AHB memory interfaces (MpmcAhbTop) should be tied to zero. In the ARM PrimeCell MPMC (PL172), the signals related to AHB ports 4 to 7 have been tied to zero in MpmcAhbTop.

To support more than 8 AHB memory interface ports, modifications will be required in the arbiter block (MpmcArbiter). Currently, the arbiter in MPMC (PL172) arbitrates among 8 AHB ports.

11.2 Reducing memory data bus width

Reducing the databus width of the MPMC from 32-bits to 16-bits can reduce the pin counts.

Table 11.2-1 shows the pin counts that can be achieved for 32-bit and 16-bit systems containing dynamic and static memory and a TIC. The MPMC (PL172) currently supports a 32-bit data bus.

Data bus	Control	Address	Total
16	32	28	76
32	32	28	92

Table 11.2-1 Pin counts for systems containing dynamic and static memory and TIC

No RTL changes are required to the MPMC for this pin count reduction. Only the lower 16-bits need to be bonded out in this case.

11.3 Removal of TIC

If TIC testing is not required in a system, the MPMC's TIC interface can be removed.

Table 11.3-1 shows the pin counts that can be achieved for 32-bit and 16-bit systems containing dynamic and static memory. The MPMCTESTIN, MPMCTESTREQA and MPMCTESTREQB signals are not required since the TIC Controller is not used.

Data bus	Control	Address	Total
16	29	28	73
32	29	28	89

Table 11.3-1 Pin counts for systems containing dynamic and static memory

To exclude the TIC controller, the following modifications are required to ensure that the external memory interface signals that are shared by the static memory and dynamic memory and TIC functions correctly:

- Remove the instantiation of MpmcTIC from the top-level of the MPMC (PL172).
- Tie-off the TBUSOUT[31:0] signal input to the MPMC core logic (MpmcCore) to zero at the top-level of the MPMC (PL172).
- Drive a HIGH on nTicDataEn[3:0] and TESTACK at the top-level of the MPMC (PL172).
- Tie-off the TICBUSREQ signal input to the MPMC core logic (MpmcCore) to zero at the top-level of the MPMC (PL172).

11.4 Removal of Static Memory Controller

If the static memory is not being accessed through the MPMC in a system, the static memory controller in the MPMC can be removed.

Table 11.4-1 shows the pin counts that can be achieved for 32-bit and 16-bit systems containing dynamic memory and TIC. The MPMCADDRROUT[27:15] address signals are not required. The following static memory signals are not required:

- nMPMCSTCSOUT[3:0]
- nMPMCBLSOUT[3:0]
- nMPMCOEOUT

Data bus	Control	Address	Total
16	23	15	54
32	23	15	70

Table 11.4-1 Pin counts for systems containing dynamic memory and TIC

To remove the static memory controller from the MPMC (PL172), the following changes are required:

- Remove the instantiation of the static memory controller from the top-level of the MPMC (PL172) memory controller block
- Remove the instantiation of the memory bus granting logic (MpmcMemBGnt) and drive DyBusReq back to the dynamic memory controller block (MpmcDyMemCntl) on the DyBusGnt input port of MpmcDyMemCntl.
- Tie-Off to their inactive values, the static memory controller outputs that are connected to the memory controller interface block (MpmcMemCntlIf) in the memory controller top-level block (MpmcMemCntl)
- Modify the pad interface block (MpmcPadIf) to remove the usage of the static memory controller signals – nStWEQ, StDataOutQ[31:0], nStDataEn[3:0] and StAddrOutQ[27:0].
- Reduce the width of the memory address signals MPMCADDRROUT[27:0] to MPMCADDRROUT[14:0].

11.5 Removal of Dynamic Memory Controller

If the dynamic memory is not being accessed through the MPMC in a system, the static memory controller in the MPMC can be removed.

Table 11.5-1 shows the pin counts that can be achieved for 32-bit and 16-bit systems containing static memory and a TIC.

The following dynamic memory signals are not required:

- MPMCCCLKOUT[3:0]
- MPMCFBCLKIN[3:0]
- nMPMCDYCSOUT[3:0]
- MPMCCKEOUT[3:0]
- nMPMCRASOUT
- nMPMCCASOUT
- nMPMCRPOUT.
- MPMCDQMOUT[3:0]

Data bus	Control	Address	Total
16	11	28	55
32	11	28	71

Table 11.5-1 Pin counts for systems containing static memory and TIC

To remove the dynamic memory controller from the MPMC (PL172), the following changes are required:

- Remove the instantiation of the dynamic memory controller (MpmcDyMemCntl) from the top-level of the MPMC (PL172) memory controller block
- Remove the instantiation of the memory bus granting logic (MpmcMemBGnt) and drive StBusReq back to the static memory controller block (MpmcStMemCntl) on the StBusGnt input port of MpmcStMemCntl.
- Tie-Off to their inactive values, the dynamic memory controller outputs that are connected to the memory controller interface block (MpmcMemCntlIf) in the memory controller top-level block (MpmcMemCntl)
- Modify the pad interface block (MpmcPadIf) to remove the usage of the dynamic memory controller signals.

11.6 Removal of TIC and Static memory controller

If the static memory is not being accessed through the MPMC in a system; and if the TIC is also not being used, the static memory controller and the MpmcTIC modules in the MPMC can be removed.

Table 11.6-1 shows the pin counts that can be achieved for 32-bit and 16-bit systems containing dynamic memory.

The MPMCADDRROUT[27:15] address signals are not required. The following static memory and TIC signals are not required:

- nMPMPMCSTCSOUT[3:0]
- nMPMCBLSOUT[3:0]

- nMPMCOEOUT
- MPMCTESTIN
- MPMCTESTREQA
- MPMCTESTREQB

Data bus	Control	Address	Total
16	20	15	51
32	20	15	67

Table 11.6-1 Pin counts for systems containing dynamic memory

To remove the static memory controller and the TIC from the MPMC (PL172), the following changes are required:

- Remove the instantiation of the static memory controller from the top-level of the MPMC (PL172) memory controller block
- Remove the instantiation of MpmcTIC from the top-level of the MPMC (PL172).
- Remove the instantiation of the memory bus granting logic (MpmcMemBGnt) and drive DyBusReq to the output MPMCEBIREQ port. Drive MPMCEBIGNT input port value on the DyBusGnt input to the dynamic memory controller block (MpmcDyMemCntl).
- Tie-Off to their inactive values, the static memory controller outputs that are connected to the memory controller interface block (MpmcMemCntlIf) in the memory controller top-level block (MpmcMemCntl)
- Modify the pad interface block (MpmcPadIf) to remove the usage of the static memory controller signals and the TIC signals— nStWEQ, StDataOutQ[31:0], nStDataEn[3:0], StAddrOutQ[27:0], TESTACK, nTicRead and TBUSOUT[31:0].
- Reduce the width of the memory address signals MPMCADDRROUT[27:0] to MPMCADDRROUT[14:0].

11.7 Removal of TIC and Dynamic memory controller

If the dynamic memory is not being accessed through the MPMC in a system; and if the TIC is also not being used, the dynamic memory controller and the MpmcTIC modules in the MPMC can be removed.

Table 9.7-1 shows the pin counts that can be achieved for 32-bit and 16-bit systems containing static memory.

The following dynamic memory and TIC signals are not required:

- MPMCCLKOUT[3:0]
- MPMCFBCLKIN[3:0]
- nMPMCDYCSOUT[3:0]
- MPMCCKEOUT[3:0]
- nMPMCRASOUT
- nMPMCCASOUT
- nMPMCRPOUT.
- MPMCTESTIN
- MPMCTESTREQA

- MPMCTESTREQB

Data bus	Control	Address	Total
16	8	28	52
32	8	28	68

Table 11.7-1 Pin counts for systems containing static memory

To remove the dynamic memory controller and the TIC from the MPMC (PL172), the following changes are required:

- Remove the instantiation of the dynamic memory controller (MpmcDyMemCntl) from the top-level of the MPMC (PL172) memory controller block
- Remove the instantiation of MpmcTIC from the top-level of the MPMC (PL172).
- Remove the instantiation of the memory bus granting logic (MpmcMemBGnt) and drive StBusReq to the output MPMCEBIREQ port. Assert StBusGnt input to the static memory controller module (MpmcStMemCntl) only if MPMCEBIGNT is asserted and MPMCEBIBACKOFF is not asserted.
- Tie-Off to their inactive values, the dynamic memory controller outputs that are connected to the memory controller interface block (MpmcMemCntlIf) in the memory controller top-level block (MpmcMemCntl)
- Modify the pad interface block (MpmcPadIf) to remove the usage of the dynamic memory controller and TIC signals.

11.8 Reducing memory device support

Some of the address pins can be removed depending upon the memory devices to be supported. For example, if 32Kx32 or 32Kx16 or smaller devices static memory devices are to be supported then address lines MPMCADDROUT[27:15] are not required. In this case removing these address lines would not reduce the number of dynamic memory devices supported.

No RTL changes are required to the MPMC for this pin count reduction.